claude-windows / 56b7f849-5d56-4770-93e0-b3b4da943700

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700.jsonl`
- SHA-256: `555ac4d0258465c28015689daa1bdaeeb15c075ee78ddf4b820de924eb5c120f`
- Source modified: `2026-06-23T10:17:26+00:00`
- Imported at: `2026-07-05T16:48:17+00:00`
- project: `C--Users-avidu-Projects-badminton-highlight-indexer`
- session_id: `56b7f849-5d56-4770-93e0-b3b4da943700`

Transcript

1. user (2026-06-07T06:40:39.480Z)

Hi. Can you run that skill which I added today regarding github repos at session start? I want you to "pick up from you left off" last time and I have a starter project for you which is to plumb a very rich "observability output per processed video" feature in the highlights generation tool and also the data collector tool. i.e. once the data collector or sports highlighter ingests / processes a new video, they should also emit a very rich side output which captures all the stages the video went through anfd the details eg. fps, resolution, frames extracted, rallies etc. all the details assuming it will be used to catch some inconsistencies and evaluate output differences as well. Show me a plan for both and then I want you to ensure that you iterate on the quality of the report by evaluating it as if "someone who is reading the README file and some FAQs is running the tool and he should be able to make sense of the report and do an initial debugging if required" as a stretch goal, we should also ensure we produce a "customer friendly video report" where we would remove all the system processing details and the user will know metadata about the video and some meta level highlights emitted by the tools processings.
I am happy to clarify any questions you have and make sure you plumb it well enough that this 'house of cards' doesn't fall!

2. assistant (2026-06-07T06:40:46.440Z)

I'll start by running the session-start skill you added today, then read my handoff/memory to pick up where we left off.

3. user (2026-06-07T06:40:46.459Z)

Launching skill: anthropic-skills:new-session-hello

4. user (2026-06-07T06:40:46.453Z)

Base directory for this skill: C:\Users\avidu\AppData\Roaming\Claude\local-agent-mode-sessions\skills-plugin\b4180250-5146-4b86-9001-c9f3dd74258c\87f748ce-e72b-44f3-8ef0-78e3212a99c3\skills\new-session-hello

Read the github folders linked in the session and build a context based on it and mostly there could be a code map file which you write to index the code so that re-read is faster and cheaper. Always look to build context from where I had left off, all this only when the session is initiated with >=1 github repositories.

5. user (2026-06-07T06:40:50.522Z)

```
1       ---
2       name: new-session-hello
3       description: "I want claude to read the github repos linked and have some context when I start a new coding session"
```

```
4       ---
5
6       Read the github folders linked in the session and build a context based on it and mostly
there could be a code map file which you write to index the code so that re-read is faster and
cheaper. Always look to build context from where I had left off, all this only when the session
is initiated with >=1 github repositories.
7
```

6. assistant (2026-06-07T06:40:57.137Z)

```
Skill loaded. Per its instructions and my memory's "? Start here" pointers, let me ramp up from
where I left off. Reading the live checkpoint, handoff, and code-map pointers in parallel.
```

7. user (2026-06-07T06:40:58.629Z)

```
1       ---
2       name: current-state
3       description: "Live micro-checkpoint state for Claude ? updated every checkpoint (not
only on request). What just landed, what's next, open decisions."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: b13673cc-f087-43b1-9527-35dc0b67b301
8       ---
9
10      # Current state ? updated continuously (see [[working-agreement-checkpoints]])
11
12      **Checkpoint:** 2026-06-07 · ? **SESSION SUNSET.** master clean (PRs #25?#35 merged; 209
tests green; only
13      feat/khelacharya-shot-intelligence retained; no open PRs). Audio E1 done ? classical
onset = ~2.2× ceiling, NOT
14      adopted (detector kept, PR #35); quantitative findings + lessons in
[[audio-e1-findings]] (+ plan §10). VLC
15      annotator = standalone repo github.com/avidullu/rally-annotator (MIT, multi-sport).
16      **?? RESUME NEXT SESSION on EITHER:** (1) **GOLDEN EVALS** ? owner self-rates clips in
the VLC annotator ?
17      `import-local`/`--labels` ? unblocks distilled-model LOVO (n?3) + human-gold E1 re-
confirm = the highest-leverage
18      unblock; OR (2) **AUDIO E2** ? learned timbre classifier (needs labeled hits). Also
queued: pose/stance #2, owner's
19      "other hunch", Option B (owned end-to-end model). Read [[audio-e1-findings]], [[code-
map-artifact]] (docs/CODE_MAP.md),
20      docs/RALLY_DETECTION_RESEARCH.md, ADR-007 to ramp.
21      ? **PR #36 (docs/AUDIO_E1_FINDINGS.md) MERGED to master 2026-06-07** (owner restored the
branch after an accidental
22      close; reopened + squash-merged). Record now in-repo + [[audio-e1-findings]] (memory) +
plan §10. master fully clean, no open PRs.
23
24      --- (prior checkpoint) CLEAN SLATE. Indexer PRs #25?#32 MERGED; master 202 tests green,
tree clean, branches pruned
25      (kept feat/khelacharya-shot-intelligence). Audio decided (ADR-007). VLC annotator ?
standalone public repo
26      github.com/avidullu/rally-annotator (MIT, multi-sport). **ALL PRs #25?#34 MERGED; slate
FULLY clean** (master only + retained feat/khelacharya-shot-intelligence; no open
27      PRs; tree clean). #33 removed the stale annotator copy; #34 expanded ADR-007 (fusion +
Option B) + repointed the
28      ref ? both merged safely. **PARKED: modeling waits on golden labels (user self-rates via
annotator); next code
29      lever = AUDIO (E1 go/no-go probe).**
30      **ARCHITECTURE DECISION captured (ADR-007 §Integration architecture, PR #34):** audio
fuses in OUR rally layer
31      (WASB stays frozen black box ? feature-level into distill_local + decision-level recall-
recovery); **Option B = the
32      intended end-state** ? once a healthy golden corpus exists, train an OWNED end-to-end
multimodal model (frames+audio+
```

pose+??rallies) fine-tuned on our data, dropping WASB-weight dependence + recall ceiling. Modular cues now (audio? stance?other) are how we earn Option B (each generates the labels it needs). See resumption runbook ?.

DISTILLED MODEL BUILT (2026-06-06) ? user wants this approach; honest result = needs more data

- **ALL PRs #27-#31 MERGED to master** (0c6f2c1). master now has: code-map, gate-A/rally_gate, gemini-refine (+provider hardening + extract_video_chunk scale_width), calibrate_local, distill_local. This session's 5 branches PRUNED (local+remote). Retained: feat/khelacharya-shot-intelligence (PR #11). Pre-existing merged stragglers still on origin (left, optional prune): docs/ending-reason-forced-unforced(#19), feat/rally-slicer(#20), feat/wasb-substrate(#21).
- **User will RATE the clips ? cached artifacts LEFT IN PLACE** (do NOT delete): Temp\{adarsh,testlarge}_frames + trajectories (~/clips + Temp), Temp\verylarge_frames (partial 12246/46080). Adarsh+TestLargeVideo trajectories ready ? when user import-locals golden labels, calibrate_local/distill_local use them with zero re-extraction. (#30 stranded calibrate_local on gate branch ? RECOVERED in PR #31.)
- `backend/eval/distill_local.py` (PR #31, OPEN): learned model ? WASB recall-oriented candidates (0.005,1.0,0.5) ? features [duration, peak_velocity, mean_velocity, active_ratio, net_crossings] (resolution/fps-INVARIANT) ? classifier.LogisticRegression trained on Gemini labels (training.label_candidates IoU). Honest LOVO + saves output/local_rally_model.json. Reuses classifier/training/rally_gate/metrics/gemini_refine.
- **HONEST RESULT (2 videos): LEARNED UNDERPERFORMS baseline** ? mean held-out F1 0.359 (learned) vs 0.438 (threshold baseline). Causes: (1) n=2 too few ? no transferable model; (2) WASB proposal recall CEILING 0.43 on TestLargeVideo (its windows don't align with Gemini rallies; Adarsh ceiling 0.81). Framework READY; needs more teacher videos + better WASB recall to beat baseline. Final weights: mean_velocity + duration dominate, net_crossings barely used (unreliable at n=2).
- ? **VeryLargeTestVideo (4K, 46080 frames, 12m49s, 11.5GB) = held-out FINAL test for USER MANUAL EVAL.** Extraction running (task bzt8hc4ks, frames?1920). Then WASB ? baseline + learned rally lists ? user manually evals (manual = GT; no Gemini needed on it). User's manual eval becomes a 3rd-video human GT ? strengthens model.
- PR #31 (feat/distill-local) OPEN ? merge to get distill_local + calibrate_local onto master.

? AUDIO SCAFFOLDING + PRELIMINARY E1 DONE (2026-06-07) ? PR #35 (feat/audio-hit-detector-e1, OPEN). Verdict = AMBER.

- Built `backend/audio/hit_detector.py` (ffmpeg?numpy spectral-flux onset?adaptive peak; NO scipy; 5 tests) + `backend/eval/audio_e1_probe.py` (E1 diagnostic). 207 tests green. WASB untouched (Option A).
- Ran preliminary E1 on EXISTING GoPro audio vs **Gemini SILVER** labels (no wait for human labels): Adarsh (36 rallies) + TestLargeVideo (7).
- **RESULT (honest, AMBER):** ? audio fires in **100% of rallies** and **100% of WASB-missed rallies have hit-clusters** ? real RECALL potential (the bottleneck). ?? but **discrimination weak**: 1.3x @k=2.5, peaks **~2.8x @k=4**, then a CLIFF (k=6 ? ~0 hits); hit rate high (56/min @k=2.5 = many non-shuttle transients: footwork/voices/ambient); **audio-only F1 0.13-0.27 < WASB-only (0.33-0.54).** So

audio is NOT clean enough
69   to stand alone ? it's a **fusion/recall cue**, not a standalone segmenter.
70   - **OPTION A DONE (band-pass) ? NEGATIVE RESULT (2026-06-07, committed to PR #35):**
added a `band` knob to the
71   onset detector + swept low/mid/high bands × k at 32kHz on both videos. **Band-pass
does NOT help** ? HF
72   "shuttle-click" bands gave LOWER discrimination, not higher. **Ceiling stays ~2.2x**
(in-rally vs MID-MATCH
73   dead-time). **Not a warm-up/label artifact** (mid-match dead alone ~0.6 hits/s vs ~1.3
in-rally on both).
74   Root cause: between-point activity (shuttle tosses=our false-fire, footwork, talk) =
transients a classical
75   onset detector can't separate from rally hits. `k` only trades coverage for
discrimination. Kept the band knob
76   (cheap/optional) but it's NOT the fix. Full writeup: AUDIO_RALLY_DETECTION_PLAN.md
§10.
77   - **HONEST VERDICT: classical-onset audio is too weak to be a clean cue (~2.2x); band-
pass won't rescue it.**
78   Remaining audio path = **E2 learned TIMBRE classifier** (MFCC ? shuttle-thwack vs
toss/footstep/voice; needs
79   labeled hits ? real work, the only plausible way past 2.2x). Cheaper: re-confirm on
human-gold (unlikely to
80   move much per the warm-up check). **Recommend re-prioritizing:** owner's "other hunch"
/ pose-stance #2 /
81   more golden data for WASB+distilled, rather than over-investing in classical audio.
User to decide when back.
82   - **PR #35 review addressed (2026-06-07):** owner left 2 refactor comments (PR otherwise
LGTM). (1) made
83   `backend/eval/audio/` package + moved probe ? `backend/eval/audio/e1_probe.py` (folded
INTO #35 since it's a
84   new unmerged file ? cleaner than a stacked PR; import/doc refs updated). (2) broader
tests/ de-fragmentation
85   filed as a TODO in `docs/BACKLOG.md` (Code & test organization) ? grouping metric is
owner's design call,
86   dedicated PR later. 209 tests green; replied on the PR. **#35 still OPEN, ready to
merge as the single PR.**
87
88   ## ? DECISION DOCUMENTED (2026-06-07): AUDIO next, STANCE parked ? PR #32 (docs/audio-
plan-adr007)
89   - **ADR-007** (docs/DECISIONS.md): adopt fully-local AUDIO "thwack" hit detection FUSED
with WASB trajectory as
90   the next rally cue (attacks recall bottleneck); park pose/stance as #2 (harder);
Gemini = offline-teacher only;
91   gate experiments on golden labels. **Sequence: AUDIO now ? [TBD owner hunch] ?
POSE/STANCE.**
92   - **docs/AUDIO_RALLY_DETECTION_PLAN.md** ? detailed plan:
extract?onset/peak?confidence?hit-cluster?fuse-with-WASB;
93   experiments E1(GO/NO-GO recall probe on real GoPro audio) / E2(LR+MFCC FP suppression)
/ E3(audio features into
94   distill_local) / E4(boundary tighten); deps numpy+scipy (permissive); top risk =
shuttle SNR (quiet vs tennis).
95   New module planned: `backend/audio/hit_detector.py`.
96   - **docs/VIDEO_RECORDING_GUIDELINES.md** ? AUDIO now a first-class capture REQUIREMENT
(record w/ audio on, mic
97   unobstructed). User will add "enable audio always" to camera-setup docs.
98   - **Research checked in** (was uncommitted): docs/RALLY_DETECTION_RESEARCH.md. **VLC
annotator checked in**:
99   tools/vlc_rally_annotator/ (source+README). All in PR #32.
100   - Owner has "another hunch" to slot BEFORE stance (TBD).
101
102   ## ? RESEARCH DONE (2026-06-06) ? rally-detection methods ?
`docs/RALLY_DETECTION_RESEARCH.md` (full cited report)
103   Verdict: teacher?student is SOUND but NOT uniquely best ? augment with a COMPLEMENTARY
LOCAL multi-cue stack.
104   **#1 cheapest lever = AUDIO hit detection (shuttle "thwack") fused with the trajectory ?

directly attacks the
105   WASB recall bottleneck with inputs we already have.** #2 = pose/court serve-ready START
+ flight-gradient END
106   (our "gate B", validated). #3 = compact HitNet GRU (MonoTrack ~16K params). Audio+visual
fusion lifts rally
107   RECALL (tennis HMM: 83% fused vs 54% visual/63% audio). Keep Gemini OFFLINE-teacher
only. ? caveats: most lit
108   numbers are BROADCAST + per-frame (not IoU boundary F1, not amateur single-cam) ? won't
transfer cleanly; ?
109   LICENSE TBD for ShuttleSet/MonoTrack/WASB-weights (commercial). Cheap next experiments
E1-E4 in the doc (E1:
110   audio energy-peaks overlay on WASB windows ? recover misses; E3: add audio features to
the distill student).
111
112   ## ? VLC RALLY ANNOTATOR ? STANDALONE PUBLIC REPO (2026-06-07):
github.com/avidullu/rally-annotator
113   MOVED OUT of the indexer into its own **MIT, public, multi-sport** repo (default branch
`main`) per user ? for
114   open-source release + iteration across **net-separated racquet sports**
(badminton/tennis/table_tennis/
115   pickleball/padel). Local clone: `C:\Users\avidu\Projects\rally-annotator\` (git identity
= Avi Dullu
116   <avi.dullu@gmail.com>, mirrored from indexer). Layout: `vlc/rally_annotator.lua` (v1.2 ?
added Sport dropdown +
117   `sport` CSV column; generic naming) + README + LICENSE(MIT) + docs/CSV_FORMAT.md +
CHANGELOG. CSV now
118   `rally_number,start_time,end_time,ending_reason,sport`. **v1.2 INSTALLED** to
`%APPDATA%\vlc\lua\extensions\
119   rally_annotator.lua` (replaced v1.1). ? v1.2 multi-sport build NEEDS user's live VLC
smoke test (I can't click
120   VLC). Removed from indexer (PR #32 now docs-only; ADR-007 ref repointed to the new
repo).
121   Roadmap (in the new repo README): live test all 5 sports; per-sport hotkeys; python-
vlc/HTML5 front-ends.
122   --- historical (v1.1, badminton-only, was in indexer tools/) ---
123   Button-dialog (not pause-hook;
124   pause callback flaky macOS/broken VLC4): View menu ? "Badminton Rally Annotator" ? Mark
START / ending_reason
125   dropdown (winner/forced_error/unforced_error/service_fault/let/other) / Mark END / Undo
+ live status. Reads
126   vlc.var.get(input,"time")/1e6 ? decimal sec. Appends
`rally_number,start_time,end_time,ending_reason` to
127   `<video-stem>.rallies.csv` next to the video (continues numbering across re-enable ? no
dup rally_number).
128   CSV ingests UNCHANGED by backend/eval/calibrate_wasb.py --labels,
backend/tools/rally_slicer.py --labels, AND
129   collector import-local. Verify=high confidence (fixed dup-rally_number, HTML status
label, install-doc repo
130   mixup) but NEEDS user's live VLC test (I can't click VLC). Alternatives if cramped:
python-vlc+Tkinter (global
131   hotkeys) or HTML5 page (shareable w/ remote raters). This is the golden-data generation
tool ? feeds the model.
132
133   ## ? GOLDEN DATA via SELF-RATING (confirmed 2026-06-06) ? preferred over Gemini silver
134   User can be a RATER: make a CSV `rally_number,start_time,end_time` (sec; JSON ok;
optional shots_count/ending_reason)
135   ? `python -m src.cli.curate import-local --sport badminton --video-path <vid>
--annotations-path <csv> --license Proprietary`
136   (validates rallies via validate_badminton_rallies, registers CURATED) ? `curate export`
? indexer eval format ?
137   add to calibrate_local/distill_local manifest. HUMAN golden > Gemini silver ? directly
fixes the 2 caps (too few
138   videos + label quality). Collector `import_local` @src/cli/curate.py:52; CSV parse
@_parse_local_annotations:144.
139   **Pause decision 2026-06-06:** modeling PAUSED until more golden data (user self-rating

+ next week's vendor videos).
140   VeryLargeTestVideo 4K extraction STOPPED at 12246/46080 (partial frames
Temp\verylarge_frames deletable). Research
141   workflow wxzp6h7xk still running ? post report then fully paused.
142
143   ## ?? NEXT-WEEK RESUMPTION RUNBOOK (more teacher videos ? re-run calibrate_local +
distill_local; learned beats baseline once n grows)
144   **Goal:** more teacher-labeled videos ? stable leave-one-video-out ? then build the
LEARNED distilled model
145   (classifier.py on Gemini labels) ? the real local-precision lever (threshold-tuning
ceiling'd at ~0.44 P/F1, overfits on 2 videos).
146   **FIRST: merge the open PR stack** so master has the tools: #27 (code-map), #28
(gate-A/rally_gate), #29
147   (gemini-refine + provider hardening + extract_video_chunk scale_width), #30
(calibrate_local, stacked on #28 ? retarget to master after #28).
148   **Per NEW video (repeat for each):**
149   1. Probe: `ffprobe ... <video>` ? note fps, width. (TestLargeVideo was 5.3K ? extract
frames downscaled to 1920.)
150   2. Extract frames: Windows `ffmpeg -i <video> [-vf scale=1920:-2] -q:v 2
<Temp>/<name>_frames/%08d.jpg`.
151   3. WASB trajectory (WSL, checkpointed): re-stage wasb_infer.py to `~/models/WASB-
SBDT/src`, then
152   `conda activate wasb && cd ~/models/WASB-SBDT/src && python wasb_infer.py
--frames_dir /mnt/c/.../<name>_frames
153   --weights ../pretrained_weights/wasb_badminton_best.pth.tar --out
~/clips/<name>_traj.csv --batch-size 8 --num-workers 4 --flush-every 500`
154   then copy CSV to a Windows path.
155   4. Gemini teacher labels (paid key in env GEMINI_API_KEY; NEVER store key): `python -m
backend.eval.gemini_refine
156   --video <video> --full-video --out output/<name>_gemini_fullvideo.csv` (model gemini-
flash-latest; clips auto-downscaled <500MB).
157   5. Add `{name,trajectory,fps,frame_width,labels}` to the calibration manifest (see
output/local_calib_manifest.json).
158   **Then re-run calibration + build the learned model:**
159   6. `python -m backend.eval.calibrate_local --manifest <manifest> --metric f1` (and
--metric precision) ? honest LOVO across N videos.
160   7. **NEW (the real lever):** train `backend/eval/classifier.py` (LogisticRegression) on
pooled Gemini labels: label each WASB
161   candidate rally-vs-not by IoU-match to Gemini labels (training.py::label_candidates),
features = candidate stats
162   (peak/mean velocity, active_frame_count) + net-crossings (rally_gate). Score held-out
(LOVO). Compare to threshold-calib.
163   **Artifacts that persist (don't redo):** trajectories
`~/clips/{adarsh,testlarge,sample_long}_traj.csv` (+ Windows Temp copies);
164   Gemini labels output/{testlarge,adarsh}_gemini_fullvideo.csv; manifest
output/local_calib_manifest.json.
165   **Deletable (big, regenerable from trajectories):** Temp/{adarsh_frames(7.9GB),
testlarge_frames, sample_long_frames(4.1GB)}.
166
167   ## OLD checkpoint context below ? (this session's journey)
168
169   ## DEMO RUN ? 2nd video "Adarsh and Vasanth.MP4" (2026-06-06, stress + checkpoint test)
170   - Video: `C:\Users\avidu\Projects\Annotation Setup\Collect\Adarsh and Vasanth.MP4`,
1920x1080, 59.94fps,
171   40,536 frames, 11m16s, 5.5GB. Stress-tested infra (1.9x Sample Long).
172   - Frames: ffmpeg -> `C:\?\Temp\adarsh_frames` (40,536 JPGs, 7.9GB, deletable). Inference
40,534 windows
173   in 1272s @ 31.9 win/s; 13,224/40,536 visible (32.6%). Checkpoint cache kept pace (no
crash, so resume
174   not exercised, but cache fully built). Trajectory: WSL `~/clips/adarsh_traj.csv` +
`C:\?\Temp\adarsh_traj.csv`.
175   - Tuned export (cfg 0.05/1.0/1.0, no golden labels for this video): **65 rally segments,
251.4s play** ->
176   `output/adarsh_tuned_segments.csv`. User eyeballing; baseline NOT yet generated (held
until user anchors).

NOTE tuned cfg was fit on Sample Long (different video) -> transfer is the open question.

BASELINE vs TUNED on Adarsh (2026-06-06) ? tuned config does NOT transfer + over-fire pattern

- Baseline (0.02/2.0/2.0): 45 segs, 303.8s, mean 6.8s. Tuned (0.05/1.0/1.0): 65 segs, 251.4s, mean 3.9s.
- Tuned = +44% segments but -17% play: 30 tuned-only segs (almost all 1-2s = false fires); 10 baseline-only
  segs that tuned LOST are the LONG rallies (17.7/10.9/9.6/9.2/8.9/7.6s) ? merge_gap=1.0 fragments them.
- **Honest verdict: no "% improvement" (no GT for this video); tuned REGRESSES here. Baseline is the better
  operating point on Adarsh.** Concrete proof the Sample-Long-tuned cfg overfits -> need leave_one_video_out
  (2nd labelled video). `output/adarsh_baseline_segments.csv` written.
- **OVER-FIRE PATTERN (user-documented, data-confirmed):** loser tossing/flicking shuttle back for service =
  single short trajectory -> WASB detects as rally start. Real rally = shuttle crosses net MULTIPLE times.
- **FIX SPECTRUM (design, grounded in current substrate):** (A) shuttle-only: gate candidates on net/midline
  crossings >=K (?shot count) / direction-reversals ? cheap, no new model, kills single-toss fires. (B) user's
  START-STANCE STATE MACHINE: IDLE->await-stance(players settled+diagonal in service zones)->ARMED->shuttle->
  RALLY->end->IDLE; reuse the existing person detector (yolo_hybrid torchvision FRCNN+ByteTrack) as a VERIFIER
  run only ~2s around each candidate start (not full-video, which was ~50min/video). (C) pose-based serve
  detection ? richest, new model + license review, likely overkill. Rec: A now, B robust target; B needs research.
- ? **GATE A SHIPPED ? PR #28** (`feat/rally-start-gate`): `backend/detectors/rally_gate.py` (pure,
  camera-agnostic, no new model) ? estimate play axis (larger shuttle spread) + net (median coord),
  count (coord-net) sign-changes per window, keep crossings>=K. Wired into export via `--min-crossings`
  (0=off, 2 rec). 6 tests, suite 193 green. **Validated on Adarsh (side camera auto-detected, net x?716):
  real rallies 5-13 crossings, junk 0-1; K>=2 kills 18 of ~19 short tuned false fires, keeps all long
  rallies.** `output/adarsh_baseline_gated.csv` = BASELINE+gate(K=2) = 38 segs (45->38) = the "stronger
  baseline now". 1 yellow flag to eyeball: 4:39.5-4:44.7 (5.2s,1 crossing) removed ? real rally or one-sided?
- **STRATEGIC (user, 2026-06-06): paid-Gemini as a cost-controlled QUALITY fallback.** User OK slicing
  videos <1GB + paid Gemini to build a stronger baseline NOW + as a product "calculated-bit-expensive,
  quality-significantly-improving" fallback. KEY: the hybrid Phase-3 ALREADY does this (WASB candidates ->
  Gemini verifies each PADDED CANDIDATE CLIP, not whole video -> cost bounded by #candidates×cliplen, not
  duration); it died only on FREE-tier quota. So paid key re-enables it. Recommended stack: WASB propose ->
  gate A (free, cuts junk -> fewer Gemini calls) -> Gemini verify survivors (paid, precision+boundaries).
  ? PRIVACY TRADE-OFF: sending video to Gemini breaks the "vendors/AI never see user uploads" privacy lever
  ([[monetization-plan]]) ? fine for OWNED/test/golden video; for end users make it an explicit opt-in
  premium tier. More annotated videos arrive ~next week (-> leave_one_video_out for honest cross-video).

```
211     ## GEMINI REFINE (2026-06-06) ? WASB candidates -> Gemini verify/refine (in progress)
212     - `backend/eval/gemini_refine.py` (NEW, uncommitted on master working tree; + provider
retry edits in
213       `backend/providers/gemini.py`). Reuses GeminiProvider + badminton prompt +
extract_video_chunk +
214       make_predict_fn + write_segments_csv (no reinvent). Drives Gemini over SHORT WASB
candidate clips
215       (~baseline 45, padded 3s, reencoded ? tiny uploads ~450MB total, NOT GB chunks).
Gemini returns the
216       true rally per clip or [] (rejects between-point feeds it can see). **Envelope per
candidate** (1 WASB
217       candidate ? 1 rally) so Gemini fragments don't shatter a long rally below min_dur.
User has a PAID key
218       (env GEMINI_API_KEY only ? NEVER store the key in memory/disk/commits).
219     - **MODEL GOTCHA (verified 2026-06-06):** `gemini-2.5-flash` works but hits frequent
**503 "high demand"**
220       spikes; `gemini-2.0-flash` is **404 (deprecated for generateContent)** despite
appearing in models.list.
221       ? use **`gemini-flash-latest`** (clean, no 503, current stable flash). Account also
has gemini-3.x /
222       gemini-3.5-flash (newer than my training). Added upload+generate retries (3x backoff)
to the provider;
223       per-call client per worker (genai client not thread-safe ? socket errors when shared).
224     - Smoke (6 cands) clean on flash-latest: tightens boundaries (e.g. WASB 21.3-27.5 ?
Gemini 23.8-25.8),
225       keeps long rally whole. Full 45-candidate run launched (task bkap9jri2) ->
output/adarsh_gemini_refined.csv.
226     - NEXT after run: compare Gemini-refined vs WASB baseline/baseline+gate (Gemini as
SILVER-GT to finally
227       quantify Adarsh); commit gemini_refine + provider retries as a PR. Recall ceiling =
WASB candidates
228       (Gemini only sees proposed windows; rallies WASB missed entirely won't appear).
229
230     ## PRECISION FINDING (2026-06-06): full-context Gemini > candidate-refine (envelope
over-merges)
231     - Added `full_video_rallies()` to gemini_refine + `--full-video` (whole clip, downscaled
1280, chunked
232       <=120s to stay under 500MB) + clip downscaling in extract_video_chunk (scale_width).
Reuses provider/prompt.
233     - **TestLargeVideo (5.3K, 100s) head-to-head:** candidate-refine = 5 rallies/49s;
**full-context = 7 rallies/
234       34.5s**. Refine collapsed the 0:23-0:43 region into ONE 22s blob; full-context split
it into 3 real rallies
235       (23.5-26.5, 31.5-33.5, 39-42.5) with dead time between. Root cause: WASB merge_gap=2.0
bundled 3 rallies
236       into 1 candidate, then my **envelope-per-candidate** collapsed Gemini's fragments back
into 1 ? swept ~13s
237       dead time into a "rally" = PRECISION LOSS. Full-context is a strict improvement (same
rallies, tighter, +
238       correct split). Files: output/testlarge_gemini_rallies.csv (refine),
output/testlarge_gemini_fullvideo.csv (full).
239     - ? **USER CONFIRMED (2026-06-06): full-context is correct** ? 0:23-0:43 IS three
rallies (24-26, 31-34,
240       39-42), not one blob; later rallies also right. Full-context is the precision-improved
default.
241     - ? **FIXED + SHIPPED ? PR #29** (`feat/gemini-refine`): dropped the over-merging
envelope; tiny
242       invisibility splits healed by gap-tolerant merge (<=1s), real between-rally gaps kept
separate. Provider
243       hardening (upload+503 retries, MAX_UPLOAD_MB=500 guard), extract_video_chunk
scale_width downscaling,
244       default model gemini-flash-latest, per-worker client. 7 tests, suite 192 green. Modes:
refine (WASB
245       candidates) + `--full-video` (whole clip, chunked<=120s, downscaled).
246     - ? Adarsh full-context RUNNING (task b9fs6aylf) -> output/adarsh_gemini_fullvideo.csv
```

(~7 chunks).
247   - Ensemble voting (N Gemini judgments/rally, majority+median boundaries) = next
precision lever if needed.
248   - ? Gemini sees user video = breaks privacy lever ([[monetization-plan]]); OK for
owned/test video, opt-in
249   premium tier for end users.
250
251   ## ? TEACHER?STUDENT DISTILLATION (user's framing, 2026-06-06): Gemini=offline teacher,
LOCAL-ONLY WASB+gate=student
252   - GOAL: use Gemini (cloud, offline, one-time) as silver-GT to CALIBRATE the local-only
knobs so PRODUCTION
253   runs 100% local (no Gemini at runtime ? privacy lever intact, ~0 marginal cost).
Gemini quality distilled
254   into thresholds.
255   - BUILT: `backend/eval/calibrate_local.py` (+test, 2 pass). predict_fn =
trajectory_to_action_windows
256   (velocity/merge/min_dur) + rally_gate(min_crossings); grid 5×3×3×4=180; scores vs
Gemini silver labels;
257   `leave_one_video_out` across videos = honest cross-video precision.
BASELINE_LOCAL=(0.02,2.0,2.0,0).
258   **DEPENDS ON PR #28 (rally_gate)** ? built on branch `feat/local-calibration` (stacked
on feat/rally-start-gate).
259   - INPUTS: WASB trajectories (testlarge_traj.csv, adarsh_traj.csv on disk) + Gemini full-
context label CSVs
260   (output/testlarge_gemini_fullvideo.csv done=7 rallies;
output/adarsh_gemini_fullvideo.csv RUNNING b9fs6aylf).
261   - NEXT: when Adarsh labels land ? manifest of 2 videos ? run calibrate_local ? tuned
local cfg + honest LOVO
262   precision (baseline-untuned vs calibrated held-out). Commit/PR (note #28 dependency).
More videos next week
263   ? stronger LOVO.
264   - BRANCH STACK now: #27 code-map, #28 gate, #29 gemini-refine, feat/local-calibration
(on #28). Suggest user
265   merges #28+#29 so local-calibration rebases onto clean master.
266
267   ## ? RAN local calibration vs Gemini silver-GT (2 videos: TestLargeVideo=7 rallies,
Adarsh=36) ? HONEST result
268   - **Held-out (LOVO) ? the trustworthy number:**
269   · F1-optimized: baseline 0.438 ? calibrated **0.483** ±0.17, overfit gap +0.035 (small
?) = modest real lift.
270   · Precision-optimized: baseline 0.444 ? calibrated **0.320** ? ±0.10, gap **+0.245
(severe overfit)** = does NOT transfer.
271   - Per-video fit-on-self (optimistic): Adarsh P 0.489?0.731 / F1 0.543?0.683;
TestLargeVideo barely moves (F1 0.33).
272   - **CONCLUSION (honest, don't oversell):** local-only pipeline agrees with Gemini only
~0.44 P/F1; threshold-
273   tuning hits a ceiling fast and OVERFITS for precision with only 2 dissimilar videos
(5.3K phone vs Adarsh);
274   they want opposite configs. TestLargeVideo also RECALL-capped (WASB ~5 windows vs 7
Gemini rallies).
275   - **REAL LEVERS for local precision (next):** (1) MORE teacher-labeled videos (next
week) ? stable LOVO;
276   (2) LEARNED distilled model ? train existing `classifier.py` on Gemini labels using
candidate features
277   (peak/mean velocity, active_frame_count, net-crossings) instead of 4 thresholds; (3)
raise WASB recall.
278   Recommend learned-distillation over more threshold-tuning.
279
280   ## CODE_MAP artifact ? PR #27 (`docs/code-map`, OPEN) ? see [[code-map-artifact]]
281   `docs/CODE_MAP.md` = dense cold-ramp navigation map
(pipeline/subsystems/contracts/gotchas/ramp-paths/
282   test-map). Built by a map->synthesize->adversarial-verify Workflow; verify caught real
errors (validator
283   order, nonexistent config.json indexing.wasb, extract_yolo_features gotcha). Regenerate
via the

`build-code-map` workflow. Read it FIRST for code ramp-up.

? NEXT-SESSION pickup
- **User to eyeball** the tuned segments (`output/wasb_tuned_segments_vs_golden.csv`; regenerate via
  `python -m backend.eval.export_wasb_segments --trajectory C:\?\Temp\sample_long_traj.csv --labels "Annotation Setup\golden_labels_badminton.csv" --fps 59.94 --frame-width 1920 --baseline`; add
  `--video <mp4> --slice` for clips). Verdict drives whether to tune merge_gap (fragmentation) /
  min_window_duration (short rallies).
- **Real precision verdict** still needs a **2nd labelled video** ? `leave_one_video_out`.
- Optional housekeeping: stale MERGED remote branches `feat/rally-slicer` (#20) + `feat/wasb-substrate`
  (#21) still on origin (never deleted; harmless, all on master). NOT deleted this session given the
  #21-branch-delete-lost-calibrate_wasb history ? ask before pruning. KEEP
  `feat/khelacharya-shot-intelligence` (PR #11 closed, intentionally retained).

? (1) EXPORT TUNED SEGMENTS DONE ? PR #26 MERGED to master (`feat/wasb-export-segments`)
`backend/eval/export_wasb_segments.py` (+5 tests, suite 187 green). Reuses make_predict_fn +
metrics.match_intervals/score + rally_slicer (exported CSV is in the golden/slicer schema ? feeds
straight into clip cutting via `--slice --video`). Ran on REAL trajectory + golden labels:
- **34 tuned segments** (cfg 0.05/1.0/1.0); vs 25 golden: TP 22, FN 3, FP 12 ? precision 0.647,
  recall 0.880, **F1 0.746**, meanIoU 0.753 (reproduces calibration EXACTLY). Mean boundary err:
  start 0.64s / end 0.91s. Artifacts in `output/` (gitignored): `wasb_tuned_segments.csv`,
  `_baseline.csv`, `_vs_golden.csv`.
- **EYEBALL FINDINGS (honest):** recall strong, boundaries mostly <1s, but precision 0.647 = tuned
  config OVER-segments. The 3 "misses" + 12 "FPs" are partly artifacts not pure errors:
  · golden 3 (44.5-51.1s) FRAGMENTED into 2 sub-segments (44.8-46.9 + 48.4-50.8) ? neither hits
    IoU 0.5 ? 1 miss + 2 FPs from ONE rally (a merge_gap problem).
  · golden 8 (118.1-119.6, 1.5s) detected as 118.1-123.0 but boundary too long ? 1 miss + 1 FP.
  · golden 7 (95.6-98.0, 2.4s) genuinely dropped (short rally).
  · boundary overruns: golden 1 start +6.16s (missed first 6s of a long rally), g21 end +4.76s,
    g23 end +3.05s.
  · ~9 genuine extra segments to eyeball: 3.35-7.96, 36.9-39.4, 53.9-58.4, 60.5-62.9, 79.5-81.2,
    107.2-111.2, 144.5-146.6, 188.2-190.3, 339.1-343.3 (warmup / between-point motion / real
    unlabeled rallies?).
  · LEVERS if user wants better precision: raise merge_gap (fix fragmentation), revisit
    min_window_duration for short rallies; real precision verdict needs a 2nd labelled video
    (`leave_one_video_out`).

? PR status: #25 (checkpointing) + #26 (export) BOTH MERGED to master. No open indexer PRs.

NEW session 2026-06-06 (late) ? user asked for tasks in order: (4) repo cleanup ? (2) WASB checkpointing ? (1) export tuned segments
- ? **(4) repo cleanup DONE:** collector local checkout was stranded on the merged `feat/validate-delivery`

```
      branch ? synced to `master` (ff to d68c5da = PR #9 merge) + deleted the merged branch.
Both repos now
      genuinely clean on master. Untracked local `data/roora_*` artifacts left in place (not
committed).
    - ? **PR #11 (KhelAcharya shot-intel) CLOSED, branch retained.** Correction: there is NO
separate
      khelacharya GitHub repo ? that folder is a 2nd clone of THIS repo. Separation is
branch-level; #11
      kept shot-work off master. Closed to declutter; `feat/khelacharya-shot-intelligence`
branch persists,
      reopenable. See [[sibling-internship-repo]] (corrected).
    - ? **(2) WASB-pass checkpointing DONE ? PR #25 MERGED to master** (`feat/wasb-
checkpointing`).
      `wasb_infer.py` refactored: per-window detector cache `<out>_wasbcache/det_raw.jsonl`
(fsync every
      --flush-every=1000 windows) + atomic `manifest.json`; resume replays cache + skips
DataLoader to first
      un-cached window; tracker re-run over full ordered cache (cheap, deterministic);
atomic tmp+replace
      writes; trailing-corruption truncation; auto-invalidate on input change; idempotent
extract_frames.
      11 new unit tests (`tests/test_wasb_cache.py`), suite **182 green**. WSL-validated:
tracker-only re-run
      byte-identical; incremental resume matches full run except 3/300 rows ?6e-5 px (cuDNN
float noise from
      re-batching, NOT a bug); --limit change correctly invalidates. Runner gets resume for
free (cache
      derives from --out, not %TEMP%). **GOTCHA:** must re-stage `wasb_infer.py` to
`~/models/WASB-SBDT/src/`
      in WSL when running directly (runner does this automatically).
    - ? **NEXT: (1) export tuned segments** for user eyeball ("are rallies better?"). Use
tuned cfg
      (velocity 0.05, merge_gap 1.0, min_dur 1.0) on the cached trajectory ? cut rally
clips.
      Artifacts confirmed intact: `~/clips/sample_long_traj.csv` (21,284 frames) + Windows
Temp copy;
      21,284 frames in `Temp\sample_long_frames`; golden labels at `Annotation
Setup\golden_labels_badminton.csv`.

    ## Merged to master (both repos clean, branches deleted)
    - collector #6 (CVAT?canonical adapter), #7 (forced/unforced vocab), indexer #19 (docs).
    - Local master pulled + clean in both repos.

    ## WASB inference API (reverse-engineered 2026-06-06 ? for the wrapper)
    Repo `~/models/WASB-SBDT` (Hydra configs). Usage: `cd src && python3 main.py --config-
name=eval
    dataset=badminton model=wasb
detector.model_path=../pretrained_weights/wasb_badminton_best.pth.tar`.
    Inference core (`src/runners/eval.py::inference_video`): build detector + tracker +
dataloader ?
    loop batches `(imgs, hms, trans, ...)` ? `detector.run_tensor(imgs, trans)` ? per-frame
preds ?
    `tracker.refresh()` then `tracker.update(preds)` per frame ? `result_dict[img_path] =
{x,y,visi,score}`.
    That dict IS the trajectory we want.
    - `TracknetV2Detector(cfg)` (`src/detectors/detector.py`): builds model via
`build_model(cfg)`,
      loads `cfg.detector.model_path` (`checkpoint['model_state_dict']`), DataParallel,
eval. Needs
      CUDA (asserts GPU). `frames_in`=cfg.model.frames_in, `input_wh`=(model.inp_width,
inp_height).
      `run_tensor(imgs, affine_mats)` ? model ? postprocessor ? per-frame (x,y,visi,score).
      Preprocessing helpers: `dataloaders.build_img_transforms/read_image/get_transform`,
      `utils.image.get_affine_transform/affine_transform`.
    - eval.yaml defaults: model=wasb, detector=tracknetv2, tracker=online,
```

transform=default.
364   - **Wrapper plan:** extract video frames ? build `frames_in` sliding windows + affine
`trans`
365   (mirror dataset `__getitem__`) ? run detector+tracker ? emit CSV
`Frame,Visibility,X,Y`
366   (EXACTLY what existing `tracknet_runner.parse_trajectory_csv` +
`trajectory_to_action_windows`
367   already consume ? big reuse). Use weights `wasb_badminton_best.pth.tar` (NOT
monotrack=noncommercial).
368   - **Build steps:** (1) `backend/detectors/wasb_infer.py` ? **DONE + VALIDATED** (branch
369   `feat/wasb-substrate`, pushed, not yet PR'd). Runs in WSL `wasb` env; reuses WASB
ImageDataset
370   transform + detector + online tracker; emits Frame,Visibility,X,Y. Verified on
rally-18 clip
371   (121 frames ? 107 visible, X439-1036/Y64-586, smooth = real shuttle track).
372   **GOTCHAS:** must run from `~/models/WASB-SBDT/src` in `wasb` env; Hydra overrides
need
373   `runner.gpus=[0]` (single-GPU box) + `runner.device=cuda`; `__getitem__` lives in
374   `dataloaders/dataset_loader.py::ImageDataset` (samples =
`{'frames':[paths],'annos':[{frame_path,
375   center:Center(is_visible,x,y)}]}`); read frames via /mnt (Google-Drive G: extraction
done on
376   Windows ffmpeg). To run: `conda activate wasb && cd ~/models/WASB-SBDT/src && python
wasb_infer.py
377   --frames_dir <dir> --weights ../pretrained_weights/wasb_badminton_best.pth.tar --out
<csv>`.
378   (2) `wasb_runner.py` ? DONE (Win adapter: stage video+script to WSL ? run `wasb` env ?
CSV;
379   REUSES `TrackNetRunner.parse_trajectory_csv`/`trajectory_to_action_windows` as
staticmethods ?
380   those are class staticmethods, NOT module funcs; only
`to_wsl_mnt_path`/`TrajectoryPoint` are
381   module-level). (3) `wasb_hybrid.py` ? DONE (mirrors tracknet_hybrid; registered
"wasb_hybrid";
382   reads thresholds from idx_cfg.wasb). All on branch `feat/wasb-substrate` (pushed, NOT
PR'd).
383   Suite 159 green; registry lists wasb_hybrid. **NOT yet end-to-end run**: full-video
WSL pass +
384   AI handoff untested (wasb_infer core IS validated on real frames). config.json needs
an
385   `indexing.wasb` section (defaults exist in WasbConfig). (4) per-video calibration
Workflow
386   = DEFERRED (user will call it). Calibration only needs Phase-2 windows (no AI
handoff/API).
387   Preprocessing (RESOLVED): `configs/model/wasb.yaml` ? name=hrnet, frames_in=3,
frames_out=3,
388   inp 512×288 (W×H). `dataloaders/__init__.py build_img_transforms`: T.ToTensor +
Normalize
389   mean[0.485,0.456,0.406]/std[0.229,0.224,0.225] (ImageNet). Resize via affine
(`utils.image.
390   get_affine_transform` ? input_wh; the affine matrix is the `trans`/`affine_mats`
passed to
391   run_tensor and used to map heatmap coords back to original frame). So wrapper: per
sliding window
392   of 3 consecutive frames ? affine-resize to 512×288 ? ToTensor+Normalize ? stack ?
detector.run_tensor
393   ? tracker ? (x,y,visi,score) per frame ? CSV Frame,Visibility,X,Y. Weight variant:
wasb_badminton_best.
394
395   ## GOAL CORRECTION (user clarified 2026-06-06 ? important)
396   User wants to "make the WASB pretrained WEIGHTS more robust with the 1 annotated video."
Honest reality:
397   - Our annotation = **temporal rally [start,end]** labels, NOT per-frame shuttle (x,y).
WASB is a shuttle
398   DETECTOR (needs x,y heatmap supervision). **So the rally labels CANNOT fine-tune

WASB's detector weights.**
399   - What the 1 video CAN quantifiably improve = the **rally-SEGMENTATION layer on top of
WASB**: the 3
400   windowing thresholds (velocity_thresh/merge_gap/min_window_duration) + WASB variant
choice. Measure
401   IoU/F1 on HELD-OUT golden rallies (calibrate on K, test on rest). That's the
"quantifiable improvement".
402   - To genuinely fine-tune WASB DETECTOR weights ? need per-frame shuttle (x,y) labels (a
different
403   annotation, or pseudo-labels) ? separate track, not this annotation.
404   - Also: neural weights are found by backprop, not by agents "trying weights +
coordinating". Agents are
405   the right tool for the CONFIG search (variant × 3 thresholds × strategy), not weight
space.
406
407   ## Tuning plan = a Workflow (per user's ultracode ask), runs AFTER the wrapper exists
408   Once WASB trajectories exist: run a parallel **calibration Workflow** ? agents
independently search
409   {WASB variant × velocity_thresh × merge_gap × min_window_duration × search-strategy},
each scored on
410   held-out golden rally IoU, coordinate ? pick max-precision config. (Can't run before a
trajectory exists.)
411
412   ## ? PLAN: checkpoint the heavyweight WASB pass (resume across restarts) ? IMPLEMENT
NEXT SESSION
413   Problem: the GPU detector pass (21k forward passes) writes the trajectory ONLY at the
end, so a
414   reboot mid-pass loses all of it (happened 2026-06-06).
415   Key insight: **decouple the expensive GPU detector pass (checkpoint incrementally) from
the cheap,
416   stateful CPU tracker pass (just re-run from the cache).** The online tracker can't
resume
417   mid-stream, but if per-frame DETECTOR outputs are cached, the tracker re-runs over them
in seconds.
418   Plan (prioritised):
419   1. **Stable cache dir keyed by video MD5** (not %TEMP%): `output/wasb_cache/<md5>/` with
`frames/`,
420   `det_raw.jsonl` (incremental detector outputs), `trajectory.csv`, `manifest.json`
421   {video_md5, frames_total, frames_done, cfg}. (Ties to the indexer MD5-keying gotcha.)
422   2. **Idempotent extraction:** skip if frame count already == expected.
423   3. **Incremental detector cache (the big win):** in wasb_infer's batch loop, append each
batch's
424   per-frame raw detections to `det_raw.jsonl` and FLUSH every ~1000 frames; on startup
read
425   manifest/det_raw ? skip already-done frames ? resume. Loss bounded to <1000 frames
(~secs of GPU).
426   4. **Tracker re-run:** once all frames are detected (fresh or resumed), run the tracker
over the full
427   `det_raw.jsonl` in order ? `trajectory.csv`. Cheap, deterministic, re-runnable.
428   5. **Atomic writes** (write .tmp then rename) so a crash mid-write can't corrupt the
cache.
429   6. (Optional) expose as one resumable CLI stage.
430   Effort: moderate ? mostly a wasb_infer refactor (incremental det cache + resume); the
tracker
431   decoupling is the core design. Calibration grid stays cheap (operates on the cached
trajectory) ?
432   no checkpointing needed there.
433
434   ## ? STEP 4 ? FIRST CALIBRATION DONE (2026-06-06) ? RESULT
435   Full WASB trajectory of the long badminton video: 21,284 frames, 37% visible-shuttle
(sane).
436   `calibrate_wasb` on 25 golden rallies (IoU 0.5):
437   - **baseline** thresholds (0.02,2.0,2.0): precision 0.577, recall 0.600, **F1 0.588**,
meanIoU 0.765.
438   - **tuned** (0.05,1.0,1.0): **F1 0.746** ? precision +12.2%, **F1 +26.8%** (OPTIMISTIC ?

grid fit on full GT).
439   - **HONEST (5-fold cross-val): held-out recall 0.88 ± 0.16, generalization_gap +0.000**
(no overfit).
440   - Caveats: +X% is optimistic upper bound; honest within-video metric is RECALL;
precision/F1
441   generalization needs a 2nd labelled video ? `leave_one_video_out`. std ±0.16 = small
sample.
442   - Driver `backend/eval/calibrate_wasb.py` was LOST from master in #21's branch-delete ?
RECOVERED
443   in PR #24 (merged). Trajectory cached at `~/clips/sample_long_traj.csv` (+ `C:\?\Temp\
444   sample_long_traj.csv`); frames at `C:\?\Temp\sample_long_frames` (4.1GB, deletable).
445   - NEXT: export tuned segments for user manual-verify; 2nd labelled video for honest
precision;
446   implement WASB-pass checkpointing (plan below); optional multi-agent tuning Workflow.
447
448   ## (historical) STEP 4 IN PROGRESS (2026-06-06)
449   - PR **#21** open (feat/wasb-substrate: substrate + calibration.py + `calibrate_wasb.py`
driver +
450   HOW_RALLY_DETECTION_WORKS doc). Suite 171 green.
451   - **Calibration driver DONE + tested:** `backend/eval/calibrate_wasb.py` (grid over
452   velocity_thresh/merge_gap/min_window_duration ? improvement_report + cross_validate).
Run:
453   `python -m backend.eval.calibrate_wasb --trajectory ~/clips/sample_long_traj.csv (copy
to Win)
454   --labels "<Annotation Setup>/golden_labels_badminton.csv" --fps 59.94 --frame-width
1920`.
455   - **GOTCHA:** WSL CANNOT see Google Drive `/mnt/g` (first pass b04odl4jl failed: "cannot
stat
456   /mnt/g/..."). Fix: copy the video G:?C: on the Windows side first (`cp "/g/My
Drive/.../Sample
457   Long Video - Baadminton.MP4" /c/Users/avidu/AppData/Local/Temp/sample_long.mp4`, ~9s,
cached),
458   then WSL reads it via `/mnt/c`.
459   - **PERF GOTCHA:** wasb_infer's cv2 `imwrite` PNG extraction is pathologically slow
(~5fps ? ~1hr
460   for 21k frames, GPU idle the whole time). FIX: extract with **ffmpeg** (Windows, reads
the C:
461   copy): `ffmpeg -i C:\...\sample_long.mp4 -q:v 2 C:\...\sample_long_frames\%08d.jpg`
(21284 frames
462   in **3.5 min**, 4.1GB JPG). Then `wasb_infer --frames_dir` (now defaults
**batch_size=8,
463   num_workers=4** ? GPU ~80-90%). ffmpeg NOT in WSL; use Windows ffmpeg. wasb_infer
globs *.jpg too.
464   - **RESTART RECOVERY (2026-06-06):** laptop rebooted mid-inference. The 21,284 extracted
frames
465   (`C:\Users\avidu\AppData\Local\Temp\sample_long_frames`) + staged video survived (Temp
persisted);
466   trajectory CSV was NOT written (writes only at end). Resume = just RE-RUN the
inference command
467   below on the existing frames (NO re-extraction). Relaunched as task brmhzav6x. The
468   `wasb_infer.py` batch/workers change is uncommitted on master working tree (commit
after verify).
469   - **Full-video WASB inference RUNNING** (task brmhzav6x; orig b4pqhqkte died on reboot):
`wasb_infer.py --frames_dir
470   /mnt/c/Users/avidu/AppData/Local/Temp/sample_long_frames --weights
471   ../pretrained_weights/wasb_badminton_best.pth.tar --out ~/clips/sample_long_traj.csv
472   --batch-size 8 --num-workers 4`. GPU confirmed ~80-90%, 6.4/8GB. Writes CSV at the
end. When done:
473   copy CSV to Windows (or read /mnt) ? run the driver above ? the "+X%" + cross-val.
Then clean
474   up `~/clips/sample_long_frames` (~30-40GB). Calibrate ONLY on the labelled Sample Long
Video
475   (only one with golden labels); a 2nd labelled video later ? leave_one_video_out.
476
477   ## NEXT-SESSION RUNBOOK ? step 4 (the actual calibration run)

Substrate (wasb_infer/runner/hybrid) + eval harness (calibration.py) are DONE on
`feat/wasb-substrate` (pushed). To summit:
1. **Full-video WASB trajectory (the slow long pole ? do first, cache it).** Untested at
full
   scale; expect a debug lap. Two paths:
   (a) via runner: `WasbRunner(WasbConfig()).run_predict(<video>, "output/wasb")` ?
stages 2.8GB
       into WSL (off G: = possibly slow/flaky) + extracts frames + runs. OR
   (b) safer: extract frames on Windows (ffmpeg from G:, proven) ? run in WSL:
       `conda activate wasb && cd ~/models/WASB-SBDT/src && python wasb_infer.py
--frames_dir <dir>
       --weights ../pretrained_weights/wasb_badminton_best.pth.tar --out
~/clips/full_traj.csv`.
   Watch: disk for ~21k PNGs, GPU mem, /mnt read speed. Validate visible-shuttle % is
high during
   known rallies, ~0 during dead time.
2. **GT** from `Annotation Setup/golden_labels_badminton.csv` ? rally [start,end] in
seconds
   (use the verified rows; skip the 1 missed row whose end is still TBD = rally 7.5).
3. **Write the calibration driver** (not yet written): `predict_fn(cfg)` =
parse_trajectory_csv(full_traj)
   ? `TrackNetRunner.trajectory_to_action_windows(points, fps=59.94, frame_width=1920,
**cfg)` ?
   [start,end]; cfg = {velocity_thresh, merge_gap, min_window_duration}. Grid over
those.
4. **Run** `backend/eval/calibration.py`: `improvement_report(predict_fn, baseline_cfg,
best_cfg, GT)`
   for the "+X%"; `cross_validate` (within-video recall + gap) for the overfit check.
(Only 1 video
   now ? leave_one_video_out waits for more videos = the real generalization metric.)
5. Optional: the **multi-agent tuning Workflow** (token-heavy) instead of a plain grid
in step 3.

Eval/overfit harness BUILT + license soul-check (2026-06-06)
- `backend/eval/calibration.py` (on `feat/wasb-substrate`, 9 tests; segmenter-agnostic
predict_fn):
  `improvement_report` = baseline-vs-tuned %? precision/F1 (the "+X%"); `cross_validate`
=
  within-video k-fold, default metric **recall** (global windows vs a GT *subset* can't
fairly
  score precision), reports `generalization_gap` (train?test = overfit alarm);
`leave_one_video_out`
  = the honest metric as videos arrive (pick config on others, score held-out). This is
the
  overfitting guard for "more videos coming."
- **LICENSE:** CODE clean ? WASB-SBDT MIT (verified), torchvision/supervision
permissive, AGPL
  Ultralytics gone (PR #8), MonoTrack never referenced (default = wasb_badminton_best).
?? OPEN:
  WASB *weights* trained on academic NYCU/TrackNetV2 data ? confirm terms or retrain
before
  COMMERCIAL deploy (fine for R&D). See [[tracknet-wasb-wsl2-decision]].

Step-3 substrate finding (2026-06-06)
- WASB IS the plan and is ~ready: `wasb` WSL env + WASB-SBDT repo + pretrained badminton
weights
  all present (verified); healthcheck OK (TF2.17/GPU). The wired `tracknet_runner`
targets
  TrackNetV4 = NOT runnable (no weights, repo not cloned). To use WASB: build inference
wrapper +
  runner adapter + `wasb_hybrid` segmenter, then calibrate
`velocity_thresh/merge_gap/min_window_duration`
  vs golden labels (eval held-out). See [[tracknet-wasb-wsl2-decision]] STATUS
2026-06-06.

```
518     ## Merged: #6, #7, #19, #8, #20, **#9 (validate-delivery)**
519     - collector master has: adapter, forced/unforced vocab, last_shot_outcome (two-field).
520     - indexer master has: docs mirror, rally slicer (`backend/tools/rally_slicer.py`).
521
522     ## Open PR (pushed, awaiting merge)
523     - **collector #9** `feat/validate-delivery` ? per-video QA gate. `curate validate-
delivery`
524       (CVAT or CSV) ? `<id>_report.md`/`.json` + `<id>_review.csv` (auto manual-verify
sheet);
525       exit!=0 on blocker. New `possible_missed_rally` gap check.
`docs/INGESTION_PIPELINE.md` FAQ.
526       104 tests; verified on real delivery (BLOCKED on 1?2 overlap, flags 2/7/16).
527
528     ## Pilot status ? DONE verifying
529     - User verified **all** rallies: only the previously-flagged rows are issues;
**everything else
530       is clean trainable data**. The [start,end] intervals are settled ? **segmentation GT
is ready**
531       (ending_reason/outcome specifics for flagged rows are metadata, not needed for
temporal seg).
532     - Golden master: `C:\Users\avidu\Projects\Annotation Setup\golden_labels_badminton.csv`
533       (24 + 2 missed = rallies; 7.5 @1:46.031 confirmed via screenshot; 16.5 = 3:34?3:43).
534
535     ## Decisions locked
536     - **Two-field ending**: `ending_reason` (why) + `last_shot_outcome` ? {in,net,out,n_a}
(where).
537     - **Accept Roora's CVAT export** ? adapter ingests it; do NOT ask them to switch to CSV.
538     - **Absorb timestamps** ? adapter recomputes `seconds = frame/fps`; mm:ss also parsed
everywhere
539       (indexer parse_timestamp / collector parse_time_string / compiler / slicer). Minor
slack
540       absorbed by VideoCompiler padding (0.5/1.0s). Don't ask Roora to fix time computation.
541     - Roora report (`Annotation Setup\Roora Pilot Review ? Badminton (2026-06-06).md`)
revised to
542       only ask for what code can't fix: **completeness (no missed rallies), one-box-per-
rally
543       boundaries, add `last_shot_outcome` dropdown, pickleball/TT scope.**
544
545     ## Next steps (sequence 1?2?3, user approved)
546     > User will get **5?6 30-min+ videos** annotated soon ? the **validation phase must be
ready**
547     > to auto-check each delivery against our conventions BEFORE that lands.
548     1. ? **last_shot_outcome vertical** ? MERGED (#8).
549     2. ? **validate-delivery framework** ? DONE (collector PR #9, awaiting merge). Label-
only gap
550        heuristic for missed rallies (a motion-based 2nd opinion over raw video is a future
enhancement).
551     3. (claude) **NEXT: per-video calibration loop** ? slicer merged (#20); tune segmenter
thresholds
552        on K labeled rallies, measure IoU on the HELD-OUT rest (no leakage). Then learned
fine-tuning.
553     4. (claude) Build segmentation GT from golden labels ? convert-cvat/import-local/export
(the
554        actual "train on my own video" step; needs the corrected golden labels, overlap
already resolved).
555
556     ## Open design threads
557     - Per-video few-shot adaptation: calibration first (static camera = good case), honest
eval
558       (held-out rallies). cf. train/eval guardrail in [[collector-indexer-bridge]].
559
```

8. user (2026-06-07T06:40:58.691Z)

```
1       ---
```

```
2       name: session-handoff
3       description: "Living \"you are here\" handoff for this project ? current state, next
steps, open decisions, ramp-up kit. AUTO-REFRESHED at meaningful checkpoints (PR merged, key
decision/ADR, phase shift, session wrap); \"update the handoff\" forces an immediate full
rewrite."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: eb4b7b53-2f77-4c4d-83e1-4fa5e77f3e3f
8       ---
9
10      # Session handoff ? badminton-highlight-indexer (+ siblings: sports-data-collector,
rally-annotator)
11
12      **Last updated:** 2026-06-07 · **Refresh policy: AUTO-refresh at meaningful
checkpoints** ? PR merged,
13      key decision/ADR, phase shift, session wrap-up (NOT on-request-only). "update the
handoff" forces an
14      immediate full rewrite. **Staleness tripwire:** if ?3 substantive [[current-state]]
checkpoints ? or any
15      PR-merge / new ADR / phase shift ? have landed since the "Last updated" date above,
refresh this handoff now.
16      Pointer, not a duplicate. **For the freshest blow-by-blow state, read [[current-state]]
first**
17      (updated every micro-checkpoint); this handoff is the slower ~1-page orientation.
18
19      ## You are here (2026-06-07)
20      - **Rally-detection stack matured & all merged.** WASB shuttle-trajectory substrate is
**resumable/
21        checkpointed** (md5-ish cache, incremental detector flush, tracker re-run). On top: a
rally layer =
22        windowing thresholds + a **net-crossing "gate A"** + a **distilled classifier**
(`distill_local`).
23        Eval harness: `calibrate_wasb` / `calibration.py` (LOVO) / `metrics.py`.
24      - **Honest core finding:** WASB-only threshold tuning **ceilings ~F1 0.44** and the
learned distilled
25        model **underperforms the baseline at n=2 videos**. The blocker is **golden data +
WASB candidate
26        recall**, not the method. (Gemini-as-teacher gave us the first real measurement.)
27      - **Strategy decided (ADR-007):** Gemini = **offline teacher only**; **AUDIO** hit-
detection is the next
28        local cue, **fused in our rally layer** (WASB stays a frozen black box) ? Option A.
**Pose/stance =
29        parked #2.** **Option B = the eventual end-state**: one OWNED end-to-end multimodal
model
30        (frames+audio+pose?rallies) fine-tuned on our data, dropping WASB-weight dependence +
the recall ceiling.
31      - **Golden data path:** owner **self-rates** clips with the new **standalone VLC
annotator repo**
32        (github.com/avidullu/rally-annotator, MIT, multi-sport) ? CSV ? in-repo `--labels`
loaders / collector
33        `import-local`. Human labels > the Gemini silver labels the model was starving on.
34      - **? SESSION SUNSET (2026-06-07).** Indexer PRs **#25?#35 merged**, master green (**209
tests**), tree clean
35        (only `feat/khelacharya-shot-intelligence` retained; **no open PRs**). Annotator repo
public/clean. **Audio E1
36        ran ? classical onset = ~2.2× discrimination ceiling, NOT adopted for fusion**
(detector kept; quantitative
37        gains/losses + reusable lessons in [[audio-e1-findings]]). Modeling **PARKED** pending
golden labels.
38        ? PR #36 (`docs/AUDIO_E1_FINDINGS.md`) MERGED to master (2026-06-07); record also in
[[audio-e1-findings]] + plan §10.
39
40      ## Next steps / open threads
41      1. **Owner:** rate a few clips in VLC (annotator v1.2 ? needs a live multi-sport smoke
```

test), and/or merge
42   nothing pending (none open).
43   2. **Audio E1 ? DONE ? PR #35. Verdict: classical-onset audio is too weak (~2.2x
ceiling).** Built
44   `backend/audio/hit_detector.py` + `audio_e1_probe.py` (numpy/ffmpeg, no scipy, 209
tests). On existing GoPro
45   audio vs Gemini silver: audio fires in 100% of rallies + recovers 100% of WASB-missed
rallies (real recall
46   signal), BUT discrimination only ~2.2x and audio-only F1 < WASB. **Band-pass (Option
A) tried ? does NOT help**
47   (HF bands worse; ceiling holds; NOT a warm-up artifact ? between-point
tosses/footwork/talk fool a classical
48   onset detector). **Next (user decides):** E2 learned TIMBRE classifier (needs labeled
hits ? only plausible way
49   past 2.2x) OR re-prioritize to the owner's "other hunch" / pose-stance #2 / more
golden data for WASB+distilled.
50   Details: AUDIO_RALLY_DETECTION_PLAN.md §10 + [[current-state]].
51   3. Stance heuristic (#2) after the owner's "other hunch"; Option B once the corpus is
healthy.
52   4. Cached artifacts LEFT IN PLACE for re-use (owner will rate):
`Temp\{adarsh,testlarge}_frames` + trajectories;
53   `Temp\verylarge_frames` (partial). Adarsh/TestLargeVideo trajectories are ready to
feed calibration.
54
55   ## Key decisions (don't relitigate without reason)
56   - **Audio now ? stance parked ? Option B eventual** (ADR-007). Modular cues are how we
earn Option B
57   (each is cheap/debuggable and generates the labels the end-to-end model needs).
58   - **Fusion is in OUR rally layer, never inside WASB** (WASB = frozen vision-only shuttle
detector; no audio
59   path; retraining needs x,y+audio labels we lack). Two modes: feature-level (?
distilled classifier) +
60   decision-level recall-recovery. (ADR-007 §Integration architecture.)
61   - **Local-only inference = privacy lever**; **permissive licensing only** (ADR-005);
Gemini paid+offline only.
62   - **Distilled model > threshold tuning** is the direction, but **needs more videos**
(n=2 overfits).
63   - Per-video adaptation = calibration on held-out; never mix train/eval. Frequent **small
PRs** + continuous
64   [[current-state]] updates (this handoff on request only).
65
66   ## Ramp-up kit (read these to get current)
67   - [[current-state]] (live blow-by-blow), this handoff, [[audio-e1-findings]] (audio
gains/losses + lessons), [[working-agreement-checkpoints]], [[working-agreement-attribution]],
[[feedback-tail-long-job-logs]].
68   - **CODE/architecture:** `docs/CODE_MAP.md` FIRST (dense map). See [[code-map-
artifact]].
69   - **Strategy/decisions:** `docs/DECISIONS.md` (esp. **ADR-007** audio),
`docs/RALLY_DETECTION_RESEARCH.md`
70   (cited methods survey), `docs/AUDIO_RALLY_DETECTION_PLAN.md`,
`docs/VIDEO_RECORDING_GUIDELINES.md` (audio now required).
71   - **Key code:** `backend/eval/{distill_local,calibrate_local,calibrate_wasb,gemini_refin
e,metrics,calibration}.py`,
72   `backend/detectors/{wasb_infer,wasb_runner,rally_gate,tracknet_runner}.py`.
73   - **Annotator repo:** `C:\Users\avidu\Projects\rally-annotator\` (+ installed at
`%APPDATA%\vlc\lua\extensions\`).
74   - Memory notes: golden-set-vendoring, collector-indexer-bridge, candidate-segments-
finetuning, sibling-internship-repo.
75

9. user (2026-06-07T06:40:59.224Z)

1   ---
2   name: code-map-artifact
3   description: Where the dense cold-start CODE MAP of the indexer lives ? read it FIRST to

```
ramp up on codebase structure instead of crawling files
4       metadata:
5         node_type: memory
6         type: reference
7         originSessionId: 42211f80-2169-4025-ad9d-99c1aa29ff70
8       ---
9
10      For **code/architecture ramp-up** on `badminton-highlight-indexer`, read
**`docs/CODE_MAP.md`** in
11      the repo FIRST ? a dense, anchor-rich (`@path::symbol`), machine-first map: pipeline
(runtime +
12      calibration/eval), per-subsystem file/symbol index, canonical data contracts, WASB WSL
boundary,
13      cross-repo collector contract, cross-cutting gotchas, "to do X -> open Y" ramp paths,
and a
14      test->subsystem map. It replaces a DFS/BFS crawl of 40+ files.
15
16      - Landed via PR #27 (`docs/code-map`).
17      - **Regenerate** after any registry/contract/WSL-interface change by rerunning the
`build-code-map`
18        workflow (script saved under the session's `workflows/scripts/`): parallel subsystem
readers ->
19        synthesize -> adversarial verify (the verify phase re-checks every anchor against real
files).
20      - Caveat baked into the doc: Sections 4 (WASB-SBDT) & 5 (collector) describe EXTERNAL
repos and are
21        unverified against their source ? only the repo-side callers were checked.
22      - This is for CODE structure (which is repo-derivable, hence a doc not a memory). Non-
code project
23        state stays in [[current-state]] / [[session-handoff]].
24
```

10. assistant (2026-06-07T06:41:15.184Z)

Memory ramped me up well. I'm picking up from a clean master (PRs #25?#36 merged, 209 tests green, modeling parked pending golden labels). The new task ? rich per-video observability output in **both** the indexer (highlights tool) and the collector ? is greenfield. Let me read the code map and survey both repos' pipelines + their README/FAQ docs (since the report must be debuggable by a README-reader).

11. user (2026-06-07T06:41:16.059Z)

```
1       # CODE_MAP.md
2
3       **Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-
hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a
segmenter-agnostic eval/calibration stack.
4
5       **Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this
doc). Re-run after any registry/contract/WSL-interface change.
6
7       **LEGEND**
8       - `@path` = file (always repo-relative)
9       - `::sym` = symbol (class/func/const) in the nearest preceding `@path`
10      - `->` = dataflow / call / "produces"
11      - `§` = data contract (canonical shape)
12      - `?` = gotcha / footgun
13      - `?` = registry / plugin extension point
14      - `WSL` = runs inside WSL2 conda env, NOT the Windows Python process
15
16      ---
17
18      ## 1. PIPELINE
19
20      ### 1A. Runtime: video file -> highlight reel
```

```
config.json (json.load; absent -> {})
  -> video_id = MD5(whole file)            @backend/main.py::get_file_md5  (? chunk size varies per entrypoint: 64KB main, 1MB run_process, 8KB run_eval, 64KB phase3 ? all hash WHOLE file so values agree)
  -> validate @backend/validators/base.py::registry.run_all(video_path, config)
       FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity -> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see @backend/validators/__init__.py)
  -> db.add_video(video_id, filepath, status, [r.dict()])   @backend/database.py
  -> seg = segmenter_registry.get(name)(db, config)         ? @backend/indexer/base.py::segmenter_registry
  -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)
       [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without stitching]
  -> select segment ids -> [(start,end)] (+ stitching padding)
  -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name) @backend/stitcher/compiler.py
       per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy -> output/<name>.mp4
```

Hybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):

```
P1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)
 -> P2 candidate "action windows" from motion/trajectory  [STOP: candidates persisted via db.add_candidate_segment]
 -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip, prompt)   ? provider_registry
 -> P4 db.add_play_segment + db.link_candidate_to_segment
```

Pure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) -> add_play_segment.
Legacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).

1B. WASB shuttle substrate (P2 of wasb_hybrid)

```
WasbHybridSegmenter.process_video                @backend/indexer/wasb_hybrid.py
 -> WasbRunner.healthcheck()  (gate; not ok -> return []) @backend/detectors/wasb_runner.py
 -> WasbRunner.run_predict(video, out_dir)
      stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python wasb_infer.py ...'`
      -> @backend/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV
 -> TrackNetRunner.parse_trajectory_csv(csv)      @backend/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)
 -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start, velocity_thresh, merge_gap, min_window_duration, chunk_index)  [the model-agnostic WINDOWING BRAIN]
```

(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width` fallback (30.0, 1280.0).)

1C. Calibration / eval flow (NO pipeline run unless told)

```
GT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]
DB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]   stage ? {candidates, final}
  -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score
  -> harness.evaluate -> EvalReport -> report_to_dict
       -> batch.aggregate (corpus micro/macro)            @backend/eval/batch.py
```

63   -> regression.regress(_with_provider) vs baseline   @backend/eval/regression.py
[CI gate]
64   Calibration: calibrate_wasb.run -> calibration.{select_best, improvement_report,
cross_validate} @backend/eval/calibration.py
65   Learned segmenter (offline): training.build_training_set(X,y) ->
classifier.LogisticRegression.fit/save
66   ```
67   Entrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch,
can segment), `@run_regression.py` (baseline gate, exit 1=regression),
`@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning
drivers).
68
69   ---
70
71   ## 2. SUBSYSTEMS
72
73   ### 2.0 Orchestration & config
74   - `@backend/main.py` ? FastAPI app + argparse CLI (modes via FLAGS, not subcommands:
`--server`/`--video-path`(process)/`--debug-clip`/`--trim-start`+`--trim-end`/`--setup`).
75   - `::app` FastAPI; `::db_instance`,`::global_config` module globals set ONLY in
`main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.
76   - `::process_video` `@POST /api/process` -> validate -> add_video -> segment.
`::compile_highlights` `@POST /api/compile`. `::run_cli_diagnose_clip` lazy `import cv2`,
samples ±3s vs `indexing.motion_threshold`.
77   - ? uvicorn host hardcoded `127.0.0.1`; CLI default_segmenter fallback `'motion'`.
78   - `@run_process_and_stitch.py` ? one-shot demo; ? hardcoded paths, interactive `input()`
(blocks automation), `default_segmenter` fallback `'yolo_hybrid'`, skips validation.
79   - `@run_eval.py::main` ? score one video's DB preds vs GT CSV.
`--stage{candidates,final,both}`. Exit 2=arg/IO. ? pure scorer; errors if video never processed.
80   - `@run_phase3_eval.py::main` ? walk manifest, segment-if-needed, score, aggregate.
`load_dotenv()` at import. ? DB key = file MD5, NOT manifest's `video_id` (kept as label
`collector_video_id`).
81   - `@run_regression.py::main` ? `regress_with_provider` vs baseline. Exit codes semantic:
0=ok, 1=REGRESSION (CI gate), 2=missing DB. ? `--update-baseline` runs AFTER compare
(snapshots even a regressing run).
82   - `@config.json` ? single config. Sections: `validation` (+ `.relevance`), `indexing` (+
`.tracknet`; ? NO `.wasb` block in checked-in config ? WASB runs use `WasbConfig` defaults),
`output`, `stitching`, root `sport/ai_provider/ai_model`. ? **three default segmenters
diverge**: config ships `gemini`, setup.py writes `yolo_hybrid`, main.py fallback `motion`.
83   - `@setup.py` ? write default config (skips if present), ffmpeg/torch/CUDA diagnostics,
GPU-aware pip (`nvidia-smi` -> cu124 else cpu). ? generated config diverges from checked-in (no
`relevance`/`tracknet` blocks, blur 100.0 vs 20.0).
84
85   ### 2.1 Segmenters (indexer) ?
86   - `@backend/indexer/base.py`
87   - `::BasePlaySegmenter` ABC ? `__init__(db, config)`; abstract
`name`,`description`,`process_video`. § `process_video(video_id, video_path, player_pool=None,
max_frames=None) -> List[dict{id,start_time,end_time,duration,metadata}]`.
88   - `::SegmenterRegistry`, `::segmenter_registry` (singleton). ? **register a new
segmenter**: subclass `BasePlaySegmenter`, call `segmenter_registry.register(MyCls)` at module
bottom, AND import the module in `@backend/indexer/__init__.py` (registration is an IMPORT side-
effect). ? `register()` builds `cls(None, {})` just to read `.name` -> name/description MUST NOT
touch `self.db`/`self.config`.
89   - `@backend/indexer/__init__.py` ? imports all 5
(`motion,gemini,yolo_hybrid,tracknet_hybrid,wasb_hybrid`) = the de-facto registration manifest.
90   - `@backend/indexer/wasb_hybrid.py::WasbHybridSegmenter` (name `'wasb_hybrid'`) ? WSL
WASB substrate. Config `indexing.wasb.velocity_threshold(0.02)` + shared
`dead_time_merge_gap/min_action_window_duration/ai_handoff_padding/ai_max_workers`. ? whole-
video single pass; reuses YOLO-named keys `log_yolo_candidates/store_yolo_candidates`; hardcoded
relative out dir `output/wasb`; needs `{PROVIDER}_API_KEY` (missing -> []). Registered at module
bottom.
91   - `@backend/indexer/tracknet_hybrid.py::TrackNetHybridSegmenter` (name
`'tracknet_hybrid'`) ? copy-paste twin of wasb_hybrid (swaps runner; `detection_params` adds
`queue_length`). ? any P3/P4 fix must be applied to BOTH (and arguably yolo_hybrid).
92   - `@backend/indexer/yolo_hybrid.py::HybridYoloSegmenter` (name `'yolo_hybrid'`, no YOLO

remains ? torchvision detector + supervision ByteTrack). `::_PERSON_CLASS_ID=1` ? (ultralytics used 0; wrong value -> zero detections). `::_lazy_load_model`, `::_extract_action_windows_from_chunk`, `::_DETECTOR_FACTORIES`/`::_DEFAULT_DETECTOR='fasterrcnn_resnet50_fpn'`. ? velocity computed at `effective_fps = fps/frame_skip` -> `yolo_velocity_threshold` coupled to `frame_skip`; trackers per-chunk (windows merged by TIME not ID); `import torch` at module top.
93 - `@backend/indexer/gemini.py::GeminiPlaySegmenter` (name `'gemini'`) ? pure-AI, no P2. ? chunks only if `duration>chunk_duration`; `max_workers=5` hardcoded (ignores `ai_max_workers`); `max_segment_duration=90.0` hardcoded; `debug_duration` silently truncates analysis; `parse_time` accepts colon timestamps (warns).
94 - `@backend/indexer/motion.py::MotionPlaySegmenter` (name `'motion'`) ? pixel absdiff baseline. ? **metadata + serve/smash/foul/winner events are RANDOM** (random.sample/randint/uniform/choice); only segmenter using `db.add_event` and passing server/receiver; uses `print()`; honors `max_frames`.
95
96 ### 2.2 Detectors & windowing (WSL-quarantined)
97 - `@backend/detectors/__init__.py` ? docstring only (refs `docs/DECISIONS.md` ADR-001). ? does NOT re-export runners; import submodules directly.
98 - `@backend/detectors/tracknet_runner.py`
99 - `::TrackNetConfig`(+`.from_indexing_cfg`), `::TrackNetRunner` ? runs TrackNet `predict.py` in WSL.
100 - `::to_wsl_mnt_path` (`C:\x`->`/mnt/c/x`), `::TrajectoryPoint`(frame,visible,x,y).
101 - **`::parse_trajectory_csv` @staticmethod** + **`::trajectory_to_action_windows` @staticmethod** = MODEL-AGNOSTIC brain, re-exported verbatim by WasbRunner. ? `weights_path` defaults `''` -> `run_predict` returns None; predict.py only `.mp4/.avi`; `visible==False` resets prev (occlusion gap breaks track); `fps<=0->30`, `frame_width<=0->1280` silent; `track_id_count` hardcoded 1; `timeout_sec=0` -> NO timeout (hangs forever).
102 - `@backend/detectors/wasb_runner.py::WasbRunner` (+`::WasbConfig`) ? stages `wasb_infer.py` + video into WSL, runs in `wasb` env. `::parse_trajectory_csv`/`::trajectory_to_action_windows` = `staticmethod(TrackNetRunner.*)`. ? `run_predict` does NOT pass cache flags (uses wasb_infer defaults flush=1000/batch=8/workers=4); `wasb_infer.py` MUST be re-staged each run (editing Windows copy is inert until staged); `_wsl_bash` duplicated (drift risk).
103 - `@backend/detectors/wasb_infer.py` (WSL) ? `::run` core. Reboot-resumable detector cache: `::_DET_FILE='det_raw.jsonl'`,`::_MANIFEST_FILE='manifest.json'`, `::CACHE_VERSION=1`, `::load_and_clean_cache` (reads longest CONTIGUOUS prefix where `w==line index`, rewrites to heal crash-truncated tail), `::manifest_compatible`, `::_atomic_write_text/_atomic_write_trajectory`, `::_dets_to_tracker` (xy MUST be np.array). ? GPU pass resumable; CPU tracker STATEFUL -> always fully re-run; `runner.gpus=[0]` hardcoded single-GPU; `extract_frames` cv2-PNG SLOW (prefer host ffmpeg).
104
105 ### 2.3 Eval & calibration (all pure; I/O lives in CLI drivers)
106 - `@backend/eval/metrics.py` ? scoring primitive. `::interval_iou`, `::match_intervals` (greedy, deterministic), `::score`->`::Score`(`.as_dict()` = wire shape), `::pr_curve`, `::Match`, `::MatchResult`. ? greedy not Hungarian; `score()` indexes the SAME preds/gts used to build `match_result` or boundary errors are silently wrong.
107 - `@backend/eval/annotations.py` ? generic GT loader. `::load_annotations` (`start,end` CSV; RAISES on bad rows/`start>=end`), `::parse_timestamp` (decimal or MM:SS/HH:MM:SS), `::write_scaffold`.
108 - `@backend/eval/calibration.py` ? overfit guard. `::cross_validate` (within-video k-fold; ? defaults `metric='recall'` deliberately), `::leave_one_video_out` (honest cross-video; needs ?2 labelled videos), `::improvement_report` (OPTIMISTIC ? selected on full GT), `::select_best`, `::kfold_indices`, `::pct_improvement`, `::evaluate`. ? `generalization_gap >~0.1` = overfit alarm.
109 - `@backend/eval/calibrate_wasb.py` ? WASB tuning CLI. `::BASELINE=(0.02,2.0,2.0)` (? hardcoded dup of real defaults), `::default_grid` (45 cfgs = 5×3×3), `::make_predict_fn`, `::load_golden_gts` (? SILENTLY skips bad rows ? opposite of annotations.load_annotations). ? CLI defaults `fps=59.94/frame-width=1920` baked in.
110 - `@backend/eval/export_wasb_segments.py` ? trajectory+cfg -> golden/slicer CSV + comparison. `::TUNED=(0.05,1.0,1.0)` (? hardcoded from ONE video), `::SEG_FIELDS`,`::COMP_FIELDS`, `::build_comparison`. `--slice` requires `--video`.
111 - `@backend/eval/batch.py` ? corpus aggregation. `::aggregate` (micro pool TP/FP/FN, TP-weighted mean_iou, macro F1), `::default_stages_for` (`auto`; `::_FINAL_ONLY_SEGMENTERS={motion,gemini}`), `::resolve_video_path`.
112 - `@backend/eval/harness.py` ? DB-pred scorer. `::STAGES=('candidates','final')`,

`::get_predictions` (candidates->`query_candidate_segments(detector=)`;
final->`query_play_segments({})` ALL segs, detector arg ignored),
`::evaluate`->`::EvalReport`(`::StageReport`)->`::report_to_dict` (input contract for
batch.aggregate).
113   - `@backend/eval/training.py` ? learned-segmenter data layer. `::YOLO_FEATURE_NAMES`
(5-d), `::extract_yolo_features` (? assumes `row['stats']` is already a dict ? the DB layer
deserializes it; a raw JSON STR would AttributeError, not zero-fill; missing fields default
0.0), `::label_candidates` (same IoU matching as evaluator),
`::build_training_set`->(X,y,intervals).
114   - `@backend/eval/classifier.py::LogisticRegression` ? numpy-only logreg (no sklearn).
`fit/predict_proba/predict/to_dict/from_dict/save/load`. ? `class_weight='balanced'` default;
feature ORDER at predict must match fit (names are metadata only, not used in math); fixed 1000
epochs.
115   - `@backend/eval/regression.py` ? CI gate. `::DEFAULT_TOLERANCE=0.05`,
`::DEFAULT_BASELINE_PATH=output/regression_baseline.json`, `::regress`,
`::regress_with_provider`, `::save_baseline`/`::load_baselines`, `::RegressionResult`. ? imports
`annotation_registry` from **`backend.annotations`** NOT `backend.eval.annotations` (two
different "annotations"); gate fires on precision OR recall drop (F1 reported, not gated); first
run on a video silently passes (no baseline).
116
117   ### 2.4 Stitcher / tools / utils
118   - `@backend/stitcher/compiler.py::VideoCompiler` ? `compile_highlights(raw,
segments:[(start,end)], out_name)`. Per-clip re-encode -> `-f concat -c copy`. § skipped tuple
`(idx,start,end,reason)`. ? if duration probe fails (0.0) NO boundary validation; `padded_start`
uses `begin_padding`(0.5)/`end_padding`(1.0) defaults; temp clips in a TemporaryDirectory under
output_dir; raises ValueError/FileNotFoundError/RuntimeError; uses `print()`.
119   - `@backend/tools/rally_slicer.py` ? standalone CLI. `::compute_clip_windows` (pure,
unit-tested), `::load_rally_labels` (golden schema; ? silently skips degenerate rows),
`::slice_rallies`, `::BEGIN_PADDING=0.5`/`::END_PADDING=1.0` (? duplicate compiler's by copy,
kept in sync manually). `_read_time -> backend/eval/annotations.parse_timestamp`.
120   - `@backend/utils/video.py` ? `::get_video_duration` (ffprobe),
`::extract_video_chunk(...,reencode=False)`. ? uses `-t DURATION` (length, not end ts); copy
mode cuts at nearest keyframe (drift) ? pass `reencode=True` for short/accurate clips.
121   - `@backend/utils/geometry.py` ? `::get_velocity_normalised` (used by yolo_hybrid),
`::calculate_iou` (? +1 px convention inflates IoU), `::get_center/get_distance/get_velocity`.
122
123   ### 2.5 Validators ?
124   - `@backend/validators/base.py::VideoValidator` ABC, `::ValidationResult` (pydantic),
`::ValidationRegistry`, `::registry` (singleton). § `validate(video_path, config, context) ->
ValidationResult`. ? **add a validator**: subclass + `registry.register(MyValidator())`
(registration order = execution order). ? validators share a per-run `context` dict; exceptions
wrapped as `passed=False` (never abort batch).
125   - `@backend/validators/__init__.py` ? registers in EXEC order: FormatValidator,
ResolutionFPSValidator, PTSContinuityValidator, SceneCutDensityValidator, BlurValidator,
RelevanceValidator (? producers before consumers ? format_check first).
126   - `@backend/validators/format.py::FormatValidator` (`format_check`) ? **PRODUCER** of
`context['ffprobe_meta']`; ? MUST run before resolution_fps; container check is substring
`'mp4'` (lowercased).
127   - `@backend/validators/resolution_fps.py::ResolutionFPSValidator`
(`resolution_fps_check`) ? CONSUMES `ffprobe_meta` (? hard-fails if format_check didn't run
first). `min_fps` default 29.9; min res 1280x720.
128   - `@backend/validators/pts_continuity.py::PTSContinuityValidator`
(`pts_continuity_check`) ? ? 10.0s keyframe-gap threshold hardcoded, comment says 8.0s (drift);
any backward jump fails.
129   - `@backend/validators/scene_cut.py::SceneCutDensityValidator` (`scene_cut_check`) ? ?
`cut_threshold=0.6` hardcoded; samples at 2fps and caps at 100 SAMPLED frames (~first 50s of
video, not first 100 raw frames).
130   - `@backend/validators/blur.py::BlurValidator` (`blur_check`) ? Laplacian var <
`min_blur_threshold`(100.0) = fail; samples 15 frames.
131   - `@backend/validators/relevance.py::RelevanceValidator` (`relevance_check`) ? HSV
court-green; lazy `import backend.sports`; ? unknown sport -> empty sport cfg, falls back to
defaults, does NOT fail.
132
133   ### 2.6 Sports / Providers / Annotations / DB ?
134   - `@backend/sports/base.py::SportProfile` ABC

(`name`,`get_prompt`,`get_relevance_config`);
`@backend/sports/badminton.py::BadmintonSportProfile` (the Gemini prompt + HSV relevance cfg);
`@backend/sports/__init__.py::sport_registry` (+`::SportRegistry`, auto-registers badminton). ?
add a sport: subclass `SportProfile`, `sport_registry.register(MyProfile)`. § prompt emits
JSON array `{start_time(float DECIMAL SEC), end_time, shuttle_exchange_count(int),
shot_count_confidence('Low'|'Medium'|'High')}`. ? prompt actively defends against MM:SS
timestamps; `get()` returns a fresh instance each call.
135   - `@backend/providers/base.py::AIVideoProvider` ABC (`__init__(api_key, model_name)`);
`@backend/providers/gemini.py::GeminiProvider` (upload->poll->generate JSON temp 0->delete);
`@backend/providers/__init__.py::provider_registry` (+`::ProviderRegistry`, auto-registers
`'gemini'`). ? **add a provider**: subclass, `provider_registry.register('name', MyCls)` (key
passed EXPLICITLY, unlike sport_registry); `get(name, api_key, model_name)`. §
`analyze_video(path, prompt, chunk_duration=0.0, label='') -> (segments:list,
metrics:dict{upload_time,process_time,gen_time})`. ? blocking `time.sleep(10)` poll; returns
`([], metrics)` on ANY failure (empty != error); lazy client (auth surfaces on first use).
136   - `@backend/annotations/base.py::AnnotationProvider` ABC (`submit/poll/fetch/annotate`),
`::Interval` (= metrics.Interval), `::STATUSES`, `::AnnotationProviderRegistry`,
`::annotation_registry`. ? **add a tier**: subclass (define `name`), call
`annotation_registry.register(MyCls)` at module bottom + import in `__init__.py` (registry keys
off `provider_cls().name`). § `fetch -> List[Interval]` sorted; contract says RAISE (not `[]`)
on unavailable (`[]` silently zeroes recall).
137   - `@backend/annotations/file_provider.py::FileProvider` (`'file'`; job_id IS the
resolved CSV path; -> `backend.eval.annotations.load_annotations`).
138   - `@backend/annotations/automated_provider.py::AutomatedProvider` (`'auto'`; injectable
`labeler`, `::warn_if_same_lineage` ORACLE RULE, default `_not_configured` RAISES
NotImplementedError). ? get via `annotation_registry.get('auto', labeler=fn, lineage='...')`.
139   - `@backend/annotations/__init__.py` ? ? `noqa F401` imports are load-bearing (removing
de-registers providers).
140   - `@backend/database.py::Database` ? SQLite, 4 tables
(`videos/play_segments/events/candidate_segments`), `PRAGMA user_version` migration ladder in
`_init_db` (currently =1). Methods: `add_video`,`add_play_segment`,`add_event`,`add_candidate_se
gment`,`link_candidate_to_segment`,`query_candidate_segments`,`query_play_segments`,`get_video`,
`clear_video_data`. ? every conn re-issues `PRAGMA foreign_keys=ON` (else CASCADE silently
skipped); `add_video` is `INSERT OR REPLACE` -> cascade-DELETES dependent segments on re-ingest;
write helpers swallow exceptions (return False/None); events column is `type` not `event_type`;
`duration` stored not derived.
141
142   ---
143
144   ## 3. DATA CONTRACTS (canonical shapes ? one place)
145
146   § **Trajectory CSV** (produced by tracknet/wasb predict, consumed by
`parse_trajectory_csv`):
147   `Frame,Visibility,X,Y` ? header+column order fixed; `Visibility==1`=visible; X,Y pixel
coords. Row = `(fid, 1|0, x or -1, y or -1)`.
148   -> `TrajectoryPoint{frame:int, visible:bool, x:float, y:float}` (frame-sorted).
149
150   § **Candidate window dict** (produced by `trajectory_to_action_windows` AND yolo
`_extract_action_windows_from_chunk`):
151   `{start:float(sec), end:float(sec), active_frame_count:int, peak_velocity:float,
mean_velocity:float, track_id_count:int(==1 for trajectory), chunk_index:Optional[int]}`.
152   `start/end = group_frame/fps + chunk_start`.
153
154   § **candidate_segments DB row** (`db.add_candidate_segment` /
`query_candidate_segments`):
155   `{id, video_id, detector:str, chunk_index, start_time:REAL, end_time:REAL,
duration:REAL, confidence:REAL(min(1.0,peak_velocity)),
stats:JSON{peak_velocity,mean_velocity,active_frame_count,track_id_count},
detection_params:JSON{...}, confirmed_segment_id, human_label, notes, created_at}`
(stats/detection_params deserialized on read).
156   `detection_params` keys by detector:
wasb=`{detector,velocity_thresh,merge_gap,min_action_window_duration,weights_path}`; tracknet
adds `queue_length`; yolo=`{velocity_thresh,merge_gap,frame_skip,tracker,detector_model,detector
_score_threshold,min_action_window_duration}`.
157

§ **play_segment dict** (`db.add_play_segment` / `query_play_segments`):
`{id:int, video_id, start_time:float, end_time:float, duration:float(end-start), server, receiver, metadata:dict, events:[...]}`. Hybrid P4 metadata `{shot_count, shot_count_confidence:'Unknown', detector:f'{family}-hybrid-{model}'}`; gemini detector = bare `model_name`; motion adds random `intensity/avg_speed_kmh`.

§ **AI segment dict** (provider output, consumed by hybrid P3 / gemini): `{start_time, end_time, shuttle_exchange_count?, shot_count_confidence?}` (offset by chunk/window start; `_candidate_id` stamped internally).

§ **GT / golden CSVs** (TWO conventions ? do not conflate):
- generic (`annotations.load_annotations`): `start,end` 2-col; optional header; decimal or MM:SS/HH:MM:SS; RAISES on bad row.
- golden/slicer (`calibrate_wasb.load_golden_gts`, `rally_slicer.load_rally_labels`, `export.SEG_FIELDS`): `rally_number,start_time,end_time[,duration,start_mmss,end_mmss]`; SILENTLY skips bad rows.

§ **det_raw.jsonl** (one window/line): `{"w":<window_index>, "f":[[frame_id,[[x,y,score,scale],...]], ...]}`. Lines MUST be contiguous (line N has `w==N`); first violation/JSONDecodeError -> truncate-heal.

§ **manifest.json** (cache): `{version, frames_dir(abspath), weights, sport, frames_in, frames_total, windows_total, windows_done}`. Reuse only if ALL match + `version==CACHE_VERSION`.

§ **collector manifest.json** (eval): `{eval_sets:[{video_id(label), local_path, csv}, ...]}`; ? DB key = file MD5, manifest `video_id` is a label.

§ **eval primitives**: `Interval=Tuple[float,float]` (start<end). `Match{pred_index,gt_index,iou}`. `MatchResult{matches, unmatched_pred_indices(=FP), unmatched_gt_indices(=FN)}`. `Score{iou_threshold,num_pred,num_gt,true_positives,false_positives,false_negatives,precision,recall,f1,mean_iou,mean_start_error,mean_end_error}.as_dict()`.

§ **WSL detector candidate** (in-WSL, model<->tracker): `{'xy':np.array([x,y]), 'score':float, 'scale':int}`; plain JSON form `[x,y,score,scale]`. ? `_dets_to_tracker` MUST rebuild `xy` as np.array (tracker does `np.linalg.norm`).

---

4. WASB WSL CONTRACT (external dep `~/models/WASB-SBDT`, MIT; NOT in this repo ? claims below are unverified against external source, only the repo-side caller wasb_infer.py was checked)

- Env: WSL conda `wasb`, cwd `~/models/WASB-SBDT/src`. Hydra `compose(config_name='eval', overrides=[dataset=<sport>, model=wasb, detector.model_path=<weights>, runner.device=cuda, runner.gpus=[0]])`. `gpus=[0]` because box is single-GPU. (overrides VERIFIED in @backend/detectors/wasb_infer.py::_build_cfg.)
- `model=wasb` -> `name=hrnet`, `frames_in=3`, `frames_out=3`, input 512x288 (WxH), ImageNet normalize; affine resize maps heatmap coords -> original-frame coords. (model dims read from cfg["model"] at runtime in wasb_infer.run.)
- `detectors/detector.py::TracknetV2Detector.run_tensor(imgs, affine_mats) -> (results, hms_vis)`. `results[bid][eid]` = list of `{xy:np.array (ORIGINAL coords), score, scale}`. (caller uses `detector.run_tensor(imgs, trans)` returning `(batch_results, _)` ? consistent.)
- `trackers/online.py::OnlineTracker` ? STATEFUL, sequential per `update()` (assumes every frame fed once, in order, no gaps). `update(frame_dets)->{x,y,visi,score}`; `refresh()` resets. `det['xy']` MUST be np.array. Full ordered re-run is deterministic. (caller calls `build_tracker(cfg)`, `tracker.refresh()`, `tracker.update(...)` returning dict with `visi/x/y` ? consistent.)
- `dataloaders/dataset_loader.py::ImageDataset(cfg, samples, input_wh, output_wh, transform, seq_transform, is_train)`; `samples=[{frames:[paths], annos:[{frame_path, center:Center(is_visible,x,y)}]}]`. (caller constructs exactly this ? VERIFIED in wasb_infer.run.)
- Weights `pretrained_weights/wasb_badminton_best.pth.tar` ? NOT committed, academic-data-trained -> ? confirm terms before COMMERCIAL deploy. monotrack weights = noncommercial, avoided.

---

5. CROSS-REPO (sibling private repo `sports-data-collector` ? external, unverified

here)
- Indexer keys video by **FILE MD5**; collector keys by human `video_id`. ? re-encoding/re-downloading changes bytes -> MD5; acquisition MUST precede processing; manifest tracks the exact file used.
- `python -m src.cli.curate export` (collector) -> `<out>/<sport>/<video_id>.csv` (indexer `start,end` format) + `manifest.json` -> consumed by `@backend/eval/batch.py` + `@run_phase3_eval.py`.
- Collector also emits golden rally labels (`rally_number,start_time,end_time,...`) -> consumed by `@backend/eval/calibrate_wasb.py` / `export_wasb_segments.py` / `@backend/tools/rally_slicer.py`.

---

6. GOTCHAS (cross-cutting footguns)
- **MD5 keying:** `video_id` = MD5 of WHOLE file everywhere ? but each entrypoint uses a different chunk SIZE (64KB/1MB/8KB); values agree, code does not. Manifest `video_id` is a label, NOT the DB key.
- **Three default segmenters:** config.json `gemini` vs setup.py `yolo_hybrid` vs main.py `motion`. You may run a different detector than expected.
- **Gemini free-tier / API-key dead path:** hybrids + gemini return `[]` (not error) if `{PROVIDER}_API_KEY` missing OR WSL healthcheck fails ? a silent no-op for callers. `analyze_video` also returns `[]` on real failure -> empty != "no rallies".
- **fps / frame_width must be PROBED not assumed:** `_probe_fps_width` / runner fallbacks (30.0, 1280.0) silently mis-scale velocity thresholds if probing is skipped; calibrate_wasb CLI bakes `59.94/1920`.
- **WSL `/mnt` can't see Google Drive / network mounts;** stage video into native WSL fs (`~/clips`) ? `/mnt/c` reads are slow and Drive-backed paths invisible.
- **ffmpeg NOT in WSL** by design: all ffmpeg/ffprobe run Windows-side (must be on PATH); WSL only runs the GPU model. `extract_frames` uses cv2 PNG (slow) ? prefer host ffmpeg + `--frames_dir`.
- **GPU float nondeterminism:** detector pass is checkpointed/resumable but not bit-reproducible; the CPU tracker re-run over the ordered cache IS deterministic.
- **`output/` is gitignored:** DB, baselines, cache, reels live there. Predictions MUST exist in `output/<db_name>` before any eval/regression.
- **Validator ordering coupling:** exec order is Format -> ResolutionFPS -> PTSContinuity -> SceneCut -> Blur -> Relevance; `resolution_fps_check` hard-fails unless `format_check` ran earlier (shared `context['ffprobe_meta']`).
- **`add_video` INSERT OR REPLACE + FK CASCADE** -> re-ingesting a `video_id` cascade-deletes its segments. Use `clear_video_data` deliberately.
- **Twin segmenters:** `wasb_hybrid.py` / `tracknet_hybrid.py` are copy-paste; fixes to P3/P4 must land in both.
- **Two "annotations" modules:** `backend.eval.annotations` (GT CSV loader) vs `backend.annotations` (provider registry). regression.py imports the latter.
- **Hardcoded tuned constants:** `calibrate_wasb.BASELINE`, `export.TUNED`, slicer/compiler padding ? duplicated by copy, not import; re-tuning won't propagate.
- **`timeout_sec=0` = no timeout:** a hung WSL/GPU call blocks forever.
- **Stitch concat assumes identical encode:** per-clip re-encode then `-c copy` concat; if probe fails (0.0) no boundary validation.

---

7. RAMP PATHS (to do X -> open Y::symbol)
Task	Open
Tune candidate windowing (velocity/merge/min-dur)	`@backend/detectors/tracknet_runner.py::trajectory_to_action_windows`; cfg `config.json indexing.{wasb,tracknet}.velocity_threshold`, `dead_time_merge_gap`, `min_action_window_duration`
Add a segmenter	subclass `@backend/indexer/base.py::BasePlaySegmenter`; `segmenter_registry.register(...)` at module bottom; import in `@backend/indexer/__init__.py`
Change stitching padding	`config.json stitching.{begin_padding,end_padding}`; impl `@backend/stitcher/compiler.py::VideoCompiler.compile_highlights`; mirror `@backend/tools/rally_slicer.py::BEGIN_PADDING/END_PADDING`
Add a sport	subclass `@backend/sports/base.py::SportProfile`; `sport_registry.register(...)` in `@backend/sports/__init__.py`; define prompt+HSV in new file

Add an AI provider	subclass `@backend/providers/base.py::AIVideoProvider`; `provider_registry.register('name', Cls)`
Add an annotation/GT tier	subclass `@backend/annotations/base.py::AnnotationProvider`; register + import in `@backend/annotations/__init__.py`
Run calibration (WASB thresholds)	`@backend/eval/calibrate_wasb.py::main` (`--trajectory --labels --fps --frame-width`); trust `cv` recall not `fit`
Export tuned rally segments / clips	`@backend/eval/export_wasb_segments.py::main` (`--slice` needs `--video`)
Run a video end-to-end	`@backend/main.py::main` (`--video-path` or `--server`) or `@run_process_and_stitch.py::main` (edit hardcoded paths)
Score DB preds vs GT (one video)	`@run_eval.py::main`
Batch score a collector manifest	`@run_phase3_eval.py::main` (`--manifest`)
Guard a regression (CI)	`@run_regression.py::main` (`--video-id`); exit 1=regress, 2=missing DB; snapshot via `--update-baseline`
Resume a crashed WASB pass	nothing to do ? `@backend/detectors/wasb_infer.py::run` auto-resumes from `det_raw.jsonl`+`manifest.json` next to `--out`; `--fresh` forces restart; cache invalidates on weights/sport/frames change
Add a validator	subclass `@backend/validators/base.py::VideoValidator`; `registry.register(instance)` in `@backend/validators/__init__.py` (mind order; producers before consumers)
Add a DB column/table	`@backend/database.py::Database._init_db` migration ladder + bump `PRAGMA user_version` (else version-assert tests fail)
Train the learned segmenter	`@backend/eval/training.py::build_training_set` -> `@backend/eval/classifier.py::LogisticRegression.fit/save`

---

8. TEST MAP (`tests/test_*.py` -> subsystem)

Test	Covers
`test_base.py`	`indexer.base.SegmenterRegistry` (register/get/list via DummySegmenter)
`test_yolo_hybrid.py` / `test_tracknet_hybrid.py`	hybrid segmenters (yolo torchvision+ByteTrack; tracknet 4-phase)
`test_gemini.py`	`indexer.gemini.GeminiPlaySegmenter` (parse/validate/chunk)
`test_tracknet_runner.py`	`detectors.tracknet_runner` parse_trajectory_csv + trajectory_to_action_windows (shared brain)
`test_wasb_cache.py`	`detectors.wasb_infer` resumable cache (serialize, contiguity heal, manifest invalidation, atomic writes)
`test_eval_metrics.py`	`eval.metrics` IoU/matching/score/pr_curve
`test_eval_annotations.py`	`eval.annotations` GT CSV parse/validate
`test_eval_harness.py`	`eval.harness` DB-pred fetch + evaluate + report_to_dict
`test_eval_batch.py`	`eval.batch` path resolve / stage select / aggregate
`test_calibration.py`	`eval.calibration` kfold/select_best/cross_validate/LOVO
`test_calibrate_wasb.py`	`eval.calibrate_wasb` CLI/run/grid
`test_export_wasb_segments.py`	`eval.export_wasb_segments` segment+comparison export
`test_eval_training.py`	`eval.training` feature extraction + labeling
`test_eval_classifier.py`	`eval.classifier.LogisticRegression` fit/predict/save/load
`test_eval_regression.py`	`eval.regression` baseline compare/gate
`test_rally_slicer.py`	`tools.rally_slicer.compute_clip_windows` (pure) + label parse
`test_video.py`	`utils.video` duration/chunk
`test_geometry.py`	`utils.geometry` center/distance/velocity/IoU
`test_database.py`	`database.Database` schema/CRUD/migrations/version
`test_sport_profiles.py`	`sports` registry + BadmintonSportProfile
`test_providers.py` / `test_provider_gemini.py` / `test_gemini.py`	`providers` registry + GeminiProvider lifecycle
`test_annotation_providers.py` / `test_automated_provider.py`	`annotations` registry, FileProvider, AutomatedProvider (oracle/lineage)
(no dedicated test)	`validators/*`, `stitcher.compiler`, `main.py`, `run_*.py` orchestrators, `wasb_runner`, `wasb_hybrid`/`tracknet_hybrid` (covered indirectly) ? ? exercised only indirectly

261

12. user (2026-06-07T06:41:16.649Z)

```
.pytest_cache\README.md
TEST_RUN_SUMMARY.md
docs\EVALUATION.md
docs\MONETIZATION_AUDIT.md
docs\TRACKNET_WSL_SETUP.md
README.md
docs\GOLDEN_SET_IMPLEMENTATION.md
docs\ROADMAP.md
docs\GOLDEN_SET_PHASE1_VIDEOS.md
docs\GOLDEN_SET_VENDORING.md
docs\HOW_RALLY_DETECTION_WORKS.md
docs\CODE_MAP.md
docs\RALLY_DETECTION_RESEARCH.md
docs\VIDEO_RECORDING_GUIDELINES.md
docs\DECISIONS.md
docs\AUDIO_RALLY_DETECTION_PLAN.md
docs\BACKLOG.md
docs\AUDIO_E1_FINDINGS.md
```

13. user (2026-06-07T06:41:17.111Z)

```
C:\Users\avidu\Projects\sports-data-collector\.pytest_cache\README.md
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\NDA_TEMPLATE.md
C:\Users\avidu\Projects\sports-data-collector\docs\DECISIONS.md
C:\Users\avidu\Projects\sports-data-collector\README.md
C:\Users\avidu\Projects\sports-data-collector\docs\PIPELINE_ROLE.md
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ANNOTATION_GUIDE.md
C:\Users\avidu\Projects\sports-data-collector\docs\annotation\ROORA_BRIEF.md
C:\Users\avidu\Projects\sports-data-collector\data\validation\pilot_badminton_01_report.md
C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md
```

14. assistant (2026-06-07T06:41:26.357Z)

```
Indexer architecture is clear. Now I need the collector's pipeline + both READMEs (to calibrate
the "README-reader debuggability" bar). Let me read those and fan out an Explore agent to map
the collector's processing stages and current output-emission points.
```

15. user (2026-06-07T06:41:29.679Z)

```
1       # Badminton Highlight & Rally Indexer
2
3       A local, AI-powered web application and CLI utility designed to process long-form
badminton recordings (e.g., from GoPro), run comprehensive pluggable validations (quality,
integrity, and relevance checks), index all rallies into a relational SQLite database, and
dynamically compile custom highlight reels using high-speed FFmpeg stitching.
4
5       The pipeline is built around three pluggable registries ? **validators**,
**segmenters**, and **AI providers** ? plus a **sport profile** registry so the same engine can
target other racket/court sports in the future. Badminton is the default profile.
6
7       > **Where this is heading:** the project is evolving from an AI-API-dependent prototype
into a self-improving tool that learns rally detection *offline* from a commercially-licensed,
annotated corpus. See **[docs/ROADMAP.md](docs/ROADMAP.md)** for the four-phase plan and
**[docs/DECISIONS.md](docs/DECISIONS.md)** (ADR-002?004) for the rationale.
8
9       ---
10
11      ## ? Key Features
12
13      *   **Pluggable Video Validation Framework**: Easily extendable base-class architecture
```

that automatically runs checks on:
14   *   *Static Format & Resolution*: Verifies container is MP4, resolution >= 720p, and framerate >= 30 FPS.
15   *   *PTS Timeline Continuity*: Scans keyframe timestamps for timeline gaps, jumps, or editing splices to detect tampering or splices.
16   *   *Scene Cut Density Check*: Measures sub-sampled HSV frame histogram correlation to verify visual cuts. A continuous raw camera stream should have near 0 cuts.
17   *   *Blurriness Check*: Calculates average frame Laplacian variance to reject out-of-focus footage.
18   *   *Relevance (Is it Badminton?)*: Uses high-performance HSV color court matching to identify court layouts.
19   *   **Three Pluggable Rally Segmenters** (`backend/indexer/`):
20   *   `yolo_hybrid` **(default)** ? two-stage pipeline. Stage 1 runs **torchvision person detection** (Faster R-CNN, BSD-licensed) with **ByteTrack** tracking (supervision, MIT-licensed) to cheaply filter the video down to a small set of high-motion candidate windows. Stage 2 hands each candidate (with a small padding) to the AI provider for high-precision semantic boundaries and shot counts. Massively cheaper than running the LLM on the full video. *(The registry key remains `yolo_hybrid` for back-compat; the AGPL-licensed YOLOv8 was replaced with permissive components ? see docs/MONETIZATION_AUDIT.md and ADR-005.)*
21   *   `gemini` ? sends the entire video (or a `chunk_duration` slice) to Google Gemini directly. Highest precision per call, highest cost.
22   *   `motion` ? zero-dependency OpenCV pixel-difference heuristic. No API key required. Useful as a baseline or for offline/air-gapped use.
23   *   **Pluggable AI Provider Registry** (`backend/providers/`): The Gemini provider ships in-tree; new providers (e.g. an OpenAI or local-model backend) can be registered without touching the segmenters.
24   *   **Pluggable Sport Profile Registry** (`backend/sports/`): The per-sport LLM prompt and relevance config live in a `SportProfile`. Add a new sport by subclassing `SportProfile` and registering it.
25   *   **SQLite Indexing**: Each processed video is keyed by MD5 hash. Play segments (start/end/duration/server/receiver/metadata) and per-segment events are persisted in `output/sports_indexer.db`. Re-running on the same file is a cache hit.
26   *   **Query System**: Filter indexed play segments by `server`, `receiver`, `duration_min`, and `event_type` (the schema supports rich event types like smashes/serves/fouls; today the shipped segmenters mainly populate timing + estimated `shot_count`).
27   *   **Zero-Dependency Local UI**: Served statically directly by FastAPI (Vite + Node.js NOT required!) featuring a glassmorphic dashboard, responsive filtering, and immediate clip stitching.
28   *   **High-Speed FFmpeg Stitching** with configurable `begin_padding`, `end_padding`, and `min_shot_count` filtering.
29   *   **Detailed CLI Debugging Tools**: `--debug-clip <t>` inspects frame metrics, validator scores, and motion values around a timestamp. `--trim-start/--trim-end` extracts an arbitrary segment via stream-copy ffmpeg for fast iteration.
30
31   ---
32
33   ## ?? Setup & Diagnostics
34
35   ### 1. External System Requirements
36   The indexer depends on **FFmpeg** and **FFprobe** to parse media containers, extract keyframes, and stitch clips.
37
38   *   **On Windows (PowerShell as Admin)**:
39   ```powershell
40   winget install Gyan.FFmpeg
41   ```
42   *Restart your terminal afterwards to register the path changes.*
43
44   ### 2. Auto-Installer & Diagnostics Checker
45   Run the diagnostics checker script in the root directory to verify your dependencies, initialize default configs, and install PIP packages:
46   ```bash
47   python setup.py

```
This script will:
*   Validate `ffmpeg` and `ffprobe` are present in your PATH.
*   Initialize a default `config.json` (if one does not exist) with thresholds for blur, court HSV bounds, motion signals, YOLO velocity, chunking, and stitching.
*   Detect whether an NVIDIA GPU is available (via `nvidia-smi`) and select the matching PyTorch wheel index automatically (`cu124` if GPU present, `cpu` otherwise).
*   Install everything from `requirements.txt`: `fastapi`, `uvicorn`, `opencv-python`, `numpy`, `pydantic`, `jinja2`, `python-dotenv`, `google-genai`, `torch`, `torchvision`, `supervision`.

> The default `yolo_hybrid` segmenter uses **torchvision**'s COCO-pretrained Faster R-CNN, whose weights are downloaded and cached automatically by torchvision on first run ? nothing is bundled in the repo. (Pick a lighter/heavier detector via `indexing.detector_model`: `fasterrcnn_resnet50_fpn` (default), `fasterrcnn_mobilenet_v3_large_fpn`, or `retinanet_resnet50_fpn`.)

---

## ? Running the Application

### 1. Launching the Local Web App Dashboard
To start the FastAPI server and open the web dashboard:
```bash
python -m backend.main --server --port 8000
```
Open your browser and navigate to **http://localhost:8000**.
Simply input your local raw video file path in the sidebar to process, validate, index, and compile highlights instantly.

2. Running in CLI Processing Mode
To process and index a video completely via terminal commands:
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4"
```
This outputs a validation summary and indexes all rallies directly into the local SQLite database, using whichever segmenter is set as `default_segmenter` in `config.json` (`yolo_hybrid` by default).

To explicitly pick a segmenter for one run:
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter yolo_hybrid
python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter gemini
python -m backend.main --video-path "C:/path/to/my_video.mp4" --segmenter motion
```

3. Rich CLI Debugging (`--debug-clip`)
To diagnose exactly why a video segment was registered as active/dead play, or inspect detailed focus/continuity metrics at a particular time in seconds:
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4" --debug-clip 125.5
```
Outputs granular validation check logs and motion ratios within a +/- 3 second frame window around the timestamp.

4. Extracting an Arbitrary Trimmed Segment (`--trim-start` / `--trim-end`)
For fast iteration on a tiny slice of a multi-GB recording, you can stream-copy a segment to disk without re-encoding:
```bash
python -m backend.main --video-path "C:/path/to/my_video.mp4" \
    --trim-start 600 --trim-end 660 --trim-output slice_10m_to_11m.mp4
```
Output lands in `--output-dir` (default `output/`) unless `--trim-output` is an absolute path.
```

5. End-to-End "Process + Stitch" Script

`run_process_and_stitch.py` is an interactive convenience runner: it MD5-hashes a hardcoded video path, reuses cached indexed segments if available (offering to re-index from scratch), and then prompts for stitching parameters (`begin_padding`, `end_padding`, `min_shot_count`) before compiling the final highlight reel. Useful when iterating on stitching settings against a fixed large recording without re-running expensive segmentation.

?? Threshold Tuning (`config.json`)

You can adjust the parameters of your validators, segmenters, AI provider, and stitcher by editing `config.json` in the root folder. The default config emitted by `setup.py`:

```json
{
    "sport": "badminton",
    "ai_provider": "gemini",
    "ai_model": "gemini-2.5-flash",
    "validation": {
        "min_resolution_width": 1280,
        "min_resolution_height": 720,
        "min_fps": 29.9,
        "min_blur_threshold": 20.0,
        "max_scene_cuts_per_minute": 0.5,
        "relevance": {
            "court_green_lower": [35, 40, 40],
            "court_green_upper": [85, 255, 255],
            "court_pixels_min_ratio": 0.05
        }
    },
    "indexing": {
        "default_segmenter": "yolo_hybrid",
        "use_gpu": true,
        "motion_threshold": 0.08,
        "min_segment_duration": 3.0,
        "dead_time_merge_gap": 2.0,
        "chunk_duration": 90,
        "chunk_overlap": 10,
        "yolo_velocity_threshold": 0.15,
        "yolo_frame_skip": 3,
        "yolo_tracker": "bytetrack.yaml",
        "yolo_parallel_chunks": false,
        "yolo_chunk_workers": 2,
        "min_action_window_duration": 2.0,
        "ai_handoff_padding": 2.0,
        "ai_max_workers": 5
    },
    "output": {
        "default_highlights_name": "highlights.mp4",
        "db_name": "sports_indexer.db"
    },
    "stitching": {
        "begin_padding": 0.5,
        "end_padding": 0.5,
        "use_cache": false,
        "min_shot_count": 1
    }
}
```

Notable knobs:

*   `validation.min_blur_threshold` ? defaults to `20.0` (deliberately permissive for noisy indoor-arena GoPro footage; see `TEST_RUN_SUMMARY.md` for why).
*   `indexing.default_segmenter` ? `yolo_hybrid` | `gemini` | `motion`.
*   `indexing.use_gpu` ? when true and CUDA is available, YOLO runs on GPU.

*   `indexing.yolo_velocity_threshold` ? fraction of frame width per second; lower values catch slower play, higher values are stricter about what counts as "action".
*   `indexing.yolo_frame_skip` ? process every Nth frame during YOLO tracking. `3` at 30 fps ? 10 samples/sec.
*   `indexing.chunk_duration` / `chunk_overlap` ? how to split long videos for the YOLO phase.
*   `indexing.ai_handoff_padding` ? seconds of slack added around each YOLO candidate window before sending to the AI provider.
*   `indexing.ai_max_workers` ? parallel calls to the AI provider during phase 3.
*   `stitching.min_shot_count` ? drop play segments whose estimated shot count is below this threshold when compiling highlights.

? Frequently Asked Questions (FAQ)

Q1: The server complains that `ffprobe` is not found.
Ensure FFmpeg is installed and added to your system environment variables. You can check this by typing `ffmpeg -version` or `ffprobe -version` in your terminal. If running `setup.py` outputs a `[FAIL]`, re-install FFmpeg or manually copy the binary files to a PATH folder.

Q2: Why does my video fail the Relevance check?
Our default relevance check scans for the typical indoor green badminton court floor color space (HSV). If your court is blue, or uses standard wood grain flooring, simply adjust the HSV values inside your `config.json` under `validation.relevance.court_green_lower` and `court_green_upper` to match your court, or lower the `court_pixels_min_ratio` to `0.0` to bypass.

Q3: How do I add a new custom validator to the pipeline?
1. Create a new python file in `backend/validators/` (e.g. `my_check.py`).
2. Subclass `VideoValidator` and implement `validate(self, video_path, config, context)`.
3. Register your class inside `backend/validators/__init__.py` using `registry.register(MyCheckValidator())`.
It's that simple! The app will automatically run it, display it in the CLI diagnostics summary, and render it in the frontend checklist.

Q4: Why does the CLI or server output freeze or take a long time on large videos?
*   **Print Buffering**: Standard Python buffers console output when stdout is redirected or running in background shells on Windows. To get real-time unbuffered log statements immediately, run commands with Python's unbuffered flag:
    ```bash
    python -u -m backend.main --video-path "C:/path/to/video.mp4"
    ```
*   **Video Seeking Overhead**: Seeking to random frames in high-bitrate long H.265/HEVC videos using OpenCV's `cap.set(cv2.CAP_PROP_POS_FRAMES)` incurs massive I/O decoding delays. The `motion` segmenter reads sequentially and skips frame decodes using container-level `cap.grab()`, which delivers a **100x speedup** on multi-gigabyte raw files. `yolo_hybrid` sidesteps the problem entirely by extracting fixed chunks via ffmpeg first.

Q5: Why did the video fail with "No keyframes / packet timestamps could be extracted"?
Some H.265/HEVC streams (e.g. GoPro footage) encode frames as GDR or CRA keyframes. Standard FFmpeg frame-level skipping (`-skip_frame nokey`) fails to identify them, returning empty logs. We resolved this by extracting packet indexes (`packet=pts_time,flags`) at the container demuxer level instead, which is incredibly robust and executes instantly without decoding a single pixel.

Q6: How can I quickly test the indexing without waiting to parse an entire multi-GB video file?
You can pass the `--max-frames <N>` argument in CLI or `max_frames` in the POST API payload. This restricts the segmenter frame loop to process only the first N sub-sampled frames, enabling you to test motion filters and court relevance checks in seconds:
```bash
python -m backend.main --video-path "C:\Users\avidu\Videos\LargeTestVideo.MP4" --max-
```

frames 300
194   ```
195   *(At 5 sub-sampled frames per second, `--max-frames 300` analyzes exactly the first 60
seconds of video).*
196
197   ### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
198   Both `yolo_hybrid` (default) and `gemini` need an API key for the AI provider. To use
Google Gemini:
199   1. Create a local `.env` file in the project root (this is already in `.gitignore` to
prevent secret leaks).
200   2. Set your Google AI Studio API key in the file:
201   ```env
202   GEMINI_API_KEY=your_key_here
203   ```
204   3. Run with whichever segmenter you want:
205   ```bash
206   # Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini
207   python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid
208
209   # Full-video Gemini analysis (more expensive, highest semantic precision per call)
210   python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini
211
212   # Pure CV baseline ? no API key needed
213   python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
214   ```
215   *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under
`"indexing"` in `config.json`.*
216
217   The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` +
`ai_model` at the top level of `config.json`. Today only `"gemini"` is registered; new providers
can be added under `backend/providers/`.
218
219   ### Q8: How does the `yolo_hybrid` pipeline actually save money?
220   Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo_hybrid` runs
in four phases:
221
222   1. **Chunking** ? splits the video into overlapping windows (`chunk_duration`,
`chunk_overlap`) so we can process incrementally.
223   2. **Detection + tracking** ? torchvision person detection (Faster R-CNN) + ByteTrack
runs on each chunk locally (GPU-accelerated when available), measuring per-track velocity.
Frames whose normalised player velocity exceeds `yolo_velocity_threshold` are kept; the rest are
dropped. Adjacent active frames are merged into candidate windows, with
sub-`min_action_window_duration` windows discarded.
224   3. **AI handoff** ? only the surviving candidate windows (padded by `ai_handoff_padding`
seconds on each side) are extracted and sent to the AI provider for precise rally boundaries and
shot-count estimation. These calls run in parallel up to `ai_max_workers`.
225   4. **DB population** ? confirmed segments are written to `play_segments` along with
`shot_count`, `shot_count_confidence`, and a `detector` tag like `yolo-hybrid-gemini-2.5-flash`.
226
227   In practice this typically cuts AI-provider runtime/cost by an order of magnitude versus
uploading the full video to `gemini` directly, while preserving the LLM's strength at semantic
boundary detection.
228
229   ### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without
uploading a huge multi-GB file?
230   Ingesting and uploading multi-gigabyte video files to Google Cloud can be slow and
expensive. We built a native **fast local trimming** developer debugging feature:
231   1. Open your `config.json` file.
232   2. Under the `"indexing"` configuration block, add a `"debug_duration"` key (in
seconds):
233   ```json
234   "indexing": {
235   "default_segmenter": "gemini",
236   "debug_duration": 15.0,
237   ...

```
      }
      ```
3. When `"debug_duration"` is set, the Gemini segmenter will instantly run a zero-re-encode, sub-second `ffmpeg` slice to create a tiny temporary clip of the first 15 seconds, upload only that tiny clip to Google AI Studio (which takes <1 second to upload and <5 seconds to process), run the model query, and clean up the temporary files automatically. This allows you to iterate on model prompts, evaluate validation overlaps, and verify rally boundaries in under 10 seconds!

? Annotation & Evaluation

Once a video is processed, you can measure **how well the pipeline detected its rallies** against hand-annotated ground truth. You annotate each rally's `[start, end)` time range in a small CSV; the harness scores the detected rallies straight from the SQLite DB ? no API calls, fully deterministic.

```bash
python run_eval.py --scaffold --annotations rallies.csv          # make an empty template
# ...fill in start,end rows, then:
python run_eval.py --video "C:/videos/match.mp4" --annotations rallies.csv --show-failures
```

It reports precision/recall/F1, mean IoU, and boundary error for **two stages** ? raw detector `candidates` vs. AI-confirmed `final` segments ? which pinpoints whether a weak result is a *detection* problem or a *segmentation* problem.

> Rally timestamps evaluate and tune **segmentation**; they do **not** train the shuttle detector (that needs per-frame coordinate labels). See **docs/EVALUATION.md** for the annotation format, CLI flags, and how to read the output.

? Running the Test Suite

The test suite uses `pytest` and lives in `tests/`. From the project root:

```bash
pip install pytest pytest-cov --user
pytest                              # run everything
pytest tests/test_yolo_hybrid.py    # run a single file
pytest -k yolo --maxfail=1 -v       # filter by keyword, fail fast, verbose
pytest --cov=backend --cov-report=term-missing   # coverage report (uses .coveragerc)
```

The suite mocks external dependencies (the Gemini client, ffprobe/ffmpeg, OpenCV `VideoCapture`, the YOLO model) so it runs offline and without a GPU. The tests cover:

*   `tests/test_base.py` ? segmenter registry
*   `tests/test_database.py` ? SQLite schema, video/segment insert/query
*   `tests/test_geometry.py` ? bbox center / distance / velocity / IoU helpers
*   `tests/test_video.py` ? ffprobe duration + ffmpeg chunk extraction helpers
*   `tests/test_providers.py` + `tests/test_provider_gemini.py` ? AI provider registry and the Gemini provider
*   `tests/test_sport_profiles.py` ? sport profile registry + the badminton prompt
*   `tests/test_gemini.py` ? `GeminiPlaySegmenter` (single-file and chunked paths)
*   `tests/test_yolo_hybrid.py` ? `HybridYoloSegmenter` happy path, missing-API-key bailout, and timestamp-validation drops
*   `tests/test_tracknet_runner.py` + `tests/test_tracknet_hybrid.py` ? TrackNet WSL runner (mocked subprocess) and the `tracknet_hybrid` segmenter
*   `tests/test_eval_metrics.py` + `tests/test_eval_annotations.py` + `tests/test_eval_harness.py` ? rally evaluation harness (interval matching, CSV loading, DB

scoring, CLI)

    ## ? Project Structure (high level)

    ```
    backend/
      main.py                # FastAPI app + CLI orchestrator
      database.py            # SQLite schema and queries
      validators/            # Pluggable video validators (format, blur, scene-cut, relevance, ...)
      indexer/               # Pluggable rally segmenters: motion, gemini, yolo_hybrid, tracknet_hybrid
      detectors/             # TrackNet WSL2 runner (shuttle-trajectory subprocess adapter)
      providers/             # Pluggable AI providers (gemini today)
      sports/                # Pluggable sport profiles (badminton today)
      eval/                  # Rally evaluation harness: metrics, annotations, scoring
      stitcher/compiler.py   # FFmpeg-based highlight compiler
      utils/                 # ffprobe/ffmpeg + bbox geometry helpers
      static/                # Zero-build frontend served by FastAPI
    docs/                    # DECISIONS (ADRs), TRACKNET_WSL_SETUP, EVALUATION
    tests/                   # pytest suite (mocked, runs offline)
    run_process_and_stitch.py # Interactive end-to-end runner
    run_eval.py              # Rally evaluation CLI (see "Annotation & Evaluation" above)
    setup.py                 # Dependency & FFmpeg diagnostics + default config init
    config.json              # Runtime configuration (see "Threshold Tuning" above)
    ```

16. user (2026-06-07T06:41:29.691Z)

1   # Sports Data Collector & Curator

2

3   A clean, extensible Python codebase and CLI toolchain designed to discover, ingest, validate, and curate sports video datasets. The repository stores annotations and video metadata in an SQLite database, supports sport-specific tables and schemas, manages video storage outside of Git, and enforces strict commercial license compliance.

4

5   > **Pipeline role:** this repo is the *data producer* feeding the **badminton-highlight-indexer**, which trains/evaluates an offline rally segmenter on this curated, commercially-licensed data. See docs/PIPELINE_ROLE.md for why `export`/`fetch` exist and how the two repos fit together.

6

7   ---

8

9   ## Features

10

11   - **SQLite Backend**: Centralized `sports_metadata.db` with a generic `videos` table and sport-specific annotation tables (`badminton_rallies`, `tennis_points`, `cricket_deliveries`, `pickleball_rallies`, `tabletennis_rallies`).

12   - **Commercial License Flagging**: Each ingested source license is checked against a configured whitelist (`MIT`, `Apache-2.0`, `BSD`, `CC0`, `CC-BY`, `Public-Domain`, `Proprietary`) and the resulting `is_commercial` flag is stored alongside the video. Non-commercial sources (like `CC-BY-NC-SA`) are flagged as non-commercial but still ingested ? nothing is discarded automatically. The whitelist is read from `config/config.yaml` consistently across the parser, validator, and CLI.

13   - **Overlap & Duration Validation**: Ensures chronological event bounds (e.g. rally start/end times) do not overlap and that durations are positive.

14   - **Stroke-level CSV Parser**: Ingests ShuttleSet/CoachAI-style stroke CSV logs and aggregates them into rally durations using configurable start/end time buffers.

15   - **Local Ingestion Mode**: Manually registers local videos and associated CSV/JSON annotation files directly into the database as curated entries.

16   - **Interactive Curation CLI & Visual Play Harness**: Let's you query DB status, list staging entries, review sample timestamps, and approve/reject them. It includes a built-in play

function that opens the video in your default browser at the exact rally segment (using HTML5 media fragments `#t=start,end` for local files or YouTube start time markers `&t=start` for remote videos) for manual visual verification.

Installation

Ensure you have Python 3.9+ installed.

1. Clone or download the repository.
2. Install the package and dependencies:
   ```bash
   pip install -e .            # core: curate / import-local / export / status
   pip install -e ".[fetch]"   # adds yt-dlp for video acquisition (fetch / crawler)
   pip install -e ".[dev]"     # test tooling (pytest, pytest-cov)
   ```

External requirements (not pip-installable)

The `fetch` command and the ShuttleSet crawler shell out to external binaries. They degrade gracefully (a clear error / skipped download) when missing, but are **required** for video acquisition:

- **FFmpeg / ffprobe** ? used to probe the real resolution/fps of downloaded files.
- **A JS runtime (e.g. [deno](https://deno.com))** ? needed by yt-dlp to solve YouTube's challenge for high-resolution formats. `fetch` auto-detects a deno install at `~/.deno/bin`.

Core curation/import/export/status commands work without any of these.

Usage

All commands should be executed from the root of the project. Make sure `PYTHONPATH` is set to the current directory:

```bash
# On Windows PowerShell:
$env:PYTHONPATH="."
```

1. View Database Status
To check the current statistics of staging vs. curated items:
```bash
python -m src.cli.curate status
```

2. Ingest External Datasets to Staging
To parse external raw stroke-level datasets (e.g. CoachAI format) and register them in staging:
```bash
python -m src.cli.ingest --sport badminton --parser coachai --file-path data/staging_coachai_match.csv --license CC-BY-NC-SA --video-id staging_match_non_comm
```

3. Launch the Interactive Curation Tool & Play Harness
To review staging items, inspect their timestamps, play individual rallies in the browser to visually verify their start and end bounds, and approve or reject them:
```bash
python -m src.cli.curate curate
```

4. Manually Import Local Videos & Annotations

To register a local video file alongside its JSON/CSV annotation file (bypasses staging):

```bash
python -m src.cli.curate import-local --sport badminton --video-path "data/videos/my_custom_match.mp4" --annotations-path "data/mock_local_badminton_anno.json" --match-name "My Custom Local Match" --license Proprietary
```

`import-local` works for every configured sport ? `badminton`, `tennis`, `cricket`, `pickleball`, and `tabletennis` ? from either a JSON or a CSV annotations file. For the canonical rally CSV schema and the rater-facing instructions used to produce these files (e.g. via an annotation vendor), see [`docs/annotation/`](docs/annotation/).

5. Fetch / Upgrade Videos to Best Resolution

Videos are **always downloaded at the best resolution available** (via yt-dlp `bv*+ba/b -S res,...`, merged to MP4); the actual width/height/fps is ffprobed and stored ? never assumed. The `fetch` command (re)downloads videos for entries already in the DB and updates `local_path`/`resolution`/`fps` **in place**, preserving all rally annotations and curation status (no clean slate needed):

```bash
# Upgrade every sub-720p video to best resolution, using a logged-in browser's cookies:
python -m src.cli.curate fetch --cookies-from-browser firefox

# A single video, forcing a re-download even if a good local copy exists:
python -m src.cli.curate fetch --video-id shuttleset_1 --force --cookies-from-browser firefox
```

Flags: `--sport`, `--video-id`, `--status {staging,curated,all}` (default `all`), `--min-height` (skip videos already at/above it unless `--force`; default `720`), `--max-height` (cap resolution, e.g. `1080`; default best available), `--force`, `--include-noncommercial`, `--cookies-from-browser`/`--cookies`, `--player-client`. A global resolution cap can also be set via `video_storage.max_height` in `config/config.yaml`. Re-runs are cheap and resumable; a download that falls below `--min-height` is treated as a **failure** (discarded, DB left unchanged) so a flaky low-res result never overwrites a good entry.

> **Authenticated cookies are strongly recommended.** Anonymous YouTube downloads are throttled and non-deterministic (often falling back to 144p). Pass `--cookies-from-browser firefox` (or `chrome`/`edge`/`brave`, fully closed) ? or `--cookies cookies.txt` ? to use a logged-in (ideally Premium) session for fast, reliable best-resolution downloads. Chrome's app-bound cookie encryption may block direct reads on Windows; Firefox reads cleanly even while open.
>
> **JS runtime:** yt-dlp needs one (e.g. [deno](https://deno.com)) to solve YouTube's challenges for high-res formats; `fetch` auto-detects a deno install at `~/.deno/bin`. The default `--player-client` avoids `web`/`web_safari`, which YouTube forces into SABR streaming (drops to 144p).

6. Export Annotations as Indexer-Ready Eval/Training Sets

To turn curated annotations into ground-truth files the badminton-highlight-indexer can score and train against, export per-video `start,end` CSVs plus a `manifest.json`:

```bash
# Curated + commercially-safe videos across all supported sports (default):
python -m src.cli.curate export --out-dir data/eval_export

# One sport, also including staging entries:
python -m src.cli.curate export --sport badminton --include-staging
```

The exporter writes `<out-dir>/<sport>/<video_id>.csv` (decimal-second `start,end` rows) and a `manifest.json` pairing each CSV with its video `local_path`/`video_url`, fps, resolution, and license. The indexer keys videos by **file MD5**, not by our `video_id`, so the downstream eval/training run must process the exact video file named in the manifest. Flags: `--sport` (restrict to one sport), `--include-staging` (default: curated only), `--include-noncommercial` (default: commercial-safe only).

```
## Configuration

Adjust settings in `config/config.yaml`:
- **`database_path`**: Path to the SQLite database.
- **`video_storage`**: Local folder path and automatic download size limit.
- **`commercial_licenses`**: Whitelist of licenses allowed for commercial use.
- **`supported_sports`**: Allowed sports categories.

---

## Documentation

- [docs/PIPELINE_ROLE.md](docs/PIPELINE_ROLE.md) ? how this repo feeds the **badminton-highlight-indexer** (`export` / `fetch`).
- [docs/DECISIONS.md](docs/DECISIONS.md) ? collector-local decision records (**ADR-001**: keep explicit per-sport code instead of a registry, with an "adding a new sport" checklist).
- [docs/annotation/](docs/annotation/) ? the vendor annotation package used to produce golden rally labels:
  - [ROORA_BRIEF.md](docs/annotation/ROORA_BRIEF.md) ? vendor brief + the canonical rally CSV schema.
  - [ANNOTATION_GUIDE.md](docs/annotation/ANNOTATION_GUIDE.md) ? the illustrated rater guide (singles & doubles).
  - [INTAKE_MANIFEST_TEMPLATE.csv](docs/annotation/INTAKE_MANIFEST_TEMPLATE.csv) ? per-video intake sheet.
  - [NDA_TEMPLATE.md](docs/annotation/NDA_TEMPLATE.md) ? mutual NDA template (have counsel review).

---

## Running Tests

We use `pytest` for unit and integration testing:
```bash
python -m pytest tests/
python -m pytest tests/ --cov=src --cov-report=term-missing   # with coverage (~87%)
```
```

17. user (2026-06-07T06:41:29.691Z)

```
# Role in the rally-segmenter training pipeline

This repo is the **data producer** in a two-repo system. Its job is to curate
sports videos with per-rally `[start, end)` annotations and commercial-license
metadata, then hand that data to the **badminton-highlight-indexer**, which
processes, scores, and (Phase 4) *trains* an offline rally segmenter on it.

This doc explains *why* the `export` and `fetch` commands exist and how they fit
the bigger picture. The full decision records and end-to-end narrative live in the
indexer repo:

- `badminton-highlight-indexer/docs/ROADMAP.md` ? the four-phase plan and end-state.
- `badminton-highlight-indexer/docs/DECISIONS.md` ? ADR-002 (this bridge), ADR-003
  (acquisition policy), ADR-004 (offline learned segmenter).

## Why this repo matters

The indexer's best rally mode refines local detections with a **paid Gemini API**.
The strategic goal is to replace that with an **offline model trained on our own
commercially-licensed, annotated corpus** ? making rally detection free, offline,
and self-improving. None of that is possible without a clean, licensable,
accurately-labelled, full-resolution dataset. That dataset is what this repo
produces.

```

```
25      ## What the two key commands do (and why)
26
27      - **`curate export`** ? emits the indexer's ground-truth format: per-video
28        `start,end` CSVs + a `manifest.json` pairing each CSV with its video file.
29        Sport-agnostic (all supported sports ? badminton, tennis, cricket, pickleball,
30        table tennis ? share a `(video_id, ordinal, start, end)` shape) and gated on
31        `is_commercial`, so only licensable data flows
32        downstream. The annotations are **temporal-only** ? they train/evaluate rally
33        *segmentation*, not the frame-level shuttle detector (see ADR-002).
34
35      - **`fetch`** ? (re)downloads videos at **best available resolution**, updating
36        the DB **in place** so annotations and curation status are never lost. Uses
37        authenticated cookies for reliable, unthrottled downloads. The why and the
38        hard-won pitfalls (SABR?144p, Chrome DPAPI, rate-limiting) are in ADR-003.
39
40      ## The one contract that matters: video identity
41
42      The indexer keys videos by **file MD5**, this repo by a human `video_id`. A
43      re-encode/re-download changes the bytes, hence the MD5. So the manifest carries the
44      exact `local_path`, and any downstream processing must run on **that** file. This is
45      why acquisition (`fetch`) happens *before* the indexer processes anything.
46
```

18. user (2026-06-07T06:43:26.676Z)

Now I'll provide you with a comprehensive map of the sports-data-collector repository:

COMPREHENSIVE REPOSITORY MAP: SPORTS-DATA-COLLECTOR

1. CLI ENTRYPOINTS

File: `C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py` (lines 805-921)

The `SportsDataCLI` class exposes these subcommands via argparse:

Subcommand	Handler	Lines	Purpose
`status`	`cli.status()`	38-50, 902	Display summary of videos in DB (count by sport/status)
`curate`	`cli.curate()`	619-722, 904-905	Interactive review loop for staging videos; approve/reject/play segments
`import-local`	`cli.import_local(args)`	52-142, 906-907	Manually register local video + JSON/CSV annotations into DB (any sport)
`convert-cvat`	`cli.convert_cvat(args)`	526-560, 908-909	Convert vendor CVAT XML export to canonical per-rally CSV + issues report
`validate-delivery`	`cli.validate_delivery(args)`	562-617, 910-911	Full validation suite on CVAT/CSV; emit `<id>_report.{md,json}` + `<id>_review.csv`
`fetch`	`cli.fetch(args)`	330-426, 914-915	Download/re-download videos at best resolution; update DB in place
`export`	`cli.export_eval_set(args)`	428-524, 912-913	Export curated annotations as indexer-ready eval/training CSVs + manifest.json

---

2. PER-VIDEO PROCESSING PIPELINE

Ingestion Path 1: `import-local` (Direct Local File)

Entry: `src/cli/curate.py:52-142`

1. **Parse annotations** ? `_parse_local_annotations()` (lines 144-290)
   - Reads `.json` or `.csv` files

   - Dispatches per-sport: BadmintonRally, TennisPoint, CricketDelivery, PickleballRally, TableTennisRally
   - Uses `safe_int()` / `safe_float_required()` from `src/core/parsing.py:9-29` for robust casting

2. **Validate** ? `DatasetValidator.validate_{sport}_rallies/points/deliveries()` (src/core/validator.py:29-144)
   - Checks: chronological order, no overlaps, positive duration
   - Returns `(is_valid: bool, errors: List[str])`

3. **Create VideoMetadata** ? `VideoMetadata(...)` (lines 109-121)
   - Sets `status = CurationStatus.CURATED` (bypasses staging)
   - Stores: video_id, sport, local_path, license_type, fps, resolution

4. **Insert to DB** ? `SportsDatabase.insert_video()` + sport-specific insert (lines 124-136)
   - `insert_badminton_rallies()`, `insert_tennis_points()`, etc.
   - Uses UPSERT logic to preserve child rows when updating

Ingestion Path 2: `convert-cvat` (Vendor CVAT Export ? CSV)

Entry: `src/cli/curate.py:526-560`

Key Pipeline (src/parsers/cvat/rally_cvat.py):

1. **Parse CVAT XML** ? `parse_cvat_boxes(xml_path)` (lines 140-193)
   - Extracts `<image>` (frame-mode) or `<track>` (interpolation-mode) boxes
   - Returns `(meta: {step, stop_frame}, boxes: [{frame, attrs}])`

2. **Recompute time from frames** ? `build_records(boxes, fps)` (lines 205-239)
   - **Critical:** derives `start_time = frame_min / fps`, `end_time = frame_max / fps`
   - **Ignores** vendor's `start_time`/`end_time` attributes (they're often wrong due to step/fps mismatch)
   - Resolves CVAT attribute name aliases (e.g., "score_before (a-b)" ? "score_before")

3. **Detect issues** ? `detect_issues(records, stop_frame, ...)` (lines 242-322)
   - **Blocker issues** (halt import): overlap, zero_duration
   - **High/Medium/Low issues** (flag for review): ending_reason_vocab, shot_density, possible_missed_rally, outcome_vocab, frame_out_of_range
   - **Does NOT flag** sampling density; downstream padding absorbs it

4. **Write canonical CSV** ? `write_canonical_csv(records, out_path)` (lines 356-372)
   - Columns: `rally_number`, `start_mmss`, `end_mmss`, `start_time`, `end_time`, `shots_count`, `ending_reason`, `last_shot_outcome`, `score_before`, `score_after`, `notes`
   - Output ready for `import-local`

5. **Emit issues report** ? `render_issue_report(result, source)` (lines 375-396)
   - Human-readable text grouped by severity
   - Also written to `--issues` file if specified

Ingestion Path 3: `validate-delivery` (Validation + Structured Reports)

Entry: `src/cli/curate.py:562-617`

1. **Load from CVAT or CSV:**
   - CVAT: calls `convert(...)` ? records + issues
   - CSV: calls `load_canonical_csv()` (lines 424-456) ? records + calls `detect_issues()`

2. **Build summary (JSON)** ? `build_summary(records, issues, ...)` (lines 459-476)
   - Returns dict with: `n_rallies`, `n_issues`, `counts_by_severity`, `n_blockers`, `passed`, `flagged_rallies`, `issues` (detailed array)

3. **Render validation report (MD)** ? `render_validation_md(records, issues, ...)` (lines 479-509)
   - Human scoreboard + issues by severity + list of flagged rally numbers

4. **Generate review sheet (CSV)** ? `write_review_csv(records, issues, out_path)` (lines 512-534)
   - Annotatable spreadsheet with `auto_flags` column + blank `true_*` / `reviewer_comment` cols for human review

5. **Output files** (lines 600-610):
   - `{vid}_report.json` ? machine summary
   - `{vid}_report.md` ? human report
   - `{vid}_review.csv` ? annotation sheet

Exit code: 0 if no blockers; 2 if blockers exist (gates pipeline)

Fetch Path: `fetch` (Download or Re-download Videos)

Entry: `src/cli/curate.py:330-426`

1. **Collect target videos** by status/sport (lines 356-369)
2. **For each video:**
   - Skip if no URL or already-good local file (unless `--force`)
   - Call `_fetch_one_with_retry()` (lines 292-328) ? `download_best_video()` from `src/core/downloader.py`
   - Probe resolution/fps ? `probe_resolution_fps()` from `src/core/downloader.py`
3. **Update DB in place** (line 407) via UPSERT (preserves rally annotations, status)
4. **Emit fetch summary table** (lines 416-426)

Export Path: `export` (Evaluation/Training Sets)

Entry: `src/cli/curate.py:428-524`

1. **For each video/sport:**
   - Check commercial flag + status (default: CURATED + commercial-safe)
   - Call `get_segment_intervals()` ? returns `[(start_time, end_time), ...]` (lines 527-546)
   - Write `<sport>/<video_id>.csv` with columns `start,end` (decimal seconds)
2. **Write manifest.json** (lines 500-509)
   - Per-video entry: `video_id`, `sport`, `csv` (relative path), `local_path`, `video_url`, `fps`, `resolution`, `license_type`, `is_commercial`, `status`, `num_segments`
   - Metadata: `commercial_only`, `statuses`, `total_videos`, `total_segments`

---

3. DATA MODEL / STORE

Database Schema (src/core/db.py:52-166)

Location: `data/sports_metadata.db` (configurable via `config/config.yaml:database_path`)

Core Table: `videos` (lines 57-71)
```sql
CREATE TABLE videos (
    video_id TEXT PRIMARY KEY,
    sport TEXT NOT NULL,                    -- 'badminton'|'tennis'|'cricket'|'pickleball'|'tabletennis'
    video_url TEXT,                         -- remote source (e.g., YouTube)
    local_path TEXT,                        -- downloaded video file path
    license_type TEXT NOT NULL,             -- enum: MIT, Apache-2.0, CC-BY, Proprietary, Unknown, etc.
    license_url TEXT,
```

```
    is_commercial INTEGER NOT NULL,         -- 1/0 (computed from license whitelist)
    status TEXT NOT NULL DEFAULT 'staging', -- 'staging'|'curated'|'rejected'
    match_name TEXT NOT NULL,
    fps REAL,                               -- frames per second
    resolution TEXT                         -- e.g., '1080p', '720p'
)
```

Sport-Specific Tables (all with `ON DELETE CASCADE`):

- **`badminton_rallies`** (lines 74-88): PK(video_id, rally_number)
  - Columns: `rally_number INT`, `start_time REAL`, `end_time REAL`, `score_before TEXT`,
`score_after TEXT`, `shots_count INT`, `ending_reason TEXT`, `last_shot_outcome TEXT`

- **`tennis_points`** (lines 92-104): PK(video_id, point_number)
  - Columns: `point_number INT`, `start_time REAL`, `end_time REAL`, `set_score TEXT`,
`game_score TEXT`, `server TEXT`, `ending_type TEXT`

- **`cricket_deliveries`** (lines 108-122): PK(video_id, over_num, ball_num)
  - Columns: `over_num INT`, `ball_num INT`, `start_time REAL`, `end_time REAL`, `batsman TEXT`,
`bowler TEXT`, `runs INT`, `wicket INT`, `extras INT`

- **`pickleball_rallies`** (lines 126-139): PK(video_id, rally_number)
  - Same as badminton (rally-based sport)

- **`tabletennis_rallies`** (lines 143-156): PK(video_id, rally_number)
  - Same as badminton (rally-based sport)

Generic Reader (lines 16-22, 527-546):
```python
SEGMENT_TABLES = {
    SportType.BADMINTON: ("badminton_rallies", "rally_number"),
    SportType.TENNIS: ("tennis_points", "point_number"),
    SportType.CRICKET: ("cricket_deliveries", "over_num, ball_num"),
    SportType.PICKLEBALL: ("pickleball_rallies", "rally_number"),
    SportType.TABLE_TENNIS: ("tabletennis_rallies", "rally_number"),
}
```

Canonical Data Shapes (src/core/schemas/)

VideoMetadata (src/core/schemas/base.py:29-40)
```python
class VideoMetadata(BaseModel):
    video_id: str                           # unique identifier
    sport: SportType                        # enum:
badminton|tennis|cricket|pickleball|tabletennis
    video_url: Optional[str]                # remote source
    local_path: Optional[str]               # local file
    license_type: LicenseType               # enum: MIT|Apache-2.0|CC-BY|CC-BY-
NC|Proprietary|Unknown...
    license_url: Optional[str]
    is_commercial: bool                     # license whitelist match
    status: CurationStatus                  # enum: staging|curated|rejected
    match_name: str                         # human name
    fps: Optional[float]                    # video metadata
    resolution: Optional[str]               # video metadata (e.g., '1080p')
```

BadmintonRally (src/core/schemas/badminton.py:4-21)
```python
class BadmintonRally(BaseModel):
    video_id: str
```

```
    rally_number: int                       # chronological index
    start_time: float                       # seconds (must be >= 0, < end_time)
    end_time: float
    score_before: Optional[str]             # e.g., '0-0'
    score_after: Optional[str]              # e.g., '1-0'
    shots_count: Optional[int]              # stroke count
    ending_reason: Optional[str]            #
'winner'|'forced_error'|'unforced_error'|'service_fault'|'let'|'other'
    last_shot_outcome: Optional[str]        # 'in'|'net'|'out'|'n_a'
```

TennisPoint, CricketDelivery, PickleballRally, TableTennisRally follow similar patterns.

---

4. EXISTING OUTPUT ARTIFACTS

A. Export CSVs (curate.py:478-482)

Location: `<out_dir>/<sport>/<video_id>.csv`

Format: 2-column CSV (decimal seconds)
```
start,end
10.5,20.3
25.1,30.8
```

Producer: `export_eval_set()` calls `get_segment_intervals()` (db.py:527-546)

B. Manifest JSON (curate.py:500-509)

Location: `<out_dir>/manifest.json`

Shape:
```json
{
  "commercial_only": true,
  "statuses": ["curated"],
  "total_videos": 50,
  "total_segments": 1200,
  "eval_sets": [
    {
      "video_id": "shuttleset_1",
      "sport": "badminton",
      "csv": "badminton/shuttleset_1.csv",
      "local_path": "/path/to/video.mp4",
      "video_url": "https://youtube.com/...",
      "match_name": "...",
      "fps": 60.0,
      "resolution": "1080p",
      "license_type": "Proprietary",
      "is_commercial": true,
      "status": "curated",
      "num_segments": 24
    }
  ]
}
```

Producer: `export_eval_set()` (curate.py:500-509)

C. Validation Reports (validate-delivery command)

Location: `<out_dir>/{vid}_report.md`, `<out_dir>/{vid}_report.json`,
`<out_dir>/{vid}_review.csv`

`{vid}_report.md` (rally_cvat.py:479-509)
```
# Delivery validation ? <source>

- rallies: **24**
- issues: **4** (blocker 1, high 0, medium 3, low 0)
- structurally-clean rallies: **21/24** (87%)
- gate: **BLOCKED** | **PASS**

## BLOCKER
- (overlap) Rally 1 (ends 21.02s) overlaps Rally 2 (starts 2.50s) by 18.52s...
## MEDIUM
- (shot_density) Rally 2: 2 shots over 30.0s (0.07/s) is implausibly low...
## Rallies to review: [2, 7, 16]
```

`{vid}_report.json` (rally_cvat.py:459-476)
```json
{
  "source": "pilot_badminton_01",
  "fps": 59.94,
  "n_rallies": 24,
  "n_issues": 4,
  "counts_by_severity": {"blocker": 1, "high": 0, "medium": 3, "low": 0},
  "n_blockers": 1,
  "passed": false,
  "flagged_rallies": [2, 7, 16],
  "issues": [
    {
      "severity": "medium",
      "code": "shot_density",
      "rally_number": 2,
      "message": "Rally 2: 2 shots over 30.0s (0.07/s)..."
    }
  ]
}
```

`{vid}_review.csv` (rally_cvat.py:512-534)
```
rally_number,start_mmss,end_mmss,dur_s,shots_count,ending_reason,last_shot_outcome,auto_flags,VE
RIFIED(Y/N),true_start_mmss,true_end_mmss,true_ending_reason,true_last_shot_outcome,reviewer_com
ment
1,0:00.000,0:05.000,5.0,5,winner,in,,,,,,
2,0:02.500,0:32.500,30.0,2,n/a,n/a,shot_density,,,,,,
```

Producers: `build_summary()`, `render_validation_md()`, `write_review_csv()`

D. CVAT Conversion Issues Report (convert-cvat command)

Location: `--issues` file (optional; also printed to stdout)

Format: Human-readable text from `render_issue_report()` (rally_cvat.py:375-396)
```
CVAT rally conversion report ? input.xml
```

```
  fps used        : 59.94
  sampling step   : 30 frames (~0.50s)
  rallies parsed  : 24
  issues          : 4 (1 blocker)

[BLOCKER]
  - (overlap) Rally 1 (ends 21.02s) overlaps Rally 2...

[MEDIUM]
  - (shot_density) Rally 2: 2 shots over 30.0s...
```

Producer: `render_issue_report()` (rally_cvat.py:375-396)

---

5. VALIDATION: `validate-delivery` IMPLEMENTATION

Entry Point: `src/cli/curate.py:562-617`

Validation Logic: `src/parsers/cvat/rally_cvat.py:242-322`

Key Checks:

Issue Code	Severity	Condition	Blocker?
`zero_duration`	blocker	`end_time <= start_time`	YES
`overlap`	blocker	Rally i starts before Rally i-1 ends	YES
`ending_reason_vocab`	medium	`ending_reason not in {winner, forced_error, unforced_error, service_fault, let, other}`	NO
`outcome_vocab`	low	`last_shot_outcome not in {in, net, out, n_a}`	NO
`shot_density`	medium	`shots_count/duration < 0.3 (for duration >= 5s)` ? inflated span	NO
`possible_missed_rally`	medium	Gap > 15s between rallies ? check for unlabeled segment	NO
`frame_out_of_range`	high	`frame_max > stop_frame`	NO

Exit behavior:
- `validate-delivery` exits with code **2** if any blockers exist (gates pipeline)
- Code **0** otherwise (PASS gate)

---

6. CANONICAL DATA SHAPES & FPS/TIMESTAMP HANDLING

Time Representation:

- **Primary:** Decimal seconds (float) ? stored in DB and used in all calculations
- **mm:ss format:** Human-readable output via `_mmss(seconds)` (rally_cvat.py:126-130)
  ```python
  def _mmss(seconds: float) -> str:
      m = int(seconds // 60)
      s = seconds - m * 60
      return f"{m}:{s:06.3f}"  # e.g., "0:02.500"
  ```

Frame ? Seconds Conversion:

- **Frame ? seconds:** `seconds = frame_number / fps`
- **Example:** Frame 120 at 60 fps ? 120/60 = 2.0 seconds
- **Critical in CVAT adapter:** Always use **video's true fps** (probed from ffprobe), never guess 30 fps
  ```python
  # From rally_cvat.py:325-354
  fps = fps_arg or ffprobe(video_path) or raise ValueError(...)
```

  ```

#### **Sport Profile Abstraction:**

The `SEGMENT_TABLES` mapping (db.py:16-22) allows generic iteration over any sport:
```python
SEGMENT_TABLES = {
    SportType.BADMINTON: ("badminton_rallies", "rally_number"),
    SportType.TENNIS: ("tennis_points", "point_number"),
    # ...
}
```

Usage (db.py:527-546):

```
table, order_by = SEGMENT_TABLES[sport]
cursor.execute(f"SELECT start_time, end_time FROM {table} WHERE video_id = ? ORDER BY
{order_by}", ...)
```

---

7. TESTS

Location: `C:\Users\avidu\Projects\sports-data-collector\tests\`

Test File	Coverage	Key Tests
`test_cli.py`	CLI commands (import-local, export, fetch, curate)	All sports (badminton, tennis, cricket, pickleball, tabletennis); JSON/CSV parsing
`test_cvat_parser.py`	CVAT adapter (frame?time, issue detection)	Time recomputation, overlap/zero-duration blockers, vocabulary checks, output serialization
`test_db.py`	Database insert/query	UPSERT logic, cascade delete, per-sport queries
`test_validator.py`	DatasetValidator	Overlap detection, duration validation per sport
`test_local_import.py`	Local ingestion pipeline	JSON/CSV ? BadmintonRally ? DB
`test_parser.py`	Generic parser utilities	?
`test_parsing.py`	`safe_int()`, `safe_float_required()`, `parse_time_string()`	Robust casting from mixed types
`test_schemas.py`	Pydantic models	BadmintonRally validation (start < end, non-negative)
`test_downloader.py`	Video download (yt-dlp wrapper)	?
`test_config.py`	Configuration loader	?

---

8. DESIGN RECOMMENDATIONS FOR NEW "RICH OBSERVABILITY REPORT" FEATURE

Based on the architecture, here's where/how to integrate a per-video observability report:

Ideal Placement:

1. **Hook into `export_eval_set()` pipeline** (curate.py:428-524)
   - Called after all videos have been collected and CSVs written
   - Can iterate all exported videos, aggregate metrics, emit final report
   - Write to `<out_dir>/observability_report.{json,md}`

2. **Alternative: Standalone command `export-with-observability`**
   - Wrap `export_eval_set()` and add observability layer
   - Or create a separate `--emit-observability` flag to `export` command

What to Observe (candidate metrics):

- **Per-video level:**
  - Segment count, total annotated duration, density (segments/hour)
  - FPS distribution, resolution distribution
  - License/commercial breakdown
  - Status distribution (staging vs. curated)
```

```
- **Per-sport level:**
  - Total videos, total segments, coverage stats
  - Segment duration stats (min, max, mean, median)

- **Cross-cutting:**
  - Data quality scoreboard (validation pass rate from `validate-delivery`)
  - Temporal coverage (do time ranges overlap? gaps?)
  - Download success/failure rates (from `fetch` command)

#### **Output Format (consistent with existing reports):**

**`observability_report.json`** (machine-readable for dashboards)
```json
{
  "timestamp": "2026-06-07T...",
  "total_videos": 50,
  "total_segments": 1200,
  "by_sport": {
    "badminton": {
      "count": 30,
      "segments": 800,
      "avg_duration_s": 4.5,
      "fps_distribution": {"60": 25, "30": 5}
    }
  },
  "quality": {
    "validation_passed": 48,
    "validation_blocked": 2
  }
}
```

`observability_report.md` (human-readable summary)
```markdown
# Data Export Observability Report

## Summary
- Total videos exported: 50
- Total segments: 1200
- Coverage: 5400 seconds of annotation

## By Sport
| Sport | Videos | Segments | Avg Duration |
...

## Quality Metrics
- Validation pass rate: 96% (48/50)
...
```

Integration with Existing Patterns:

- **Report generation:** Mirror `build_summary()` + `render_validation_md()` from rally_cvat.py
- **File writing:** Use same patterns as validate-delivery (lines 606-610 in curate.py)
- **Data aggregation:** Pull from manifest.json entries already built (lines 484-497)
- **Test structure:** Add `test_observability_report.py` alongside `test_cvat_parser.py`

Key Files to Modify:

1. **`src/cli/curate.py`**
   - Add `_emit_observability_report(manifest, out_dir)` method
   - Call from `export_eval_set()` after manifest is finalized (post-line 509)
```

2. **`src/core/observability.py`** (NEW)
   - `build_observability_summary(manifest: List[Dict]) -> Dict` (analogous to `build_summary()`)
   - `render_observability_md(summary: Dict) -> str` (analogous to `render_validation_md()`)
   - `write_observability_report(summary: Dict, out_dir: str)` (analogous to validate-delivery output logic)

3. **`tests/test_observability.py`** (NEW)
   - Test aggregation logic, format consistency
   - Test with various manifest shapes (empty, single sport, multi-sport, mixed statuses)

SUMMARY: WHERE THE PIPELINE WRITES OUTPUT

Output	Command	Location	Producer Function	Format	Lines
Per-video eval CSVs	`export`	`<out_dir>/<sport>/<video_id>.csv`	`export_eval_set()`	start,end (decimal seconds)	curate.py:478-482
Manifest	`export`	`<out_dir>/manifest.json`	`export_eval_set()`	JSON: video metadata + segment count	curate.py:500-509
Validation MD report	`validate-delivery`	`<out_dir>/<id>_report.md`	`render_validation_md()`	Human-readable scoreboard + issues	rally_cvat.py:479-509
Validation JSON summary	`validate-delivery`	`<out_dir>/<id>_report.json`	`build_summary()`	Machine-readable: severity counts, blockers, flagged rallies	rally_cvat.py:459-476
Review sheet	`validate-delivery`	`<out_dir>/<id>_review.csv`	`write_review_csv()`	Annotatable CSV with auto-flags + blank true_* columns	rally_cvat.py:512-534
CVAT issues report	`convert-cvat`	`--issues` file (optional)	`render_issue_report()`	Text: grouped by severity	rally_cvat.py:375-396
Canonical rally CSV	`convert-cvat`	`--out` file	`write_canonical_csv()`	CSV: rally_number, start/end times, metadata	rally_cvat.py:356-372
Database	All ingestion	`data/sports_metadata.db`	`SportsDatabase.insert_*()`	SQLite: videos + sport tables	db.py:168-349

This architecture is clean, modular, and sports-agnostic. To add a new observability report, you'd follow the exact pattern used by `validate-delivery` (build summary ? render markdown/JSON ? write files), pulling data from the already-computed manifest rather than re-querying the DB.

19. assistant (2026-06-07T06:44:34.211Z)

Excellent ? I now have a full picture of both repos. Before I draft the plan, let me ground the central integration points by reading the exact orchestration code I'd hook into (so the plan isn't hand-wavy). Reading the indexer's orchestrator + validation result shape, and the collector's report-builder pattern I'd mirror.

20. user (2026-06-07T06:44:35.168Z)

```
1       import os
2       import sys
3       import argparse
4       import hashlib
5       import json
6       import uvicorn
7       from dotenv import load_dotenv
8
9       load_dotenv()
10      from fastapi import FastAPI, HTTPException
11      from fastapi.middleware.cors import CORSMiddleware
12      from pydantic import BaseModel
13      from typing import List, Dict, Any, Optional
14
```

```
    from fastapi.staticfiles import StaticFiles
    from fastapi.responses import HTMLResponse
    from backend.validators.base import registry
    from backend.database import Database
    from backend.indexer.base import segmenter_registry, BasePlaySegmenter
    from backend.stitcher.compiler import VideoCompiler

    app = FastAPI(title="Sports Highlight Indexer API")

    # Enable CORS for local web interface
    app.add_middleware(
        CORSMiddleware,
        allow_origins=["*"],
        allow_credentials=True,
        allow_methods=["*"],
        allow_headers=["*"],
    )

    # Serves static frontend files directly
    os.makedirs("backend/static", exist_ok=True)
    app.mount("/static", StaticFiles(directory="backend/static"), name="static")

    @app.get("/", response_class=HTMLResponse)
    def serve_index():
        index_path = "backend/static/index.html"
        if os.path.exists(index_path):
            with open(index_path, "r", encoding="utf-8") as f:
                return f.read()
        return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/static/</h3>"

    # Global variables initialized at startup
    db_instance: Optional[Database] = None
    global_config: Optional[Dict[str, Any]] = None

    def get_file_md5(filepath: str) -> str:
        """Calculates file hash to uniquely identify videos and prevent duplicate
indexes."""
        hasher = hashlib.md5()
        with open(filepath, "rb") as f:
            # Read in chunk sizes of 64kb
            for chunk in iter(lambda: f.read(65536), b""):
                hasher.update(chunk)
        return hasher.hexdigest()

    def load_config(config_path: str = "config.json") -> Dict[str, Any]:
        if os.path.exists(config_path):
            with open(config_path, "r") as f:
                return json.load(f)
        return {}

    # --- API Models ---
    class ProcessRequest(BaseModel):
        video_path: str
        player_pool: Optional[List[str]] = None
        max_frames: Optional[int] = None
        segmenter: Optional[str] = None

    class QueryRequest(BaseModel):
        video_id: str
        server: Optional[str] = None
        receiver: Optional[str] = None
        duration_min: Optional[float] = None
        event_type: Optional[str] = None

```

```
78      class CompileRequest(BaseModel):
79          video_id: str
80          segment_ids: List[int]
81          output_filename: str
82
83      # --- API Endpoints ---
84      @app.get("/api/config")
85      def get_config():
86          return global_config
87
88      @app.post("/api/process")
89      def process_video(req: ProcessRequest):
90          if not os.path.exists(req.video_path):
91              raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
92
93          video_id = get_file_md5(req.video_path)
94
95          # 1. Run pluggable validators
96          print(f"[API] Running validations for {req.video_path}...")
97          val_results = registry.run_all(req.video_path, global_config)
98
99          # Check if any validator failed
100         failures = [r for r in val_results if not r.passed]
101         if failures:
102             # Save failed processing log
103             db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
104             return {
105                 "video_id": video_id,
106                 "status": "failed",
107                 "message": "Video failed validation checks.",
108                 "results": [r.dict() for r in val_results]
109             }
110
111         # 2. Run segmenter & indexer
112         print(f"[API] Video passed checks. Segmenting and indexing...")
113         db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
114
115         seg_name = req.segmenter or global_config.get("indexing",
{}).get("default_segmenter", "motion")
116         segmenter_cls = segmenter_registry.get(seg_name)
117         if not segmenter_cls:
118             available = list(segmenter_registry.list_available().keys())
119             raise HTTPException(
120                 status_code=400,
121                 detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
122             )
123
124         print(f"[API] Using segmenter: {seg_name}")
125         segmenter = segmenter_cls(db_instance, global_config)
126         segments = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
127
128         return {
129             "video_id": video_id,
130             "status": "success",
131             "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
132             "results": [r.dict() for r in val_results],
133             "play_segments": segments
134         }
135
```

```
    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]

        # Run compiler
        output_dir = global_config.get("output", {}).get("output_dir", "output")
        compiler = VideoCompiler(output_dir)
        try:
            out_path = compiler.compile_highlights(video["filepath"], time_pairs,
req.output_filename)
            return {"status": "success", "filepath": out_path}
        except Exception as e:
            raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}")

    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
        for r in results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"  {status_symbol} {r.validator_name}: {r.message}")
            if r.details:
                print(f"         Details: {r.details}")

        # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
        print("-" * 60)
        print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            print("[FAIL] OpenCV could not open video.")
            return
```

```
198
199         fps = cap.get(cv2.CAP_PROP_FPS)
200         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
201
202         start_frame = max(0, int((timestamp - 3.0) * fps))
203         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
204
205         # Measure frame-by-frame changes in sample region
206         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
207         prev_gray = None
208         motions = []
209
210         for f in range(start_frame, end_frame):
211             ret, frame = cap.read()
212             if not ret:
213                 break
214             gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
215             gray = cv2.GaussianBlur(gray, (21, 21), 0)
216
217             if prev_gray is not None:
218                 diff = cv2.absdiff(prev_gray, gray)
219                 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
220                 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
221                 motions.append(motion_ratio)
222             prev_gray = gray
223
224         cap.release()
225
226         if motions:
227             avg_motion = sum(motions) / len(motions)
228             threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)
229             print(f"  Sample Frame Count: {len(motions)}")
230             print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
231             if avg_motion > threshold:
232                 print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
233             else:
234                 print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
235         else:
236             print("  [ERROR] No visual frames were extracted in the sample range.")
237
238         print("=" * 60)
239
240     def main():
241         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
242         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
243         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
244         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
245         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
246         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
247         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
248         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
249
250         available_segmenters = list(segmenter_registry.list_available().keys())
251         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
```

```
detection segmenter. Available: {available_segmenters}")
252         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
253         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
254         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
255
256         args = parser.parse_args()
257
258         if args.setup:
259             # Invoke setup diagnostics
260             import subprocess
261             print("[INFO] Invoking setup.py...")
262             subprocess.run([sys.executable, "setup.py"])
263             return
264
265         if args.trim_start is not None and args.trim_end is not None:
266             if not args.video_path:
267                 print("[ERROR] Please provide --video-path to extract a segment.")
268                 return
269
270             # Determine output path
271             out_filename = args.trim_output or "trimmed_segment.mp4"
272             if os.path.isabs(out_filename):
273                 out_path = out_filename
274                 out_dir = os.path.dirname(out_path)
275                 out_name = os.path.basename(out_path)
276             else:
277                 out_dir = args.output_dir
278                 out_path = os.path.join(out_dir, out_filename)
279                 out_name = out_filename
280
281             os.makedirs(out_dir, exist_ok=True)
282             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
283
284             compiler = VideoCompiler(out_dir)
285             try:
286                 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
287                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
288             except Exception as e:
289                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
290             return
291
292         if args.debug_clip is not None:
293             if not args.video_path:
294                 print("[ERROR] Please provide --video-path to debug a specific clip.")
295                 return
296             run_cli_diagnose_clip(args.video_path, args.debug_clip)
297             return
298
299         # Initialize Global Config and DB
300         global global_config, db_instance
301         global_config = load_config()
302         # Merge output_dir into config
303         if "output" not in global_config:
304             global_config["output"] = {}
305         global_config["output"]["output_dir"] = args.output_dir
306         os.makedirs(args.output_dir, exist_ok=True)
307
308         db_path = os.path.join(args.output_dir, global_config.get("output",
{}).get("db_name", "sports_indexer.db"))
309         db_instance = Database(db_path)
```

```
310
311         # CLI processing mode (no web UI needed)
312         if args.video_path and not args.server:
313             print(f"[CLI] Processing video file: {args.video_path}")
314             if not os.path.exists(args.video_path):
315                 print(f"[FAIL] Video file not found: {args.video_path}")
316                 return
317
318             video_id = get_file_md5(args.video_path)
319
320             # Validate
321             print("[CLI] Validating...")
322             val_results = registry.run_all(args.video_path, global_config)
323             failures = [r for r in val_results if not r.passed]
324
325             # Save video status
326             status = "failed" if failures else "processed"
327             db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
val_results])
328
329             print("\n" + "="*40)
330             print("VALIDATION SUMMARY")
331             print("="*40)
332             for r in val_results:
333                 status_symbol = "[PASS]" if r.passed else "[FAIL]"
334                 print(f"{status_symbol} {r.validator_name}: {r.message}")
335             print("="*40 + "\n")
336
337             if failures:
338                 print("[FAIL] Video failed checks. Indexing halted.")
339                 return
340
341             # Index
342             seg_name = args.segmenter or global_config.get("indexing",
{}).get("default_segmenter", "motion")
343             segmenter_cls = segmenter_registry.get(seg_name)
344             if not segmenter_cls:
345                 print(f"[FAIL] Segmenter '{seg_name}' not found.")
346                 return
347
348             print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
349             segmenter = segmenter_cls(db_instance, global_config)
350             segments = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
351             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments inside
database: {db_path}")
352             return
353
354         # Start Local Server Mode
355         if args.server or not args.video_path:
356             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
357             uvicorn.run(app, host="127.0.0.1", port=args.port)
358
359     if __name__ == "__main__":
360         main()
361
```

21. user (2026-06-07T06:44:35.437Z)

```
1       from abc import ABC, abstractmethod
2       from pydantic import BaseModel
3       from typing import List, Dict, Any, Optional
4
5       class ValidationResult(BaseModel):
```

```
6           validator_name: str
7           passed: bool
8           message: str
9           details: Dict[str, Any] = {}
10
11      class VideoValidator(ABC):
12          @property
13          @abstractmethod
14          def name(self) -> str:
15              """Unique identifier for this validator."""
16              pass
17
18          @property
19          @abstractmethod
20          def description(self) -> str:
21              """Human-readable explanation of what this validator checks."""
22              pass
23
24          @abstractmethod
25          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
26              """
27              Executes validation.
28              - video_path: Local path to the MP4 file.
29              - config: Dict parsed from config.json.
30              - context: Dynamic context dict shared across validators (stores shared values
like ffprobe metadata).
31              """
32              pass
33
34      class ValidationRegistry:
35          def __init__(self):
36              self._validators: List[VideoValidator] = []
37
38          def register(self, validator: VideoValidator):
39              """Registers a new validator."""
40              self._validators.append(validator)
41
42          def run_all(self, video_path: str, config: Dict[str, Any]) ->
List[ValidationResult]:
43              """Runs all registered validators in sequence, accumulating results."""
44              results = []
45              context = {} # Shared context dictionary (e.g. for caching ffprobe outputs)
46              for val in self._validators:
47                  try:
48                      res = val.validate(video_path, config, context)
49                      results.append(res)
50                  except Exception as e:
51                      results.append(ValidationResult(
52                          validator_name=val.name,
53                          passed=False,
54                          message=f"Validator threw an unexpected error: {str(e)}"
55                      ))
56              return results
57
58      # Shared global registry
59      registry = ValidationRegistry()
60
```

22. assistant (2026-06-07T06:45:04.389Z)

I now have precise grounding. One important design reality I can see in the code: `registry.run_all()` builds the rich `context` dict (with `ffprobe_meta`) internally and **discards it**, and `segmenter.process_video()` returns only the final segment list ? the juicy per-phase internals (candidate counts, Gemini upload/process timing, frames extracted,

trajectory visibility %, WSL healthcheck) are either persisted to the DB or only `print()`-ed. So the observability feature has a real plumbing question (harvest post-hoc from DB+artifacts vs. thread a lightweight recorder through the run). That shapes the plan.

Let me load plan-mode tools so I can present this properly and confirm a couple of decisions with you.

23. user (2026-06-07T06:45:08.822Z)

Entered plan mode. You should now focus on exploring the codebase and designing an implementation approach.

In plan mode, you should:
1. Thoroughly explore the codebase to understand existing patterns
2. Identify similar features and architectural approaches
3. Consider multiple approaches and their trade-offs
4. Use AskUserQuestion if you need to clarify the approach
5. Design a concrete implementation strategy
6. When ready, use ExitPlanMode to present your plan for approval

Remember: DO NOT write or edit any files yet. This is a read-only exploration and planning phase.

24. assistant (2026-06-07T06:45:30.492Z)

I've completed a thorough exploration of both repos (pipelines, data shapes, existing report patterns, and the exact orchestration hooks). Before I finalize the plan, three decisions genuinely change the design ? let me confirm your intent rather than assume.

25. user (2026-06-07T06:48:57.716Z)

Your questions have been answered: "How should the rich observability report be emitted on each run?"="Always-on and we should define a convention i.e. its written always next to the video path's location or could be configured independently.", "The 'customer-friendly' report (strip system internals, show video metadata + meta-highlights) ? which tool(s) should produce it?"="Both tools", "Both repos are separate (one private). How unified should the report schema be across the two tools?"="1. with the additional feedback that maybe make that an independent github private repo which provides a "config language" to produce reports which can be reusable across these two today and other tools later on. Which also adds that, you may do some research to find if something "commercially safe" can be imported/installed which will reduce some of the "reinvention" work.". You can now continue with these answers in mind.

26. assistant (2026-06-07T06:49:40.953Z)

That third answer meaningfully expands the scope ? a **separate private repo providing a reusable "report config language"** consumed by both tools (and future ones), plus a research pass for commercially-safe building blocks to avoid reinvention. That's the right instinct for not building a house of cards, but it deserves grounding. Let me research the landscape before I commit the plan to a foundation.

27. user (2026-06-07T06:49:50.865Z)

Web search results for query: "soda-core SodaCL data quality YAML config language license Apache"

Links: [{"title":"GitHub - sodadata/soda-core: Data Contracts engine for the modern data stack. https://www.soda.io · GitHub","url":"https://github.com/sodadata/soda-core"},{"title":"Run Soda Core checks with Airflow | Astronomer Documentation","url":"https://www.astronomer.io/docs/learn/2.x/soda-data-quality"},{"title":"soda-core/README.md at main · sodadata/soda-core","url":"https://github.com/sodadata/soda-core/blob/main/README.md?plain=1"},{"title":"Soda Moved to ELv2. Provero Is Apache 2.0. - DEV Community","url":"https://dev.to/andreahlert/soda-moved-to-elv2-provero-is-apache-20-42l9"},{"title":"Using Soda Core to test data quality | by Maksim Kazartsev | Medium","url":"https://medium.com/@kazarmax/using-soda-core-to-check-data-

quality-07b370da2df3"},{"title":"Soda Core (v3) | Soda v3 | Documentation","url":"https://docs.soda.io/soda-documentation/soda-v3/overview-main"},{"title":"SodaCL tutorial | Soda Documentation","url":"https://docs.soda.io/soda/quick-start-sodacl.html"},{"title":"How To Use Soda For Data Quality Checks With Apache Airflow | by Ahmed Mokbel | Medium","url":"https://ahmed-mokbel.medium.com/how-to-use-soda-for-data-quality-checks-with-apache-airflow-cf249a737b5a"},{"title":"soda-core/docs/configuration.md at main · sodadata/soda-core","url":"https://github.com/sodadata/soda-core/blob/main/docs/configuration.md"},{"title":"Creating a Fabric integration for Soda and checking data quality","url":"https://dataroots.io/blog/creating-a-fabric-integration-for-soda-and-checking-data-quality"}]

Based on the search results, here's what I found about soda-core, SodaCL, and its license:

Soda Core Overview

Soda Core is a data quality and data contract verification engine that lets you define data quality contracts in YAML and automatically validate both schema and data across your data stack.

SodaCL Language

Soda Core uses the Soda Checks Language (SodaCL) to run checks defined in a YAML file. SodaCL is a YAML-based, domain-specific language for data reliability.

License Information

The project is licensed under Apache 2.0, though it's important to note a significant recent change: Soda Core was originally released under Apache License 2.0, but in 2023, Soda switched to the Elastic License v2 (ELv2), with the change applied to all new versions of Soda Core and its associated packages.

ELv2 is not an open source license by the OSI definition and adds two restrictions that Apache 2.0 does not have: you cannot offer the software as a managed service, and you cannot modify the license key functionality.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

28. user (2026-06-07T06:49:52.545Z)

Web search results for query: "Python declarative report generation library YAML/JSON spec to markdown HTML 2025 MIT license"

Links: [{"title":"GitHub - Nicceboy/python-markdown-generator: Python library for dynamically generating HTML sanitised Markdown syntax. · GitHub","url":"https://github.com/Nicceboy/python-markdown-generator"},{"title":"yaml-to-markdown - PyPI Package Security Analysis - Socket","url":"https://socket.dev/pypi/package/yaml-to-markdown"},{"title":"YAML to Markdown Converter","url":"https://pypi.org/project/yaml-to-markdown/"},{"title":"Developer Documentation | yaml-to-markdown","url":"https://anevis.github.io/yaml-to-markdown/docs/DEVELOPER.html"},{"title":"json-schema-for-humans · PyPI","url":"https://pypi.org/project/json-schema-for-humans/"},{"title":"yamldoc","url":"http://chrisbcole.me/yamldoc/"},{"title":"GitHub - yaml/pyyaml: Canonical source repository for PyYAML · GitHub","url":"https://github.com/yaml/pyyaml"},{"title":"markdown-full-yaml-metadata · PyPI","url":"https://pypi.org/project/markdown-full-yaml-metadata/"},{"title":"GitHub - lauritzh/md-report-generator: Python-based report generation toolchain based on Markdown input files. To start a fresh report, clone this repo and you are ready to go! ?","url":"https://github.com/lauritzh/md-report-generator"}]

Based on the search results, I found several relevant Python libraries for declarative report generation with YAML/JSON specifications:

Key Findings:

1. yaml-to-markdown
A library to convert YAML files to Markdown format, and a Python utility to take a JSON/YAML file or a python dict/list and create a Markdown file. This project is licensed under the MIT License.

2. yamldoc
This package converts a YAML file into markdown, formatting values and associated metadata in a doxygen-like way. It provides an easy interface for developers to document data in their YAML files as well as the expected types from a JSON YAML schema validator.

3. python-markdown-generator
A Python library for generating HTML sanitised Markdown documents that aims to bring a modular approach for dynamically building pure Markdown documents with pure Python so that they could consist of multiple reusable components.

4. json-schema-for-humans
Accepts JSON or YAML configuration files and can generate documentation in HTML and Markdown formats.

5. md-report-generator
A toolchain to generate pentest reports based on Markdown inputs, with YAML-based configuration support.

The **yaml-to-markdown** library appears to be the most directly aligned with your requirements, as it specifically handles YAML/JSON to Markdown/HTML conversion with MIT licensing.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

29. user (2026-06-07T06:49:56.839Z)

Web search results for query: "evidently ydata-profiling great-expectations frictionless license comparison data observability report"

Links: [{"title":"GitHub - evidentlyai/evidently: Evidently is ??an open-source ML and LLM observability framework. Evaluate, test, and monitor any AI-powered system or data pipeline. From tabular data to Gen AI. 100+ metrics.","url":"https://github.com/evidentlyai/evidently"},{"title":"How to Use Ydata-Profiling with Great Expectations V3 API | by Bvolodarskiy | Provectus Articles | Medium","url":"https://medium.com/provectus-articles/how-to-use-ydata-profiling-with-great-expectations-v3-api-fc6c2d0035cd"},{"title":"Great Expectations - YData Profiling","url":"https://docs.profiling.ydata.ai/4.5/integrations/great_expectations/"},{"title":"Complete Guide to Effortless ML Monitoring with Evidently.ai","url":"https://www.analyticsvidhya.com/blog/2024/03/complete-guide-to-effortless-ml-monitoring-with-evidently-ai/"},{"title":"Data Quality & Observability: The Hidden Pillars of AI Success | by Amit Kharche | Medium","url":"https://medium.com/@amitkharche/data-quality-observability-the-hidden-pillars-of-ai-success-a4ee1545d2a6"},{"title":"Building Reliable Data Pipelines with Databricks and Great Expectations / Evidently AI | by André Castro | Medium","url":"https://medium.com/@andreluizfc/building-reliable-data-pipelines-with-databricks-and-great-expectations-evidently-ai-95f681c53b6b"},{"title":"GitHub - jorson-chen/evidently-LLM-observability: Evidently is ??an open-source ML and LLM observability framework. Evaluate, test, and monitor any AI-powered system or data pipeline. From tabular data to Gen AI. 100+ metrics.","url":"https://github.com/jorson-chen/evidently-LLM-observability"},{"title":"There are a lot of freaking data quality and data observability vendors | DataKitchen","url":"https://datakitchen.io/blog/there-are-a-lot-of-freaking-data-quality-and-data-observability-vendors/"},{"title":"The 2026 Open-Source Data Quality and Data Observability Landscape | DataKitchen","url":"https://datakitchen.io/blog/the-2026-open-source-data-quality-and-data-observability-landscape/"},{"title":"Data Observability Tool Comparison: Monte Carlo vs. great ...","url":"https://www.castordoc.com/tool-comparison/data-observability-tool-comparison-monte-carlo-vs-great-expectations"}]

I've found some relevant information about these tools, but I haven't found a comprehensive

license comparison report yet. Let me search more specifically for license information.

Links: [{"title":"Great Expectations - YData Profiling","url":"https://docs.profiling.ydata.ai/4
.5/integrations/great_expectations/"},{"title":"Great Expectations - YData-Synthetic","url":"htt
ps://docs.synthetic.ydata.ai/1.2/integrations/gx_integration/"},{"title":"GX Core: Open Source
Data Quality Platform | Great Expectations ? Great
Expectations","url":"https://greatexpectations.io/"},{"title":"How to validate your synthetic
data quality","url":"https://ydata.ai/resources/how-to-validate-the-quality-of-your-synthetic-
data"},{"title":"GitHub - Data-Centric-AI-Community/ydata-profiling: 1 Line of code data quality
profiling & exploratory data analysis for Pandas and Spark DataFrames. ·
GitHub","url":"https://github.com/ydataai/ydata-profiling"},{"title":"YData ·
GitHub","url":"https://github.com/ydataai"},{"title":"3 Open Source Data Quality Tools You Can't
Ignore (Compared) - Telmai","url":"https://www.telm.ai/blog/open-source-data-quality-
tools/"},{"title":"How to Use Ydata-Profiling with Great Expectations V3 API | by Bvolodarskiy |
Provectus Articles | Medium","url":"https://medium.com/provectus-articles/how-to-use-ydata-
profiling-with-great-expectations-v3-api-fc6c2d0035cd"},{"title":"ydata-profiling ·
PyPI","url":"https://pypi.org/project/ydata-profiling/"},{"title":"ydata-profiling download |
SourceForge.net","url":"https://sourceforge.net/projects/ydata-profiling.mirror/"}]

Links: [{"title":"Open Source Licenses: Which One Should You Pick? MIT, GPL, Apache, AGPL and
More (2026 Guide) - DEV Community","url":"https://dev.to/juanisidoro/open-source-licenses-which-
one-should-you-pick-mit-gpl-apache-agpl-and-more-2026-guide-p90"},{"title":"What are the most
common open-source licenses?","url":"https://milvus.io/ai-quick-reference/what-are-the-most-
common-opensource-licenses"},{"title":"Open Source Licensing Considerations for Artificial
...","url":"https://www.knobbe.com/wp-content/uploads/2025/04/Legaltech-News-Open-Source-
Licensing.pdf"},{"title":"The Open Source Legacy and AI?s Licensing
Challenge","url":"https://www.linuxfoundation.org/blog/the-open-source-legacy-and-ais-licensing-
challenge"},{"title":"Top Open Source licenses in 2025 ? Open Source
Initiative","url":"https://opensource.org/blog/top-open-source-licenses-
in-2025"},{"title":"GitHub - evidentlyai/evidently: Evidently is ??an open-source ML and LLM
observability framework. Evaluate, test, and monitor any AI-powered system or data pipeline.
From tabular data to Gen AI. 100+
metrics.","url":"https://github.com/evidentlyai/evidently"},{"title":"Open Source Licensing
Simplified: A Comparative Overview of Popular Licenses | Blog | Endor
Labs","url":"https://www.endorlabs.com/learn/open-source-licensing-simplified-a-comparative-
overview-of-popular-licenses"},{"title":"Understanding AI Licenses: Key Types and
Considerations","url":"https://viso.ai/deep-learning/ai-licenses/"},{"title":"Quick Guide to
Popular AI Licenses","url":"https://www.mend.io/blog/quick-guide-to-popular-ai-licenses/"}]

I found information about Great Expectations being Apache 2.0 licensed, but I need more specific
license information for the other tools. Let me search more specifically for Frictionless Data's
license and confirm Evidently's license.

Links: [{"title":"Read and Write Frictionless Data Packages","url":"https://cran.r-
project.org/web/packages/frictionless/frictionless.pdf"},{"title":"Frictionless Data |
Frictionless Data","url":"https://frictionlessdata.io/"},{"title":"GitHub -
frictionlessdata/frictionless-r: R package to read and write Frictionless Data
Packages","url":"https://github.com/frictionlessdata/frictionless-r"},{"title":"Read and Write
Frictionless Data Packages ?
frictionless","url":"https://docs.ropensci.org/frictionless/"},{"title":"frictionless-ci |
Frictionless Repository","url":"https://repository.frictionlessdata.io/"},{"title":"Frictionless
Data · GitHub","url":"https://github.com/frictionlessdata"},{"title":"IDB |
Frictionlessdata","url":"https://knowledge.iadb.org/en/open-knowledge/code-development/open-
source-solution/frictionlessdata"},{"title":"GitHub - frictionlessdata/software: The
Frictionless Data project provides a rich set of open source software for working with
data","url":"https://github.com/frictionlessdata/software"},{"title":"frictionless: Python
library for Data Packages","url":"https://zenodo.org/records/15085933"},{"title":"Resources","ur
l":"https://fellows.frictionlessdata.io/resources/"}]

Links: [{"title":"ydata-profiling · PyPI","url":"https://pypi.org/project/ydata-
profiling/"},{"title":"pandas-profiling · PyPI","url":"https://pypi.org/project/pandas-
profiling/"},{"title":"ydata-profiling 4.16.1 on PyPI - Libraries.io - security & maintenance
data for open source software","url":"https://libraries.io/pypi/ydata-
profiling"},{"title":"ydata-profiling-infoworks · PyPI","url":"https://pypi.org/project/ydata-
profiling-infoworks/"},{"title":"ydata-profiling-thorrester ·

PyPI","url":"https://pypi.org/project/ydata-profiling-thorrester/"},{"title":"streamlit-ydata-profiling · PyPI","url":"https://pypi.org/project/streamlit-ydata-profiling/"},{"title":"ydata-profiling - conda-forge | Anaconda.org","url":"https://anaconda.org/conda-forge/ydata-profiling"},{"title":"MIT license - ydataai/ydata-profiling","url":"https://github.com/ydataai/ydata-profiling/blob/develop/LICENSE"},{"title":"ydata-profiling/pyproject.toml at develop · Data-Centric-AI-Community/ydata-profiling","url":"https://github.com/ydataai/ydata-profiling/blob/develop/pyproject.toml"},{"title":"ydata-profiling Python Guide [2026] | PyPI Tutorial","url":"https://generalistprogrammer.com/tutorials/ydata-profiling-python-package-guide"}]

Based on my web search results, I can now provide you with a summary of the license information for these data observability and quality tools:

License Comparison Summary

Here's what I found about the licenses for these data quality and observability tools:

Great Expectations is licensed under an Apache 2.0 license.

ydata-profiling is licensed under the MIT License.

Frictionless Data (frictionless-py) is licensed under the MIT License.

Evidently is licensed under an Apache 2.0 license.

Summary Table:

Tool	License
Evidently	Apache 2.0
ydata-profiling	MIT
Great Expectations	Apache 2.0
Frictionless Data	MIT

All four of these tools are open-source with permissive licenses. Apache 2.0 licenses include explicit patent grants and protections, making them popular for projects involving corporate contributors, while MIT-licensed projects prioritize flexibility. Both license types allow commercial use and modification, though Apache 2.0 provides additional patent protections.

Note: I couldn't locate a specific comprehensive comparison report combining all four tools, but the individual license information has been confirmed from official sources.

REMINDER: You MUST include the sources above in your response to the user using markdown hyperlinks.

30. assistant (2026-06-07T06:50:53.171Z)

Research is decisive. Key findings (with licenses):

- **Soda Core / SodaCL** ? the closest thing to a "YAML config language that produces data-quality reports" ? **relicensed to Elastic License v2 (ELv2), which is NOT OSI-open-source** and forbids offering it as a managed service. That directly violates your permissive-only / commercial-hosted constraint (ADR-005). ? Disqualified.
- **Great Expectations** (Apache-2.0 ?) and **Evidently** (Apache-2.0 ?) are license-safe but heavy and built for **tabular/dataframe + ML-monitoring**, not per-video pipeline-stage reports. Wrong shape, big dependency trees, telemetry.
- **ydata-profiling** (MIT ?) and **Frictionless Data** (MIT ?) are clean but also dataframe/data-package oriented.
- Thin MIT renderers exist (`yaml-to-markdown`, `json-schema-for-humans`, `python-markdown-generator`) ? useful primitives, not frameworks.
- The real reusable primitives you already depend on are all permissive: **pydantic (MIT)**, **Jinja2 (BSD, already in the indexer)**, **PyYAML (MIT, already in the collector)**,

jsonschema (MIT).

So: no off-the-shelf framework fits cleanly *and* is license-safe. The right move is a **small in-house "report config language" library built on those permissive primitives** ? borrowing thin MIT renderers rather than reinventing them, and explicitly rejecting the heavy frameworks. Let me pressure-test that design + the two integrations with a Plan agent before I commit the final plan.

31. user (2026-06-07T06:57:07.870Z)

Write isn't available in this read-only planning context. I'll deliver the plan directly as my response, which is what the parent agent reads.

I explored both repos and verified every load-bearing fact in your brief against the actual code. The plan below is hardened, file-level concrete, and phased.

Rich Per-Video Observability Report ? Implementation Plan

0. Facts I verified in code (not assumed)

- **Indexer already has the deps**: `requirements.txt` has `pydantic>=2.7.1` and `jinja2>=3.1.4`. Collector `pyproject.toml` has `pydantic>=2.0.0`, `pyyaml>=6.0`, `tabulate` ? no Jinja2 (the shared lib brings it transitively, BSD, fine).
- **`run_all` discards context**: `backend/validators/base.py:42-56` builds a local `context={}`, FormatValidator fills `context['ffprobe_meta']`, and only `List[ValidationResult]` is returned. Confirmed: to get probe metadata you must surface context or re-probe.
- **AI timing is dropped at the segmenter**: `backend/indexer/wasb_hybrid.py:167` is literally `segments, _ = provider.analyze_video(...)` ? the `{upload_time,process_time,gen_time}` dict is thrown away. Same in the twins.
- **Candidates ARE persisted**: `wasb_hybrid.py:144` calls `db.add_candidate_segment(...)`; harvestable via `db.query_candidate_segments(detector=)`. WASB manifest at `output/wasb/manifest.json` (cwd-relative).
- **Both entrypoints confirmed**: CLI `backend/main.py:312-352`, API `process_video @ main.py:88-134` (note the API has TWO return points: validation-fail at :104-109 and success at :128-134).
- **Collector report module to reuse**: `src/parsers/cvat/rally_cvat.py` ? `build_summary()` (JSON dict), `render_validation_md()`, `write_review_csv()`, `detect_issues()` with `Issue{severity in blocker/high/medium/low, code, message, rally_number}`, `ConversionResult{records,issues,fps,step,meta}`. `validate_delivery` (`curate.py:562-617`) already writes `<id>_report.json/.md` + `_review.csv`.
- **Collector Video model** `src/core/schemas/base.py:29-40`: `video_id, video_url, local_path, license_type, is_commercial, status, match_name, fps, resolution`. `probe_resolution_fps` + `DownloadResult{success,size_mb,width,height,fps,resolution_label,error}` in `src/core/downloader.py`.
- **License posture (ADR-005, `docs/DECISIONS.md:232`)**: hard permissive-only; pydantic/Jinja2/PyYAML/jsonschema all clean.
- **FAQ Q1-Q9** (`README.md:166+`) maps directly to footgun flags: Q5?PTS/keyframe & fps, Q7?AI provider/API key, Q2?relevance, Q8?yolo economics. These are the `faq_ref` citation targets.

---

1. Library-vs-framework decision ? VALIDATED

Build the small in-house library. It is the right call, and not only for licensing:

1. **Shape mismatch is decisive.** GE/Evidently/ydata/Frictionless model *tabular datasets / data packages*. Your domain object is a **per-video run = an ordered list of heterogeneous pipeline stages**, each with a free-form metrics blob and a verdict, plus a **declarative footgun?flag rule layer**. None of those tools expresses that naturally; forcing it in becomes its own house of cards.
2. **License is an independent hard gate** (ADR-005) that already kills Soda (ELv2).

3. **House-of-cards containment**: ~6-8 tiny modules, a frozen pydantic envelope, pure renderers, no IO at import. Far less fragile than wrapping a heavy framework whose API churn you don't control.

Refinements to your stated stack:
- **Drop `jsonschema` from the runtime core (defer to Phase 2+).** pydantic v2 is your single source of truth and emits JSON Schema via `model_json_schema()`. Only add `jsonschema` if/when you validate *externally authored* spec files against a published schema.
- **Do NOT take `json-schema-for-humans`/`yaml-to-markdown` as runtime deps.** Hand-write the Jinja2 templates ? the FAQ-debuggability eval *requires* you to control exact wording. Treat those MIT tools as optional dev helpers for generating the schema doc only.
- **Keep the "config language" sub-Turing.** No `eval`, no DSL parser. A restricted list of typed comparison clauses (§2.4) covers every footgun you listed.

---

2. The shared library

2.1 Repo / package name

Repo **`sports-obsreport`**, package **`obsreport`**, private GitHub, MIT. (Use generic `obsreport` package name so non-sports reuse later is clean.)

```
src/obsreport/
  envelope.py   spec.py   rules.py   recorder.py   io.py   version.py
  render/{json.py, markdown_technical.py, markdown_customer.py, templates/*.j2}
specs/{indexer.yaml, collector.yaml}
tests/
```

2.2 Typed envelope (`envelope.py`, pydantic v2) ? single source of truth

Both Markdown renderers consume the SAME envelope, so the two report types can never disagree on facts (serves goal a: catch inconsistencies).

- `Identity{schema_version, tool, tool_version, video_id, video_id_kind ("file_md5"|"human_id"), video_path, video_url, generated_at, run_id}`
- `Metric{key, value, unit, audience: technical|customer|both}`
- `Stage{name, status: ok|warn|error|skipped|not_run, started_at, duration_s, metrics[], details: dict, messages[]}`
- `Flag{code, severity: error|warn|info, message, stage, faq_ref, evidence}` ? `faq_ref` REQUIRED for error/warn (enforced by test).
- `VideoMeta{duration_s, fps, fps_source: probed|fallback|declared|unknown, resolution, width, height, container, size_mb}`
- `HighlightSummary{segment_count, total_highlight_s, coverage_pct, notes[]}` ? customer-safe, meta-level only.
- `ReportEnvelope{identity, video_meta, stages[], flags[], highlights, verdict: pass|pass_with_warnings|fail, extras}`

Audience views are derived, not stored twice. Customer renderer emits only `video_meta + highlights + customer-safe flags + verdict`; technical renderer emits everything. One source of truth = the two reports physically cannot diverge.

2.3 Declarative config language (`spec.py`, YAML, loaded into pydantic)

A spec = `sections[]` (layout + audience + which fields/stage) and `rules[]`. Pure data. Example footgun rule:

```yaml
rules:
  - code: silent_provider_noop
    severity: error
    when:
      - {path: "stages.ai_handoff.details.api_key_present", op: eq, value: false}
      - {path: "stages.segmentation.metrics.segment_count", op: eq, value: 0}
    message: "0 segments AND no API key ? likely a silent provider no-op, not 'no rallies'."
    faq_ref: "README#Q7"
```

```

### 2.4 Rule engine (`rules.py`) ? restricted & safe
- Clause `{path, op, value}`; `path` = dotted accessor into serialized envelope (missing path ? False unless `op=is_missing`).
- Ops only: `eq, ne, gt, ge, lt, le, in, contains, is_missing, is_present, eq_path, ne_path`. Each is a tiny pure function. NO `eval`, NO expression grammar.
- `when` = AND of clauses (add `when_any` later only if OR ever needed).
- Verdict: any `error`?`fail`; any `warn`(no error)?`pass_with_warnings`; else `pass`.

### 2.5 RunRecorder (`recorder.py`) ? two modes
- **Mode A (in-flight)**: `rec.stage("segmentation")` context manager auto-times, auto-sets status, records exceptions WITHOUT re-raising by default (so reporting never breaks the run). `rec.finalize()` runs the rule engine; `rec.write(...)` emits all 3 files.
- **Mode B (post-hoc harvest)**: pure functions reconstruct the envelope from persisted artifacts (DB candidates/play_segments, WASB manifest, compiler skips). Guarantees a report is ALWAYS produceable and enables backfilling old runs for cross-run diff (goal b/c). The indexer uses A live and B as the gap-filler inside the same write.

### 2.6 Renderers
- `json.py`: `model_dump_json(indent=2)`, deterministic ordering + sorted lists ? clean cross-run diffs.
- `markdown_technical.py` / `markdown_customer.py`: hand-written Jinja2. HTML customer twin deferrable to a later PR with a second template (no new dep).

### 2.7 `io.py` ? convention + safe write
- Next-to-video default: `/x/match.mp4` ? `match.report.json`, `match.report.md` (technical), `match.summary.md` (customer).
- Override via `report.output_dir` config key.
- Remote/no-local-file (collector): fall back to `report.output_dir` or `reports/<video_id>/`. Never crash on missing local path.
- Atomic temp+rename; ALL writes wrapped to return a result object, never raise (see §7).

### 2.8 Install across two private repos (git+ssh)
- Indexer `requirements.txt`: `obsreport @ git+ssh://git@github.com/<org>/sports-obsreport.git@v0.1.0`
- Collector `pyproject.toml` dep: same `@v0.1.0`.
- **Pin to a tag, never a branch** ? this is the primary cross-repo house-of-cards guard. Bump deliberately. Lib has no network/IO at import, so it adds nothing to mock in either repo's offline pytest.

---

## 3. Indexer integration ? exact hooks

**3.1 Surface ffprobe_meta (backward-compatible).** Add `run_all_with_context(...) -> (list[ValidationResult], dict)` to `backend/validators/base.py`; make `run_all` call it and return `[0]`. Existing callers and tests stay green. The two `main.py` callers switch to `run_all_with_context` to grab `context['ffprobe_meta']` ? no re-probe, no contract break. Fallback: if missing, harvest re-probes (same cv2 fallback the segmenters use) and raises `fps_fallback_used`.

**3.2 RunRecorder placement.**
- CLI (`main.py:312-352`): create `rec` after `video_id = get_file_md5(...)`; wrap validation/add_video/segmentation in `rec.stage(...)`; `rec.write(...)` in a `finally` so even a failed/exception run emits a partial report with the error flag.
- API (`main.py:88-134`): instantiate after `video_id`; route BOTH exits (fail :104-109, success :128-134) through one finalize+write; add report paths to the JSON response (`"reports": {...}`).

**3.3 Harvest sources (Mode B):** probe meta from context; validation from the `List[ValidationResult]` in hand; segmentation from `segments` (+ `segmenter_requested` vs `segmenter_actual`); candidates from `db.query_candidate_segments`; WASB `{frames_total,windows_total,windows_done}` from `output/wasb/manifest.json` (resolve against
```

configured output dir, not raw cwd); compiler skips deferred (compile is a separate endpoint ? record `not_run`). AI timing: start by recording `not_captured` (cheap); add a recorder callback through `analyze_video`?`process_video` in a LATER isolated PR (touches the twins, so defer).

3.4 Indexer flags: `silent_provider_noop`(error, Q7), `ai_empty_vs_no_rallies`(warn, Q7), `fps_fallback_used`(warn, Q5), `synthetic_motion_metadata`(warn, CODE_MAP#motion), `segmenter_divergence`(info ? the three-defaults footgun), `reingest_cascade`(warn ? INSERT OR REPLACE deleted prior segments), `validation_failed`(error, Q2/Q5), `wsl_healthcheck_failed`(error, Q7).

---

4. Collector integration ? wrap, don't duplicate

Add a thin `src/reporting/envelope_adapter.py`; keep `build_summary/detect_issues/render_validation_md` as the authoritative content producers.

- **Issue?Flag**: `blocker?error`, `high/medium?warn`, `low?info`; carry `code` verbatim; map `code?faq_ref` via a small table; `evidence={"rally_number":...}`.
- **Stages**: `acquisition` (from `DownloadResult`), `parse` (format + n records), `time_recompute` (CVAT `fps/step/meta` ? flag guessed fps, the exact bug rally_cvat exists to undo), `validation` (existing `detect_issues`/sport validators), `registration` (DB insert), `export`.
- **Wire-in points** (add envelope write AFTER existing work, never replacing): `import_local` (`curate.py:52-142`, after insert :124), `convert_cvat` (`:526-560`), `validate_delivery` (`:562-617` ? lowest risk, already writes a report; reuse `build_summary` content, add customer summary, keep existing files for back-compat), `fetch` (`:330+`, acquisition report), `export` (`:428-524`, optional per-video delivery summary).
- **Customer "delivery summary"**: `VideoMeta` (fps/resolution/match_name/license) + `HighlightSummary` (n rallies, total rally seconds, coverage) + only error/warn flags with `code` stripped, message kept.

---

5. README/FAQ-reader debuggability ? concrete eval loop

1. **Broken-run fixtures** (offline, mocked): no-API-key, skipped-probe, motion-segmenter, re-ingest, collector overlap/zero-duration. Each emits a real report.
2. **"Fresh reader" subagent**: given ONLY README+FAQ+the technical report (no source), must name the root cause and the fix. Pass = correct cause + cites the FAQ section the flag's `faq_ref` points to.
3. **Iterate** flag `message`/`faq_ref` (and FAQ text if needed) until diagnosis succeeds. Enforce `faq_ref` required for error/warn via a test.
4. **Lock as golden regression**: fixture ? expected `{flag codes, faq_refs}`; pytest asserts exact set. Prevents wording/flag drift.
5. **Cross-tool/cross-run diff test**: same logical video through both tools / two runs ? JSON envelopes diff only where expected (goals b/c).

---

6. Phasing into small PRs

Riskiest: (i) cross-repo dep ? contained by tag-pin + pure lib; (ii) touching `run_all` ? contained by additive `run_all_with_context`; (iii) AI-timing through 3 twin segmenters ? deferred/isolated.

Library (land + tag first): C1 scaffold + envelope + JSON renderer + tests ? C2 spec + rules + verdict tests ? C3 MD renderers + io safe-write/path tests ? C4 RunRecorder + harvest scaffolding + tests ? **tag v0.1.0**.

Indexer: A1 add dep + `run_all_with_context` (prove `run_all` unchanged) ? A2 pure `harvest_indexer_run` (mocked DB) ? A3 wire CLI path (`finally`-guarded) + flags + broken-run tests ? A4 wire API (both branches) + report paths in response ? A5 (later) thread AI timing via callback, one segmenter then twins.

Collector: B1 add dep + `envelope_adapter` (Issue?Flag, stage builders) + tests ? B2 wire `validate_delivery` (lowest risk) ? B3 wire `import_local` + `convert_cvat` ? B4 wire `fetch` + remote-no-file handling + optional `export` summary.

Cross-cutting: D1 broken-run fixtures + fresh-reader eval + golden flag/faq assertions in CI for both repos (after A3/B2).

Order rationale: lib tagged before any consumer; every consumer PR is independently shippable and reversible; lowest-risk/highest-value wiring (`validate_delivery`, indexer CLI) lands before the trickier API/twin work.

---

7. Risks & edge cases

- **Report write must never break the run**: writes wrapped to return a result, called in `finally`; inject-IOError test asserts pipeline result unaffected.
- **Remote videos / no local file**: `io.py` resolves to `report.output_dir`/`reports/<video_id>/`; `video_meta.video_url` carried; dedicated test.
- **Schema versioning**: `SCHEMA_VERSION` in identity; renderers assert major compatibility; additive-only within a major; consumers pin a tag so a bump can't surprise them.
- **Idempotency**: default overwrite per latest run next to the video (matches `add_video` INSERT-OR-REPLACE semantics); optional `report.history_dir` archives `<video_id>/<run_id>.json` for diffing; compile (separate step) writes a sibling `*.compile.report.json` to avoid read-modify-write races.
- **Large reports**: cap list lengths in the technical renderer (first N + "?M more"); full data only in JSON; `report.max_detail_items` knob; customer report never includes raw lists.
- **Cross-repo coupling**: tag-pin + pure lib + a contract-test CI job in the lib repo that runs both consumers' relevant tests against `HEAD` before tagging.
- **Twin-segmenter drift**: prefer harvest-from-DB (A2) so you don't touch wasb/tracknet copies at all initially; if A5 lands, apply to both.
- **`video_id` semantics differ** (indexer file-MD5 vs collector human id): `identity.video_id_kind` records which so cross-tool diffs don't mis-join.
- **cwd-relative paths** (`output/wasb`): harvest resolves against the configured output dir, not raw cwd.

Critical Files for Implementation
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py (CLI :312-352 + API :88-134 recorder hooks)
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\validators\base.py (add `run_all_with_context` to surface ffprobe_meta)
- C:\Users\avidu\Projects\badminton-highlight-indexer\backend\indexer\wasb_hybrid.py (candidate / AI-timing / WASB-manifest harvest reference; twin = tracknet_hybrid.py)
- C:\Users\avidu\Projects\sports-data-collector\src\parsers\cvat\rally_cvat.py (reuse build_summary/detect_issues/Issue; wrap into envelope)
- C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py (import_local / convert_cvat / validate_delivery / fetch / export wire-in points)

Note: the plan file write was blocked (read-only planning context disables Write), so the full plan is delivered above for you to adapt into your final plan document.

32. assistant (2026-06-07T06:57:20.776Z)

The Plan agent verified every load-bearing fact against the actual code and hardened the design with several good refinements (drop `jsonschema` from the runtime core, hand-write the Jinja2 templates so I control exact wording for the FAQ-debuggability eval, keep the rule language sub-Turing/no-`eval`, tag-pin the cross-repo dependency, make `run_all` backward-compatible, and defer the risky AI-timing-through-twin-segmenters work). Let me write the final plan.

33. assistant (2026-06-07T06:58:27.860Z)

Plan ? Rich per-video observability reports (indexer + collector + shared lib)

Context (why)

Today neither tool emits a per-video record of *what happened during processing*. The
indexer's pipeline computes a lot of signal (ffprobe metadata, validator results, candidate
windows, AI-provider timing, WASB frame/window counts, stitch skips) but **throws most of it
away**: `registry.run_all()` builds a `context` dict with `ffprobe_meta` and discards it
(`backend/validators/base.py:42-56`); `segmenter.process_video()` returns only the final
segment list; AI timing is dropped at `wasb_hybrid.py:167` (`segments, _ =
provider.analyze_video(...)`);
much detail is only `print()`-ed. The collector has a good report pattern for *delivery
validation* (`rally_cvat.py` `build_summary`/`render_validation_md`/`write_review_csv`) but
nothing that captures the **per-video processing journey** (acquire ? parse ? recompute time ?
validate ? register ? export).

We want both tools to **always** emit a rich "side output" per processed video that (a) captures
every stage + details to **catch inconsistencies**, (b) is **diffable across runs and across the
two tools** to evaluate output differences, and (c) lets **someone who has only read the README
+
FAQ self-debug a bad run**. Plus a second **customer-friendly** report that strips system
internals
and shows only video metadata + meta-level highlights. Reports are written **next to the video
by
convention**, with a configurable override.

Per the user, the schema is unified via a **new separate private repo providing a reusable
"report
config language."** Research (below) confirms no off-the-shelf framework fits both the
licensing constraint and the per-video-pipeline-stage shape ? so we build a small library on
permissive primitives the repos already use.

Decisions locked (user + research)

- **Always-on emission.** Written next to the video path by default (`<video>.report.json`
etc.);
  overridable via a `report.output_dir` config key. (Not flag-gated.)
- **Both tools** produce **both** report types (technical + customer-friendly).
- **Shared schema via a new private library repo** that exposes a declarative "report config
  language," reusable by both tools today and others later.
- **Library, not a heavy framework.** Research findings:
  - Soda Core / SodaCL (the closest "YAML?data-quality-report" DSL) is **ELv2 ? not OSI
    open-source, forbids managed-service use ? violates ADR-005. Disqualified.**
  - Great Expectations (Apache-2.0) & Evidently (Apache-2.0) are license-safe but heavy and
    tabular/ML-monitoring shaped ? wrong fit for "an ordered list of heterogeneous pipeline
    stages + a footgun?flag rule layer." ydata-profiling (MIT) / Frictionless (MIT) are
    dataframe/data-package shaped ? also wrong fit.
  - Build on permissive primitives the repos **already depend on**: **pydantic (MIT)** (indexer
    `pydantic>=2.7.1`, collector `pydantic>=2.0.0`), **Jinja2 (BSD)** (indexer dep), **PyYAML
    (MIT)** (collector dep). `jsonschema` deferred ? pydantic v2 `model_json_schema()` is the
    contract source. Thin MIT renderers (`yaml-to-markdown`, `json-schema-for-humans`) are
    dev-only helpers at most; templates are **hand-written** so we control exact wording for
    the FAQ-debuggability eval.

Architecture

New repo: `sports-obsreport` (package `obsreport`), private, MIT ? name TBD by owner

```
src/obsreport/
  envelope.py      # pydantic v2 typed report envelope (single source of truth)
  spec.py          # YAML "report config language" loader -> pydantic
  rules.py         # restricted, sub-Turing footgun->flag engine (NO eval)
  recorder.py      # RunRecorder: in-flight capture + post-hoc harvest
  io.py            # next-to-video path convention + safe (never-raising) writes
  version.py       # SCHEMA_VERSION
  render/{json.py, markdown_technical.py, markdown_customer.py, templates/*.j2}
```

```
specs/{indexer.yaml, collector.yaml}
tests/
```

Envelope (`envelope.py`) ? both renderers consume the *same* object, so the technical and customer reports can never disagree on facts:
- `Identity{schema_version, tool, tool_version, video_id, video_id_kind("file_md5"|"human_id"), video_path, video_url, generated_at, run_id}`
- `VideoMeta{duration_s, fps, fps_source(probed|fallback|declared|unknown), resolution, width, height, container, size_mb}`
- `Stage{name, status(ok|warn|error|skipped|not_run), started_at, duration_s, metrics[], details, messages[]}`
- `Flag{code, severity(error|warn|info), message, stage, faq_ref, evidence}` ? `faq_ref` **required for error/warn** (enforced by a test).
- `HighlightSummary{segment_count, total_highlight_s, coverage_pct, notes[]}` ? customer-safe.
- `ReportEnvelope{identity, video_meta, stages[], flags[], highlights, verdict(pass|pass_with_warnings|fail), extras}`
- **Audience views are derived, not stored twice.** Customer renderer emits only `video_meta + highlights + customer-safe flags + verdict`.

Config language (`spec.py`, YAML) ? pure data: `sections[]` (layout + audience + which stage/fields) and `rules[]`. Example footgun rule:
```yaml
rules:
  - code: silent_provider_noop
    severity: error
    when:
      - {path: "stages.ai_handoff.details.api_key_present", op: eq, value: false}
      - {path: "stages.segmentation.metrics.segment_count", op: eq, value: 0}
    message: "0 segments AND no API key ? likely a silent provider no-op, not 'no rallies'."
    faq_ref: "README#Q7"
```

Rule engine (`rules.py`) ? clause `{path, op, value}`; `path` = dotted accessor into the serialized envelope. Ops only: `eq, ne, gt, ge, lt, le, in, contains, is_missing, is_present, eq_path, ne_path` (each a tiny pure fn; **no `eval`, no grammar**). `when` = AND of clauses. Verdict: any error?`fail`; any warn (no error)?`pass_with_warnings`; else `pass`.

RunRecorder (`recorder.py`) ? two modes:
- *In-flight*: `with rec.stage("segmentation"):` auto-times, sets status, captures exceptions **without re-raising** (reporting never breaks the run). `rec.finalize()` runs rules; `rec.write()` emits all files.
- *Post-hoc harvest*: pure functions rebuild the envelope from persisted artifacts (DB candidates/play_segments, WASB manifest, compiler skips) ? guarantees a report is always produceable and enables backfilling old runs for cross-run diffs.

io convention (`io.py`): `/x/match.mp4` ? `match.report.json` + `match.report.md` (technical) + `match.summary.md` (customer). Override via `report.output_dir`. Remote/no-local (collector) ? `report.output_dir` or `reports/<video_id>/`. Atomic temp+rename; all writes return
a result object, **never raise**.

Cross-repo install (git+ssh, tag-pinned ? the main coupling guard):
- indexer `requirements.txt`: `obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.0`
- collector `pyproject.toml`: same `@v0.1.0`. **Pin a tag, never a branch.** Lib has no IO/network
  at import ? nothing new to mock in either repo's offline pytest.

Indexer integration (`badminton-highlight-indexer`)
- **Surface ffprobe_meta backward-compatibly:** add `run_all_with_context(...) -> (results, context)`
  to `backend/validators/base.py`; keep `run_all` returning `[0]` (existing callers/tests stay green). The two `main.py` callers switch to it to grab `context['ffprobe_meta']` (no re-probe).

- **RunRecorder placement:** CLI `backend/main.py:312-352` ? create after `video_id`, wrap
  validate/add_video/segment in stages, `rec.write()` in a `finally` (partial report even on
  crash). API `process_video` `main.py:88-134` ? route **both** exits (validation-fail :104-109,
  success :128-134) through one finalize+write; add report paths to the JSON response.
- **Harvest (Mode B):** probe meta from context; validation from the `ValidationResult` list;
  segmentation from `segments` (+ `segmenter_requested` vs `segmenter_actual`); candidates from
  `db.query_candidate_segments`; WASB `{frames_total,windows_total,windows_done}` from
  `output/wasb/manifest.json` (resolve against configured output dir, not raw cwd); compiler
skips
  = `not_run` (compile is a separate endpoint). AI timing = `not_captured` initially; thread a
  recorder callback through `analyze_video`?`process_video` in a **later isolated PR** (touches
the
  twin segmenters, so deferred).
- **Footgun flags:** `silent_provider_noop`(error,Q7), `ai_empty_vs_no_rallies`(warn,Q7),
  `fps_fallback_used`(warn,Q5), `synthetic_motion_metadata`(warn), `segmenter_divergence`(info ?
  config/setup/main disagree on default), `reingest_cascade`(warn ? INSERT OR REPLACE deleted
  prior segments), `wsl_healthcheck_failed`(error,Q7), `validation_failed`(error,Q2/Q5).

Collector integration (`sports-data-collector`)

Add `src/reporting/envelope_adapter.py`; **reuse** `build_summary`/`detect_issues`/
`render_validation_md` as the authoritative content producers (no duplication).
- **Issue?Flag:** `blocker?error`, `high/medium?warn`, `low?info`; keep `code`; `code?faq_ref`
via
  a small table; `evidence={"rally_number":...}`.
- **Stages:** `acquisition` (from `DownloadResult` in `src/core/downloader.py`), `parse`
  (format + n records), `time_recompute` (CVAT `fps/step/meta`; flag guessed fps), `validation`
  (existing `detect_issues`/sport validators), `registration` (DB insert), `export`.
- **Wire-in points (append a report write after existing work):** `import_local`
  (`curate.py:52-142`, after insert), `convert_cvat` (`:526-560`), `validate_delivery`
  (`:562-617` ? lowest risk, already writes a report; keep existing files for back-compat),
  `fetch` (`:330+`), `export` (`:428-524`, optional per-video delivery summary).
- **Customer "delivery summary":** `VideoMeta` + `HighlightSummary` (n rallies, total rally
  seconds, coverage) + error/warn flag messages (code stripped).

README/FAQ-reader debuggability ? eval loop (the quality bar)

1. **Broken-run fixtures** (offline, mocked): no-API-key, skipped-probe, motion-segmenter,
   re-ingest cascade (indexer); overlap, zero-duration, guessed-fps (collector). Each emits a
real
   report.
2. **"Fresh reader" subagent** given **only** README+FAQ+the technical report (no source): must
   name root cause + fix. Pass = correct cause AND cites the FAQ section the flag's `faq_ref`
points
   to.
3. **Iterate** flag `message`/`faq_ref` (and FAQ text where thin) until diagnosis succeeds.
4. **Lock as golden regression:** fixture ? expected `{flag codes, faq_refs}`; pytest asserts
the
   exact set (prevents wording/flag drift). Enforce `faq_ref` required for error/warn.
5. **Cross-tool/cross-run diff test:** same logical video through both tools / two runs ? JSON
   envelopes differ only where expected.

> The fresh-reader loop can optionally be run as a multi-agent **Workflow** for breadth, but
only
> with explicit opt-in ("ultracode"); default is a lightweight subagent loop.

Phasing (small PRs; lib tagged before any consumer)

Library: C1 scaffold + envelope + JSON renderer + tests ? C2 spec + rules + verdict tests ?
C3 MD renderers + io safe-write/path tests ? C4 RunRecorder + harvest scaffolding + tests ?
tag v0.1.0.
Indexer: A1 add dep + `run_all_with_context` (prove `run_all` unchanged) ? A2 pure
`harvest_indexer_run` (mocked DB) ? A3 wire CLI (`finally`-guarded) + flags + broken-run tests ?
A4 wire API (both branches) + report paths in response ? A5 (later) thread AI timing via
callback
(one segmenter, then twins).
Collector: B1 add dep + `envelope_adapter` + tests ? B2 wire `validate_delivery` (lowest

risk)
? B3 wire `import_local` + `convert_cvat` ? B4 wire `fetch` (+ remote-no-file) + optional
`export`.
Cross-cutting: D1 broken-run fixtures + fresh-reader eval + golden flag/faq assertions in CI
for both repos (after A3/B2).

Risks & mitigations

- **Report write must never break the run:** writes return a result, called in `finally`;
  inject-IOError test asserts the pipeline result is unaffected.
- **Remote videos / no local file:** `io.py` resolves to
`report.output_dir`/`reports/<video_id>/`.
- **Schema versioning:** `SCHEMA_VERSION` in identity; additive-only within a major; consumers
pin a
  tag so a bump can't surprise them.
- **Idempotency:** default overwrite per latest run next to the video; optional
  `report.history_dir` archives `<video_id>/<run_id>.json` for diffing; compile writes a sibling
  `*.compile.report.json` (no read-modify-write race).
- **Twin-segmenter drift:** prefer harvest-from-DB (A2) so wasb/tracknet copies aren't touched
  initially; if A5 lands, apply to both.
- **`video_id` semantics differ** (indexer file-MD5 vs collector human id):
`identity.video_id_kind`
  records which so cross-tool diffs don't mis-join.
- **cwd-relative `output/wasb`:** harvest resolves against the configured output dir.

Critical files

- `badminton-highlight-indexer/backend/main.py` ? CLI :312-352 + API :88-134 recorder hooks
- `badminton-highlight-indexer/backend/validators/base.py` ? add `run_all_with_context`
- `badminton-highlight-indexer/backend/indexer/wasb_hybrid.py` ? candidate/AI-timing/manifest
  harvest reference (twin: `tracknet_hybrid.py`)
- `sports-data-collector/src/parsers/cvat/rally_cvat.py` ? reuse
`build_summary`/`detect_issues`/`Issue`
- `sports-data-collector/src/cli/curate.py` ? import_local / convert_cvat / validate_delivery /
fetch / export wire-ins
- NEW repo `sports-obsreport` ? the shared library above

Verification

- **Lib:** `pytest` in `sports-obsreport` (envelope round-trip, rule engine truth tables,
verdict,
  renderer snapshots, io safe-write incl. injected IOError, path convention). Generate the JSON
  Schema from pydantic and check it in.
- **Indexer:** `pytest` (209 green today) stays green after A1 (`run_all` unchanged); new tests
for
  harvest + flags + `finally`-guarded write; manual smoke `python -m backend.main --video-path
<mp4>
  --max-frames 300` ? assert `<video>.report.json/.md` + `.summary.md` appear and flags read
  sensibly; `--segmenter motion` ? `synthetic_motion_metadata`; no API key ?
`silent_provider_noop`.
- **Collector:** `pytest` (~87% cov) stays green; new adapter tests; manual `curate validate-
delivery`
  + `import-local` ? reports appear next to the video / in `report.output_dir`.
- **Debuggability:** the fresh-reader golden test passes for every broken-run fixture in both
repos.
- **Cross-tool diff:** envelope JSON for the same logical video differs only where expected.
```
```

34. user (2026-06-07T06:58:29.041Z)

File created successfully at: C:\Users\avidu\.claude\plans\drifting-snuggling-squirrel.md (file
state is current in your context ? no need to Read it back)

35. assistant (2026-06-07T06:58:33.948Z)

Let me clean up a stray code-fence artifact at the end of the plan file.

36. user (2026-06-07T06:58:35.104Z)

The file C:\Users\avidu\.claude\plans\drifting-snuggling-squirrel.md has been updated successfully. (file state is current in your context ? no need to Read it back)

37. assistant (2026-06-07T06:58:56.824Z)

The plan is written. Here's the summary for your review.

Plan: rich per-video observability reports (both tools + a shared library)

The core problem I found: both tools already *compute* rich signal but throw it away. In the indexer, `registry.run_all()` builds the `ffprobe_meta` context and discards it, `process_video()` returns only the final segments, and AI timing is literally dropped (`segments, _ = provider.analyze_video(...)`). The collector has a good *delivery-validation* report but nothing capturing the per-video *processing journey*. So this is mostly about **surfacing data that already exists**, plus encoding the known footguns as readable warnings.

Three decisions from your answers, baked in:
- **Always-on**, written **next to the video** by convention (`match.report.json` / `.report.md` / `.summary.md`), with a `report.output_dir` override.
- **Both tools** emit **technical + customer-friendly** reports (one shared envelope ? the two views physically can't disagree on facts).
- **A new private library repo** (`sports-obsreport`, proposed name) providing a declarative **"report config language"** (YAML spec + sub-Turing footgun?flag rules), consumed by both repos via a **tag-pinned** `git+ssh` dep.

Research result on "don't reinvent": the closest off-the-shelf DSL ? **Soda/SodaCL ? is ELv2, not OSI-open-source, and forbids managed-service use ? disqualified by ADR-005.** Great Expectations / Evidently (Apache-2.0) and ydata-profiling / Frictionless (MIT) are license-safe but tabular/ML-shaped ? wrong fit for "ordered pipeline stages + flag rules." So we build a small lib on permissive primitives the repos **already** depend on (pydantic MIT, Jinja2 BSD, PyYAML MIT) and **hand-write the templates** so I control exact wording for the debuggability eval.

The debuggability bar is a real test, not a vibe: broken-run fixtures (no API key, skipped probe, motion-segmenter, collector overlap/zero-duration) ? a **"fresh reader" subagent given only README+FAQ+report** must self-diagnose and cite the FAQ section each flag points to ? locked as a golden regression so wording can't drift.

Phasing keeps it from becoming a house of cards: ship the library and **tag v0.1.0 first**; every consumer PR is small, independently shippable, and `finally`-guarded so a report write can never break a run; the risky AI-timing-through-twin-segmenters work is **deferred** (harvest from the DB instead).

Full detail (envelope shape, file-level hooks, flag list, risks, verification) is in the plan file.

38. user (2026-06-07T07:04:45.331Z)

User has approved your plan. You can now start coding. Start with updating your todo list if applicable

Your plan has been saved to: C:\Users\avidu\.claude\plans\drifting-snuggling-squirrel.md
You can refer back to it if needed during implementation.

Approved Plan (edited by user):

Plan ? Rich per-video observability reports (indexer + collector + shared lib)

Context (why)

Today neither tool emits a per-video record of *what happened during processing*. The
indexer's pipeline computes a lot of signal (ffprobe metadata, validator results, candidate
windows, AI-provider timing, WASB frame/window counts, stitch skips) but **throws most of it
away**: `registry.run_all()` builds a `context` dict with `ffprobe_meta` and discards it
(`backend/validators/base.py:42-56`); `segmenter.process_video()` returns only the final
segment list; AI timing is dropped at `wasb_hybrid.py:167` (`segments, _ =
provider.analyze_video(...)`);
much detail is only `print()`-ed. The collector has a good report pattern for *delivery
validation* (`rally_cvat.py` `build_summary`/`render_validation_md`/`write_review_csv`) but
nothing that captures the **per-video processing journey** (acquire ? parse ? recompute time ?
validate ? register ? export).

We want both tools to **always** emit a rich "side output" per processed video that (a) captures
every stage + details to **catch inconsistencies**, (b) is **diffable across runs and across the
two tools** to evaluate output differences, and (c) lets **someone who has only read the README
+
FAQ self-debug a bad run**. Plus a second **customer-friendly** report that strips system
internals
and shows only video metadata + meta-level highlights. Reports are written **next to the video
by
convention**, with a configurable override.

Per the user, the schema is unified via a **new separate private repo providing a reusable
"report
config language."** Research (below) confirms no off-the-shelf framework fits both the
licensing constraint and the per-video-pipeline-stage shape ? so we build a small library on
permissive primitives the repos already use.

Decisions locked (user + research)

- **Always-on emission.** Written next to the video path by default (`<video>.report.json`
etc.);
  overridable via a `report.output_dir` config key. (Not flag-gated.)
- **Both tools** produce **both** report types (technical + customer-friendly).
- **Shared schema via a new private library repo** that exposes a declarative "report config
  language," reusable by both tools today and others later.
- **Library, not a heavy framework.** Research findings:
  - Soda Core / SodaCL (the closest "YAML?data-quality-report" DSL) is **ELv2 ? not OSI
    open-source, forbids managed-service use ? violates ADR-005. Disqualified.**
  - Great Expectations (Apache-2.0) & Evidently (Apache-2.0) are license-safe but heavy and
    tabular/ML-monitoring shaped ? wrong fit for "an ordered list of heterogeneous pipeline
    stages + a footgun?flag rule layer." ydata-profiling (MIT) / Frictionless (MIT) are
    dataframe/data-package shaped ? also wrong fit.
  - Build on permissive primitives the repos **already depend on**: **pydantic (MIT)** (indexer
    `pydantic>=2.7.1`, collector `pydantic>=2.0.0`), **Jinja2 (BSD)** (indexer dep), **PyYAML
    (MIT)** (collector dep). `jsonschema` deferred ? pydantic v2 `model_json_schema()` is the
    contract source. Thin MIT renderers (`yaml-to-markdown`, `json-schema-for-humans`) are
    dev-only helpers at most; templates are **hand-written** so we control exact wording for
    the FAQ-debuggability eval.

Architecture

New repo: `sports-obsreport` (package `obsreport`), private, MIT ? name TBD by owner

```
src/obsreport/
  envelope.py      # pydantic v2 typed report envelope (single source of truth)
  spec.py          # YAML "report config language" loader -> pydantic
  rules.py         # restricted, sub-Turing footgun->flag engine (NO eval)
  recorder.py      # RunRecorder: in-flight capture + post-hoc harvest
  io.py            # next-to-video path convention + safe (never-raising) writes
  version.py       # SCHEMA_VERSION
  render/{json.py, markdown_technical.py, markdown_customer.py, templates/*.j2}
specs/{indexer.yaml, collector.yaml}
tests/
```

```

**Envelope (`envelope.py`)** ? both renderers consume the *same* object, so the technical and customer reports can never disagree on facts:
- `Identity{schema_version, tool, tool_version, video_id, video_id_kind("file_md5"|"human_id"), video_path, video_url, generated_at, run_id}`
- `VideoMeta{duration_s, fps, fps_source(probed|fallback|declared|unknown), resolution, width, height, container, size_mb}`
- `Stage{name, status(ok|warn|error|skipped|not_run), started_at, duration_s, metrics[], details, messages[]}`
- `Flag{code, severity(error|warn|info), message, stage, faq_ref, evidence}` ? `faq_ref` **required for error/warn** (enforced by a test).
- `HighlightSummary{segment_count, total_highlight_s, coverage_pct, notes[]}` ? customer-safe.
- `ReportEnvelope{identity, video_meta, stages[], flags[], highlights, verdict(pass|pass_with_warnings|fail), extras}`
- **Audience views are derived, not stored twice.** Customer renderer emits only `video_meta + highlights + customer-safe flags + verdict`.

**Config language (`spec.py`, YAML)** ? pure data: `sections[]` (layout + audience + which stage/fields) and `rules[]`. Example footgun rule:
```yaml
rules:
  - code: silent_provider_noop
    severity: error
    when:
      - {path: "stages.ai_handoff.details.api_key_present", op: eq, value: false}
      - {path: "stages.segmentation.metrics.segment_count", op: eq, value: 0}
    message: "0 segments AND no API key ? likely a silent provider no-op, not 'no rallies'."
    faq_ref: "README#Q7"
```

Rule engine (`rules.py`) ? clause `{path, op, value}`; `path` = dotted accessor into the serialized envelope. Ops only: `eq, ne, gt, ge, lt, le, in, contains, is_missing, is_present, eq_path, ne_path` (each a tiny pure fn; **no `eval`, no grammar**). `when` = AND of clauses. Verdict: any error?`fail`; any warn (no error)?`pass_with_warnings`; else `pass`.

RunRecorder (`recorder.py`) ? two modes:
- *In-flight*: `with rec.stage("segmentation"):` auto-times, sets status, captures exceptions **without re-raising** (reporting never breaks the run). `rec.finalize()` runs rules; `rec.write()` emits all files.
- *Post-hoc harvest*: pure functions rebuild the envelope from persisted artifacts (DB candidates/play_segments, WASB manifest, compiler skips) ? guarantees a report is always produceable and enables backfilling old runs for cross-run diffs.

io convention (`io.py`): `/x/match.mp4` ? `match.report.json` + `match.report.md` (technical) + `match.summary.md` (customer). Override via `report.output_dir`. Remote/no-local (collector) ? `report.output_dir` or `reports/<video_id>/`. Atomic temp+rename; all writes return
a result object, **never raise**.

Cross-repo install (git+ssh, tag-pinned ? the main coupling guard):
- indexer `requirements.txt`: `obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.0`
- collector `pyproject.toml`: same `@v0.1.0`. **Pin a tag, never a branch.** Lib has no IO/network
  at import ? nothing new to mock in either repo's offline pytest.

Indexer integration (`badminton-highlight-indexer`)
- **Surface ffprobe_meta backward-compatibly:** add `run_all_with_context(...) -> (results, context)`
  to `backend/validators/base.py`; keep `run_all` returning `[0]` (existing callers/tests stay green). The two `main.py` callers switch to it to grab `context['ffprobe_meta']` (no re-probe).
- **RunRecorder placement:** CLI `backend/main.py:312-352` ? create after `video_id`, wrap validate/add_video/segment in stages, `rec.write()` in a `finally` (partial report even on
```

  crash). API `process_video` `main.py:88-134` ? route **both** exits (validation-fail :104-109,
  success :128-134) through one finalize+write; add report paths to the JSON response.
- **Harvest (Mode B):** probe meta from context; validation from the `ValidationResult` list;
  segmentation from `segments` (+ `segmenter_requested` vs `segmenter_actual`); candidates from
  `db.query_candidate_segments`; WASB `{frames_total,windows_total,windows_done}` from
  `output/wasb/manifest.json` (resolve against configured output dir, not raw cwd); compiler
skips
  = `not_run` (compile is a separate endpoint). AI timing = `not_captured` initially; thread a
  recorder callback through `analyze_video`?`process_video` in a **later isolated PR** (touches
the
  twin segmenters, so deferred).
- **Footgun flags:** `silent_provider_noop`(error,Q7), `ai_empty_vs_no_rallies`(warn,Q7),
  `fps_fallback_used`(warn,Q5), `synthetic_motion_metadata`(warn), `segmenter_divergence`(info ?
  config/setup/main disagree on default), `reingest_cascade`(warn ? INSERT OR REPLACE deleted
  prior segments), `wsl_healthcheck_failed`(error,Q7), `validation_failed`(error,Q2/Q5).

Collector integration (`sports-data-collector`)

Add `src/reporting/envelope_adapter.py`; **reuse** `build_summary`/`detect_issues`/
`render_validation_md` as the authoritative content producers (no duplication).
- **Issue?Flag:** `blocker?error`, `high/medium?warn`, `low?info`; keep `code`; `code?faq_ref`
via
  a small table; `evidence={"rally_number":...}`.
- **Stages:** `acquisition` (from `DownloadResult` in `src/core/downloader.py`), `parse`
  (format + n records), `time_recompute` (CVAT `fps/step/meta`; flag guessed fps), `validation`
  (existing `detect_issues`/sport validators), `registration` (DB insert), `export`.
- **Wire-in points (append a report write after existing work):** `import_local`
  (`curate.py:52-142`, after insert), `convert_cvat` (`:526-560`), `validate_delivery`
  (`:562-617` ? lowest risk, already writes a report; keep existing files for back-compat),
  `fetch` (`:330+`), `export` (`:428-524`, optional per-video delivery summary).
- **Customer "delivery summary":** `VideoMeta` + `HighlightSummary` (n rallies, total rally
  seconds, coverage) + error/warn flag messages (code stripped).

README/FAQ-reader debuggability ? eval loop (the quality bar)

1. **Broken-run fixtures** (offline, mocked): no-API-key, skipped-probe, motion-segmenter,
   re-ingest cascade (indexer); overlap, zero-duration, guessed-fps (collector). Each emits a
real
   report.
2. **"Fresh reader" subagent** given **only** README+FAQ+the technical report (no source): must
   name root cause + fix. Pass = correct cause AND cites the FAQ section the flag's `faq_ref`
points
   to.
3. **Iterate** flag `message`/`faq_ref` (and FAQ text where thin) until diagnosis succeeds.
4. **Lock as golden regression:** fixture ? expected `{flag codes, faq_refs}`; pytest asserts
the
   exact set (prevents wording/flag drift). Enforce `faq_ref` required for error/warn.
5. **Cross-tool/cross-run diff test:** same logical video through both tools / two runs ? JSON
   envelopes differ only where expected.

> The fresh-reader loop can optionally be run as a multi-agent **Workflow** for breadth, but
only
> with explicit opt-in ("ultracode"); default is a lightweight subagent loop.

Phasing (small PRs; lib tagged before any consumer)

Library: C1 scaffold + envelope + JSON renderer + tests ? C2 spec + rules + verdict tests ?
C3 MD renderers + io safe-write/path tests ? C4 RunRecorder + harvest scaffolding + tests ?
tag v0.1.0.
Indexer: A1 add dep + `run_all_with_context` (prove `run_all` unchanged) ? A2 pure
`harvest_indexer_run` (mocked DB) ? A3 wire CLI (`finally`-guarded) + flags + broken-run tests ?
A4 wire API (both branches) + report paths in response ? A5 (later) thread AI timing via
callback
(one segmenter, then twins).
Collector: B1 add dep + `envelope_adapter` + tests ? B2 wire `validate_delivery` (lowest
risk)
? B3 wire `import_local` + `convert_cvat` ? B4 wire `fetch` (+ remote-no-file) + optional

`export`.
Cross-cutting: D1 broken-run fixtures + fresh-reader eval + golden flag/faq assertions in CI for both repos (after A3/B2).

Risks & mitigations

- **Report write must never break the run:** writes return a result, called in `finally`; inject-IOError test asserts the pipeline result is unaffected.
- **Remote videos / no local file:** `io.py` resolves to `report.output_dir`/`reports/<video_id>/`.
- **Schema versioning:** `SCHEMA_VERSION` in identity; additive-only within a major; consumers pin a tag so a bump can't surprise them.
- **Idempotency:** default overwrite per latest run next to the video; optional `report.history_dir` archives `<video_id>/<run_id>.json` for diffing; compile writes a sibling `*.compile.report.json` (no read-modify-write race).
- **Twin-segmenter drift:** prefer harvest-from-DB (A2) so wasb/tracknet copies aren't touched initially; if A5 lands, apply to both.
- **`video_id` semantics differ** (indexer file-MD5 vs collector human id): `identity.video_id_kind` records which so cross-tool diffs don't mis-join.
- **cwd-relative `output/wasb`:** harvest resolves against the configured output dir.

Critical files

- `badminton-highlight-indexer/backend/main.py` ? CLI :312-352 + API :88-134 recorder hooks
- `badminton-highlight-indexer/backend/validators/base.py` ? add `run_all_with_context`
- `badminton-highlight-indexer/backend/indexer/wasb_hybrid.py` ? candidate/AI-timing/manifest harvest reference (twin: `tracknet_hybrid.py`)
- `sports-data-collector/src/parsers/cvat/rally_cvat.py` ? reuse `build_summary`/`detect_issues`/`Issue`
- `sports-data-collector/src/cli/curate.py` ? import_local / convert_cvat / validate_delivery / fetch / export wire-ins
- NEW repo `sports-obsreport` ? the shared library above

Verification

- **Lib:** `pytest` in `sports-obsreport` (envelope round-trip, rule engine truth tables, verdict, renderer snapshots, io safe-write incl. injected IOError, path convention). Generate the JSON Schema from pydantic and check it in.
- **Indexer:** `pytest` (209 green today) stays green after A1 (`run_all` unchanged); new tests for harvest + flags + `finally`-guarded write; manual smoke `python -m backend.main --video-path <mp4> --max-frames 300` ? assert `<video>.report.json/.md` + `.summary.md` appear and flags read sensibly; `--segmenter motion` ? `synthetic_motion_metadata`; no API key ? `silent_provider_noop`.
- **Collector:** `pytest` (~87% cov) stays green; new adapter tests; manual `curate validate-delivery` + `import-local` ? reports appear next to the video / in `report.output_dir`.
- **Debuggability:** the fresh-reader golden test passes for every broken-run fixture in both repos.
- **Cross-tool diff:** envelope JSON for the same logical video differs only where expected.

39. assistant (2026-06-07T07:05:12.543Z)

Plan approved. Let me set up progress tracking and mirror the existing packaging conventions before scaffolding the new library. Loading the task tools and reading the collector's packaging/test setup to match its style.

40. user (2026-06-07T07:05:13.499Z)

```
1       [build-system]
2       requires = ["setuptools>=61.0.0"]
3       build-backend = "setuptools.build-meta"
```

```
4
5       [project]
6       name = "sports-data-collector"
7       version = "0.1.0"
8       description = "A collector and curator for sports video datasets with sport-specific
schemas and license checks."
9       readme = "README.md"
10      requires-python = ">=3.9"
11      dependencies = [
12          "pydantic>=2.0.0",
13          "pyyaml>=6.0",
14          "tabulate>=0.9.0"
15      ]
16
17      [project.optional-dependencies]
18      # Video acquisition (`fetch` / crawler). Also requires the external binaries
19      # ffprobe (FFmpeg) and, for YouTube high-res, a JS runtime such as deno ?
20      # neither is pip-installable; see README "External requirements".
21      fetch = [
22          "yt-dlp>=2024.1.0"
23      ]
24      dev = [
25          "pytest>=7.0.0",
26          "pytest-cov>=4.0.0"
27      ]
28
```

41. assistant (2026-06-07T07:05:24.757Z)

Loaded. Let me mark the implementation chapter, set up phase tracking, and verify the Python environment for the new library.

42. user (2026-06-07T07:05:25.642Z)

Chapter marked: "Observability reports impl". Continue your current work.

43. user (2026-06-07T07:05:27.230Z)

```
Python 3.13.13
pydantic 2.13.4
jinja2 3.1.6
yaml ok
```

44. assistant (2026-06-07T07:05:31.598Z)

Environment is ready (Python 3.13, pydantic 2.13, jinja2 3.1, yaml). Let me create the phase task list, then start building the library.

45. user (2026-06-07T07:05:34.506Z)

Task #1 created successfully: C1: Scaffold sports-obsreport lib (envelope + JSON renderer + tests)

46. user (2026-06-07T07:05:36.830Z)

Task #2 created successfully: C2: Spec loader + rule engine + verdict

47. user (2026-06-07T07:05:39.137Z)

Task #3 created successfully: C3: Markdown renderers (technical + customer) + safe io

48. user (2026-06-07T07:05:41.960Z)

```
Task #4 created successfully: C4: RunRecorder + harvest scaffolding; tag v0.1.0
```

49. user (2026-06-07T07:05:49.212Z)

```
Task #7 created successfully: D: README/FAQ-reader debuggability eval + golden regression
```

50. user (2026-06-07T07:05:44.336Z)

```
Task #5 created successfully: A: Indexer integration (run_all_with_context, harvest, CLI+API
wiring, flags)
```

51. user (2026-06-07T07:05:46.670Z)

```
Task #6 created successfully: B: Collector integration (envelope_adapter + wire-ins + delivery
summary)
```

52. assistant (2026-06-07T07:07:07.456Z)

```
Starting **C1** ? scaffolding the shared `obsreport` library. Let me build the package skeleton,
the typed envelope, the deterministic JSON renderer, and tests.
```

53. user (2026-06-07T07:07:07.865Z)

```
Updated task #1 status
```

54. assistant (2026-06-07T07:07:14.226Z)

```
[build-system]
requires = ["setuptools>=61.0.0"]
build-backend = "setuptools.build-meta"

[project]
name = "obsreport"
version = "0.1.0"
description = "Reusable per-run observability report toolkit: a small config-language for
emitting technical + customer-friendly reports across data/video pipelines."
readme = "README.md"
requires-python = ">=3.9"
license = { text = "MIT" }
authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
dependencies = [
    "pydantic>=2.0.0",
    "jinja2>=3.1.0",
    "pyyaml>=6.0",
]

[project.optional-dependencies]
dev = [
    "pytest>=7.0.0",
    "pytest-cov>=4.0.0",
]

[tool.setuptools.packages.find]
where = ["src"]

[tool.pytest.ini_options]
pythonpath = ["src"]
testpaths = ["tests"]
```

55. user (2026-06-07T07:07:14.244Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\pyproject.toml (file
state is current in your context ? no need to Read it back)
```

56. assistant (2026-06-07T07:07:21.551Z)

```
MIT License

Copyright (c) 2026 Avi Dullu

Permission is hereby granted, free of charge, to any person obtaining a copy
of this software and associated documentation files (the "Software"), to deal
in the Software without restriction, including without limitation the rights
to use, copy, modify, merge, publish, distribute, sublicense, and/or sell
copies of the Software, and to permit persons to whom the Software is
furnished to do so, subject to the following conditions:

The above copyright notice and this permission notice shall be included in all
copies or substantial portions of the Software.

THE SOFTWARE IS PROVIDED "AS IS", WITHOUT WARRANTY OF ANY KIND, EXPRESS OR
IMPLIED, INCLUDING BUT NOT LIMITED TO THE WARRANTIES OF MERCHANTABILITY,
FITNESS FOR A PARTICULAR PURPOSE AND NONINFRINGEMENT. IN NO EVENT SHALL THE
AUTHORS OR COPYRIGHT HOLDERS BE LIABLE FOR ANY CLAIM, DAMAGES OR OTHER
LIABILITY, WHETHER IN AN ACTION OF CONTRACT, TORT OR OTHERWISE, ARISING FROM,
OUT OF OR IN CONNECTION WITH THE SOFTWARE OR THE USE OR OTHER DEALINGS IN THE
SOFTWARE.
```

57. user (2026-06-07T07:07:21.568Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\LICENSE (file state is
current in your context ? no need to Read it back)
```

58. assistant (2026-06-07T07:07:25.655Z)

```
# Python
__pycache__/
*.py[cod]
*.egg-info/
.eggs/
build/
dist/
.venv/
venv/

# Test / coverage
.pytest_cache/
.coverage
htmlcov/
coverage.xml

# Editor
.vscode/
.idea/
```

59. user (2026-06-07T07:07:25.665Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\.gitignore (file state is
current in your context ? no need to Read it back)
```

60. assistant (2026-06-07T07:07:30.419Z)

```
"""Version + report-schema version.

`SCHEMA_VERSION` is the contract version of the report ENVELOPE (bump the major
when a change is not backward compatible). `__version__` is the package release.
```

```
Consumers pin a git tag, so a schema bump can never surprise them silently.
"""

SCHEMA_VERSION = "1.0"
__version__ = "0.1.0"
```

61. user (2026-06-07T07:07:30.432Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\src\obsreport\version.py
(file state is current in your context ? no need to Read it back)
```

62. assistant (2026-06-07T07:07:58.252Z)

```
"""Typed report envelope ? the single source of truth for one observability report.

A report describes ONE processed video / one pipeline run as an ordered list of
heterogeneous *stages*, plus video metadata, derived *flags* (warnings/errors),
a customer-safe *highlight summary*, and an overall *verdict*.

Both the technical and the customer-friendly renderers consume this SAME object,
so the two reports can never disagree on facts. This module is pure data + a few
pure helpers ? there is NO file/network IO here (see `io.py`) and nothing runs at
import time.
"""
from __future__ import annotations

from typing import Any, Dict, List, Literal, Optional

from pydantic import BaseModel, Field

from .version import SCHEMA_VERSION
```

--- closed vocabularies (kept as Literals so typos fail fast in consumers) ---

```
Audience = Literal["technical", "customer", "both"]
StageStatus = Literal["ok", "warn", "error", "skipped", "not_run"]
Severity = Literal["error", "warn", "info"]
Verdict = Literal["pass", "pass_with_warnings", "fail"]
FpsSource = Literal["probed", "fallback", "declared", "unknown"]
VideoIdKind = Literal["file_md5", "human_id", "unknown"]


class Metric(BaseModel):
    """A single named measurement attached to a stage.

    `audience` controls which report it surfaces in: technical-only by default;
    use ``"both"`` for things a customer should also see (e.g. duration).
    """

    key: str
    value: Any = None
    unit: Optional[str] = None
    audience: Audience = "technical"


class Stage(BaseModel):
    """One step the video went through (validate, segment, stitch, acquire, ...)."""

    name: str
    status: StageStatus = "ok"
    started_at: Optional[str] = None
    duration_s: Optional[float] = None
    metrics: List[Metric] = Field(default_factory=list)
    details: Dict[str, Any] = Field(default_factory=dict)
    messages: List[str] = Field(default_factory=list)
```

```
    def add_metric(
        self,
        key: str,
        value: Any,
        unit: Optional[str] = None,
        audience: Audience = "technical",
    ) -> "Stage":
        """Append a metric and return self (chainable)."""
        self.metrics.append(Metric(key=key, value=value, unit=unit, audience=audience))
        return self


class Flag(BaseModel):
    """A derived warning/error ? the debugging intelligence of the report.

    `faq_ref` points at the README/FAQ section a reader should open to self-debug
    (e.g. ``"README#Q7"``). It is REQUIRED for error/warn severities; this is
    enforced by `ReportEnvelope.lint`, not by the constructor, so harvest code can
    build a flag incrementally before deciding its citation.
    """

    code: str
    severity: Severity
    message: str
    stage: Optional[str] = None
    faq_ref: Optional[str] = None
    evidence: Dict[str, Any] = Field(default_factory=dict)


class Identity(BaseModel):
    """Who/what this report is about. `video_id_kind` records the id SEMANTICS so a
    cross-tool diff (indexer file-MD5 vs collector human id) never mis-joins."""

    tool: str
    schema_version: str = SCHEMA_VERSION
    tool_version: Optional[str] = None
    video_id: Optional[str] = None
    video_id_kind: VideoIdKind = "unknown"
    video_path: Optional[str] = None
    video_url: Optional[str] = None
    generated_at: Optional[str] = None
    run_id: Optional[str] = None


class VideoMeta(BaseModel):
    """Probed/declared media facts. `fps_source` distinguishes a real probe from a
    silent fallback (a known footgun the report exists to surface)."""

    duration_s: Optional[float] = None
    fps: Optional[float] = None
    fps_source: FpsSource = "unknown"
    resolution: Optional[str] = None
    width: Optional[int] = None
    height: Optional[int] = None
    container: Optional[str] = None
    size_mb: Optional[float] = None


class HighlightSummary(BaseModel):
    """Customer-safe, meta-level summary of what the tool produced (no internals)."""

    segment_count: int = 0
    total_highlight_s: float = 0.0
    coverage_pct: Optional[float] = None
    notes: List[str] = Field(default_factory=list)
```

```
def verdict_from_flags(flags: List[Flag]) -> Verdict:
    """Pure verdict derivation: any error -> fail; any warn -> pass_with_warnings."""
    severities = {f.severity for f in flags}
    if "error" in severities:
        return "fail"
    if "warn" in severities:
        return "pass_with_warnings"
    return "pass"


class ReportEnvelope(BaseModel):
    """The whole report. Serialize with `obsreport.to_json` (machine source of
    truth) or render via the markdown renderers (human/customer views)."""

    identity: Identity
    video_meta: VideoMeta = Field(default_factory=VideoMeta)
    stages: List[Stage] = Field(default_factory=list)
    flags: List[Flag] = Field(default_factory=list)
    highlights: Optional[HighlightSummary] = None
    verdict: Verdict = "pass"
    extras: Dict[str, Any] = Field(default_factory=dict)

    # --- pure helpers (no IO) ---
    def get_stage(self, name: str) -> Optional[Stage]:
        for s in self.stages:
            if s.name == name:
                return s
        return None

    def recompute_verdict(self) -> "ReportEnvelope":
        """Set `verdict` from current flags. Call in finalize, after rules run."""
        self.verdict = verdict_from_flags(self.flags)
        return self

    def lint(self) -> List[str]:
        """Structural self-checks. Empty list == clean. Used by tests and the
        recorder's finalize to guarantee a well-formed, debuggable report."""
        issues: List[str] = []
        for f in self.flags:
            if f.severity in ("error", "warn") and not f.faq_ref:
                issues.append(
                    f"flag '{f.code}' has severity '{f.severity}' but no faq_ref "
                    f"(error/warn flags must cite a README/FAQ section)"
                )
        expected = verdict_from_flags(self.flags)
        if self.verdict != expected:
            issues.append(
                f"verdict '{self.verdict}' does not match flag-derived '{expected}' "
                f"(did you forget recompute_verdict()?)"
            )
        return issues
```

63. user (2026-06-07T07:07:58.281Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\src\obsreport\envelope.py
(file state is current in your context ? no need to Read it back)
```

64. assistant (2026-06-07T07:08:02.161Z)

```
"""Renderers: turn a ReportEnvelope into JSON / Markdown / customer Markdown."""
```

65. user (2026-06-07T07:08:02.183Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-
```

```
obsreport\src\obsreport\render\__init__.py (file state is current in your context ? no need to
Read it back)
```

66. assistant (2026-06-07T07:08:10.853Z)

```
"""JSON renderer ? the machine source of truth.

Keys are sorted so the output is byte-stable regardless of the order in which
free-form dict fields (details / extras / evidence) were populated. That makes
cross-run and cross-tool diffs clean, which is a core goal of the feature.
"""
from __future__ import annotations

import json

from ..envelope import ReportEnvelope


def to_json(envelope: ReportEnvelope, indent: int = 2) -> str:
    data = envelope.model_dump(mode="json")
    return json.dumps(data, indent=indent, sort_keys=True, ensure_ascii=False)
```

67. user (2026-06-07T07:08:10.864Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-
obsreport\src\obsreport\render\json_render.py (file state is current in your context ? no need
to Read it back)
```

68. assistant (2026-06-07T07:08:19.512Z)

```
"""obsreport ? reusable per-run observability reports.

Build a `ReportEnvelope` (typed, single source of truth), optionally run a
declarative rule spec to derive flags, then render it to JSON (machine) and
Markdown (technical + customer). Designed to be shared across pipelines so their
reports are diffable with each other and across runs.
"""
from .version import SCHEMA_VERSION, __version__
from .envelope import (
    Flag,
    HighlightSummary,
    Identity,
    Metric,
    ReportEnvelope,
    Stage,
    VideoMeta,
    verdict_from_flags,
)
from .render.json_render import to_json

__all__ = [
    "SCHEMA_VERSION",
    "__version__",
    "ReportEnvelope",
    "Identity",
    "VideoMeta",
    "Stage",
    "Metric",
    "Flag",
    "HighlightSummary",
    "verdict_from_flags",
    "to_json",
]
```

69. user (2026-06-07T07:08:19.523Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\src\obsreport\__init__.py
(file state is current in your context ? no need to Read it back)
```

70. assistant (2026-06-07T07:08:24.253Z)

Now the C1 tests ? envelope round-trip, verdict/lint logic, and deterministic JSON:

71. assistant (2026-06-07T07:08:35.504Z)

```
import pytest

from obsreport import (
    Flag,
    HighlightSummary,
    Identity,
    ReportEnvelope,
    Stage,
    VideoMeta,
    verdict_from_flags,
)


def _envelope(**kw) -> ReportEnvelope:
    return ReportEnvelope(identity=Identity(tool="test"), **kw)


def test_round_trips_through_json():
    env = _envelope(
        video_meta=VideoMeta(fps=59.94, fps_source="probed", width=1920, height=1080),
        stages=[Stage(name="validation", status="ok").add_metric("checks", 6)],
        flags=[Flag(code="x", severity="info", message="hi")],
        highlights=HighlightSummary(segment_count=3, total_highlight_s=12.5),
    )
    again = ReportEnvelope.model_validate_json(env.model_dump_json())
    assert again == env


def test_add_metric_is_chainable_and_records():
    s = Stage(name="seg").add_metric("segment_count", 5).add_metric("dur", 1.2, unit="s",
audience="both")
    assert [m.key for m in s.metrics] == ["segment_count", "dur"]
    assert s.metrics[1].audience == "both"


@pytest.mark.parametrize(
    "severities,expected",
    [
        ([], "pass"),
        (["info"], "pass"),
        (["info", "warn"], "pass_with_warnings"),
        (["warn", "error"], "fail"),
        (["error"], "fail"),
    ],
)
def test_verdict_from_flags(severities, expected):
    flags = [Flag(code=f"c{i}", severity=s, message="m", faq_ref="README#Q1") for i, s in
enumerate(severities)]
    assert verdict_from_flags(flags) == expected


def test_recompute_verdict_sets_field():
    env = _envelope(flags=[Flag(code="boom", severity="error", message="m",
faq_ref="README#Q7")])
```

```
    assert env.verdict == "pass"  # default before recompute
    env.recompute_verdict()
    assert env.verdict == "fail"


def test_lint_flags_missing_faq_ref_on_error_and_warn():
    env = _envelope(
        flags=[
            Flag(code="e", severity="error", message="m"),  # missing faq_ref
            Flag(code="w", severity="warn", message="m"),  # missing faq_ref
            Flag(code="i", severity="info", message="m"),  # info is allowed to omit
        ]
    ).recompute_verdict()
    issues = env.lint()
    assert any("'e'" in i for i in issues)
    assert any("'w'" in i for i in issues)
    assert not any("'i'" in i for i in issues)


def test_lint_clean_when_faq_refs_present_and_verdict_recomputed():
    env = _envelope(
        flags=[Flag(code="w", severity="warn", message="m", faq_ref="README#Q5")]
    ).recompute_verdict()
    assert env.lint() == []


def test_lint_catches_stale_verdict():
    env = _envelope(flags=[Flag(code="e", severity="error", message="m", faq_ref="README#Q7")])
    # verdict left at default "pass" but flags imply "fail"
    assert any("verdict" in i for i in env.lint())


def test_get_stage():
    env = _envelope(stages=[Stage(name="a"), Stage(name="b")])
    assert env.get_stage("b").name == "b"
    assert env.get_stage("missing") is None
```

72. user (2026-06-07T07:08:35.514Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\tests\test_envelope.py
(file state is current in your context ? no need to Read it back)
```

73. assistant (2026-06-07T07:08:44.605Z)

```
import json

from obsreport import Flag, Identity, ReportEnvelope, Stage, to_json


def _env_with_details(details):
    return ReportEnvelope(
        identity=Identity(tool="test"),
        stages=[Stage(name="s", details=details)],
    )


def test_to_json_is_valid_and_round_trips():
    env = ReportEnvelope(
        identity=Identity(tool="indexer", video_id="abc", video_id_kind="file_md5"),
        flags=[Flag(code="c", severity="info", message="m")],
    )
    text = to_json(env)
    parsed = json.loads(text)
    assert parsed["identity"]["tool"] == "indexer"
    # round-trips back into the model unchanged
```

```
    assert ReportEnvelope.model_validate(parsed) == env


def test_to_json_is_key_order_stable():
    """Free-form dict fields inserted in different orders must serialize identically
    so cross-run diffs stay clean."""
    a = _env_with_details({"b": 1, "a": 2, "c": 3})
    b = _env_with_details({"c": 3, "a": 2, "b": 1})
    assert to_json(a) == to_json(b)


def test_to_json_keys_are_sorted():
    text = to_json(_env_with_details({"zzz": 1, "aaa": 2}))
    assert text.index('"aaa"') < text.index('"zzz"')
```

74. user (2026-06-07T07:08:44.616Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\tests\test_render_json.py
(file state is current in your context ? no need to Read it back)
```

75. assistant (2026-06-07T07:08:48.425Z)

Now a brief README for the library, then run the C1 tests:

76. assistant (2026-06-07T07:08:57.538Z)

obsreport

A small, reusable toolkit for emitting **rich per-run observability reports** from data/video pipelines ? designed so multiple tools produce reports in the **same shape**, making them diffable across runs and across tools.

It is deliberately **not** a heavy data-quality framework (Great Expectations, Evidently, Soda, ?). Those model tabular datasets; this models *a single pipeline run as an ordered list of heterogeneous stages* plus a declarative "footgun ? flag" rule layer. It is built only on permissive primitives (`pydantic`, `Jinja2`, `PyYAML`) so it is safe for commercial use.

What you get

- **One typed envelope** (`ReportEnvelope`) that is the single source of truth. Both the technical and the customer-friendly report render from the same object, so they can never disagree on facts.
- **A declarative report config language** (YAML): describe report sections and *rules* that turn known failure modes into flags, each citing a FAQ section so a reader can self-debug. *(spec.py / rules.py ? added in C2.)*
- **Renderers**: deterministic JSON (machine source of truth, today) + Markdown technical & customer views *(C3)*.
- **A RunRecorder** to capture stages in-flight, plus post-hoc harvest helpers, with writes that **never raise** so reporting can't break the host run *(C4)*.

Status

Early scaffolding. Phase C1 (envelope + JSON renderer) is in place; C2?C4 follow. See the consuming repos (`badminton-highlight-indexer`, `sports-data-collector`) for the integration.

Develop

```bash
pip install -e ".[dev]"
pytest
```

```

**77. user (2026-06-07T07:08:57.553Z)**

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\README.md (file state is
current in your context ? no need to Read it back)
```

**78. user (2026-06-07T07:09:02.197Z)**

```
..............                                                           [100%]
15 passed in 0.22s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

79. assistant (2026-06-07T07:09:26.244Z)

C1 green ? 15 passed. Let me initialize the new repo's git history and checkpoint C1 locally (I'll hold remote/GitHub creation for your go-ahead).

80. user (2026-06-07T07:09:29.479Z)

```
Initialized empty Git repository in C:/Users/avidu/Projects/sports-obsreport/.git/
warning: in the working copy of '.gitignore', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'LICENSE', LF will be replaced by CRLF the next time Git touches
it
warning: in the working copy of 'README.md', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'pyproject.toml', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'src/obsreport/__init__.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'src/obsreport/envelope.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'src/obsreport/render/__init__.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'src/obsreport/render/json_render.py', LF will be replaced by
CRLF the next time Git touches it
warning: in the working copy of 'src/obsreport/version.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'tests/test_envelope.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'tests/test_render_json.py', LF will be replaced by CRLF the
next time Git touches it
687b52e C1: scaffold obsreport ? typed envelope + deterministic JSON renderer
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

81. assistant (2026-06-07T07:11:07.429Z)

C1 committed. Now **C2** ? the declarative "report config language" (YAML spec) + the restricted, no-`eval` rule engine that turns known footguns into FAQ-citing flags. Marking C1 done, C2 in progress.

82. user (2026-06-07T07:11:07.861Z)

```
Updated task #1 status
```

83. user (2026-06-07T07:11:08.121Z)

```
Updated task #2 status
```

84. assistant (2026-06-07T07:11:39.083Z)

```
"""The report config language: a YAML spec describing report *sections* (layout)
```
```

```
and *rules* (footgun -> flag).

A spec is pure data validated into pydantic models. Specs are authored by us (one
per tool, under ``specs/``), so we validate strictly and fail fast: every
actionable rule (error/warn) MUST cite a FAQ section, and every rule MUST have at
least one condition. That guarantee is what makes the resulting reports
self-debuggable.
"""
from __future__ import annotations

from pathlib import Path
from typing import Any, List, Literal, Optional, Union

import yaml
from pydantic import BaseModel, Field, model_validator

from .envelope import Audience, Severity
```

Comparison operators the rule engine understands. Deliberately a closed, sub-Turing set ? there is NO expression grammar and NO eval anywhere.

```
ClauseOp = Literal[
    "eq", "ne", "gt", "ge", "lt", "le",
    "in", "contains",
    "is_missing", "is_present",
    "eq_path", "ne_path",
]


class Clause(BaseModel):
    """One condition: compare the value at ``path`` against ``value``.

    ``path`` is a dotted accessor into the rule context (see ``rules.to_rule_context``),
    e.g. ``stages.ai_handoff.metrics.segment_count``. For ``eq_path``/``ne_path``,
    ``value`` is itself a dotted path (compare two fields)."""

    path: str
    op: ClauseOp
    value: Any = None


class Rule(BaseModel):
    """A footgun detector. If every clause in ``when`` holds, emit a flag."""

    code: str
    severity: Severity
    message: str
    when: List[Clause] = Field(default_factory=list)
    faq_ref: Optional[str] = None
    stage: Optional[str] = None

    @model_validator(mode="after")
    def _validate(self) -> "Rule":
        if self.severity in ("error", "warn") and not self.faq_ref:
            raise ValueError(
                f"rule '{self.code}' has severity '{self.severity}' but no faq_ref "
                f"(actionable flags must cite a README/FAQ section so a reader can self-debug)"
            )
        if not self.when:
            raise ValueError(
                f"rule '{self.code}' has no conditions; a rule must have >=1 clause "
                f"(use is_present/is_missing for existence checks)"
            )
        return self
```

```
SectionKind = Literal[
    "identity", "video_meta", "highlights", "verdict", "flags", "stages", "stage"
]


class Section(BaseModel):
    """A block in the rendered report. The renderer walks these in order."""

    id: str
    title: str
    audience: Audience = "technical"
    kind: SectionKind = "stages"
    stage: Optional[str] = None  # required when kind == "stage"
    metrics: Optional[List[str]] = None  # optional whitelist of metric keys to show

    @model_validator(mode="after")
    def _validate(self) -> "Section":
        if self.kind == "stage" and not self.stage:
            raise ValueError(f"section '{self.id}' is kind=stage but names no 'stage'")
        return self


class ReportSpec(BaseModel):
    name: str
    title: Optional[str] = None
    sections: List[Section] = Field(default_factory=list)
    rules: List[Rule] = Field(default_factory=list)


def load_spec_dict(data: dict) -> ReportSpec:
    return ReportSpec.model_validate(data or {})


def load_spec_text(text: str) -> ReportSpec:
    return load_spec_dict(yaml.safe_load(text) or {})


def load_spec(path: Union[str, Path]) -> ReportSpec:
    return load_spec_text(Path(path).read_text(encoding="utf-8"))
```

85. user (2026-06-07T07:11:39.103Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\src\obsreport\spec.py
(file state is current in your context ? no need to Read it back)
```

86. assistant (2026-06-07T07:12:02.322Z)

```
"""The rule engine: evaluate a spec's rules against an envelope to derive flags.

Everything here is pure and dependency-free. A missing path makes a clause False
for every operator except ``is_missing`` ? so a typo or an absent stage can never
*accidentally* fire a footgun flag.
"""
from __future__ import annotations

from typing import Any, Dict, List

from .envelope import Flag, ReportEnvelope
from .spec import Clause, Rule, ReportSpec


class _Missing:
    """Sentinel for an unresolved path (distinct from a real ``None`` value)."""

    _singleton = None
```

```
    def __repr__(self) -> str:  # pragma: no cover - cosmetic
        return "<MISSING>"


MISSING = _Missing()


def to_rule_context(env: ReportEnvelope) -> Dict[str, Any]:
    """Project an envelope into an ergonomic dict for dotted-path lookups.

    Lists that you'd want to address by name are re-keyed: ``stages`` by stage
    name, and each stage's ``metrics`` by metric key. So a rule can say
    ``stages.segmentation.metrics.segment_count`` or
    ``stages.ai_handoff.details.api_key_present``.
    """
    stages: Dict[str, Any] = {}
    for s in env.stages:
        stages[s.name] = {
            "status": s.status,
            "duration_s": s.duration_s,
            "messages": list(s.messages),
            "details": dict(s.details),
            "metrics": {m.key: m.value for m in s.metrics},
        }
    return {
        "identity": env.identity.model_dump(mode="json"),
        "video_meta": env.video_meta.model_dump(mode="json"),
        "highlights": env.highlights.model_dump(mode="json") if env.highlights else {},
        "verdict": env.verdict,
        "extras": dict(env.extras),
        "stages": stages,
        "flags": [f.model_dump(mode="json") for f in env.flags],
    }


def get_path(data: Any, dotted: str) -> Any:
    """Navigate ``data`` by a dotted path; return MISSING if any hop is absent.
    Numeric path parts index into lists."""
    cur = data
    for part in dotted.split("."):
        if isinstance(cur, dict):
            if part in cur:
                cur = cur[part]
            else:
                return MISSING
        elif isinstance(cur, list):
            try:
                idx = int(part)
            except ValueError:
                return MISSING
            if -len(cur) <= idx < len(cur):
                cur = cur[idx]
            else:
                return MISSING
        else:
            return MISSING
    return cur


def _ordered(op: str, a: Any, b: Any) -> bool:
    try:
        if op == "gt":
            return a > b
        if op == "ge":
            return a >= b
```

```
        if op == "lt":
            return a < b
        if op == "le":
            return a <= b
    except TypeError:
        return False
    return False


def eval_clause(ctx: Dict[str, Any], clause: Clause) -> bool:
    actual = get_path(ctx, clause.path)
    op = clause.op

    if op == "is_missing":
        return actual is MISSING
    if op == "is_present":
        return actual is not MISSING
    if actual is MISSING:
        # Every other operator is False against a missing path (no accidental fires).
        return False

    if op == "eq":
        return actual == clause.value
    if op == "ne":
        return actual != clause.value
    if op in ("gt", "ge", "lt", "le"):
        return _ordered(op, actual, clause.value)
    if op == "in":
        try:
            return actual in clause.value
        except TypeError:
            return False
    if op == "contains":
        try:
            return clause.value in actual
        except TypeError:
            return False
    if op in ("eq_path", "ne_path"):
        other = get_path(ctx, clause.value) if isinstance(clause.value, str) else MISSING
        if other is MISSING:
            return False
        return actual == other if op == "eq_path" else actual != other
    return False


def eval_rule(ctx: Dict[str, Any], rule: Rule) -> bool:
    """A rule fires when ALL its clauses hold (logical AND)."""
    return all(eval_clause(ctx, c) for c in rule.when)


def _evidence(ctx: Dict[str, Any], rule: Rule) -> Dict[str, Any]:
    """Snapshot the values that triggered the rule, so the flag is self-explaining
    in the report (MISSING rendered as None for JSON-safety)."""
    ev: Dict[str, Any] = {}
    for c in rule.when:
        val = get_path(ctx, c.path)
        ev[c.path] = None if val is MISSING else val
    return ev


def apply_rules(env: ReportEnvelope, spec: ReportSpec) -> List[Flag]:
    """Return the flags a spec's rules produce for an envelope (no mutation)."""
    ctx = to_rule_context(env)
    flags: List[Flag] = []
    for rule in spec.rules:
```

```
        if eval_rule(ctx, rule):
            flags.append(
                Flag(
                    code=rule.code,
                    severity=rule.severity,
                    message=rule.message,
                    stage=rule.stage,
                    faq_ref=rule.faq_ref,
                    evidence=_evidence(ctx, rule),
                )
            )
    return flags


def run_spec(env: ReportEnvelope, spec: ReportSpec) -> ReportEnvelope:
    """Apply a spec's rules to ``env`` in place, then recompute the verdict."""
    env.flags.extend(apply_rules(env, spec))
    env.recompute_verdict()
    return env
```

87. user (2026-06-07T07:12:02.345Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\src\obsreport\rules.py
(file state is current in your context ? no need to Read it back)
```

88. assistant (2026-06-07T07:12:06.143Z)

Now export the new symbols and add the shipped spec files:

89. user (2026-06-07T07:12:09.617Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\__init__.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

90. assistant (2026-06-07T07:12:30.669Z)

Tool-specific specs reference that tool's FAQ, so they belong **with each tool** (added in phases A/B). The library ships only an illustrative example spec that doubles as a test fixture:

91. assistant (2026-06-07T07:12:38.349Z)

Illustrative report spec ? demonstrates the config language and doubles as a test fixture. Real tools ship their own spec (referencing their own FAQ) in their own repo; the LANGUAGE + engine + renderers live here in obsreport.

```
name: example
title: Example Pipeline ? Video Report

sections:
  - id: identity
    title: Run
    kind: identity
    audience: both
  - id: media
    title: Video
    kind: video_meta
    audience: both
  - id: highlights
    title: Highlights
    kind: highlights
    audience: customer
  - id: stages
    title: Processing stages
```

```
    kind: stages
    audience: technical
  - id: flags
    title: Findings
    kind: flags
    audience: technical
  - id: verdict
    title: Verdict
    kind: verdict
    audience: both

rules:
  # The canonical footgun: a tool returns 0 segments not because there were no
  # rallies, but because the AI provider silently no-op'd on a missing API key.
  - code: silent_provider_noop
    severity: error
    when:
      - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: false }
      - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
    message: >-
      0 segments AND no API key ? this is almost certainly a silent provider
      no-op, not 'the video has no rallies'. Set the provider API key and re-run.
    faq_ref: "README#Q7"

  # fps was a silent fallback rather than a real probe -> velocity thresholds and
  # frame/second timing may be wrong.
  - code: fps_fallback_used
    severity: warn
    when:
      - { path: "video_meta.fps_source", op: eq, value: fallback }
    message: >-
      fps was assumed (fallback), not probed from the file. Timing and motion
      thresholds may be miscalibrated. Confirm the real fps with ffprobe.
    faq_ref: "README#Q5"
```

92. user (2026-06-07T07:12:38.375Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\specs\example.yaml (file
state is current in your context ? no need to Read it back)
```

93. assistant (2026-06-07T07:12:48.302Z)

```
from pathlib import Path

import pytest

from obsreport import load_spec, load_spec_text
from obsreport.spec import ReportSpec

SPECS_DIR = Path(__file__).resolve().parent.parent / "specs"


def test_shipped_specs_all_validate():
    yamls = list(SPECS_DIR.glob("*.yaml"))
    assert yamls, "expected at least the example spec to ship"
    for p in yamls:
        spec = load_spec(p)
        assert isinstance(spec, ReportSpec)
        assert spec.name


def test_actionable_rule_requires_faq_ref():
    bad = """
    name: bad
    rules:
```

```
      - code: x
        severity: error
        when: [{path: a.b, op: eq, value: 1}]
        message: boom
    """
    with pytest.raises(ValueError, match="faq_ref"):
        load_spec_text(bad)


def test_info_rule_may_omit_faq_ref():
    ok = """
    name: ok
    rules:
      - code: note
        severity: info
        when: [{path: a.b, op: is_present}]
        message: just an fyi
    """
    spec = load_spec_text(ok)
    assert spec.rules[0].faq_ref is None


def test_rule_requires_at_least_one_clause():
    bad = """
    name: bad
    rules:
      - code: x
        severity: info
        when: []
        message: boom
    """
    with pytest.raises(ValueError, match="no conditions"):
        load_spec_text(bad)


def test_stage_section_requires_stage_name():
    bad = """
    name: bad
    sections:
      - {id: s, title: S, kind: stage}
    """
    with pytest.raises(ValueError, match="names no 'stage'"):
        load_spec_text(bad)
```

94. user (2026-06-07T07:12:48.321Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\tests\test_spec.py (file
state is current in your context ? no need to Read it back)
```

95. assistant (2026-06-07T07:13:10.239Z)

```
from pathlib import Path

import pytest

from obsreport import Flag, Identity, ReportEnvelope, Stage, load_spec, run_spec
from obsreport.rules import MISSING, apply_rules, eval_clause, get_path, to_rule_context
from obsreport.spec import Clause

SPECS_DIR = Path(__file__).resolve().parent.parent / "specs"


# --- get_path -------------------------------------------------------
def test_get_path_dict_list_and_missing():
```

```
    data = {"a": {"b": [10, 20, 30]}}
    assert get_path(data, "a.b.1") == 20
    assert get_path(data, "a.b.9") is MISSING
    assert get_path(data, "a.c") is MISSING
    assert get_path(data, "a.b.x") is MISSING  # non-int index into list
```

--- eval_clause truth tables ----------

```
@pytest.mark.parametrize(
    "op,value,actual_path,expected",
    [
        ("eq", 0, "n.zero", True),
        ("eq", 1, "n.zero", False),
        ("ne", 1, "n.zero", True),
        ("gt", 1, "n.five", True),
        ("ge", 5, "n.five", True),
        ("lt", 10, "n.five", True),
        ("le", 4, "n.five", False),
        ("in", [1, 5, 9], "n.five", True),
        ("contains", "ell", "s.word", True),
        ("is_present", None, "n.five", True),
        ("is_missing", None, "n.nope", True),
        ("is_present", None, "n.nope", False),
    ],
)
def test_eval_clause_present(op, value, actual_path, expected):
    ctx = {"n": {"zero": 0, "five": 5}, "s": {"word": "hello"}}
    assert eval_clause(ctx, Clause(path=actual_path, op=op, value=value)) is expected


@pytest.mark.parametrize("op", ["eq", "ne", "gt", "ge", "lt", "le", "in", "contains", "eq_path",
"ne_path"])
def test_missing_path_never_fires_except_is_missing(op):
    ctx = {"present": 1}
    # value chosen to be "true-ish" if it were present; must still be False when missing
    assert eval_clause(ctx, Clause(path="absent", op=op, value="present")) is False


def test_eq_path_and_ne_path():
    ctx = {"a": 5, "b": 5, "c": 9}
    assert eval_clause(ctx, Clause(path="a", op="eq_path", value="b")) is True
    assert eval_clause(ctx, Clause(path="a", op="eq_path", value="c")) is False
    assert eval_clause(ctx, Clause(path="a", op="ne_path", value="c")) is True
    # comparing against a missing other-path is False (can't compare)
    assert eval_clause(ctx, Clause(path="a", op="eq_path", value="missing")) is False


def test_ordered_ops_typeerror_is_false():
    ctx = {"s": "text"}
    assert eval_clause(ctx, Clause(path="s", op="gt", value=3)) is False
```

--- to_rule_context ----------

```
def test_to_rule_context_rekeys_stages_and_metrics():
    env = ReportEnvelope(
        identity=Identity(tool="t"),
        stages=[Stage(name="seg").add_metric("segment_count", 0)],
    )
    ctx = to_rule_context(env)
    assert ctx["stages"]["seg"]["metrics"]["segment_count"] == 0
```

--- apply_rules / run_spec end-to-end ----------

```
def _crafted(api_key_present: bool, segment_count: int) -> ReportEnvelope:
    return ReportEnvelope(
        identity=Identity(tool="indexer"),
        stages=[
            Stage(name="ai_handoff", details={"api_key_present": api_key_present}),
            Stage(name="segmentation").add_metric("segment_count", segment_count),
        ],
    )


def test_silent_provider_noop_fires_and_carries_evidence_and_faq():
    spec = load_spec(SPECS_DIR / "example.yaml")
    env = _crafted(api_key_present=False, segment_count=0)
    flags = apply_rules(env, spec)
    codes = {f.code for f in flags}
    assert "silent_provider_noop" in codes
    flag = next(f for f in flags if f.code == "silent_provider_noop")
    assert flag.severity == "error"
    assert flag.faq_ref == "README#Q7"
    assert flag.evidence["stages.segmentation.metrics.segment_count"] == 0


def test_silent_provider_noop_does_not_fire_when_api_key_present():
    spec = load_spec(SPECS_DIR / "example.yaml")
    env = _crafted(api_key_present=True, segment_count=0)
    assert all(f.code != "silent_provider_noop" for f in apply_rules(env, spec))


def test_run_spec_appends_flags_and_sets_verdict():
    spec = load_spec(SPECS_DIR / "example.yaml")
    env = _crafted(api_key_present=False, segment_count=0)
    run_spec(env, spec)
    assert env.verdict == "fail"
    assert any(f.code == "silent_provider_noop" for f in env.flags)
    assert env.lint() == []  # faq_refs present + verdict consistent
```

96. user (2026-06-07T07:13:10.256Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\tests\test_rules.py (file
state is current in your context ? no need to Read it back)
```

97. user (2026-06-07T07:13:15.187Z)

```
.............................................                            [100%]
49 passed in 0.25s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

98. assistant (2026-06-07T07:13:22.426Z)

C2 green ? 49 passed. Committing, then moving to **C3** (renderers + safe IO).

99. user (2026-06-07T07:13:24.051Z)

```
warning: in the working copy of 'src/obsreport/__init__.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'specs/example.yaml', LF will be replaced by CRLF the next time
Git touches it
warning: in the working copy of 'src/obsreport/rules.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'src/obsreport/spec.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'tests/test_rules.py', LF will be replaced by CRLF the next time
Git touches it
warning: in the working copy of 'tests/test_spec.py', LF will be replaced by CRLF the next time
```

```
Git touches it
918a728 C2: report config language ? spec loader + restricted rule engine
687b52e C1: scaffold obsreport ? typed envelope + deterministic JSON renderer
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

100. user (2026-06-07T07:13:24.304Z)

```
Updated task #2 status
```

101. assistant (2026-06-07T07:14:51.819Z)

```
C2 committed. Now **C3** ? the spec-driven Markdown renderers (technical + customer views from
the *same* envelope) and the never-raising IO layer with the next-to-video path convention.
```

102. user (2026-06-07T07:14:52.189Z)

```
Updated task #3 status
```

103. assistant (2026-06-07T07:15:24.858Z)

```
"""Markdown renderers ? technical and customer views of the SAME envelope.

Section selection/order/titles come from the report spec (or sensible defaults).
The technical view shows everything (stages, all flags with code+evidence+faq_ref);
the customer view shows only video metadata, meta-level highlights, and the
human-readable text of actionable flags ? never internal codes, evidence, or raw
stage dumps. Rendering is pure and deterministic (good for snapshot diffs).
"""
from __future__ import annotations

import os
from typing import List, Optional

from ..envelope import Flag, ReportEnvelope, Stage
from ..spec import ReportSpec, Section

# Default layout when a spec ships no sections. Covers both audiences; the
# renderer filters by the requested audience.
DEFAULT_SECTIONS: List[Section] = [
    Section(id="run", title="Run", kind="identity", audience="both"),
    Section(id="video", title="Video", kind="video_meta", audience="both"),
    Section(id="highlights", title="Highlights", kind="highlights", audience="customer"),
    Section(id="stages", title="Processing stages", kind="stages", audience="technical"),
    Section(id="findings", title="Findings", kind="flags", audience="technical"),
    Section(id="issues", title="Things to know", kind="flags", audience="customer"),
    Section(id="verdict", title="Verdict", kind="verdict", audience="both"),
]

_VERDICT_LABEL = {
    "pass": "PASS",
    "pass_with_warnings": "PASS (with warnings)",
    "fail": "FAIL",
}
_SEVERITY_MARK = {"error": "[ERROR]", "warn": "[WARN]", "info": "[info]"}


# --- small formatters ------------------------------------------------------

def _num(v) -> str:
    if isinstance(v, float):
        return f"{v:.2f}".rstrip("0").rstrip(".")
    return str(v)
```

```
def _secs(v) -> str:
    try:
        f = float(v)
    except (TypeError, ValueError):
        return "?"
    m, s = divmod(int(round(f)), 60)
    return f"{m}m{s:02d}s" if m else f"{f:.1f}s"


def _kv_table(rows: List[tuple]) -> str:
    rows = [(k, v) for k, v in rows if v is not None and v != ""]
    if not rows:
        return "_(none)_"
    out = ["| Field | Value |", "| --- | --- |"]
    out += [f"| {k} | {v} |" for k, v in rows]
    return "\n".join(out)
```

--- per-kind builders ----------

```
def _identity(env: ReportEnvelope, audience: str) -> str:
    i = env.identity
    if audience == "customer":
        name = os.path.basename(i.video_path) if i.video_path else (i.video_url or i.video_id or
"?")
        return _kv_table([("Video", name), ("Report generated", i.generated_at)])
    return _kv_table([
        ("Tool", f"{i.tool} {i.tool_version or ''}".strip()),
        ("Video id", f"{i.video_id} ({i.video_id_kind})" if i.video_id else None),
        ("Video path", i.video_path),
        ("Video url", i.video_url),
        ("Run id", i.run_id),
        ("Generated", i.generated_at),
        ("Schema", i.schema_version),
    ])


def _video_meta(env: ReportEnvelope, audience: str) -> str:
    m = env.video_meta
    if audience == "customer":
        return _kv_table([
            ("Duration", _secs(m.duration_s) if m.duration_s is not None else None),
            ("Resolution", m.resolution or (f"{m.width}x{m.height}" if m.width and m.height else
None)),
            ("Frame rate", f"{_num(m.fps)} fps" if m.fps else None),
        ])
    fps = f"{_num(m.fps)} fps ({m.fps_source})" if m.fps is not None else f"? ({m.fps_source})"
    return _kv_table([
        ("Duration", _secs(m.duration_s) if m.duration_s is not None else None),
        ("Resolution", m.resolution or (f"{m.width}x{m.height}" if m.width and m.height else
None)),
        ("FPS", fps),
        ("Container", m.container),
        ("Size", f"{_num(m.size_mb)} MB" if m.size_mb is not None else None),
    ])


def _highlights(env: ReportEnvelope, audience: str) -> str:
    h = env.highlights
    if not h:
        return "_(no highlights produced)_"
    lines = [
        f"- **{h.segment_count}** highlight{'s' if h.segment_count != 1 else ''}",
        f"- **{_secs(h.total_highlight_s)}** of play",
    ]
```

```
    if h.coverage_pct is not None:
        lines.append(f"- **{_num(h.coverage_pct)}%** of the video is highlights")
    lines += [f"- {n}" for n in h.notes]
    return "\n".join(lines)


def _stage_block(s: Stage, audience: str) -> str:
    head = f"**{s.name}** ? `{s.status}`"
    if s.duration_s is not None:
        head += f" ({_secs(s.duration_s)})"
    lines = [head]
    metrics = [m for m in s.metrics if audience != "customer" or m.audience in ("customer",
"both")]
    for m in metrics:
        unit = f" {m.unit}" if m.unit else ""
        lines.append(f"  - {m.key}: {_num(m.value)}{unit}")
    if audience != "customer":
        for k, v in s.details.items():
            lines.append(f"  - _{k}_: {v}")
    for msg in s.messages:
        lines.append(f"  - {msg}")
    return "\n".join(lines)


def _stages(env: ReportEnvelope, audience: str, only: Optional[str] = None) -> str:
    stages = env.stages if only is None else [s for s in env.stages if s.name == only]
    if not stages:
        return "_(no stages recorded)_"
    return "\n\n".join(_stage_block(s, audience) for s in stages)


def _flags(env: ReportEnvelope, audience: str) -> str:
    if audience == "customer":
        actionable = [f for f in env.flags if f.severity in ("error", "warn")]
        if not actionable:
            return "_Nothing needs your attention._"
        mark = {"error": "?", "warn": "??"}
        return "\n".join(f"- {mark[f.severity]} {f.message}" for f in actionable)
    if not env.flags:
        return "_No findings._"
    blocks = []
    for f in env.flags:
        line = f"- {_SEVERITY_MARK[f.severity]} **{f.code}** ? {f.message}"
        if f.faq_ref:
            line += f"  \n  ? see **{f.faq_ref}**"
        if f.evidence:
            ev = ", ".join(f"{k}={v}" for k, v in f.evidence.items())
            line += f"  \n  ? evidence: {ev}"
        blocks.append(line)
    return "\n".join(blocks)


def _verdict(env: ReportEnvelope, audience: str) -> str:
    return f"**{_VERDICT_LABEL.get(env.verdict, env.verdict)}**"


_BUILDERS = {
    "identity": _identity,
    "video_meta": _video_meta,
    "highlights": _highlights,
    "flags": _flags,
    "verdict": _verdict,
}
```

```
def _build_section(env: ReportEnvelope, section: Section, audience: str) -> str:
    if section.kind == "stages":
        return _stages(env, audience)
    if section.kind == "stage":
        return _stages(env, audience, only=section.stage)
    return _BUILDERS[section.kind](env, audience)


def _render(env: ReportEnvelope, spec: Optional[ReportSpec], audience: str) -> str:
    sections = spec.sections if (spec and spec.sections) else DEFAULT_SECTIONS
    title = (spec.title if spec and spec.title else None) or f"{env.identity.tool} ? video
report"
    if audience == "customer":
        title += " (summary)"
    parts = [f"# {title}", ""]
    for sec in sections:
        if sec.audience not in (audience, "both"):
            continue
        body = _build_section(env, sec, audience)
        parts.append(f"## {sec.title}")
        parts.append("")
        parts.append(body)
        parts.append("")
    return "\n".join(parts).rstrip() + "\n"


def render_technical(env: ReportEnvelope, spec: Optional[ReportSpec] = None) -> str:
    return _render(env, spec, "technical")


def render_customer(env: ReportEnvelope, spec: Optional[ReportSpec] = None) -> str:
    return _render(env, spec, "customer")
```

104. user (2026-06-07T07:15:24.875Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-
obsreport\src\obsreport\render\markdown.py (file state is current in your context ? no need to
Read it back)
```

105. assistant (2026-06-07T07:15:38.237Z)

Now the never-raising IO layer with the path convention:

106. assistant (2026-06-07T07:15:42.303Z)

```
"""File output: the path convention + safe writes.

Convention (next to the video by default):
    /videos/match.mp4  ->  /videos/match.report.json     (machine, source of truth)
                           /videos/match.report.md        (technical, debuggable)
                           /videos/match.summary.md       (customer-friendly)

Overridable with ``output_dir``. For remote/no-local-file runs (e.g. the
collector before download), output falls back to ``output_dir`` or
``reports/<video_id>/``.

THE CARDINAL RULE: writing a report must NEVER break the host pipeline run.
Every write is wrapped; failures are collected into a ``WriteResult`` and
returned, never raised.
"""
from __future__ import annotations

import os
from dataclasses import dataclass, field
```

```
from typing import Dict, List, Optional

from .envelope import ReportEnvelope
from .render.json_render import to_json
from .render.markdown import render_customer, render_technical
from .spec import ReportSpec


@dataclass
class WriteResult:
    ok: bool
    paths: Dict[str, str] = field(default_factory=dict)
    errors: List[str] = field(default_factory=list)


def _base(video_path: Optional[str], output_dir: Optional[str], video_id: Optional[str], stem:
Optional[str]):
    """Return (directory, basename) for the report files."""
    if stem:
        name = stem
    elif video_path:
        name = os.path.splitext(os.path.basename(video_path))[0]
    elif video_id:
        name = video_id
    else:
        name = "report"

    if output_dir:
        return output_dir, name
    if video_path:
        return os.path.dirname(os.path.abspath(video_path)) or ".", name
    # remote / no local file
    return os.path.join("reports", video_id or "run"), name


def report_paths(
    video_path: Optional[str] = None,
    output_dir: Optional[str] = None,
    video_id: Optional[str] = None,
    stem: Optional[str] = None,
) -> Dict[str, str]:
    """Resolve the three report file paths (no IO)."""
    directory, name = _base(video_path, output_dir, video_id, stem)
    return {
        "json": os.path.join(directory, f"{name}.report.json"),
        "technical_md": os.path.join(directory, f"{name}.report.md"),
        "customer_md": os.path.join(directory, f"{name}.summary.md"),
    }


def _write_text(path: str, text: str) -> None:
    """Atomic write: temp file in the same dir, then os.replace. Patchable in tests."""
    directory = os.path.dirname(path) or "."
    os.makedirs(directory, exist_ok=True)
    tmp = f"{path}.tmp"
    with open(tmp, "w", encoding="utf-8", newline="\n") as f:
        f.write(text)
    os.replace(tmp, path)


def write_report(
    envelope: ReportEnvelope,
    spec: Optional[ReportSpec] = None,
    *,
    video_path: Optional[str] = None,
```

```
    output_dir: Optional[str] = None,
    video_id: Optional[str] = None,
    stem: Optional[str] = None,
    write_customer: bool = True,
) -> WriteResult:
    """Render + write json / technical md / (optional) customer md. Never raises."""
    result = WriteResult(ok=True)
    try:
        paths = report_paths(
            video_path=video_path,
            output_dir=output_dir,
            video_id=video_id or envelope.identity.video_id,
            stem=stem,
        )
    except Exception as e:  # pragma: no cover - defensive
        return WriteResult(ok=False, errors=[f"path resolution failed: {e}"])

    jobs = [("json", paths["json"], lambda: to_json(envelope)),
            ("technical_md", paths["technical_md"], lambda: render_technical(envelope, spec))]
    if write_customer:
        jobs.append(("customer_md", paths["customer_md"], lambda: render_customer(envelope,
spec)))

    for key, path, render in jobs:
        try:
            _write_text(path, render())
            result.paths[key] = path
        except Exception as e:
            result.ok = False
            result.errors.append(f"{key}: {e}")
    return result
```

107. user (2026-06-07T07:15:42.315Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\src\obsreport\io.py (file
state is current in your context ? no need to Read it back)
```

108. user (2026-06-07T07:15:47.017Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\__init__.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

109. user (2026-06-07T07:15:51.158Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\__init__.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

110. assistant (2026-06-07T07:16:03.297Z)

Now the C3 tests ? markdown rendering (audience separation) and safe IO (path convention + never-raises):

111. assistant (2026-06-07T07:16:07.375Z)

```
from obsreport import (
    Flag,
    HighlightSummary,
    Identity,
    ReportEnvelope,
    Stage,
    VideoMeta,
    render_customer,
    render_technical,
)
```

```
def _full_env() -> ReportEnvelope:
    return ReportEnvelope(
        identity=Identity(tool="indexer", video_id="abc123", video_id_kind="file_md5",
                          video_path="/videos/match.mp4", run_id="r1",
generated_at="2026-06-07T10:00:00Z"),
        video_meta=VideoMeta(duration_s=754.0, fps=59.94, fps_source="probed",
                             width=1920, height=1080, container="mp4", size_mb=5500.0),
        stages=[
            Stage(name="validation", status="ok").add_metric("checks_passed", 6),
            Stage(name="segmentation", status="ok", details={"segmenter": "wasb_hybrid"})
            .add_metric("segment_count", 12, audience="both"),
        ],
        flags=[
            Flag(code="fps_fallback_used", severity="warn", message="fps was assumed, not
probed.",
                 faq_ref="README#Q5", evidence={"video_meta.fps_source": "fallback"}),
            Flag(code="note_only", severity="info", message="internal note"),
        ],
        highlights=HighlightSummary(segment_count=12, total_highlight_s=251.4,
coverage_pct=33.3),
        verdict="pass_with_warnings",
    )


def test_technical_includes_internals():
    md = render_technical(_full_env())
    assert "segmentation" in md
    assert "wasb_hybrid" in md  # stage detail
    assert "fps_fallback_used" in md  # flag code
    assert "README#Q5" in md  # faq_ref shown
    assert "59.94 fps (probed)" in md  # fps_source surfaced
    assert "Processing stages" in md


def test_customer_strips_internals():
    md = render_customer(_full_env())
    # customer view: no stage dumps, no flag codes, no evidence, no fps_source label
    assert "wasb_hybrid" not in md
    assert "fps_fallback_used" not in md
    assert "README#Q5" not in md
    assert "(probed)" not in md
    assert "Processing stages" not in md
    # but it DOES carry the human-readable warning text + highlights
    assert "fps was assumed, not probed." in md
    assert "12" in md  # highlight count
    assert "Highlights" in md


def test_customer_omits_info_flags_keeps_actionable():
    md = render_customer(_full_env())
    assert "internal note" not in md  # info dropped
    assert "fps was assumed" in md  # warn kept


def test_render_is_deterministic():
    env = _full_env()
    assert render_technical(env) == render_technical(env)
    assert render_customer(env) == render_customer(env)


def test_spec_section_order_and_audience_respected():
    from obsreport import Section, ReportSpec

    spec = ReportSpec(
        name="x",
```

```
        title="Custom Title",
        sections=[
            Section(id="v", title="ZZZ Verdict", kind="verdict", audience="both"),
            Section(id="only_tech", title="Tech Only", kind="stages", audience="technical"),
        ],
    )
    tech = render_technical(_full_env(), spec)
    assert "Custom Title" in tech
    assert tech.index("ZZZ Verdict") < tech.index("Tech Only")  # order honored
    cust = render_customer(_full_env(), spec)
    assert "Tech Only" not in cust  # technical-only section excluded from customer
```

112. user (2026-06-07T07:16:07.404Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-
obsreport\tests\test_render_markdown.py (file state is current in your context ? no need to Read
it back)
```

113. assistant (2026-06-07T07:16:24.489Z)

```
import os

import obsreport.io as io_mod
from obsreport import Identity, ReportEnvelope, report_paths, write_report


def _env():
    return ReportEnvelope(identity=Identity(tool="indexer", video_id="vid42"))


# --- path convention ------------------------------------------------

def test_paths_next_to_video():
    p = report_paths(video_path=os.path.join("X:", "videos", "match.mp4"))
    assert p["json"].endswith(os.path.join("videos", "match.report.json"))
    assert p["technical_md"].endswith("match.report.md")
    assert p["customer_md"].endswith("match.summary.md")


def test_output_dir_override(tmp_path):
    p = report_paths(video_path="/videos/match.mp4", output_dir=str(tmp_path))
    assert p["json"] == os.path.join(str(tmp_path), "match.report.json")


def test_remote_no_local_file_fallback():
    p = report_paths(video_id="ytid")
    assert os.path.join("reports", "ytid", "ytid.report.json") in p["json"]


# --- writing --------------------------------------------------------

def test_write_report_creates_all_three(tmp_path):
    res = write_report(_env(), video_path=str(tmp_path / "clip.mp4"))
    assert res.ok
    for key in ("json", "technical_md", "customer_md"):
        assert os.path.exists(res.paths[key])
    assert (tmp_path / "clip.report.json").read_text(encoding="utf-8").strip().startswith("{")


def test_write_report_can_skip_customer(tmp_path):
    res = write_report(_env(), output_dir=str(tmp_path), write_customer=False)
    assert "customer_md" not in res.paths
    assert not (tmp_path / "report.summary.md").exists()
```

```
def test_write_never_raises_and_reports_errors(tmp_path, monkeypatch):
    def boom(path, text):
        raise OSError("disk full")

    monkeypatch.setattr(io_mod, "_write_text", boom)
    res = write_report(_env(), output_dir=str(tmp_path))  # must NOT raise
    assert res.ok is False
    assert len(res.errors) == 3  # all three writes failed, each captured
    assert any("disk full" in e for e in res.errors)


def test_write_is_atomic_no_tmp_left(tmp_path):
    write_report(_env(), output_dir=str(tmp_path))
    leftovers = [p for p in os.listdir(tmp_path) if p.endswith(".tmp")]
    assert leftovers == []
```

114. user (2026-06-07T07:16:24.505Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\tests\test_io.py (file
state is current in your context ? no need to Read it back)
```

115. user (2026-06-07T07:16:29.398Z)

```
.............................................................            [100%]
61 passed in 0.30s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

116. assistant (2026-06-07T07:16:38.451Z)

```
61 passed. Since report readability is central to this task, let me eyeball an actual rendered
technical + customer report before committing:
```

117. user (2026-06-07T07:16:41.437Z)

```
================ TECHNICAL ================
```

Example Pipeline ? Video Report

Run

Field	Value
Tool	badminton-highlight-indexer 0.1
Video id	9f3a... (file_md5)
Video path	/videos/Adarsh and Vasanth.mp4
Run id	run-001
Generated	2026-06-07T10:00:00Z
Schema	1.0

Video

Field	Value
Duration	11m16s
Resolution	1920x1080
FPS	59.94 fps (probed)
Container	mp4
Size	5500 MB

Processing stages

validation ? `ok`
  - checks_passed: 6

```
  - checks_total: 6

**segmentation** ? `ok`
  - candidate_windows: 64
  - segment_count: 0
  - _segmenter_requested_: wasb_hybrid
  - _segmenter_actual_: wasb_hybrid

**ai_handoff** ? `warn`
  - clips_sent: 0
  - _api_key_present_: False
  - _provider_: gemini
```

Findings

```
- [ERROR] **silent_provider_noop** ? 0 segments AND no API key ? this is almost certainly a
silent provider no-op, not 'the video has no rallies'. Set the provider API key and re-run.
  ? see **README#Q7**
  ? evidence: stages.ai_handoff.details.api_key_present=False,
stages.segmentation.metrics.segment_count=0
```

Verdict

```
**FAIL**

================ CUSTOMER ================
```

Example Pipeline ? Video Report (summary)

Run

```
| Field | Value |
| --- | --- |
| Video | Adarsh and Vasanth.mp4 |
| Report generated | 2026-06-07T10:00:00Z |
```

Video

```
| Field | Value |
| --- | --- |
| Duration | 11m16s |
| Resolution | 1920x1080 |
| Frame rate | 59.94 fps |
```

Highlights

```
- **0** highlights
- **0.0s** of play
- **0%** of the video is highlights
- No highlights were produced for this video.
```

Verdict

```
**FAIL**
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

118. assistant (2026-06-07T07:17:09.665Z)

```
Both views render cleanly. One real UX gap: the **customer** report shows a bare `FAIL` with no
explanation (because a customer shouldn't see raw verdicts, and the example spec lacked a
customer "findings" section). That's exactly the kind of thing the readability bar should catch.
Let me make the verdict audience-aware (friendly for customers) and add a customer-facing
```

findings section to the example spec.

119. user (2026-06-07T07:17:13.118Z)

The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\render\markdown.py has been updated successfully. (file state is current in your context ? no need to Read it back)

120. user (2026-06-07T07:17:18.466Z)

The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\render\markdown.py has been updated successfully. (file state is current in your context ? no need to Read it back)

121. user (2026-06-07T07:17:23.149Z)

The file C:\Users\avidu\Projects\sports-obsreport\specs\example.yaml has been updated successfully. (file state is current in your context ? no need to Read it back)

122. assistant (2026-06-07T07:17:28.539Z)

Let me add a test locking the friendlier customer verdict + customer findings surfacing, then re-run:

123. user (2026-06-07T07:17:29.738Z)

The file C:\Users\avidu\Projects\sports-obsreport\tests\test_render_markdown.py has been updated successfully. (file state is current in your context ? no need to Read it back)

124. user (2026-06-07T07:17:34.672Z)

```
............................................................           [100%]
62 passed in 0.30s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

125. assistant (2026-06-07T07:17:41.547Z)

62 passed. Committing C3.

126. user (2026-06-07T07:17:42.889Z)

```
warning: in the working copy of 'specs/example.yaml', LF will be replaced by CRLF the next time
Git touches it
warning: in the working copy of 'src/obsreport/__init__.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'src/obsreport/io.py', LF will be replaced by CRLF the next time
Git touches it
warning: in the working copy of 'src/obsreport/render/markdown.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'tests/test_io.py', LF will be replaced by CRLF the next time
Git touches it
warning: in the working copy of 'tests/test_render_markdown.py', LF will be replaced by CRLF the
next time Git touches it
911adf7 C3: spec-driven markdown renderers (technical + customer) + safe IO
918a728 C2: report config language ? spec loader + restricted rule engine
687b52e C1: scaffold obsreport ? typed envelope + deterministic JSON renderer
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

127. user (2026-06-07T07:17:43.177Z)

Updated task #3 status

128. user (2026-06-07T07:17:43.398Z)

Updated task #4 status

129. assistant (2026-06-07T07:18:33.262Z)

```
C4 ? the `RunRecorder` (in-flight capture with injected clock for deterministic tests,
exceptions captured not raised), a stable-diff helper for clean cross-run comparison, and a
checked-in JSON Schema generated from the model.
```

130. assistant (2026-06-07T07:18:49.481Z)

```
"""RunRecorder ? capture an envelope incrementally during a pipeline run.

A tool creates a recorder at the top of a run, opens a ``stage(...)`` per pipeline
step (auto-timed; exceptions are captured into the stage, NOT re-raised, so
reporting can't crash the run), fills in video metadata / highlights / manual
flags, then ``finalize()`` (runs the spec's rules + computes the verdict) and
``write()`` (emits the files). The clock is injectable so tests are deterministic.

Tool-specific *post-hoc harvest* (rebuilding an envelope from a DB / artifacts)
lives in each consuming repo ? it just drives this same recorder API.
"""
from __future__ import annotations

import time
from contextlib import contextmanager
from datetime import datetime, timezone
from typing import Any, Callable, Dict, Iterator, Optional

from .envelope import Flag, HighlightSummary, Identity, ReportEnvelope, Stage
from .io import WriteResult, write_report
from .rules import run_spec
from .spec import ReportSpec


def _utc_now_iso() -> str:
    return datetime.now(timezone.utc).strftime("%Y-%m-%dT%H:%M:%SZ")


class RunRecorder:
    def __init__(
        self,
        tool: str,
        *,
        tool_version: Optional[str] = None,
        video_id: Optional[str] = None,
        video_id_kind: str = "unknown",
        video_path: Optional[str] = None,
        video_url: Optional[str] = None,
        run_id: Optional[str] = None,
        spec: Optional[ReportSpec] = None,
        now: Callable[[], str] = _utc_now_iso,
        monotonic: Callable[[], float] = time.perf_counter,
    ) -> None:
        self._now = now
        self._monotonic = monotonic
        self.spec = spec
        self._finalized = False
        self.envelope = ReportEnvelope(
            identity=Identity(
                tool=tool,
                tool_version=tool_version,
                video_id=video_id,
                video_id_kind=video_id_kind,  # type: ignore[arg-type]
                video_path=video_path,
                video_url=video_url,
                run_id=run_id,
                generated_at=now(),
```

```
        )
    )

# --- media / highlights ---
def set_video_meta(self, **kw: Any) -> "RunRecorder":
    self.envelope.video_meta = self.envelope.video_meta.model_copy(update=kw)
    return self

def set_highlights(self, **kw: Any) -> "RunRecorder":
    cur = self.envelope.highlights or HighlightSummary()
    self.envelope.highlights = cur.model_copy(update=kw)
    return self

# --- stages ---
def get_or_create_stage(self, name: str) -> Stage:
    st = self.envelope.get_stage(name)
    if st is None:
        st = Stage(name=name)
        self.envelope.stages.append(st)
    return st

@contextmanager
def stage(self, name: str, *, capture_exceptions: bool = True) -> Iterator[Stage]:
    """Time a pipeline step. On exception, mark the stage 'error' and record the
    message; re-raise only if ``capture_exceptions=False``."""
    st = self.get_or_create_stage(name)
    st.started_at = self._now()
    t0 = self._monotonic()
    try:
        yield st
    except Exception as e:  # noqa: BLE001 - we deliberately swallow to protect the run
        st.status = "error"
        st.messages.append(f"{type(e).__name__}: {e}")
        if not capture_exceptions:
            raise
    finally:
        st.duration_s = round(self._monotonic() - t0, 6)

# --- flags ---
def add_flag(
    self,
    code: str,
    severity: str,
    message: str,
    *,
    faq_ref: Optional[str] = None,
    stage: Optional[str] = None,
    evidence: Optional[Dict[str, Any]] = None,
) -> "RunRecorder":
    self.envelope.flags.append(
        Flag(
            code=code,
            severity=severity,  # type: ignore[arg-type]
            message=message,
            faq_ref=faq_ref,
            stage=stage,
            evidence=evidence or {},
        )
    )
    return self

# --- finalize / write ---
def finalize(self) -> ReportEnvelope:
    """Apply the spec's rules (if any) and compute the verdict. Idempotent."""
    if not self._finalized:
```

```
            if self.spec is not None:
                run_spec(self.envelope, self.spec)
            else:
                self.envelope.recompute_verdict()
            self._finalized = True
        return self.envelope

    def write(self, **io_kwargs: Any) -> WriteResult:
        """Finalize then write all report files. Never raises (see io.write_report)."""
        self.finalize()
        kwargs: Dict[str, Any] = dict(
            video_path=self.envelope.identity.video_path,
            video_id=self.envelope.identity.video_id,
        )
        kwargs.update(io_kwargs)
        return write_report(self.envelope, self.spec, **kwargs)
```

131. user (2026-06-07T07:18:49.500Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\src\obsreport\recorder.py
(file state is current in your context ? no need to Read it back)
```

132. assistant (2026-06-07T07:18:58.198Z)

```
"""Stable serialization for cross-run / cross-tool diffing.

The plain JSON is the machine source of truth, but it carries volatile fields
(timestamps, run id, per-stage durations) that change every run and would swamp a
diff. ``stable_json`` strips those so two runs of the same logical video diff only
where the *substance* differs ? which is the whole point of "evaluate output
differences."
"""
from __future__ import annotations

import json
from typing import Any, Iterable, Set

from .envelope import ReportEnvelope

VOLATILE_KEYS: Set[str] = {"generated_at", "run_id", "started_at", "duration_s"}


def _strip(obj: Any, keys: Set[str]) -> Any:
    if isinstance(obj, dict):
        return {k: _strip(v, keys) for k, v in obj.items() if k not in keys}
    if isinstance(obj, list):
        return [_strip(v, keys) for v in obj]
    return obj


def stable_dict(envelope: ReportEnvelope, extra_volatile: Iterable[str] = ()) -> dict:
    keys = VOLATILE_KEYS | set(extra_volatile)
    return _strip(envelope.model_dump(mode="json"), keys)


def stable_json(envelope: ReportEnvelope, indent: int = 2, extra_volatile: Iterable[str] = ())
-> str:
    return json.dumps(stable_dict(envelope, extra_volatile), indent=indent, sort_keys=True,
ensure_ascii=False)
```

133. user (2026-06-07T07:18:58.228Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\src\obsreport\diff.py
(file state is current in your context ? no need to Read it back)
```

134. user (2026-06-07T07:19:02.523Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\__init__.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

135. user (2026-06-07T07:19:07.064Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\__init__.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

136. assistant (2026-06-07T07:19:14.017Z)

Now the schema-generation script and the C4 tests (recorder, diff, schema currency):

137. assistant (2026-06-07T07:19:14.831Z)

```
"""Regenerate the checked-in JSON Schema for the report envelope.

Run after changing envelope.py:  python scripts/gen_schema.py
The schema is the published contract; test_schema.py fails if it drifts.
"""
import json
import sys
from pathlib import Path

ROOT = Path(__file__).resolve().parent.parent
sys.path.insert(0, str(ROOT / "src"))

from obsreport.envelope import ReportEnvelope  # noqa: E402

OUT = ROOT / "schema" / "report_envelope.schema.json"


def schema_text() -> str:
    return json.dumps(ReportEnvelope.model_json_schema(), indent=2, sort_keys=True) + "\n"


def main() -> None:
    OUT.parent.mkdir(parents=True, exist_ok=True)
    OUT.write_text(schema_text(), encoding="utf-8")
    print(f"wrote {OUT}")


if __name__ == "__main__":
    main()
```

138. user (2026-06-07T07:19:14.843Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\scripts\gen_schema.py
(file state is current in your context ? no need to Read it back)
```

139. user (2026-06-07T07:19:18.868Z)

```
wrote C:\Users\avidu\Projects\sports-obsreport\schema\report_envelope.schema.json
{
  "$defs": {
    "Flag": {
      "description": "A derived warning/error \u2014 the debugging intelligence of the
report.\n\n`faq_ref` points at the README/FAQ section a reader should open to self-debug\n(e.g.
``\"README#Q7\"``). It is REQUIRED for error/warn severities; this is\nenforced by
`ReportEnvelope.lint`, not by the constructor, so harvest code can\nbuild a flag incrementally
before deciding its citation.",
      "properties": {
        "code": {
```

```
          "title": "Code",
          "type": "string"
        },
        "evidence": {
          "additionalProperties": true,
          "title": "Evidence",
          "type": "object"
        },
        "faq_ref": {
          "anyOf": [
            {
              "type": "string"
            },
            {
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

140. assistant (2026-06-07T07:19:32.842Z)

```
from pathlib import Path

from obsreport import RunRecorder, load_spec

SPECS_DIR = Path(__file__).resolve().parent.parent / "specs"


def _clock():
    """Deterministic monotonic clock: 0.0, 1.0, 2.0, ... per call."""
    state = {"t": 0.0}

    def tick():
        v = state["t"]
        state["t"] += 1.0
        return v

    return tick


def _recorder(**kw):
    return RunRecorder("indexer", now=lambda: "2026-01-01T00:00:00Z", monotonic=_clock(), **kw)


def test_stage_times_and_records_metrics():
    rec = _recorder()
    with rec.stage("segmentation") as s:
        s.add_metric("segment_count", 3, audience="both")
    env = rec.finalize()
    seg = env.get_stage("segmentation")
    assert seg.status == "ok"
    assert seg.started_at == "2026-01-01T00:00:00Z"
    assert seg.duration_s == 1.0  # one tick elapsed
    assert seg.metrics[0].value == 3


def test_stage_captures_exception_without_raising():
    rec = _recorder()
    with rec.stage("boom") as s:  # must NOT propagate
        raise RuntimeError("kaboom")
    env = rec.finalize()
    st = env.get_stage("boom")
    assert st.status == "error"
    assert any("kaboom" in m for m in st.messages)


def test_stage_can_reraise_when_asked():
    rec = _recorder()
```

```
    raised = False
    try:
        with rec.stage("boom", capture_exceptions=False):
            raise ValueError("x")
    except ValueError:
        raised = True
    assert raised
    assert rec.envelope.get_stage("boom").status == "error"


def test_finalize_runs_spec_rules_and_sets_verdict():
    spec = load_spec(SPECS_DIR / "example.yaml")
    rec = _recorder(spec=spec)
    with rec.stage("ai_handoff") as s:
        s.details["api_key_present"] = False
    with rec.stage("segmentation") as s:
        s.add_metric("segment_count", 0)
    env = rec.finalize()
    assert env.verdict == "fail"
    assert any(f.code == "silent_provider_noop" for f in env.flags)
    assert env.lint() == []


def test_finalize_is_idempotent():
    spec = load_spec(SPECS_DIR / "example.yaml")
    rec = _recorder(spec=spec)
    with rec.stage("ai_handoff") as s:
        s.details["api_key_present"] = False
    with rec.stage("segmentation") as s:
        s.add_metric("segment_count", 0)
    rec.finalize()
    n = len(rec.envelope.flags)
    rec.finalize()  # second call must not duplicate flags
    assert len(rec.envelope.flags) == n


def test_write_emits_files_next_to_video(tmp_path):
    rec = _recorder(video_id="abc", video_path=str(tmp_path / "match.mp4"))
    rec.set_video_meta(fps=60.0, fps_source="probed")
    rec.set_highlights(segment_count=2, total_highlight_s=10.0)
    res = rec.write()
    assert res.ok
    assert (tmp_path / "match.report.json").exists()
    assert (tmp_path / "match.summary.md").exists()
```

141. user (2026-06-07T07:19:32.859Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\tests\test_recorder.py
(file state is current in your context ? no need to Read it back)
```

142. assistant (2026-06-07T07:19:42.058Z)

```
from obsreport import Identity, ReportEnvelope, Stage, stable_json


def _env(run_id, started, dur, segs):
    return ReportEnvelope(
        identity=Identity(tool="indexer", video_id="same", run_id=run_id,
generated_at="2026-01-01T00:00:00Z"),
        stages=[Stage(name="seg", started_at=started,
duration_s=dur).add_metric("segment_count", segs)],
    )


def test_stable_json_ignores_volatile_fields():
```

```
    a = _env(run_id="r1", started="t1", dur=1.0, segs=5)
    b = _env(run_id="r2", started="t2", dur=9.9, segs=5)  # same substance, different timing/ids
    assert stable_json(a) == stable_json(b)


def test_stable_json_detects_real_change():
    a = _env(run_id="r1", started="t1", dur=1.0, segs=5)
    b = _env(run_id="r1", started="t1", dur=1.0, segs=6)  # segment count differs
    assert stable_json(a) != stable_json(b)
```

143. user (2026-06-07T07:19:42.077Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\tests\test_diff.py (file
state is current in your context ? no need to Read it back)
```

144. assistant (2026-06-07T07:19:47.741Z)

```
"""The checked-in JSON Schema is the published contract; guard it from drift."""
import json
from pathlib import Path

from obsreport.envelope import ReportEnvelope

SCHEMA = Path(__file__).resolve().parent.parent / "schema" / "report_envelope.schema.json"


def test_checked_in_schema_matches_model():
    assert SCHEMA.exists(), "run scripts/gen_schema.py"
    on_disk = json.loads(SCHEMA.read_text(encoding="utf-8"))
    current = ReportEnvelope.model_json_schema()
    assert on_disk == current, "schema drift ? run `python scripts/gen_schema.py` and commit"
```

145. user (2026-06-07T07:19:47.753Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-obsreport\tests\test_schema.py
(file state is current in your context ? no need to Read it back)
```

146. user (2026-06-07T07:19:53.312Z)

```
.............................................................  [100%]
71 passed in 0.37s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

147. assistant (2026-06-07T07:20:18.372Z)

```
71 passed, schema guarded against drift. Let me commit C4, then verify the package actually
installs and imports cleanly from an installed context (the cross-repo integration depends on
this) before tagging.
```

148. user (2026-06-07T07:20:32.399Z)

```
Exit code 1
warning: in the working copy of 'src/obsreport/__init__.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'scripts/gen_schema.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'src/obsreport/diff.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'src/obsreport/recorder.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_diff.py', LF will be replaced by CRLF the next time
Git touches it
warning: in the working copy of 'tests/test_recorder.py', LF will be replaced by CRLF the next
time Git touches it
```

```
warning: in the working copy of 'tests/test_schema.py', LF will be replaced by CRLF the next
time Git touches it
d3e2a51 C4: RunRecorder + stable-diff + JSON Schema contract
911adf7 C3: spec-driven markdown renderers (technical + customer) + safe IO
918a728 C2: report config language ? spec loader + restricted rule engine
687b52e C1: scaffold obsreport ? typed envelope + deterministic JSON renderer
--- editable install + import smoke ---
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\pip\_vendor\pyproject_hooks\_impl.py", line 402, in _call_hook
    raise BackendUnavailable(
    ...<4 lines>...
    )
pip._vendor.pyproject_hooks._impl.BackendUnavailable: Cannot import 'setuptools.build-meta'
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import obsreport; print('obsreport', obsreport.__version__, 'schema',
obsreport.SCHEMA_VERSION); print('exports ok:', all(hasattr(obsreport,n) for n in ['RunRecorder'
,'write_report','run_spec','render_technical','render_customer','load_spec','stable_json']))
    ^^^^^^^^^^^^^^^^
ModuleNotFoundError: No module named 'obsreport'
```

149. assistant (2026-06-07T07:20:39.149Z)

C4 committed. The editable install hit a build-backend error (likely pip build isolation can't fetch setuptools offline). Let me diagnose and use `--no-build-isolation`:

150. user (2026-06-07T07:20:43.011Z)

```
setuptools 81.0.0
pip 26.0.1
--- retry editable, no build isolation ---
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\pip\_vendor\pyproject_hooks\_impl.py", line 180, in _supported_features
    return self._call_hook("_supported_features", {})
           ~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\site-
packages\pip\_vendor\pyproject_hooks\_impl.py", line 402, in _call_hook
    raise BackendUnavailable(
    ...<4 lines>...
    )
pip._vendor.pyproject_hooks._impl.BackendUnavailable: Cannot import 'setuptools.build-meta'
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

151. assistant (2026-06-07T07:20:59.311Z)

Found it ? my `build-backend` had a hyphen (`setuptools.build-meta`) instead of the correct underscore (`setuptools.build_meta`). (Notably the collector's `pyproject.toml` has the same typo ? a latent install bug there worth flagging.) Let me fix and retry:

152. user (2026-06-07T07:21:00.511Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\pyproject.toml has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

153. user (2026-06-07T07:21:10.070Z)

```
[notice] A new release of pip is available: 26.0.1 -> 26.1.2
[notice] To update, run: python.exe -m pip install --upgrade pip
obsreport 0.1.0 schema 1.0
exports ok: True
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

154. assistant (2026-06-07T07:21:17.750Z)

Editable install + import both work now. Let me confirm the suite still passes, then commit the fix and tag v0.1.0.

155. user (2026-06-07T07:21:20.931Z)

```
Exit code 128
.............................................................  [100%]
71 passed in 0.37s
warning: in the working copy of 'pyproject.toml', LF will be replaced by CRLF the next time Git touches it
Committer identity unknown

*** Please tell me who you are.

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: unable to auto-detect email address (got 'avidu@AVIS-MSI.(none)')
```

156. assistant (2026-06-07T07:21:32.180Z)

The commit landed but the annotated tag needs a committer identity. Let me set the repo-local git identity (so future commits/tags are clean) and create the tag:

157. user (2026-06-07T07:21:34.091Z)

```
7c40654 fix: correct setuptools build-backend (underscore) so editable/pip install works
tags:
v0.1.0
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

158. assistant (2026-06-07T07:21:58.695Z)

Phase **C is complete**: the `obsreport` library is built (71 tests green), editable-installed, and tagged **v0.1.0** locally. This is a meaningful checkpoint (a whole new repo that isn't on GitHub yet) ? let me capture it in memory before starting integration, so a restart can resume cleanly.

159. user (2026-06-07T07:22:06.309Z)

```
The file c:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)
```

160. assistant (2026-06-07T07:22:36.695Z)

Phase C done. Marking C4 complete and starting **Phase A (indexer)**. First, branch off master and confirm the 209-test baseline is green before touching anything.

161. user (2026-06-07T07:22:37.219Z)

```
Updated task #4 status
```

162. user (2026-06-07T07:22:37.804Z)

```
Updated task #5 status
```

163. user (2026-06-07T07:22:56.050Z)

```
Switched to a new branch 'feat/observability-reports'
.................................................................... [ 34%]
.................................................................... [ 68%]
.............................................................        [100%]
209 passed in 15.39s
```

164. assistant (2026-06-07T07:23:07.830Z)

Baseline green (209). **A1** ? add the backward-compatible `run_all_with_context` so the report can capture the `ffprobe_meta` that `run_all` currently discards:

165. user (2026-06-07T07:23:11.020Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\validators\base.py has been updated successfully. (file state is current in your context ? no need to Read it back)

166. assistant (2026-06-07T07:23:21.806Z)

```
"""Tests for the validation registry, focused on the backward-compatible
`run_all_with_context` added for observability reporting."""
from backend.validators.base import ValidationRegistry, ValidationResult, VideoValidator


class _ProducerValidator(VideoValidator):
    @property
    def name(self):
        return "producer"

    @property
    def description(self):
        return "writes ffprobe_meta into the shared context"

    def validate(self, video_path, config, context):
        context["ffprobe_meta"] = {"fps": 59.94, "width": 1920, "height": 1080}
        return ValidationResult(validator_name=self.name, passed=True, message="ok")


class _ConsumerValidator(VideoValidator):
    @property
    def name(self):
        return "consumer"

    @property
    def description(self):
        return "reads ffprobe_meta the producer cached"

    def validate(self, video_path, config, context):
        seen = "ffprobe_meta" in context
        return ValidationResult(validator_name=self.name, passed=seen,
message=f"saw_meta={seen}")


def _registry():
    r = ValidationRegistry()
    r.register(_ProducerValidator())
    r.register(_ConsumerValidator())
    return r


def test_run_all_with_context_returns_results_and_context():
    results, context = _registry().run_all_with_context("v.mp4", {})
    assert [r.validator_name for r in results] == ["producer", "consumer"]
    assert context["ffprobe_meta"]["fps"] == 59.94
    assert results[1].passed is True  # consumer saw the shared context
```

```
def test_run_all_is_unchanged_and_returns_only_results():
    results = _registry().run_all("v.mp4", {})
    assert isinstance(results, list)
    assert all(isinstance(r, ValidationResult) for r in results)
    assert [r.validator_name for r in results] == ["producer", "consumer"]


def test_validator_exception_is_captured_not_raised():
    class _Boom(VideoValidator):
        @property
        def name(self):
            return "boom"

        @property
        def description(self):
            return "raises"

        def validate(self, video_path, config, context):
            raise RuntimeError("explode")

    r = ValidationRegistry()
    r.register(_Boom())
    results, _ = r.run_all_with_context("v.mp4", {})
    assert results[0].passed is False
    assert "explode" in results[0].message
```

167. user (2026-06-07T07:23:21.817Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\tests\test_validators_base.py (file state is current in your context ? no need to Read
it back)
```

168. assistant (2026-06-07T07:23:42.604Z)

```
A1 looks good. Now I need exact shapes before writing the harvest layer ? let me read the
ffprobe-metadata producer, the candidate-query API, and the segment/DB shapes so the harvest is
accurate, not guessed.
```

169. user (2026-06-07T07:23:43.107Z)

```
1       import subprocess
2       import json
3       import os
4       from backend.validators.base import VideoValidator, ValidationResult
5       from typing import Dict, Any
6
7       class FormatValidator(VideoValidator):
8           @property
9           def name(self) -> str:
10              return "format_check"
11
12          @property
13          def description(self) -> str:
14              return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
15
16          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
17              if not os.path.exists(video_path):
18                  return ValidationResult(
19                      validator_name=self.name,
20                      passed=False,
21                      message=f"File not found: {video_path}"
22                  )
```

```
23
24              cmd = [
25                  "ffprobe",
26                  "-v", "error",
27                  "-select_streams", "v:0",
28                  "-show_format",
29                  "-show_streams",
30                  "-of", "json",
31                  video_path
32              ]
33
34              try:
35                  result = subprocess.run(cmd, capture_output=True, text=True, check=True)
36                  info = json.loads(result.stdout)
37
38                  if not info.get("streams") or not info.get("format"):
39                      return ValidationResult(
40                          validator_name=self.name,
41                          passed=False,
42                          message="Failed to retrieve valid video streams from the file."
43                      )
44
45                  # Store the metadata in context for subsequent validators to utilize
46                  context["ffprobe_meta"] = info
47
48                  stream = info["streams"][0]
49                  fmt = info["format"]
50
51                  details = {
52                      "format_name": fmt.get("format_name"),
53                      "duration": float(fmt.get("duration", 0)),
54                      "size_bytes": int(fmt.get("size", 0)),
55                      "codec_name": stream.get("codec_name"),
56                  }
57
58                  # Check if container name includes mp4
59                  if "mp4" not in fmt.get("format_name", "").lower():
60                      return ValidationResult(
61                          validator_name=self.name,
62                          passed=False,
63                          message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",
64                          details=details
65                      )
66
67                  return ValidationResult(
68                      validator_name=self.name,
69                      passed=True,
70                      message="Video format is valid MP4 container.",
71                      details=details
72                  )
73
74              except subprocess.CalledProcessError as e:
75                  return ValidationResult(
76                      validator_name=self.name,
77                      passed=False,
78                      message="Video stream is corrupted or unreadable by ffprobe.",
79                      details={"error": e.stderr}
80                  )
81              except Exception as e:
82                  return ValidationResult(
83                      validator_name=self.name,
84                      passed=False,
85                      message=f"Unexpected error running ffprobe: {str(e)}"
86                  )
```

87

170. user (2026-06-07T07:23:43.497Z)

```
1       from backend.validators.base import VideoValidator, ValidationResult
2       from typing import Dict, Any
3
4       class ResolutionFPSValidator(VideoValidator):
5           @property
6           def name(self) -> str:
7               return "resolution_fps_check"
8
9           @property
10          def description(self) -> str:
11              return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."
12
13          def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
14              meta = context.get("ffprobe_meta")
15              if not meta or not meta.get("streams"):
16                  return ValidationResult(
17                      validator_name=self.name,
18                      passed=False,
19                      message="Video metadata is missing. Run format_check validator first."
20                  )
21
22              stream = meta["streams"][0]
23              width = int(stream.get("width", 0))
24              height = int(stream.get("height", 0))
25
26              # Parse frame rate (e.g. "30/1" or "30000/1001")
27              fps_str = stream.get("r_frame_rate", "0/1")
28              try:
29                  num, den = map(int, fps_str.split("/"))
30                  fps = num / den if den != 0 else 0
31              except ValueError:
32                  fps = 0.0
33
34              details = {
35                  "width": width,
36                  "height": height,
37                  "fps": round(fps, 2)
38              }
39
40              # Check bounds from config
41              val_cfg = config.get("validation", {})
42              min_w = val_cfg.get("min_resolution_width", 1280)
43              min_h = val_cfg.get("min_resolution_height", 720)
44              min_fps = val_cfg.get("min_fps", 29.9)
45
46              if width < min_w or height < min_h:
47                  return ValidationResult(
48                      validator_name=self.name,
49                      passed=False,
50                      message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
51                      details=details
52                  )
53
54              if fps < min_fps:
55                  return ValidationResult(
56                      validator_name=self.name,
57                      passed=False,
58                      message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
```

```
{min_fps} FPS.",
                    details=details
                )

            return ValidationResult(
                validator_name=self.name,
                passed=True,
                message="Video resolution and frame rate meet minimum standards.",
                details=details
            )

```

171. assistant (2026-06-07T07:23:55.059Z)

`ffprobe_meta` is the full ffprobe JSON (`streams[0]` ? width/height/r_frame_rate/codec_name; `format` ? format_name/duration/size). Now the DB query API + segment shapes:

172. user (2026-06-07T07:23:56.192Z)

```
import sqlite3
import json
import os
import logging
from typing import List, Dict, Any, Optional

logger = logging.getLogger(__name__)

class Database:
    def __init__(self, db_path: str):
        self.db_path = db_path
        self._init_db()

    def _get_connection(self):
        conn = sqlite3.connect(self.db_path)
        conn.row_factory = sqlite3.Row
        # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
        conn.execute("PRAGMA foreign_keys = ON")
        return conn

    def _init_db(self):
        """Initializes tables for videos, play_segments, and events."""
        with self._get_connection() as conn:
            cursor = conn.cursor()

            # Videos Table
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS videos (
                    id TEXT PRIMARY KEY,
                    filepath TEXT NOT NULL,
                    processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                    status TEXT NOT NULL,
                    validation_results TEXT
                )
            """)

            # Play Segments Table (Extensible metadata JSON)
            cursor.execute("""
                CREATE TABLE IF NOT EXISTS play_segments (
                    id INTEGER PRIMARY KEY AUTOINCREMENT,
                    video_id TEXT NOT NULL,
                    start_time REAL NOT NULL,
                    end_time REAL NOT NULL,
                    duration REAL NOT NULL,
                    server TEXT,
                    receiver TEXT,
```

```
                        metadata TEXT,
                        FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
                    )
                """)

                # Events Table
                cursor.execute("""
                    CREATE TABLE IF NOT EXISTS events (
                        id INTEGER PRIMARY KEY AUTOINCREMENT,
                        segment_id INTEGER NOT NULL,
                        timestamp REAL NOT NULL,
                        type TEXT NOT NULL,
                        player TEXT,
                        details TEXT,
                        FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
                    )
                """)

                # Versioned migrations live here. PRAGMA user_version is the schema
                # contract ? bump it for every change and add an ordered `if version < N`
                # block below. Tests assert the expected version, so accidental drift fails
CI.
                version = cursor.execute("PRAGMA user_version").fetchone()[0]

                if version < 1:
                    # v1: raw candidate detections (pre-confirmation), detector-agnostic.
                    # Common fields are columns; detector-specific metrics go in `stats`
JSON,
                    # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
                    cursor.execute("""
                        CREATE TABLE IF NOT EXISTS candidate_segments (
                            id INTEGER PRIMARY KEY AUTOINCREMENT,
                            video_id TEXT NOT NULL,
                            detector TEXT NOT NULL,
                            chunk_index INTEGER,
                            start_time REAL NOT NULL,
                            end_time REAL NOT NULL,
                            duration REAL NOT NULL,
                            confidence REAL,
                            stats TEXT,
                            detection_params TEXT,
                            confirmed_segment_id INTEGER,
                            human_label TEXT,
                            notes TEXT,
                            created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
                            FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
                            FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
                        )
                    """)
                    cursor.execute("""
                        CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
                            ON candidate_segments(video_id, detector)
                    """)
                    cursor.execute("PRAGMA user_version = 1")
                # future: if version < 2: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 2

                conn.commit()

        def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
            try:
```

```
                with self._get_connection() as conn:
                    conn.cursor().execute(
                        "INSERT OR REPLACE INTO videos (id, filepath, status,
validation_results) VALUES (?, ?, ?, ?)",
                        (video_id, filepath, status, json.dumps(validation_results))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add video: {e}")
                return False

        def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
            try:
                with self._get_connection() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
                        (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add play segment: {e}")
                return None

        def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
            try:
                with self._get_connection() as conn:
                    conn.cursor().execute(
                        "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
                        (segment_id, timestamp, event_type, player, json.dumps(details or
{}))
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to add event: {e}")
                return False

        def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
                                  chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
                                  stats: Optional[Dict[str, Any]] = None,
                                  detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
            """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
            detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
            try:
                with self._get_connection() as conn:
                    cursor = conn.cursor()
                    cursor.execute(
                        """INSERT INTO candidate_segments
                           (video_id, detector, chunk_index, start_time, end_time, duration,
                            confidence, stats, detection_params)
                           VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)""",
                        (video_id, detector, chunk_index, start_time, end_time, end_time -
```

```
start_time,
                        confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
                    )
                    conn.commit()
                    return cursor.lastrowid
            except Exception as e:
                logger.error(f"Failed to add candidate segment: {e}")
                return None

        def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
            """Record that a candidate detection was confirmed as a given play_segment."""
            try:
                with self._get_connection() as conn:
                    conn.cursor().execute(
                        "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
                        (segment_id, candidate_id)
                    )
                    conn.commit()
                return True
            except Exception as e:
                logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
                return False

        def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
                                     only_unlabeled: bool = False) -> List[Dict[str, Any]]:
            """Fetch candidate detections for the annotation / fine-tuning workflow.
            `stats` and `detection_params` are deserialized from JSON."""
            query = "SELECT * FROM candidate_segments WHERE video_id = ?"
            params: List[Any] = [video_id]

            if detector:
                query += " AND detector = ?"
                params.append(detector)
            if only_unlabeled:
                query += " AND human_label IS NULL"

            query += " ORDER BY start_time"

            candidates = []
            with self._get_connection() as conn:
                rows = conn.cursor().execute(query, params).fetchall()
                for row in rows:
                    d = dict(row)
                    d["stats"] = json.loads(d["stats"] or "{}")
                    d["detection_params"] = json.loads(d["detection_params"] or "{}")
                    candidates.append(d)
            return candidates

        def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
            with self._get_connection() as conn:
                row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
                if row:
                    d = dict(row)
                    d["validation_results"] = json.loads(d["validation_results"] or "[]")
                    return d
            return None

        def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
            """
            Dynamically queries play segments based on filters:
```

```
            - duration_min: float
            - server: string
            - receiver: string
            - event_type: string (e.g. smash, foul)
            """
            query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
            params = [video_id]

            if filters.get("duration_min"):
                query += " AND r.duration >= ?"
                params.append(filters["duration_min"])

            if filters.get("server"):
                query += " AND r.server = ?"
                params.append(filters["server"])

            if filters.get("receiver"):
                query += " AND r.receiver = ?"
                params.append(filters["receiver"])

            if filters.get("event_type"):
                # Check if there is an event of a specific type inside this segment
                query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
                params.append(filters["event_type"])

            segments_list = []
            with self._get_connection() as conn:
                cursor = conn.cursor()
                rows = cursor.execute(query, params).fetchall()

                for row in rows:
                    r_dict = dict(row)
                    r_id = r_dict["id"]
                    r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")

                    # Fetch associated events
                    event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
                    r_dict["events"] = []
                    for er in event_rows:
                        e_dict = dict(er)
                        e_dict["details"] = json.loads(e_dict["details"] or "{}")
                        r_dict["events"].append(e_dict)

                    segments_list.append(r_dict)

            return segments_list

        def clear_video_data(self, video_id: str):
            with self._get_connection() as conn:
                conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
                conn.commit()

```

173. assistant (2026-06-07T07:25:25.173Z)

Now I have the exact DB API and `ffprobe_meta` shape. Let me build the indexer's reporting module: a **soft-import** wrapper (so the pipeline never breaks if `obsreport` is absent), a pure harvest layer, and the indexer's own report spec (footgun rules citing its FAQ). Starting with the spec:

174. assistant (2026-06-07T07:25:47.921Z)

Observability report spec for the badminton-highlight-indexer. The report ENGINE (config language, renderers) lives in the shared `obsreport` library; this file is the indexer-specific layout + footgun rules. Every actionable rule cites a README/FAQ section so a reader can self-debug.

```
name: indexer
title: Badminton Highlight Indexer ? Video Report

sections:
  - { id: run,        title: "Run",                kind: identity,   audience: both }
  - { id: video,      title: "Video",              kind: video_meta, audience: both }
  - { id: highlights, title: "Highlights",         kind: highlights, audience: customer }
  - { id: cust_notes, title: "Things to know",     kind: flags,      audience: customer }
  - { id: stages,     title: "Processing stages",  kind: stages,     audience: technical }
  - { id: findings,   title: "Findings & warnings", kind: flags,     audience: technical }
  - { id: verdict,    title: "Verdict",            kind: verdict,    audience: both }

rules:
  # The classic silent failure: 0 segments because the AI provider no-op'd on a
  # missing API key ? NOT because the video has no rallies.
  - code: silent_provider_noop
    severity: error
    when:
      - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: false }
      - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
    message: >-
      0 segments AND no AI provider API key was set ? this is almost certainly a
      silent provider no-op, not 'the video has no rallies'. Set the provider key
      (e.g. GEMINI_API_KEY in a .env file) and re-run.
    faq_ref: "README#Q7"

  # API key WAS present but an AI segmenter still produced nothing ? likely a
  # provider error / quota / WSL healthcheck issue rather than a truly empty video.
  - code: ai_empty_vs_no_rallies
    severity: warn
    when:
      - { path: "stages.ai_handoff.details.ai_based", op: eq, value: true }
      - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: true }
      - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
    message: >-
      An AI-based segmenter returned 0 segments even though the API key is set.
      Empty can mean a provider error, exhausted quota, or (for WASB/TrackNet) a
      failed WSL healthcheck ? these all return [] silently. Re-run with -u for
      unbuffered logs and check the provider/WSL output before concluding 'no rallies'.
    faq_ref: "README#Q7"

  # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
  - code: fps_fallback_used
    severity: warn
    when:
      - { path: "video_meta.fps_source", op: eq, value: fallback }
    message: >-
      fps was assumed (a fallback default), not probed from the file. Motion
      thresholds and frame->second timing may be miscalibrated. Confirm the real
      fps with `ffprobe` and ensure validation ran (it probes the file).
    faq_ref: "README#Q5"

  # The motion baseline fabricates its per-segment metadata/events ? they are not
  # measured. Warn so nobody trusts them as ground truth.
  - code: synthetic_motion_metadata
    severity: warn
    when:
      - { path: "stages.segmentation.details.segmenter_actual", op: eq, value: motion }
```

```
    message: >-
      The 'motion' segmenter was used. Its per-segment metadata (intensity, speed,
      events, server/receiver) is HEURISTIC / synthetic, not measured ? do not
      treat it as ground truth. Use yolo_hybrid/gemini for real shot metadata.
    faq_ref: "README#Q10"

  # The segmenter that ran differs from config's default -> you may be looking at a
  # different detector than expected (config/setup/CLI defaults are known to diverge).
  - code: segmenter_divergence
    severity: info
    when:
      - { path: "stages.segmentation.details.matches_config_default", op: eq, value: false }
    message: >-
      The segmenter that ran differs from config.json's default_segmenter. This is
      fine if intentional (CLI/API override), but the indexer's defaults are known
      to diverge across config.json / setup.py / the CLI ? confirm you ran the one
      you meant.
    faq_ref: "README#Q7"

  # Re-processing an already-indexed video replaced its prior segments (INSERT OR
  # REPLACE cascades). Note it so a changed result isn't mistaken for instability.
  - code: reingest_replaced_prior
    severity: info
    when:
      - { path: "stages.segmentation.details.was_reingest", op: eq, value: true }
    message: >-
      This video was already in the database; re-processing REPLACED its previous
      segments (videos are keyed by file MD5; re-ingest cascades a delete). Expected
      when you intend to re-run; just know the earlier results are gone.
    faq_ref: "README#Q11"

  # A validation check failed -> segmentation was halted. Point the reader at the
  # report's Validation section + the per-check FAQ.
  - code: validation_failed
    severity: error
    when:
      - { path: "stages.validation.status", op: eq, value: error }
    message: >-
      One or more validation checks failed, so indexing was halted. See the
      Processing stages > validation block above: each failed check states the
      reason (relevance -> README#Q2, fps/keyframes -> README#Q5).
    faq_ref: "README#Q13"
```

175. user (2026-06-07T07:25:47.953Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\reporting\spec.yaml (file state is current in your context ? no need to Read it
back)
```

176. assistant (2026-06-07T07:26:13.506Z)

Now the pure harvest layer that records raw signals into a recorder:

177. assistant (2026-06-07T07:26:22.611Z)

```
"""Build an observability report for one indexer run.

Pure-ish harvest: given the validation results/context, the persisted segments,
and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
*footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
so this module only records facts faithfully ? it does not decide warnings.

Imported only after `backend.reporting` confirms obsreport is installed, so it may
import obsreport freely.
```

```
"""
from __future__ import annotations

import os
from pathlib import Path
from typing import Any, Dict, List, Optional

import obsreport

_AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
_SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"


def load_spec() -> "obsreport.ReportSpec":
    return obsreport.load_spec(_SPEC_PATH)
```

--- media metadata ----------

```
def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path: Optional[str]) ->
Dict[str, Any]:
    """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
    distinguishes a real probe from the absence of one (the fallback footgun)."""
    meta: Dict[str, Any] = {"fps_source": "unknown"}
    if video_path and os.path.exists(video_path):
        try:
            meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
        except OSError:
            pass

    ffmeta = (val_context or {}).get("ffprobe_meta") or {}
    streams = ffmeta.get("streams") or []
    fmt = ffmeta.get("format") or {}
    if streams:
        s = streams[0]
        w, h = s.get("width"), s.get("height")
        meta["width"], meta["height"] = w, h
        if w and h:
            meta["resolution"] = f"{w}x{h}"
        fps_str = s.get("r_frame_rate", "0/1")
        try:
            num, den = (int(x) for x in fps_str.split("/"))
            if den:
                meta["fps"] = round(num / den, 2)
                meta["fps_source"] = "probed"
        except (ValueError, ZeroDivisionError):
            pass
        if s.get("codec_name"):
            meta["container"] = fmt.get("format_name") or s.get("codec_name")
    if fmt.get("duration"):
        try:
            meta["duration_s"] = round(float(fmt["duration"]), 1)
        except (ValueError, TypeError):
            pass
    if fmt.get("format_name"):
        meta["container"] = fmt.get("format_name")
    return meta
```

--- stage builders ----------

```
def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
    with rec.stage("validation") as st:
        total = len(val_results)
        passed = sum(1 for r in val_results if getattr(r, "passed", False))
```

```
        st.add_metric("checks_passed", passed)
        st.add_metric("checks_total", total)
        failures = [r for r in val_results if not getattr(r, "passed", True)]
        for r in failures:
            st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r,
'message', '')}")
        st.status = "error" if failures else ("ok" if total else "not_run")


def _segmentation_stage(
    rec: "obsreport.RunRecorder",
    play_segments: List[Dict[str, Any]],
    candidates: List[Dict[str, Any]],
    segmenter_requested: Optional[str],
    segmenter_actual: Optional[str],
    config_default: Optional[str],
    was_reingest: bool,
    ran: bool,
) -> None:
    with rec.stage("segmentation") as st:
        if not ran:
            st.status = "not_run"
        seg_count = len(play_segments)
        total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
        detector_tag = None
        if play_segments:
            detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
        st.add_metric("candidate_windows", len(candidates))
        st.add_metric("segment_count", seg_count, audience="both")
        st.add_metric("total_play_s", total_play, unit="s")
        st.details.update({
            "segmenter_requested": segmenter_requested,
            "segmenter_actual": segmenter_actual,
            "config_default": config_default,
            "matches_config_default": (segmenter_actual == config_default) if config_default
else True,
            "was_reingest": was_reingest,
            "detector": detector_tag,
        })


def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
              play_segments: List[Dict[str, Any]], ran: bool) -> None:
    """Only meaningful for AI-based segmenters. Records whether the provider key was
    present ? the signal the silent_provider_noop rule keys on."""
    ai_based = segmenter_actual in _AI_SEGMENTERS
    provider = (config.get("ai_provider") or "gemini")
    key_env = f"{provider.upper()}_API_KEY"
    with rec.stage("ai_handoff") as st:
        if not ran or not ai_based:
            st.status = "not_run" if not ran else "skipped"
        st.details.update({
            "ai_based": ai_based,
            "provider": provider if ai_based else None,
            "model": config.get("ai_model") if ai_based else None,
            "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
        })
        if ai_based:
            st.add_metric("confirmed_segments", len(play_segments))


def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
                video_meta: Dict[str, Any]) -> None:
    total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
```

```
    coverage = None
    dur = video_meta.get("duration_s")
    if dur:
        coverage = round(100.0 * total / dur, 1)
    notes = []
    if not play_segments:
        notes.append("No highlights were produced for this video.")
    rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
                       coverage_pct=coverage, notes=notes)


## --- top-level ----------------------------------------------------------
def record_run(
    *,
    video_id: str,
    video_path: Optional[str],
    config: Dict[str, Any],
    val_results: List[Any],
    val_context: Optional[Dict[str, Any]],
    play_segments: List[Dict[str, Any]],
    candidates: List[Dict[str, Any]],
    segmenter_requested: Optional[str] = None,
    segmenter_actual: Optional[str] = None,
    was_reingest: bool = False,
    segmentation_ran: bool = True,
    tool_version: Optional[str] = None,
    spec: Optional["obsreport.ReportSpec"] = None,
    **recorder_kwargs: Any,
) -> "obsreport.RunRecorder":
    """Assemble a finalized recorder for an indexer run (does NOT write)."""
    rec = obsreport.RunRecorder(
        "badminton-highlight-indexer",
        tool_version=tool_version,
        video_id=video_id,
        video_id_kind="file_md5",
        video_path=video_path,
        spec=spec if spec is not None else load_spec(),
        **recorder_kwargs,
    )
    vm = video_meta_from_context(val_context, video_path)
    rec.set_video_meta(**vm)

    _validation_stage(rec, val_results)
    config_default = (config.get("indexing") or {}).get("default_segmenter")
    _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
                        segmenter_actual, config_default, was_reingest, segmentation_ran)
    _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran)
    _highlights(rec, play_segments, vm)

    rec.finalize()
    return rec
```

178. user (2026-06-07T07:26:22.630Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\backend\reporting\harvest.py (file state is current in your context ? no need to Read it
back)
```

179. assistant (2026-06-07T07:26:36.106Z)

```
The soft-import wrapper with a single never-raising entrypoint for `main.py` to call:
```

180. assistant (2026-06-07T07:26:40.360Z)

```
"""Per-video observability reports for the indexer.

`obsreport` is a SOFT dependency: if it is not installed, this module degrades to a
no-op and the pipeline runs exactly as before. The single entrypoint
`emit_indexer_report(...)` is designed to be called from a `finally` block and
NEVER raises ? emitting a report must not be able to break a processing run.
"""
from __future__ import annotations

import os
from typing import Any, Dict, List, Optional

try:  # soft dependency ? never hard-fail the pipeline on a reporting import
    import obsreport  # noqa: F401
    from . import harvest

    OBSREPORT_AVAILABLE = True
except Exception:  # pragma: no cover - exercised only when obsreport is absent
    OBSREPORT_AVAILABLE = False


def report_enabled(config: Optional[Dict[str, Any]]) -> bool:
    return bool((config or {}).get("report", {}).get("enabled", True))


def emit_indexer_report(
    *,
    db,
    video_id: str,
    video_path: Optional[str],
    config: Optional[Dict[str, Any]],
    val_results: List[Any],
    val_context: Optional[Dict[str, Any]] = None,
    segmenter_requested: Optional[str] = None,
    segmenter_actual: Optional[str] = None,
    was_reingest: bool = False,
    segmentation_ran: bool = True,
) -> Optional[Dict[str, str]]:
    """Harvest + write the report. Returns the written file paths, or None if
    reporting is unavailable/disabled/failed. Never raises."""
    config = config or {}
    if not OBSREPORT_AVAILABLE or not report_enabled(config):
        return None
    try:
        play_segments = db.query_play_segments(video_id, {}) if db is not None else []
        candidates = db.query_candidate_segments(video_id) if db is not None else []
        rec = harvest.record_run(
            video_id=video_id,
            video_path=video_path,
            config=config,
            val_results=val_results,
            val_context=val_context,
            play_segments=play_segments,
            candidates=candidates,
            segmenter_requested=segmenter_requested,
            segmenter_actual=segmenter_actual,
            was_reingest=was_reingest,
            segmentation_ran=segmentation_ran,
        )
        output_dir = (config.get("report") or {}).get("output_dir")
        res = rec.write(output_dir=output_dir)
        return res.paths or None
    except Exception:  # pragma: no cover - defensive; reporting must never break a run
        return None
```

181. user (2026-06-07T07:26:40.370Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting__init__.py (file state is current in your context ? no need to Read it back)

182. assistant (2026-06-07T07:27:18.516Z)

Now wire it into `main.py` ? both the CLI and API paths, `finally`-guarded so the report is emitted even on validation failure or error, and surfaced in the API response. Adding the import first:

183. user (2026-06-07T07:27:20.823Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

184. user (2026-06-07T07:27:39.915Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

185. assistant (2026-06-07T07:27:51.580Z)

Now the CLI processing path:

186. user (2026-06-07T07:27:55.288Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

187. assistant (2026-06-07T07:28:18.097Z)

Let me refine fps-source inference (a real `fallback` should be detectable when segmentation ran but no probe yielded fps), then add the README FAQ entries the flags cite, the `report` config note, and tests:

188. user (2026-06-07T07:28:21.091Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py has been updated successfully. (file state is current in your context ? no need to Read it back)

189. assistant (2026-06-07T07:28:24.987Z)

Now the README FAQ entries the flags cite (Q10?Q13), so every `faq_ref` resolves:

190. user (2026-06-07T07:28:25.575Z)

````
229     ### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without uploading a huge multi-GB file?
230     Ingesting and uploading multi-gigabyte video files to Google Cloud can be slow and expensive. We built a native **fast local trimming** developer debugging feature:
231     1. Open your `config.json` file.
232     2. Under the `"indexing"` configuration block, add a `"debug_duration"` key (in seconds):
233        ```json
234        "indexing": {
235            "default_segmenter": "gemini",
236            "debug_duration": 15.0,
237            ...
238        }
239        ```
240     3. When `"debug_duration"` is set, the Gemini segmenter will instantly run a zero-re-
````

```
encode, sub-second `ffmpeg` slice to create a tiny temporary clip of the first 15 seconds,
upload only that tiny clip to Google AI Studio (which takes <1 second to upload and <5 seconds
to process), run the model query, and clean up the temporary files automatically. This allows
you to iterate on model prompts, evaluate validation overlaps, and verify rally boundaries in
under 10 seconds!
241
242     ---
243
244     ## ? Annotation & Evaluation
245
246     Once a video is processed, you can measure **how well the pipeline detected its
rallies** against hand-annotated ground truth. You annotate each rally's `[start, end)` time
range in a small CSV; the harness scores the detected rallies straight from the SQLite DB ? no
API calls, fully deterministic.
247
248     ```bash
```

191. user (2026-06-07T07:28:47.015Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\README.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

192. assistant (2026-06-07T07:28:53.999Z)

```
Let me add the `obsreport` dependency note to `requirements.txt` (commented until the private
remote exists, since the runtime soft-imports it):
```

193. user (2026-06-07T07:28:54.425Z)

```
1       fastapi>=0.111.0
2       uvicorn>=0.30.1
3       opencv-python>=4.9.0.80
4       numpy>=1.26.4
5       pydantic>=2.7.1
6       jinja2>=3.1.4
7       python-dotenv>=1.0.0
8       google-genai>=2.4.0
9       torch>=2.12.0
10      torchvision>=0.27.0
11      supervision>=0.21.0,<0.30.0
12
```

194. user (2026-06-07T07:29:02.669Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\requirements.txt has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

195. assistant (2026-06-07T07:29:24.563Z)

```
Now the harvest/reporting tests (mocked DB, covering the footgun scenarios), then run the
indexer suite:
```

196. assistant (2026-06-07T07:29:31.392Z)

```
"""Tests for the indexer observability harvest + footgun rules.

These exercise the broken-run scenarios the report is meant to make debuggable.
obsreport is a soft dependency; skip cleanly if it isn't installed.
"""
import os

import pytest

pytest.importorskip("obsreport")
```

```
from backend.reporting import emit_indexer_report, harvest  # noqa: E402
from backend.validators.base import ValidationResult  # noqa: E402


class _FakeDB:
    def __init__(self, play=None, candidates=None, existing=False):
        self._play = play or []
        self._candidates = candidates or []
        self._existing = existing

    def query_play_segments(self, video_id, filters):
        return list(self._play)

    def query_candidate_segments(self, video_id, detector=None, only_unlabeled=False):
        return list(self._candidates)

    def get_video(self, video_id):
        return {"id": video_id} if self._existing else None


def _vr(name, passed, message="ok"):
    return ValidationResult(validator_name=name, passed=passed, message=message)


_FFPROBE_OK = {
    "streams": [{"width": 1920, "height": 1080, "r_frame_rate": "60000/1001", "codec_name":
"hevc"}],
    "format": {"format_name": "mov,mp4,m4a", "duration": "676.0", "size": "5500000000"},
}


def _record(**kw):
    base = dict(
        video_id="md5abc",
        video_path=None,
        config={"ai_provider": "gemini", "ai_model": "gemini-flash", "indexing":
{"default_segmenter": "wasb_hybrid"}},
        val_results=[_vr("format_check", True), _vr("relevance_check", True)],
        val_context={"ffprobe_meta": _FFPROBE_OK},
        play_segments=[{"duration": 5.0, "metadata": {"detector": "wasb-hybrid-gemini"}}],
        candidates=[{"id": 1}, {"id": 2}],
        segmenter_requested="wasb_hybrid",
        segmenter_actual="wasb_hybrid",
        was_reingest=False,
        segmentation_ran=True,
    )
    base.update(kw)
    return harvest.record_run(**base).envelope


def _codes(env):
    return {f.code for f in env.flags}


def test_healthy_run_is_clean(monkeypatch):
    monkeypatch.setenv("GEMINI_API_KEY", "x")
    env = _record()
    assert env.verdict == "pass"
    assert _codes(env) == set()
    assert env.video_meta.fps == 59.94
    assert env.video_meta.fps_source == "probed"
    assert env.get_stage("segmentation").get_stage if False else True
    assert env.lint() == []
```

```
def test_silent_provider_noop_fires_when_no_key_and_zero_segments(monkeypatch):
    monkeypatch.delenv("GEMINI_API_KEY", raising=False)
    env = _record(play_segments=[])
    assert "silent_provider_noop" in _codes(env)
    assert env.verdict == "fail"
    flag = next(f for f in env.flags if f.code == "silent_provider_noop")
    assert flag.faq_ref == "README#Q7"


def test_ai_empty_warns_when_key_present_but_zero_segments(monkeypatch):
    monkeypatch.setenv("GEMINI_API_KEY", "x")
    env = _record(play_segments=[])
    codes = _codes(env)
    assert "ai_empty_vs_no_rallies" in codes
    assert "silent_provider_noop" not in codes
    assert env.verdict == "pass_with_warnings"


def test_motion_segmenter_flags_synthetic_metadata(monkeypatch):
    monkeypatch.delenv("GEMINI_API_KEY", raising=False)
    env = _record(segmenter_requested="motion", segmenter_actual="motion",
                  config={"indexing": {"default_segmenter": "motion"}})
    codes = _codes(env)
    assert "synthetic_motion_metadata" in codes
    # motion is not AI-based -> the provider-noop rule must NOT fire even with no key
    assert "silent_provider_noop" not in codes


def test_segmenter_divergence_info_flag(monkeypatch):
    monkeypatch.setenv("GEMINI_API_KEY", "x")
    env = _record(segmenter_actual="gemini",
                  config={"ai_provider": "gemini", "indexing": {"default_segmenter":
"wasb_hybrid"}})
    assert "segmenter_divergence" in _codes(env)


def test_reingest_info_flag(monkeypatch):
    monkeypatch.setenv("GEMINI_API_KEY", "x")
    env = _record(was_reingest=True)
    assert "reingest_replaced_prior" in _codes(env)


def test_validation_failure_halts_and_flags(monkeypatch):
    monkeypatch.setenv("GEMINI_API_KEY", "x")
    env = _record(
        val_results=[_vr("format_check", True), _vr("relevance_check", False, "court color not
found")],
        play_segments=[],
        segmenter_actual=None,
        segmentation_ran=False,
    )
    assert env.get_stage("validation").status == "error"
    assert env.get_stage("segmentation").status == "not_run"
    assert "validation_failed" in _codes(env)
    assert env.verdict == "fail"


def test_fps_fallback_flag_when_no_probe_but_segmented(monkeypatch):
    monkeypatch.setenv("GEMINI_API_KEY", "x")
    env = _record(val_context={}, segmentation_ran=True)
    assert env.video_meta.fps_source == "fallback"
    assert "fps_fallback_used" in _codes(env)


def test_emit_writes_files_next_to_video(tmp_path, monkeypatch):
```

```
    monkeypatch.setenv("GEMINI_API_KEY", "x")
    video = tmp_path / "match.mp4"
    video.write_bytes(b"not really a video")
    db = _FakeDB(play=[{"duration": 4.0, "metadata": {"detector": "wasb-hybrid"}}],
candidates=[{"id": 1}])
    paths = emit_indexer_report(
        db=db, video_id="md5abc", video_path=str(video),
        config={"ai_provider": "gemini", "indexing": {"default_segmenter": "wasb_hybrid"}},
        val_results=[_vr("format_check", True)], val_context={"ffprobe_meta": _FFPROBE_OK},
        segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid",
        was_reingest=False, segmentation_ran=True,
    )
    assert paths is not None
    assert os.path.exists(paths["json"])
    assert (tmp_path / "match.report.md").exists()
    assert (tmp_path / "match.summary.md").exists()


def test_emit_disabled_via_config(tmp_path, monkeypatch):
    monkeypatch.setenv("GEMINI_API_KEY", "x")
    db = _FakeDB()
    paths = emit_indexer_report(
        db=db, video_id="md5abc", video_path=str(tmp_path / "m.mp4"),
        config={"report": {"enabled": False}},
        val_results=[_vr("format_check", True)], val_context={},
        segmenter_actual="motion", segmentation_ran=True,
    )
    assert paths is None
```

197. user (2026-06-07T07:29:31.422Z)

File created successfully at: C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py (file state is current in your context ? no need to Read it back)

198. assistant (2026-06-07T07:29:37.200Z)

Let me fix a stray nonsense assertion I left in the healthy-run test:

199. user (2026-06-07T07:29:38.333Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py has been updated successfully. (file state is current in your context ? no need to Read it back)

200. assistant (2026-06-07T07:29:43.169Z)

Now run the full indexer suite ? confirming the new reporting tests pass *and* the existing 209 are unbroken:

201. user (2026-06-07T07:29:57.480Z)

```
.................................................................... [ 32%]
.................................................................... [ 64%]
.................................................................... [ 97%]
......                                                               [100%]
222 passed in 12.09s
```

202. assistant (2026-06-07T07:30:27.264Z)

222 passed (209 baseline intact + 13 new). Now an end-to-end smoke through the **real** `main.py` CLI to prove the wiring writes a report next to the video. I'll synthesize a tiny MP4 with ffmpeg and run the CLI:

203. user (2026-06-07T07:30:37.740Z)

```
made video:
smoke.mp4
=== run CLI (no API key) ===
[CLI] Processing video file: C:\Users\avidu\AppData\Local\Temp/obsreport_smoke/smoke.mp4
[CLI] Validating...
C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py:347:
PydanticDeprecatedSince20: The `dict` method is deprecated; use `model_dump` instead. Deprecated
in Pydantic V2.0 to be removed in V3.0. See Pydantic V2 Migration Guide at
https://errors.pydantic.dev/2.13/migration/
  db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in val_results])

========================================
VALIDATION SUMMARY
========================================
[PASS] format_check: Video format is valid MP4 container.
[PASS] resolution_fps_check: Video resolution and frame rate meet minimum standards.
[PASS] pts_continuity_check: Video is too short to perform a PTS continuity check.
[PASS] scene_cut_check: Scene continuity check passed. No abnormal visual cuts detected.
[FAIL] blur_check: Video quality rejected: Focus check failed. Average sharpness score of 0.0 is
below the minimum required threshold of 20.0.
[PASS] relevance_check: Video identified as relevant to badminton (badminton court structures
found).
========================================

[FAIL] Video failed checks. Indexing halted.
[REPORT] Technical report : C:\Users\avidu\AppData\Local\Temp\obsreport_smoke\smoke.report.md
[REPORT] Customer summary : C:\Users\avidu\AppData\Local\Temp\obsreport_smoke\smoke.summary.md
[REPORT] Machine JSON     : C:\Users\avidu\AppData\Local\Temp\obsreport_smoke\smoke.report.json
=== files next to video ===
out
smoke.mp4
smoke.report.json
smoke.report.md
smoke.summary.md
```

204. assistant (2026-06-07T07:30:44.095Z)

End-to-end wiring works ? 3 report files written next to the video via the `finally` block, even though validation halted the run. Let me read the generated technical + customer reports to confirm they read well:

205. user (2026-06-07T07:30:44.551Z)

```
1       # Badminton Highlight Indexer ? Video Report
2
3       ## Run
4
5       | Field | Value |
6       | --- | --- |
7       | Tool | badminton-highlight-indexer |
8       | Video id | fb5d84c30b683fce27292bd2a8f80677 (file_md5) |
9       | Video path | C:\Users\avidu\AppData\Local\Temp/obsreport_smoke/smoke.mp4 |
10      | Generated | 2026-06-07T07:30:37Z |
11      | Schema | 1.0 |
12
13      ## Video
14
15      | Field | Value |
16      | --- | --- |
17      | Duration | 2.0s |
18      | Resolution | 1280x720 |
19      | FPS | 30 fps (probed) |
20      | Container | mov,mp4,m4a,3gp,3g2,mj2 |
21      | Size | 0 MB |
22
```

```
23      ## Processing stages
24
25      **validation** ? `error` (0.0s)
26        - checks_passed: 5
27        - checks_total: 6
28        - FAILED blur_check: Video quality rejected: Focus check failed. Average sharpness
score of 0.0 is below the minimum required threshold of 20.0.
29
30      **segmentation** ? `not_run` (0.0s)
31        - candidate_windows: 0
32        - segment_count: 0
33        - total_play_s: 0 s
34        - _segmenter_requested_: motion
35        - _segmenter_actual_: None
36        - _config_default_: gemini
37        - _matches_config_default_: False
38        - _was_reingest_: False
39        - _detector_: None
40
41      **ai_handoff** ? `not_run` (0.0s)
42        - _ai_based_: False
43        - _provider_: None
44        - _model_: None
45        - _api_key_present_: None
46
47      ## Findings & warnings
48
49      - [info] **segmenter_divergence** ? The segmenter that ran differs from config.json's
default_segmenter. This is fine if intentional (CLI/API override), but the indexer's defaults
are known to diverge across config.json / setup.py / the CLI ? confirm you ran the one you
meant.
50        ? see **README#Q7**
51        ? evidence: stages.segmentation.details.matches_config_default=False
52      - [ERROR] **validation_failed** ? One or more validation checks failed, so indexing was
halted. See the Processing stages > validation block above: each failed check states the reason
(relevance -> README#Q2, fps/keyframes -> README#Q5).
53        ? see **README#Q13**
54        ? evidence: stages.validation.status=error
55
56      ## Verdict
57
58      **FAIL**
59
```

206. user (2026-06-07T07:30:45.107Z)

```
1       # Badminton Highlight Indexer ? Video Report (summary)
2
3       ## Run
4
5       | Field | Value |
6       | --- | --- |
7       | Video | smoke.mp4 |
8       | Report generated | 2026-06-07T07:30:37Z |
9
10      ## Video
11
12      | Field | Value |
13      | --- | --- |
14      | Duration | 2.0s |
15      | Resolution | 1280x720 |
16      | Frame rate | 30 fps |
17
18      ## Highlights
19
```

```
20      - **0** highlights
21      - **0.0s** of play
22      - **0%** of the video is highlights
23      - No highlights were produced for this video.
24
25      ## Things to know
26
27      - ? One or more validation checks failed, so indexing was halted. See the Processing
stages > validation block above: each failed check states the reason (relevance -> README#Q2,
fps/keyframes -> README#Q5).
28
29      ## Verdict
30
31      ?? We ran into a problem processing this video, so some highlights may be missing. See
the notes above, or contact support.
32
```

207. assistant (2026-06-07T07:32:02.300Z)

The reports are readable and debuggable. Two quality issues to fix before committing:

1. **`segmenter_divergence` false-fired** even though segmentation didn't run (and would also fire on intentional `--segmenter` overrides) ? noisy.
2. The **customer** summary leaked a technical phrase ("see Processing stages > validation block") ? a customer should never see internal references.

Both are exactly what the readability bar should catch. The clean fix for #2 is a small additive feature in the library: an optional **`customer_message`** on flags, where the customer report shows *only* findings explicitly curated for customers. Let me add that to `obsreport` first.

208. user (2026-06-07T07:32:05.449Z)

The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\envelope.py has been updated successfully. (file state is current in your context ? no need to Read it back)

209. user (2026-06-07T07:32:11.745Z)

The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\spec.py has been updated successfully. (file state is current in your context ? no need to Read it back)

210. user (2026-06-07T07:32:17.122Z)

The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\rules.py has been updated successfully. (file state is current in your context ? no need to Read it back)

211. user (2026-06-07T07:32:27.172Z)

The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\render\markdown.py has been updated successfully. (file state is current in your context ? no need to Read it back)

212. user (2026-06-07T07:32:34.869Z)

The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\recorder.py has been updated successfully. (file state is current in your context ? no need to Read it back)

213. assistant (2026-06-07T07:32:48.186Z)

Now update the library tests to reflect the curated-customer-flags semantics, bump the version, and regenerate the schema:

214. user (2026-06-07T07:32:51.418Z)

The file C:\Users\avidu\Projects\sports-obsreport\tests\test_render_markdown.py has been updated

```
successfully. (file state is current in your context ? no need to Read it back)
```

215. user (2026-06-07T07:33:00.849Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\tests\test_render_markdown.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

216. assistant (2026-06-07T07:33:10.288Z)

```
"""customer_message: rules can carry a gentle, customer-facing phrasing that the
technical report ignores and the customer report uses (and shows ONLY for flags
that set it)."""
from obsreport import Identity, ReportEnvelope, Stage, load_spec_text, render_customer,
render_technical, run_spec


def test_rule_customer_message_flows_to_flag_and_customer_view():
    spec = load_spec_text(
        """
        name: t
        rules:
          - code: bad_thing
            severity: error
            when: [{path: stages.s.metrics.n, op: eq, value: 0}]
            message: "Internal: pipeline produced 0 for n."
            faq_ref: "README#Q1"
            customer_message: "We couldn't generate highlights for this video."
        """
    )
    env = ReportEnvelope(identity=Identity(tool="t"), stages=[Stage(name="s").add_metric("n",
0)])
    run_spec(env, spec)
    flag = env.flags[0]
    assert flag.customer_message == "We couldn't generate highlights for this video."

    tech = render_technical(env, spec)
    assert "Internal: pipeline produced 0 for n." in tech  # technical uses message
    assert "We couldn't generate highlights" not in tech  # ...not customer_message

    cust = render_customer(env, spec)
    assert "We couldn't generate highlights for this video." in cust
    assert "Internal:" not in cust  # technical wording never leaks
```

217. user (2026-06-07T07:33:10.310Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-
obsreport\tests\test_customer_message.py (file state is current in your context ? no need to
Read it back)
```

218. assistant (2026-06-07T07:33:14.554Z)

```
Now bump the lib version, regenerate the schema, and run the lib suite:
```

219. user (2026-06-07T07:33:15.147Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\version.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

220. user (2026-06-07T07:33:19.403Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\pyproject.toml has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

221. user (2026-06-07T07:33:25.588Z)

```
wrote C:\Users\avidu\Projects\sports-obsreport\schema\report_envelope.schema.json
.................................................................... [100%]
72 passed in 0.33s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

222. assistant (2026-06-07T07:33:39.355Z)

72 passed. Commit + tag the lib v0.1.1, then fix the indexer's `segmenter_divergence` gating and add curated customer messages to its spec.

223. user (2026-06-07T07:33:47.363Z)

```
warning: in the working copy of 'pyproject.toml', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'src/obsreport/envelope.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'src/obsreport/recorder.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'src/obsreport/render/markdown.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'src/obsreport/rules.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'src/obsreport/spec.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'src/obsreport/version.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'tests/test_render_markdown.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_customer_message.py', LF will be replaced by CRLF
the next time Git touches it
v0.1.0
v0.1.1
[notice] To update, run: python.exe -m pip install --upgrade pip
reinstalled 0.1.1
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

224. assistant (2026-06-07T07:34:04.566Z)

Lib v0.1.1 reinstalled. Now fix the indexer harvest gating for `segmenter_divergence` and add curated customer messages to its spec:

225. user (2026-06-07T07:34:05.359Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

226. user (2026-06-07T07:34:13.239Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

227. assistant (2026-06-07T07:34:19.148Z)

Now add curated `customer_message` to the two indexer findings a customer should hear about:

228. user (2026-06-07T07:34:20.572Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

229. user (2026-06-07T07:34:36.211Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml has
```

```
been updated successfully. (file state is current in your context ? no need to Read it back)
```

230. assistant (2026-06-07T07:34:46.805Z)

```
Update the divergence test for the new gating (only fires without an explicit override), then
re-run the indexer suite:
```

231. user (2026-06-07T07:34:49.242Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

232. user (2026-06-07T07:35:16.227Z)

```
.......................................................................... [ 32%]
.......................................................................... [ 64%]
.......................................................................... [ 97%]
......                                                                     [100%]
222 passed in 12.06s
=== re-smoke (failure path) ===
exit=0
```

233. user (2026-06-07T07:35:23.916Z)

```
1       # Badminton Highlight Indexer ? Video Report (summary)
2
3       ## Run
4
5       | Field | Value |
6       | --- | --- |
7       | Video | smoke.mp4 |
8       | Report generated | 2026-06-07T07:35:15Z |
9
10      ## Video
11
12      | Field | Value |
13      | --- | --- |
14      | Duration | 2.0s |
15      | Resolution | 1280x720 |
16      | Frame rate | 30 fps |
17
18      ## Highlights
19
20      - **0** highlights
21      - **0.0s** of play
22      - **0%** of the video is highlights
23      - No highlights were produced for this video.
24
25      ## Things to know
26
27      - ? This video couldn't be processed because it didn't pass our quality checks (for
example: too blurry, too low resolution, or not a badminton court).
28
29      ## Verdict
30
31      ?? We ran into a problem processing this video, so some highlights may be missing. See
the notes above, or contact support.
32
```

234. user (2026-06-07T07:35:24.191Z)

```
45        - _model_: None
46        - _api_key_present_: None
47
```

```
48      ## Findings & warnings
49
50      - [ERROR] **validation_failed** ? One or more validation checks failed, so indexing was
halted. See the Processing stages > validation block above: each failed check states the reason
(relevance -> README#Q2, fps/keyframes -> README#Q5).
51        ? see **README#Q13**
52        ? evidence: stages.validation.status=error
53
54      ## Verdict
55
56      **FAIL**
57
```

235. assistant (2026-06-07T07:35:40.725Z)

Both reports are now clean: the customer summary shows only the gentle curated message (no technical leakage), and the technical findings no longer carry the false `segmenter_divergence`. **Phase A is functionally complete.** Committing:

236. user (2026-06-07T07:35:47.015Z)

```
warning: in the working copy of 'backend/main.py', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'backend/validators/base.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/reporting/__init__.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/reporting/harvest.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'backend/reporting/spec.yaml', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'tests/test_reporting_harvest.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'tests/test_validators_base.py', LF will be replaced by CRLF the
next time Git touches it
9dbc764 feat(reporting): always-on per-video observability report (indexer)
```

237. user (2026-06-07T07:36:10.279Z)

```
The file c:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

238. assistant (2026-06-07T07:36:05.337Z)

Phase A committed (222 green, verified end-to-end). Let me mark it done, refresh the memory checkpoint, and start **Phase B (collector)**.

239. user (2026-06-07T07:36:05.395Z)

```
Updated task #5 status
```

240. user (2026-06-07T07:36:05.436Z)

```
Updated task #6 status
```

241. assistant (2026-06-07T07:36:28.068Z)

Now Phase B (collector). Let me read the exact collector code I'll wire into ? the CVAT report builders I'll reuse, and the CLI command bodies:

242. user (2026-06-07T07:36:49.008Z)

```
"""Adapter: CVAT rally exports -> our canonical per-rally records.

Why this exists
---------------
Our annotation brief (``docs/annotation/ROORA_BRIEF.md``) asks vendors for a
per-rally **CSV** that imports directly via ``curate import-local``. A vendor may
instead do the work in CVAT and hand back the *CVAT* export: per-frame bounding
boxes, with the rally fields (``rally_number``, ``shots_count``,
``ending_reason`` ...) carried as CVAT **object attributes** on each box. The box
geometry is irrelevant to us ? it is just a carrier the rater attached the rally
metadata to.

This module is the ingestion-side adapter that absorbs that vendor format so the
modeling repo never has to. It does two things, both deliberately on *our* side
so we fix mechanical problems rather than renegotiating them:

1. **Recompute time from the frame range.** A rally's interval is taken from the
   first/last *frame* a box for that rally appears on:
   ``seconds = frame / fps`` at the video's true fps. We **ignore** any
   ``start_time``/``end_time`` attributes the vendor wrote ? a CVAT job that
   samples every ``step`` frames and assumes 30 fps yields systematically wrong
   times (we have seen ``start_time == frame / (step * 30)``, i.e. off by the
   sampling step at the wrong fps). The frame index is reliable; the vendor's
   seconds are not.

2. **Surface ? never silently fix ? content defects.** Overlapping rallies,
   zero-duration rallies, ending_reason values outside the agreed set, and
   too-coarse sampling are reported as :class:`Issue`s. Guessing a corrected
   rally boundary would poison ground truth, so that stays a human/vendor call.

Supported input: the single-file CVAT XML export, in either **image mode**
(``<image><box>...``) or **track/interpolation mode** (``<track><box frame=...>``).
The two PASCAL-VOC / COCO sibling exports encode the same data and can be added
later if a vendor sends only those.
"""
from __future__ import annotations

import os
import xml.etree.ElementTree as ET
from collections import Counter
from dataclasses import dataclass, field
from typing import Any, Dict, List, Optional, Tuple

from src.core.parsing import safe_int
from src.core.downloader import probe_resolution_fps

# The agreed cross-sport ending_reason vocabulary for net-separated sports.
# Kept here as a soft check only (a warning, never a hard failure) so this
# adapter does not depend on the annotation-spec wording.
DEFAULT_ALLOWED_REASONS = {
    "winner",
    "forced_error",
    "unforced_error",
    "service_fault",
    "let",
    "other",
}

# The observational outcome of the last shot ? orthogonal to ending_reason
# (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
```

243. user (2026-06-07T07:36:49.013Z)

```

```

```
    def detect_issues(records: List[RallyRecord],
                      stop_frame: Optional[int] = None,
                      allowed_reasons: Optional[set] = None,
                      allowed_outcomes: Optional[set] = None,
                      max_gap_s: float = 15.0) -> List[Issue]:
        """Find the defects that block or degrade a clean ground-truth import.

        Note: we deliberately do NOT flag the sampling density (step/fps). A boundary
        slack of ~0.5s is acceptable for this pipeline because the downstream
        highlight compiler pads every clip on output (badminton-highlight-indexer,
        backend/stitcher/compiler.py: begin_padding=0.5s, end_padding=1.0s), which
        absorbs the sampling uncertainty when highlights are trimmed.
        """
        allowed = allowed_reasons if allowed_reasons is not None else
DEFAULT_ALLOWED_REASONS
        allowed_out = allowed_outcomes if allowed_outcomes is not None else
DEFAULT_ALLOWED_OUTCOMES
        issues: List[Issue] = []

        ordered = sorted(records, key=lambda r: r.rally_number)
        for i, r in enumerate(ordered):
            if r.end_time <= r.start_time:
                issues.append(Issue(
                    "blocker", "zero_duration",
                    f"Rally {r.rally_number}: end <= start "
                    f"(frames {r.frame_min}-{r.frame_max}); single-sample or stray box.",
                    r.rally_number))

            if r.ending_reason is not None and r.ending_reason not in allowed:
                issues.append(Issue(
                    "medium", "ending_reason_vocab",
                    f"Rally {r.rally_number}: ending_reason '{r.ending_reason}' "
                    f"not in the agreed set {sorted(allowed)}.",
                    r.rally_number))

            if r.last_shot_outcome is not None and r.last_shot_outcome not in allowed_out:
                issues.append(Issue(
                    "low", "outcome_vocab",
                    f"Rally {r.rally_number}: last_shot_outcome '{r.last_shot_outcome}' "
                    f"not in {sorted(allowed_out)}.",
                    r.rally_number))

            # Inflated span: a long rally with implausibly few shots usually means a
            # stray/duplicated box stretched the frame range past the real ending.
            if r.shots_count and r.duration >= 5.0:
                sps = r.shots_count / r.duration
                if sps < 0.3:
                    issues.append(Issue(
                        "medium", "shot_density",
                        f"Rally {r.rally_number}: {r.shots_count} shots over "
                        f"{r.duration:.1f}s ({sps:.2f}/s) is implausibly low ? the span "
                        f"may be inflated by a stray box.",
                        r.rally_number))

            if stop_frame is not None and r.frame_max > stop_frame:
                issues.append(Issue(
                    "high", "frame_out_of_range",
                    f"Rally {r.rally_number}: frame {r.frame_max} exceeds the video's "
                    f"last frame {stop_frame}.",
                    r.rally_number))

            if i > 0:
                prev = ordered[i - 1]
                if r.start_time < prev.end_time:
                    issues.append(Issue(
```

```
                        "blocker", "overlap",
                        f"Rally {prev.rally_number} (ends {prev.end_time:.2f}s) overlaps "
                        f"Rally {r.rally_number} (starts {r.start_time:.2f}s) by "
                        f"{prev.end_time - r.start_time:.2f}s ? rallies must be disjoint.",
                        r.rally_number))
                elif r.start_time - prev.end_time > max_gap_s:
                    # A long unlabeled stretch between two rallies is a cheap, label-only
                    # second opinion for the worst error (a missed rally). It won't catch a
                    # rally MERGED into its neighbour's span (shot_density/inflated catches
                    # that), but it does catch an omitted rally that left a visible hole.
                    gap = r.start_time - prev.end_time
                    issues.append(Issue(
                        "medium", "possible_missed_rally",
                        f"{gap:.1f}s of unlabeled time between Rally {prev.rally_number} "
                        f"(ends {prev.end_time:.1f}s) and Rally {r.rally_number} "
                        f"(starts {r.start_time:.1f}s) ? check for an unlabeled rally in the
gap.",
                        r.rally_number))
        return issues


    def convert(xml_path: str,
                fps: Optional[float] = None,
                video_path: Optional[str] = None,
                step: Optional[int] = None,
                allowed_reasons: Optional[set] = None) -> ConversionResult:
        """Full pipeline: parse CVAT XML -> records (+ issues) at the true fps.

        ``fps`` resolution order: explicit ``fps`` arg > ffprobe of ``video_path`` >
        the CVAT meta (none) -> error. We never fall back to a guessed 30 fps, since
        that silent guess is exactly the bug this adapter exists to undo.
        """
        meta, boxes = parse_cvat_boxes(xml_path)

        if fps is None and video_path:
            _, _, probed = probe_resolution_fps(video_path)
            fps = probed
        if fps is None or fps <= 0:
            raise ValueError(
                "Could not determine fps. Pass fps=... or a readable video_path so "
                "frames can be converted to seconds (seconds = frame / fps).")

        if step is None:
            step = meta.get("step") or _infer_step([b["frame"] for b in boxes])

        records = build_records(boxes, fps)
        issues = detect_issues(records,
                               stop_frame=meta.get("stop_frame"),
                               allowed_reasons=allowed_reasons)
        return ConversionResult(records=records, issues=issues, fps=fps, step=step,
meta=meta)


    def write_canonical_csv(records: List[RallyRecord], out_path: str) -> None:
        """Write records as the canonical per-rally CSV that ``import-local`` reads."""
        import csv
        os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
        with open(out_path, "w", newline="", encoding="utf-8") as f:
            w = csv.writer(f)
            w.writerow(CANONICAL_COLUMNS)
            for r in sorted(records, key=lambda x: x.rally_number):
                w.writerow([
                    r.rally_number,
                    _mmss(r.start_time), _mmss(r.end_time),
                    f"{r.start_time:.3f}", f"{r.end_time:.3f}",
```

```
                    r.shots_count if r.shots_count is not None else "",
                    r.ending_reason or "",
                    r.last_shot_outcome or "",
                    r.score_before or "", r.score_after or "", r.notes or "",
                ])


    def render_issue_report(result: ConversionResult, source: str) -> str:
        """A human-readable issue report (for review / sending back to the vendor)."""
        lines = [
            f"CVAT rally conversion report ? {source}",
            f"  fps used        : {result.fps:.3f}",
            f"  sampling step   : {result.step} frames (~{result.step / result.fps:.2f}s)",
            f"  rallies parsed  : {len(result.records)}",
            f"  issues          : {len(result.issues)} ({result.n_blockers} blocker)",
            "",
        ]
        if not result.issues:
            lines.append("No issues detected.")
        else:
            for sev in ("blocker", "high", "medium", "low"):
                group = [i for i in result.issues if i.severity == sev]
                if not group:
                    continue
                lines.append(f"[{sev.upper()}]")
                for i in group:
                    lines.append(f"  - ({i.code}) {i.message}")
                lines.append("")
        return "\n".join(lines)


    # ---------------------------------------------------------------------------- #
    #  Validation reporting ? used by `curate validate-delivery`.                  #
    #  These turn a (records, issues) pair into the artifacts a human reviews:     #
    #  a machine summary (JSON), a human report (md), and an annotatable sheet.    #
    # ---------------------------------------------------------------------------- #

    def _parse_secs(value: Optional[str]) -> Optional[float]:
        """Decimal seconds or mm:ss / hh:mm:ss -> float; None if blank/unparseable."""
        s = (value or "").strip()
        if not s:
            return None
        if ":" in s:
            try:
                total = 0.0
                for part in s.split(":"):
                    total = total * 60 + float(part)
                return total
            except ValueError:
                return None
        try:
            return float(s)
        except ValueError:
            return None


    def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
        """Load a canonical per-rally CSV into RallyRecords, so a *CSV* delivery (not
        just a CVAT export) can be validated. Decimal start/end preferred, mm:ss
        fallback; rows with no usable start/end are skipped. (frame_* are -1 here, as a
        CSV carries no frame info ? only the frame-derived checks become inapplicable.)
        """
        import csv as _csv
        if not os.path.exists(csv_path):
            raise FileNotFoundError(f"CSV not found: {csv_path}")
```

```
        records: List[RallyRecord] = []
        with open(csv_path, newline="", encoding="utf-8") as f:
            reader = _csv.DictReader(f)
            reader.fieldnames = [(n or "").strip().lower() for n in (reader.fieldnames or
[])]
            for raw in reader:
                row = {(k or "").strip().lower(): v for k, v in raw.items()}
                rn = safe_int(row.get("rally_number"), None)
                if rn is None:
                    continue
                start = _parse_secs(row.get("start_time")) or
_parse_secs(row.get("start_mmss"))
                end = _parse_secs(row.get("end_time")) or _parse_secs(row.get("end_mmss"))
                if start is None or end is None:
                    continue
                records.append(RallyRecord(
                    rally_number=rn, start_time=start, end_time=end,
                    shots_count=safe_int(row.get("shots_count"), None),
                    ending_reason=(row.get("ending_reason") or "").strip() or None,
                    last_shot_outcome=(row.get("last_shot_outcome") or "").strip() or None,
                    score_before=(row.get("score_before") or "").strip() or None,
                    score_after=(row.get("score_after") or "").strip() or None,
                    notes=(row.get("notes") or "").strip() or None,
                    frame_min=-1, frame_max=-1, n_boxes=0,
                ))
        return records


    def build_summary(records: List[RallyRecord], issues: List[Issue],
                      source: str = "", fps: Optional[float] = None) -> Dict[str, Any]:
        """Machine-readable validation summary (JSON / CI)."""
        by_sev = {s: [i for i in issues if i.severity == s]
                  for s in ("blocker", "high", "medium", "low")}
        flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
        return {
            "source": source,
            "fps": fps,
            "n_rallies": len(records),
            "n_issues": len(issues),
            "counts_by_severity": {s: len(v) for s, v in by_sev.items()},
            "n_blockers": len(by_sev["blocker"]),
            "passed": len(by_sev["blocker"]) == 0,
            "flagged_rallies": flagged,
            "issues": [{"severity": i.severity, "code": i.code,
                        "rally_number": i.rally_number, "message": i.message} for i in
issues],
        }


    def render_validation_md(records: List[RallyRecord], issues: List[Issue], source: str)
-> str:
        """Human validation report: scoreboard + issues by severity + rallies to review."""
        n = len(records)
        by_sev = {s: [i for i in issues if i.severity == s]
                  for s in ("blocker", "high", "medium", "low")}
        flagged = sorted({i.rally_number for i in issues if i.rally_number is not None})
        clean = n - len(flagged)
        lines = [
            f"# Delivery validation ? {source}",
            "",
            f"- rallies: **{n}**",
            f"- issues: **{len(issues)}**  (blocker {len(by_sev['blocker'])}, "
            f"high {len(by_sev['high'])}, medium {len(by_sev['medium'])}, low
{len(by_sev['low'])})",
            f"- structurally-clean rallies: **{clean}/{n}**"
```

```
493             + (f" ({100 * clean // n}%)" if n else ""),
494             f"- gate: **{'PASS' if not by_sev['blocker'] else 'BLOCKED'}**",
495             "",
496         ]
497         if issues:
498             for sev in ("blocker", "high", "medium", "low"):
499                 grp = by_sev[sev]
500                 if not grp:
501                     continue
502                 lines.append(f"## {sev.upper()}")
503                 lines += [f"- ({i.code}) {i.message}" for i in grp]
504                 lines.append("")
505         else:
506             lines.append("No issues detected. ?\n")
507         if flagged:
508             lines.append(f"## Rallies to review: {flagged}")
509         return "\n".join(lines)
510
511
512     def write_review_csv(records: List[RallyRecord], issues: List[Issue], out_path: str) ->
None:
513         """Auto-generate the annotatable manual-verification sheet (blank `true_*` cols)."""
514         import csv
515         flags: Dict[int, List[str]] = {}
516         for i in issues:
517             if i.rally_number is not None:
518                 flags.setdefault(i.rally_number, []).append(i.code)
519         os.makedirs(os.path.dirname(os.path.abspath(out_path)), exist_ok=True)
520         cols = ["rally_number", "start_mmss", "end_mmss", "dur_s", "shots_count",
521                 "ending_reason", "last_shot_outcome", "auto_flags",
522                 "VERIFIED(Y/N)", "true_start_mmss", "true_end_mmss",
523                 "true_ending_reason", "true_last_shot_outcome", "reviewer_comment"]
524         with open(out_path, "w", newline="", encoding="utf-8") as f:
525             w = csv.writer(f)
526             w.writerow(cols)
527             for r in sorted(records, key=lambda x: x.rally_number):
528                 w.writerow([
529                     r.rally_number, _mmss(r.start_time), _mmss(r.end_time),
530                     f"{r.duration:.1f}", r.shots_count if r.shots_count is not None else "",
531                     r.ending_reason or "", r.last_shot_outcome or "",
532                     ";".join(flags.get(r.rally_number, [])),
533                     "", "", "", "", "", "",
534                 ])
535
```

244. assistant (2026-06-07T07:37:00.047Z)

Let me read the `Issue`/`RallyRecord`/`ConversionResult` definitions and the collector CLI command bodies I'll wire into:

245. user (2026-06-07T07:37:01.492Z)

```
60      # (winner can be 'in'; an error can be 'net' or 'out'; service_fault/let -> 'n_a').
61      DEFAULT_ALLOWED_OUTCOMES = {"in", "net", "out", "n_a"}
62
63      # CVAT attribute names we map onto canonical columns. The vendor's CVAT label
64      # schema has used suffixed names like "score_before (A-B)"; we match leniently.
65      _ATTR_ALIASES = {
66          "rally_number": ("rally_number",),
67          "shots_count": ("shots_count",),
68          "ending_reason": ("ending_reason",),
69          "last_shot_outcome": ("last_shot_outcome", "outcome", "last_shot_outcome
(in/net/out)"),
70          "score_before": ("score_before", "score_before (a-b)"),
71          "score_after": ("score_after", "score_after (a-b)"),
```

```
        "notes": ("notes", "note"),
    }

    CANONICAL_COLUMNS = [
        "rally_number", "start_mmss", "end_mmss", "start_time", "end_time",
        "shots_count", "ending_reason", "last_shot_outcome", "score_before", "score_after",
"notes",
    ]


    @dataclass
    class RallyRecord:
        """One rally recovered from the CVAT export, time re-derived from frames."""
        rally_number: int
        start_time: float            # seconds, = frame_min / fps
        end_time: float              # seconds, = frame_max / fps (see padding note)
        shots_count: Optional[int]
        ending_reason: Optional[str]
        last_shot_outcome: Optional[str]
        score_before: Optional[str]
        score_after: Optional[str]
        notes: Optional[str]
        frame_min: int
        frame_max: int
        n_boxes: int
        vendor_start_time: Optional[str] = None   # the (discarded) attribute value
        vendor_end_time: Optional[str] = None

        @property
        def duration(self) -> float:
            return self.end_time - self.start_time


    @dataclass
    class Issue:
        """A defect found in the delivery. ``blocker`` issues stop a clean import."""
        severity: str   # 'blocker' | 'high' | 'medium' | 'low'
        code: str
        message: str
        rally_number: Optional[int] = None


    @dataclass
    class ConversionResult:
        records: List[RallyRecord]
        issues: List[Issue]
        fps: float
        step: int
        meta: Dict[str, Any] = field(default_factory=dict)

        @property
        def n_blockers(self) -> int:
            return sum(1 for i in self.issues if i.severity == "blocker")


    def _mmss(seconds: float) -> str:
        """Format seconds as ``m:ss.mmm`` (matches the brief's mm:ss columns)."""
        m = int(seconds // 60)
        s = seconds - m * 60
        return f"{m}:{s:06.3f}"


    def _attr_lookup(attrs: Dict[str, str], canonical: str) -> Optional[str]:
        for alias in _ATTR_ALIASES[canonical]:
            if alias in attrs and attrs[alias] != "":
```

```
                return attrs[alias]
        return None


    def parse_cvat_boxes(xml_path: str) -> Tuple[Dict[str, Any], List[Dict[str, Any]]]:
        """Parse a CVAT XML export into (meta, boxes).

        ``boxes`` is a flat list of ``{"frame": int, "attrs": {name: value}}`` ? one
        entry per visible box, from either image-mode or track-mode exports.
        ``meta`` carries ``step`` (frame_filter) and ``stop_frame`` when present.
        """
        if not os.path.exists(xml_path):
            raise FileNotFoundError(f"CVAT export not found: {xml_path}")

        root = ET.parse(xml_path).getroot()
        meta: Dict[str, Any] = {}

        job = root.find(".//job")
        if job is not None:
            stop = job.findtext("stop_frame")
            if stop is not None and stop.strip().isdigit():
                meta["stop_frame"] = int(stop.strip())
            ff = job.findtext("frame_filter") or ""
            if "step=" in ff:
                try:
                    meta["step"] = int(ff.split("step=")[1].split()[0])
                except (ValueError, IndexError):
                    pass

        def _attrs(el: ET.Element) -> Dict[str, str]:
            out: Dict[str, str] = {}
            for a in el.findall("attribute"):
                out[(a.get("name") or "").strip().lower()] = (a.text or "").strip()
            return out

        boxes: List[Dict[str, Any]] = []

        # Image mode: <image id=frame> ... <box>...</box>
        for img in root.findall("image"):
            try:
                frame = int(img.get("id"))
            except (TypeError, ValueError):
                continue
            for box in img.findall("box"):
                boxes.append({"frame": frame, "attrs": _attrs(box)})

        # Track mode: <track> ... <box frame=N outside=0>...</box>
        for track in root.findall("track"):
            for box in track.findall("box"):
                if box.get("outside") == "1":
                    continue  # a track's terminating keyframe carries no real label
                try:
                    frame = int(box.get("frame"))
                except (TypeError, ValueError):
                    continue
                boxes.append({"frame": frame, "attrs": _attrs(box)})

        return meta, boxes
```

246. user (2026-06-07T07:37:02.056Z)

```
        def import_local(self, args):
            """Manually registers a local video and annotation file into the database."""
            sport_str = args.sport.lower()
```

```
            if sport_str not in self.config.get("supported_sports", ["badminton"]):
                print(f"Error: Sport '{sport_str}' is not supported. Supported:
{self.config.get('supported_sports')}")
                sys.exit(1)

            # Validate video path
            if not os.path.exists(args.video_path):
                print(f"Warning: Local video path '{args.video_path}' does not exist on
disk, but continuing registration.")

            # Validate annotation file
            if not os.path.exists(args.annotations_path):
                print(f"Error: Annotations file '{args.annotations_path}' not found.")
                sys.exit(1)

            match_name = args.match_name or os.path.basename(args.video_path).split('.')[0]
            video_id = args.video_id or match_name.lower().replace(" ", "_")

            # Parse license
            try:
                license_type = LicenseType(args.license)
            except ValueError:
                print(f"Warning: Unknown license '{args.license}'. Defaulting to Unknown
(non-commercial).")
                license_type = LicenseType.UNKNOWN

            is_commercial = self.validator.check_license_commercial(license_type)

            # 1. Parse Annotations
            annotations = []
            try:
                annotations = self._parse_local_annotations(sport_str, video_id,
args.annotations_path)
            except Exception as e:
                print(f"Error parsing annotations: {e}")
                sys.exit(1)

            # 2. Run Validation
            is_valid = True
            errors = []
            if sport_str == "badminton":
                is_valid, errors = self.validator.validate_badminton_rallies(annotations)
            elif sport_str == "tennis":
                is_valid, errors = self.validator.validate_tennis_points(annotations)
            elif sport_str == "cricket":
                is_valid, errors = self.validator.validate_cricket_deliveries(annotations)
            elif sport_str == "pickleball":
                is_valid, errors = self.validator.validate_pickleball_rallies(annotations)
            elif sport_str == "tabletennis":
                is_valid, errors = self.validator.validate_tabletennis_rallies(annotations)

            if not is_valid:
                print(f"Error: Annotations failed validation!")
                for err in errors:
                    print(f"  - {err}")
                sys.exit(1)

            # 3. Create Video Metadata
            video_meta = VideoMetadata(
                video_id=video_id,
                sport=SportType(sport_str),
                video_url=None,
                local_path=args.video_path,
                license_type=license_type,
                license_url=None,
```

```
                is_commercial=is_commercial,
                status=CurationStatus.CURATED,  # Custom local imports bypass staging by
default
                match_name=match_name,
                fps=args.fps,
                resolution=args.resolution
            )

            # 4. Insert into Database
            self.db.insert_video(video_meta)

            count = 0
            if sport_str == "badminton":
                count = self.db.insert_badminton_rallies(annotations)
            elif sport_str == "tennis":
                count = self.db.insert_tennis_points(annotations)
            elif sport_str == "cricket":
                count = self.db.insert_cricket_deliveries(annotations)
            elif sport_str == "pickleball":
                count = self.db.insert_pickleball_rallies(annotations)
            elif sport_str == "tabletennis":
                count = self.db.insert_tabletennis_rallies(annotations)

            print(f"\nSuccessfully imported local video '{match_name}':")
            print(f"  - Video ID: {video_id}")
            print(f"  - Sport: {sport_str}")
            print(f"  - Commercial Use Allowed: {is_commercial} (License:
{license_type.value})")
            print(f"  - Registered {count} annotated segments.\n")

        def _parse_local_annotations(self, sport: str, video_id: str, filepath: str) ->
List[Any]:
            """Parses simple JSON or CSV annotations files."""
            parsed_items = []
            ext = os.path.splitext(filepath)[1].lower()

            if ext == '.json':
                with open(filepath, 'r', encoding='utf-8') as f:
                    data = json.load(f)

                if not isinstance(data, list):
                    raise ValueError("JSON file must contain a list of objects.")

                # Uses the same safe casts as the CSV path: a missing/blank required
                # time raises a clear ValueError (caught by the caller) instead of a
                # bare KeyError, and optional ints degrade to sensible defaults.
                for idx, obj in enumerate(data):
                    if sport == "badminton":
                        parsed_items.append(BadmintonRally(
                            video_id=video_id,
                            rally_number=_safe_int(obj.get("rally_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            score_before=obj.get("score_before"),
                            score_after=obj.get("score_after"),
                            shots_count=_safe_int(obj.get("shots_count"), None),
                            ending_reason=obj.get("ending_reason"),
                            last_shot_outcome=obj.get("last_shot_outcome")
                        ))
                    elif sport == "tennis":
                        parsed_items.append(TennisPoint(
                            video_id=video_id,
                            point_number=_safe_int(obj.get("point_number"), idx + 1),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
```

```
                            set_score=obj.get("set_score"),
                            game_score=obj.get("game_score"),
                            server=obj.get("server"),
                            ending_type=obj.get("ending_type")
                        ))
                    elif sport == "cricket":
                        parsed_items.append(CricketDelivery(
                            video_id=video_id,
                            over=_safe_int(obj.get("over"), 0),
                            ball=_safe_int(obj.get("ball"), 0),
                            start_time=_safe_float(obj.get("start_time")),
                            end_time=_safe_float(obj.get("end_time")),
                            batsman=obj.get("batsman"),
                            bowler=obj.get("bowler"),
                            runs=_safe_int(obj.get("runs"), None),
                            wicket=bool(obj.get("wicket", False)),
                            extras=_safe_int(obj.get("extras"), 0)
                        ))
                    elif sport == "pickleball":
```

247. assistant (2026-06-07T07:37:44.188Z)

Now the collector's CLI command bodies (validate_delivery, convert_cvat, export) and its FAQ doc (to cite real anchors), plus how it's packaged:

248. user (2026-06-07T07:37:45.610Z)

```
        def convert_cvat(self, args):
            """Convert a vendor CVAT rally export into our canonical per-rally CSV.

            Absorbs the CVAT bounding-box delivery format (rally fields carried as box
            attributes) on the ingestion side: each rally's time is recomputed from
            its frame range at the video's true fps, and content defects (overlaps,
            zero-duration, off-vocabulary endings, coarse sampling) are written to an
            issues report rather than silently "fixed". The emitted CSV imports via
            `import-local`; any blocker issue must be resolved first, since the
            importer's validator rejects overlapping / zero-duration rallies.
            """
            from src.parsers.cvat.rally_cvat import (
                convert, write_canonical_csv, render_issue_report,
            )
            try:
                result = convert(args.input, fps=args.fps,
                                 video_path=args.video_path, step=args.step)
            except (FileNotFoundError, ValueError) as e:
                print(f"Error: {e}")
                sys.exit(1)

            write_canonical_csv(result.records, args.out)
            report = render_issue_report(result, os.path.basename(args.input))
            if args.issues:
                with open(args.issues, "w", encoding="utf-8") as f:
                    f.write(report)
            print(report)
            print(f"Canonical CSV written: {args.out}  ({len(result.records)} rallies)")
            if result.n_blockers:
                print(f"\n  {result.n_blockers} BLOCKER issue(s) ? import-local will reject
"
                      f"this file until they are resolved (by the vendor or manual
review).")
            else:
                print("\n  No blockers. Next:")
                print(f"    python -m src.cli.curate import-local --sport <sport> "
                      f"--video-path <video> --annotations-path {args.out} --fps
{result.fps:g}")
```

```

        def validate_delivery(self, args):
            """Run the full validation suite on a rally delivery and emit a report.

            Accepts a CVAT export (--input + --video-path/--fps) or an already-canonical
            CSV (--csv). Writes, into --out-dir:
              <id>_report.md    human report (scoreboard + issues + rallies to review)
              <id>_report.json  machine summary (for CI / a bulk-ingest loop)
              <id>_review.csv   the annotatable manual-verification sheet
            Exits non-zero if any blocker issue exists, so it can gate a bulk pipeline.
            """
            import json
            from src.parsers.cvat.rally_cvat import (
                convert, load_canonical_csv, detect_issues,
                build_summary, render_validation_md, write_review_csv,
            )
            vid = args.video_id or "delivery"
            fps = None
            if args.input:
                try:
                    result = convert(args.input, fps=args.fps, video_path=args.video_path)
                except (FileNotFoundError, ValueError) as e:
                    print(f"Error: {e}")
                    sys.exit(1)
                records, issues, fps = result.records, result.issues, result.fps
            elif args.csv:
                try:
                    records = load_canonical_csv(args.csv)
                except FileNotFoundError as e:
                    print(f"Error: {e}")
                    sys.exit(1)
                if not records:
                    print(f"Error: no usable rally rows in {args.csv}")
                    sys.exit(1)
                issues = detect_issues(records)
            else:
                print("Error: provide --input <cvat.xml> or --csv <canonical.csv>")
                sys.exit(1)

            os.makedirs(args.out_dir, exist_ok=True)
            summary = build_summary(records, issues, source=vid, fps=fps)
            md = render_validation_md(records, issues, source=vid)
            json_path = os.path.join(args.out_dir, f"{vid}_report.json")
            md_path = os.path.join(args.out_dir, f"{vid}_report.md")
            review_path = os.path.join(args.out_dir, f"{vid}_review.csv")
            with open(json_path, "w", encoding="utf-8") as f:
                json.dump(summary, f, indent=2)
            with open(md_path, "w", encoding="utf-8") as f:
                f.write(md)
            write_review_csv(records, issues, review_path)

            print(md)
            print(f"\nWrote:\n  {md_path}\n  {json_path}\n  {review_path}")
            if summary["n_blockers"]:
                print(f"\n  GATE: BLOCKED ? {summary['n_blockers']} blocker(s) must be
resolved.")
                sys.exit(2)
            print("\n  GATE: PASS ? review the flagged rallies, then `import-local`.")

        def curate(self):
            """Interactive loop to review and curate staging video annotations."""
```

249. user (2026-06-07T07:37:45.948Z)

```
1       # Rally Ingestion Pipeline ? End-to-End FAQ
```

How a vendor's rally annotations become training/eval ground truth, and how we
validate each delivery against our conventions. This is the operational companion
to the annotation spec (`docs/annotation/ROORA_BRIEF.md` + `ANNOTATION_GUIDE.md`).

```
  vendor delivery                  OUR side (collector)                 modeling (indexer)
 ??????????????????   convert    ?????????????????  validate  ????????????????
 ? CVAT export    ? ???????????? ? canonical CSV ? ?????????? ? report .md   ?
 ? (boxes + attrs)?              ? (per rally,   ?            ? report .json ?
 ?   OR           ?              ?  decimal secs)?            ? review .csv  ???? human verifies
 ? canonical CSV  ?              ?????????????????            ???????????????? flagged rallies
 ??????????????????                     ?                             ?
                                         ?  import-local (validated)  ?
                                         ?                     golden labels
                                ?????????????????  export   ????????????????
                                ? SQLite (rally ? ????????? ? <id>.csv     ? start,end (indexer GT)
                                ?  tables)      ?           ? manifest.json?
                                ?????????????????           ????????????????
                                                                      ?
                                                              slice clips / calibrate / train
```

The four commands (in order)

```bash
# 1. Normalise a CVAT export ? our canonical per-rally CSV (skip if the vendor sent a CSV)
python -m src.cli.curate convert-cvat --input job.xml \
    --video-path match.mp4 --out canonical.csv --issues issues.txt

# 2. Validate the delivery ? report + annotatable review sheet (the QA gate)
python -m src.cli.curate validate-delivery --input job.xml \
    --video-path match.mp4 --video-id pilot_badminton_01 --out-dir data/validation
#   ...or validate a CSV directly:
python -m src.cli.curate validate-delivery --csv canonical.csv --video-id pilot_badminton_01

# 3. After human review, import the (corrected) CSV into the DB
python -m src.cli.curate import-local --sport badminton \
    --video-path match.mp4 --annotations-path canonical.csv --fps 59.94

# 4. Export indexer-ready ground truth (start,end CSVs + manifest)
python -m src.cli.curate export --out-dir data/eval_export
```

FAQ

What delivery formats do we accept?
Both. A **CVAT export** (image- or track-mode XML ? rally fields carried as box
attributes) is normalised by `convert-cvat`; an **already-canonical CSV** is taken
as-is. Vendors may keep working in CVAT ? we do **not** require them to hand-author
CSVs.

What is the "canonical" CSV?
One row per rally:
`rally_number, start_mmss, end_mmss, start_time, end_time, shots_count, ending_reason, last_shot_outcome, score_before, score_after, notes`.

`start_time`/`end_time` are **decimal seconds**; `import-local` reads those.

How are timestamps handled? (the fps thing)
We recompute each rally's interval from its **frame range × the true fps**
(`seconds = frame / fps`). A vendor's CVAT time attributes are **ignored** ? a
step-sampled job that assumes 30 fps writes systematically wrong times. fps comes
from `--fps` or ffprobe of `--video-path`. `mm:ss` is also accepted everywhere.

What does validation check?
code	severity	meaning
`overlap`	**blocker**	two rallies' intervals overlap (`end[N] > start[N+1]`)
`zero_duration`	**blocker**	`end <= start` (single-sample / stray box)
`frame_out_of_range`	high	a box frame is past the video's last frame
`shot_density`	medium	long rally with implausibly few shots ? likely an **inflated span** (often hides a merged rally)
`possible_missed_rally`	medium	a long unlabeled gap between two rallies ? check for an **omitted rally**
`ending_reason_vocab`	medium	`ending_reason` outside the agreed set
`outcome_vocab`	low	`last_shot_outcome` outside `{in,net,out,n_a}`

Blocker vs. review ? what's the gate?
`validate-delivery` exits **non-zero if any blocker exists** (so it can gate a bulk
loop / CI). Non-blocker issues don't stop the gate but populate the **review sheet**
for the human pass. `import-local` independently rejects overlaps/zero-duration, so
bad ground truth can't slip into the DB.

How are *missed rallies* caught? (the worst error)
Two complementary checks, because a miss shows up two ways:
1. **Clean omission** (a real hole in the timeline) ? `possible_missed_rally` (gap heuristic).
2. **Merged/inflated** (the missed rally got swallowed into a neighbour's box, leaving
   no gap) ? `shot_density` flags the over-long, under-shot rally, and the human finds
   the second rally inside it.

Neither is a guarantee ? they **surface regions to review**, they don't certify
completeness. (A future enhancement: a motion second-opinion over the raw video.)

What's the review sheet?
`<id>_review.csv` ? one row per rally with `auto_flags` (which checks fired) plus
blank `VERIFIED / true_start_mmss / true_end_mmss / true_ending_reason /
true_last_shot_outcome / reviewer_comment` columns. Open it next to the video
(timestamp overlay on) and fill the flagged rows. This is the human gate that turns
a delivery into trusted golden labels.

Why two ending fields (`ending_reason` + `last_shot_outcome`)?
They're orthogonal: `ending_reason` is *why* the point ended (winner / forced_error /
unforced_error / service_fault / let / other); `last_shot_outcome` is *where* the
last shot went (in / net / out / n_a). A shuttle that flew long is
`forced_error/unforced_error` + `out` ? both kept, no information lost.

How do I onboard a brand-new video?
1. Get fps into the intake manifest (we provide it to the vendor).
2. `convert-cvat` (if CVAT) ? `validate-delivery` ? read the report, fill the review
   sheet for flagged rallies ? correct the CSV ? `import-local` ? `export`.
3. The same suite runs for every video, so quality stays consistent at scale.

How do I pull a single rally out to eyeball it?
The slicer (indexer):
```bash
python -m backend.tools.rally_slicer --video match.mp4 \
    --labels golden_labels.csv --rallies 7,16 --out-dir output/rally_clips
```
Clips are padded to match the highlight compiler (0.5s lead / 1.0s tail).

Does coarse frame sampling hurt boundaries?

```
121     Not in practice: ~0.5s sampling slack is absorbed by the highlight compiler's clip
122     padding, so we don't flag sampling density.
123
```

250. assistant (2026-06-07T07:38:00.578Z)

The collector's `docs/INGESTION_PIPELINE.md` has a clean FAQ with a validation-codes table ? perfect `faq_ref` targets. Let me branch the collector, confirm its baseline, and (while there) fix its `build-backend` typo:

251. user (2026-06-07T07:38:04.447Z)

```
Switched to a new branch 'feat/observability-reports'
.................................................................... [ 69%]
...............................                                     [100%]
104 passed in 1.94s
```

252. assistant (2026-06-07T07:39:39.781Z)

Collector baseline green (104). Now I'll build the collector's reporting module. Starting with the adapter (Issue?Flag + recorders) and the collector spec:

253. assistant (2026-06-07T07:39:59.998Z)

```
"""Adapt collector data into an obsreport envelope.

Reuses the existing rally_cvat report primitives (``Issue``, records, severities)
rather than re-deriving anything ? this module only *maps* them onto the shared
report shape. Imported only after ``src.reporting`` confirms obsreport is present,
so it may import obsreport freely.
"""
from __future__ import annotations

from pathlib import Path
from typing import Any, List, Optional

import obsreport

_SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"

# Delivery issue severity -> report flag severity.
_SEVERITY = {"blocker": "error", "high": "warn", "medium": "warn", "low": "info"}
# Every issue code is documented in this FAQ table.
_ISSUE_FAQ = "docs/INGESTION_PIPELINE.md#what-does-validation-check"


def load_spec() -> "obsreport.ReportSpec":
    return obsreport.load_spec(_SPEC_PATH)


def issue_to_flag(issue: Any) -> "obsreport.Flag":
    """Map a rally_cvat.Issue onto a report Flag. Actionable issues (blocker/high/
    medium) also carry a customer_message so the delivery summary shows the vendor
    exactly what to fix, in plain language."""
    sev = _SEVERITY.get(issue.severity, "info")
    customer = issue.message if sev in ("error", "warn") else None
    evidence = {"rally_number": issue.rally_number} if issue.rally_number is not None else {}
    return obsreport.Flag(
        code=issue.code,
        severity=sev,
        message=issue.message,
        stage="validation",
        faq_ref=_ISSUE_FAQ,
```

```
        customer_message=customer,
        evidence=evidence,
    )


def _recorder(source: str, video_path: Optional[str], spec) -> "obsreport.RunRecorder":
    return obsreport.RunRecorder(
        "sports-data-collector",
        video_id=source,
        video_id_kind="human_id",
        video_path=video_path,
        spec=spec if spec is not None else load_spec(),
    )


def build_delivery_recorder(
    *,
    records: List[Any],
    issues: List[Any],
    fps: Optional[float],
    source: str,
    video_path: Optional[str] = None,
    video_duration_s: Optional[float] = None,
    source_format: str = "cvat",
    step: Optional[int] = None,
    spec=None,
) -> "obsreport.RunRecorder":
    """Envelope for `validate-delivery` / `convert-cvat`: parse + validation stages,
    one flag per delivery issue, a rally summary."""
    rec = _recorder(source, video_path, spec)
    if fps:
        rec.set_video_meta(fps=fps, fps_source="declared")
    if video_duration_s:
        rec.set_video_meta(duration_s=round(video_duration_s, 1))

    with rec.stage("parse") as st:
        st.add_metric("rallies_parsed", len(records))
        st.details.update({"source_format": source_format, "fps": fps, "sampling_step": step})

    blockers = [i for i in issues if i.severity == "blocker"]
    with rec.stage("validation") as st:
        for sev in ("blocker", "high", "medium", "low"):
            st.add_metric(f"issues_{sev}", sum(1 for i in issues if i.severity == sev))
        st.add_metric("n_blockers", len(blockers))
        st.details["gate"] = "BLOCKED" if blockers else "PASS"
        st.status = "error" if blockers else ("warn" if issues else "ok")

    for issue in issues:
        rec.envelope.flags.append(issue_to_flag(issue))

    _rally_summary(rec, records, video_duration_s)
    rec.finalize()
    return rec


def build_import_recorder(
    *,
    video_meta_obj: Any,
    annotations: List[Any],
    is_valid: bool,
    errors: List[str],
    count: int,
    sport: str,
    source_format: str,
    spec=None,
```

```
) -> "obsreport.RunRecorder":
    """Envelope for `import-local`: parse + validation + registration stages."""
    rec = _recorder(video_meta_obj.video_id, video_meta_obj.local_path, spec)
    fps = getattr(video_meta_obj, "fps", None)
    rec.set_video_meta(
        fps=fps,
        fps_source="declared" if fps else "unknown",
        resolution=getattr(video_meta_obj, "resolution", None),
    )

    with rec.stage("parse") as st:
        st.add_metric("records_parsed", len(annotations))
        st.details["source_format"] = source_format

    with rec.stage("validation") as st:
        st.status = "ok" if is_valid else "error"
        st.add_metric("valid", is_valid)
        for e in errors or []:
            st.messages.append(str(e))

    with rec.stage("registration") as st:
        st.add_metric("segments_registered", count)
        st.details.update({
            "sport": sport,
            "license": getattr(getattr(video_meta_obj, "license_type", None), "value",
                               str(getattr(video_meta_obj, "license_type", ""))),
            "is_commercial": bool(getattr(video_meta_obj, "is_commercial", False)),
            "status": getattr(getattr(video_meta_obj, "status", None), "value",
                              str(getattr(video_meta_obj, "status", ""))),
        })

    _rally_summary(rec, annotations, None)
    rec.finalize()
    return rec


def _rally_summary(rec: "obsreport.RunRecorder", records: List[Any], video_duration_s:
Optional[float]) -> None:
    total = 0.0
    for r in records:
        try:
            total += float(r.end_time) - float(r.start_time)
        except (AttributeError, TypeError, ValueError):
            pass
    coverage = round(100.0 * total / video_duration_s, 1) if video_duration_s else None
    notes = [] if records else ["No rallies were found in this delivery."]
    rec.set_highlights(segment_count=len(records), total_highlight_s=round(total, 1),
                       coverage_pct=coverage, notes=notes)
```

254. user (2026-06-07T07:40:00.016Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-data-
collector\src\reporting\adapter.py (file state is current in your context ? no need to Read it
back)
```

255. assistant (2026-06-07T07:40:13.267Z)

Observability report spec for the sports-data-collector.
The report ENGINE lives in the shared `obsreport` library; this is the collector-specific layout + a couple of rules. Most delivery findings come straight from rally_cvat's Issue list (mapped 1:1 to flags by the adapter);

the rules below cover signals that aren't delivery "issues".

```
name: collector
title: Sports Data Collector ? Delivery / Ingestion Report

sections:
  - { id: run,        title: "Run",               kind: identity,   audience: both }
  - { id: video,      title: "Video",             kind: video_meta, audience: both }
  - { id: summary,    title: "Rally summary",     kind: highlights, audience: customer }
  - { id: review,     title: "Things to review",  kind: flags,      audience: customer }
  - { id: stages,     title: "Processing stages", kind: stages,     audience: technical }
  - { id: findings,   title: "Findings & issues", kind: flags,     audience: technical }
  - { id: verdict,    title: "Verdict",           kind: verdict,    audience: both }

rules:
  # A non-commercial license means the video is kept but EXCLUDED from the default
  # (commercial-only) export ? easy to forget when it later "goes missing" downstream.
  - code: license_noncommercial
    severity: warn
    when:
      - { path: "stages.registration.details.is_commercial", op: eq, value: false }
    message: >-
      This video's license is non-commercial, so it is EXCLUDED from the default
      commercial-only export. It stays in the DB; pass --include-noncommercial to
      export it.
    faq_ref: "docs/INGESTION_PIPELINE.md#will-a-non-commercial-license-block-export"

  # No fps recorded at import -> downstream export/manifest carries no fps and the
  # indexer must probe the file; provide --fps for accurate frame<->second timing.
  - code: missing_fps
    severity: warn
    when:
      - { path: "video_meta.fps", op: is_missing }
      - { path: "stages.registration.metrics.segments_registered", op: is_present }
    message: >-
      No fps was recorded for this video. The export manifest will lack fps and the
      indexer will have to probe the file. Pass --fps for accurate timing.
    faq_ref: "docs/INGESTION_PIPELINE.md#how-are-timestamps-handled-the-fps-thing"
```

256. user (2026-06-07T07:40:13.285Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-data-
collector\src\reporting\spec.yaml (file state is current in your context ? no need to Read it
back)
```

257. assistant (2026-06-07T07:40:27.180Z)

The soft-import wrapper with never-raising emit functions:

258. assistant (2026-06-07T07:40:30.607Z)

```
"""Per-video observability reports for the collector.

`obsreport` is a SOFT dependency: absent -> these emitters no-op and the CLI runs
unchanged. Every emitter is safe to call from normal control flow and NEVER raises
(emitting a report must not break ingestion). Returns the written paths, or None.
"""
from __future__ import annotations

from typing import Any, Dict, List, Optional

try:  # soft dependency
    import obsreport  # noqa: F401
    from . import adapter
```

```
    OBSREPORT_AVAILABLE = True
except Exception:  # pragma: no cover - only when obsreport is absent
    OBSREPORT_AVAILABLE = False


def emit_delivery_report(
    *,
    out_dir: str,
    source: str,
    records: List[Any],
    issues: List[Any],
    fps: Optional[float],
    video_path: Optional[str] = None,
    video_duration_s: Optional[float] = None,
    source_format: str = "cvat",
    step: Optional[int] = None,
) -> Optional[Dict[str, str]]:
    """Write the delivery observability report into out_dir (alongside the existing
    <id>_report.* artifacts). Used by validate-delivery / convert-cvat. Never raises."""
    if not OBSREPORT_AVAILABLE:
        return None
    try:
        rec = adapter.build_delivery_recorder(
            records=records, issues=issues, fps=fps, source=source, video_path=video_path,
            video_duration_s=video_duration_s, source_format=source_format, step=step,
        )
        res = rec.write(output_dir=out_dir, stem=source)
        return res.paths or None
    except Exception:  # pragma: no cover - reporting must never break the command
        return None


def emit_import_report(
    *,
    video_meta_obj: Any,
    annotations: List[Any],
    is_valid: bool,
    errors: List[str],
    count: int,
    sport: str,
    source_format: str,
) -> Optional[Dict[str, str]]:
    """Write the import observability report next to the imported video. Never raises."""
    if not OBSREPORT_AVAILABLE:
        return None
    try:
        rec = adapter.build_import_recorder(
            video_meta_obj=video_meta_obj, annotations=annotations, is_valid=is_valid,
            errors=errors, count=count, sport=sport, source_format=source_format,
        )
        res = rec.write()
        return res.paths or None
    except Exception:  # pragma: no cover
        return None
```

259. user (2026-06-07T07:40:30.628Z)

File created successfully at: C:\Users\avidu\Projects\sports-data-collector\src\reporting__init__.py (file state is current in your context ? no need to Read it back)

260. assistant (2026-06-07T07:40:53.030Z)

Now wire the three collector commands. First `import_local` (success path):

261. user (2026-06-07T07:40:57.893Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

262. assistant (2026-06-07T07:41:03.750Z)

Now `validate_delivery` (capture step/stop_frame, emit alongside existing artifacts):

263. user (2026-06-07T07:41:05.236Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

264. user (2026-06-07T07:41:12.927Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

265. assistant (2026-06-07T07:41:19.300Z)

Now `convert_cvat`:

266. user (2026-06-07T07:41:20.479Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

267. assistant (2026-06-07T07:41:34.038Z)

Now fix the collector's `build-backend` typo and document the `obsreport` dependency:

268. user (2026-06-07T07:41:35.173Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\pyproject.toml has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

269. user (2026-06-07T07:41:42.132Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\pyproject.toml has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

270. assistant (2026-06-07T07:41:52.364Z)

Now add the two FAQ entries the collector flags cite (so every `faq_ref` resolves):

271. user (2026-06-07T07:41:54.810Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

272. assistant (2026-06-07T07:42:17.871Z)

Two fixes needed: `missing_fps` must match `None` (the key is always present as null, so `is_missing` won't catch it ? use `eq null`). Let me correct the spec, then write tests:

273. user (2026-06-07T07:42:19.748Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\src\reporting\spec.yaml has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

274. assistant (2026-06-07T07:42:47.862Z)

```
"""Tests for the collector observability reports. obsreport is a soft dependency;
skip cleanly if it isn't installed."""
import os
from types import SimpleNamespace

import pytest

pytest.importorskip("obsreport")

from src.parsers.cvat.rally_cvat import Issue, RallyRecord  # noqa: E402
from src.reporting import emit_delivery_report, emit_import_report  # noqa: E402
from src.reporting import adapter  # noqa: E402


def _rec(n, start, end, shots=5):
    return RallyRecord(rally_number=n, start_time=start, end_time=end, shots_count=shots,
                       ending_reason="winner", last_shot_outcome="in", score_before=None,
                       score_after=None, notes=None, frame_min=int(start * 60),
frame_max=int(end * 60), n_boxes=3)


def _codes(env):
    return {f.code for f in env.flags}


## --- issue -> flag ----------------------------------------------------
def test_issue_to_flag_severity_and_customer_message():
    blocker = adapter.issue_to_flag(Issue("blocker", "overlap", "Rally 1 overlaps Rally 2.", 2))
    assert blocker.severity == "error"
    assert blocker.customer_message == "Rally 1 overlaps Rally 2."  # actionable -> shown to
vendor
    assert blocker.faq_ref.endswith("#what-does-validation-check")
    assert blocker.evidence == {"rally_number": 2}

    low = adapter.issue_to_flag(Issue("low", "outcome_vocab", "odd outcome", 3))
    assert low.severity == "info"
    assert low.customer_message is None  # cosmetic -> not surfaced to the vendor


## --- delivery envelope ------------------------------------------------
def test_delivery_with_blocker_fails_and_surfaces_to_customer():
    records = [_rec(1, 0.0, 5.0), _rec(2, 4.0, 9.0)]
    issues = [Issue("blocker", "overlap", "Rally 1 (ends 5.00s) overlaps Rally 2 (starts
4.00s).", 2)]
    rec = adapter.build_delivery_recorder(records=records, issues=issues, fps=59.94,
                                          source="pilot_01", video_duration_s=600.0)
    env = rec.envelope
    assert env.verdict == "fail"
    assert env.get_stage("validation").status == "error"
    assert "overlap" in _codes(env)
    assert env.highlights.segment_count == 2
    assert env.lint() == []

    from obsreport import render_customer, render_technical
    tech = render_technical(env, rec.spec)
    cust = render_customer(env, rec.spec)
    assert "overlap" in tech  # code visible technically
    assert "overlap" not in cust  # code hidden from the customer/vendor view
    assert "overlaps Rally 2" in cust  # but the plain message is shown


def test_delivery_clean_passes():
    records = [_rec(1, 0.0, 5.0), _rec(2, 6.0, 9.0)]
```

```
    rec = adapter.build_delivery_recorder(records=records, issues=[], fps=59.94, source="clean")
    assert rec.envelope.verdict == "pass"
    assert _codes(rec.envelope) == set()
```

--- import envelope ----------

```
def _vmeta(fps=59.94, is_commercial=True):
    return SimpleNamespace(
        video_id="my_match", local_path="data/videos/my_match.mp4", fps=fps, resolution="1080p",
        license_type=SimpleNamespace(value="Proprietary"), is_commercial=is_commercial,
        status=SimpleNamespace(value="curated"),
    )


def test_import_clean_run():
    anns = [SimpleNamespace(start_time=1.0, end_time=6.0), SimpleNamespace(start_time=8.0,
end_time=12.0)]
    rec = adapter.build_import_recorder(video_meta_obj=_vmeta(), annotations=anns,
is_valid=True,
                                        errors=[], count=2, sport="badminton",
source_format="csv")
    env = rec.envelope
    assert env.verdict == "pass"
    assert env.get_stage("registration").details["is_commercial"] is True
    assert env.highlights.segment_count == 2
    assert env.lint() == []


def test_import_noncommercial_warns():
    rec = adapter.build_import_recorder(video_meta_obj=_vmeta(is_commercial=False),
annotations=[],
                                        is_valid=True, errors=[], count=0, sport="badminton",
source_format="json")
    assert "license_noncommercial" in _codes(rec.envelope)


def test_import_missing_fps_warns():
    rec = adapter.build_import_recorder(video_meta_obj=_vmeta(fps=None), annotations=[],
is_valid=True,
                                        errors=[], count=0, sport="badminton",
source_format="csv")
    assert "missing_fps" in _codes(rec.envelope)
    assert rec.envelope.video_meta.fps_source == "unknown"
```

--- emit (file writing) ----------

```
def test_emit_delivery_writes_into_out_dir(tmp_path):
    paths = emit_delivery_report(out_dir=str(tmp_path), source="pilot_01",
                                 records=[_rec(1, 0.0, 5.0)], issues=[], fps=59.94)
    assert paths is not None
    assert (tmp_path / "pilot_01.report.json").exists()
    assert (tmp_path / "pilot_01.report.md").exists()
    assert (tmp_path / "pilot_01.summary.md").exists()


def test_emit_import_writes_next_to_video(tmp_path):
    video = tmp_path / "clip.mp4"
    video.write_bytes(b"x")
    vm = SimpleNamespace(video_id="clip", local_path=str(video), fps=30.0, resolution="720p",
                         license_type=SimpleNamespace(value="MIT"), is_commercial=True,
                         status=SimpleNamespace(value="curated"))
    paths = emit_import_report(video_meta_obj=vm, annotations=[SimpleNamespace(start_time=0.0,
end_time=2.0)],
```

```
                                is_valid=True, errors=[], count=1, sport="badminton",
source_format="csv")
    assert paths is not None
    assert (tmp_path / "clip.report.md").exists()
    assert (tmp_path / "clip.summary.md").exists()
```

275. user (2026-06-07T07:42:47.881Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-data-
collector\tests\test_reporting.py (file state is current in your context ? no need to Read it
back)
```

276. user (2026-06-07T07:42:54.572Z)

```
        from obsreport import render_customer, render_technical
        tech = render_technical(env, rec.spec)
        cust = render_customer(env, rec.spec)
        assert "overlap" in tech  # code visible technically
>       assert "overlap" not in cust  # code hidden from the customer/vendor view
        ^^^^^^^^^^^^^^^^^^^^^^^^^^^^
E       AssertionError: assert 'overlap' not in '# Sports Da...t support.\n'
E
E         'overlap' is contained here:
E           ds 5.00s) overlaps Rally 2 (starts 4.00s).
E         ?           +++++++
E
E           ## Verdict
E
E           ?? We ran into a problem processing this video, so some highlights may be missing.
See the notes above, or contact support.

tests\test_reporting.py:55: AssertionError
========================== short test summary info ==========================
FAILED tests/test_reporting.py::test_delivery_with_blocker_fails_and_surfaces_to_customer
1 failed, 111 passed in 2.15s
```

277. assistant (2026-06-07T07:43:22.236Z)

```
Good catch from the test ? but it revealed a deeper issue: the **customer verdict text is
indexer-flavored** ("highlights may be missing, contact support"), which reads wrong for a
*collector delivery* to a vendor. The clean, reusable fix is a spec-level `customer_verdict`
override so each tool phrases its own verdict. Let me add that to the library:
```

278. user (2026-06-07T07:43:23.430Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\spec.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

279. user (2026-06-07T07:43:28.405Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\spec.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

280. assistant (2026-06-07T07:43:35.924Z)

```
Now thread the spec into the verdict renderer:
```

281. user (2026-06-07T07:43:37.240Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\render\markdown.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

282. user (2026-06-07T07:43:43.240Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\render\markdown.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

283. user (2026-06-07T07:43:48.043Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\render\markdown.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

284. assistant (2026-06-07T07:43:53.499Z)

Bump the lib version, add a test for the override, regen schema, run, and reinstall:

285. user (2026-06-07T07:43:54.475Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\version.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

286. user (2026-06-07T07:43:58.634Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\pyproject.toml has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

287. user (2026-06-07T07:44:07.982Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\tests\test_customer_message.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

288. user (2026-06-07T07:44:21.919Z)

```
wrote C:\Users\avidu\Projects\sports-obsreport\schema\report_envelope.schema.json
.................................................................... [ 98%]
.                                                                    [100%]
73 passed in 0.33s
warning: in the working copy of 'pyproject.toml', LF will be replaced by CRLF the next time Git
touches it
warning: in the working copy of 'src/obsreport/render/markdown.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'src/obsreport/spec.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'src/obsreport/version.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'tests/test_customer_message.py', LF will be replaced by CRLF
the next time Git touches it
[notice] To update, run: python.exe -m pip install --upgrade pip
reinstalled 0.1.2
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

289. assistant (2026-06-07T07:44:33.375Z)

73 green, v0.1.2 reinstalled. Now add per-tool `customer_verdict` to both specs and fix the test assertion (it tripped on the substring "overlap**s**" ? should check the bolded code token):

290. user (2026-06-07T07:44:35.914Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\src\reporting\spec.yaml has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

291. user (2026-06-07T07:44:41.446Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

292. user (2026-06-07T07:44:47.520Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\tests\test_reporting.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

293. user (2026-06-07T07:44:53.970Z)

```
.................................................................... [ 64%]
.......................................                             [100%]
112 passed in 1.83s
```

294. assistant (2026-06-07T07:45:04.402Z)

112 passed. Now an end-to-end smoke through the real collector CLI ? `validate-delivery` on a CSV with an overlap (blocker), then read the generated reports:

295. user (2026-06-07T07:45:07.981Z)

```
exit=0 (2 = blocked, expected)
=== files ===
smoke_delivery.report.json
smoke_delivery.report.md
smoke_delivery.summary.md
smoke_delivery_report.json
smoke_delivery_report.md
smoke_delivery_review.csv
```

296. assistant (2026-06-07T07:45:32.921Z)

Reports generated alongside the existing artifacts. The exit code reading looks off (expected 2 for a blocker) ? let me read the generated reports to confirm the blocker was detected, and re-check the gate exit code directly:

297. user (2026-06-07T07:45:34.155Z)

```
1       # Sports Data Collector ? Delivery / Ingestion Report (summary)
2
3       ## Run
4
5       | Field | Value |
6       | --- | --- |
7       | Video | smoke_delivery |
8       | Report generated | 2026-06-07T07:45:07Z |
9
10      ## Video
11
12      _(none)_
13
14      ## Rally summary
15
16      - **2** highlights
17      - **8.0s** of play
18
19      ## Things to review
20
21      - ?? 21.0s of unlabeled time between Rally 2 (ends 9.0s) and Rally 3 (starts 30.0s) ?
check for an unlabeled rally in the gap.
22
23      ## Verdict
24
25      This delivery is importable, but please review the flagged rallies above.
26
```

298. user (2026-06-07T07:45:34.281Z)

```
1       # Sports Data Collector ? Delivery / Ingestion Report
```

```
2
3       ## Run
4
5       | Field | Value |
6       | --- | --- |
7       | Tool | sports-data-collector |
8       | Video id | smoke_delivery (human_id) |
9       | Generated | 2026-06-07T07:45:07Z |
10      | Schema | 1.0 |
11
12      ## Video
13
14      | Field | Value |
15      | --- | --- |
16      | FPS | ? (unknown) |
17
18      ## Processing stages
19
20      **parse** ? `ok` (0.0s)
21        - rallies_parsed: 2
22        - _source_format_: csv
23        - _fps_: None
24        - _sampling_step_: None
25
26      **validation** ? `warn` (0.0s)
27        - issues_blocker: 0
28        - issues_high: 0
29        - issues_medium: 1
30        - issues_low: 0
31        - n_blockers: 0
32        - _gate_: PASS
33
34      ## Findings & issues
35
36      - [WARN] **possible_missed_rally** ? 21.0s of unlabeled time between Rally 2 (ends 9.0s)
and Rally 3 (starts 30.0s) ? check for an unlabeled rally in the gap.
37        ? see **docs/INGESTION_PIPELINE.md#what-does-validation-check**
38        ? evidence: rally_number=3
39
40      ## Verdict
41
42      **PASS (with warnings)**
43
```

299. assistant (2026-06-07T07:46:02.119Z)

```
The report is correct, but my smoke surfaced a **pre-existing collector bug**: only 2 of 3
rallies parsed, and rally 1 (which starts at `0`) was silently dropped. Cause:
`load_canonical_csv` uses `start = _parse_secs(start_time) or _parse_secs(start_mmss)` ? a
legitimate `0.0` start is falsy, so it's discarded. That's out of scope for this feature, so
I'll flag it for a separate fix and re-smoke with a nonzero overlap to verify the
blocker?error?customer path.
```

300. user (2026-06-07T07:46:08.549Z)

```
Noted (position 1, task_id: task_7a3d8081). A chip is showing for the user ? they can start it
in a fresh worktree with one click, or dismiss it. If this suggestion becomes stale or
superseded, call dismiss_task with this task_id. Currently pending: task_7a3d8081 "Fix
load_canonical_csv dropping rally at 0.0s". Continue your current work.
```

301. user (2026-06-07T07:46:17.381Z)

```
exit=2 (expect 2 = blocked)
```

302. assistant (2026-06-07T07:46:23.116Z)

Exit 2 (blocker correctly detected). Let me confirm the customer summary shows the collector-specific fail verdict and the blocker in plain language:

303. user (2026-06-07T07:46:23.647Z)

```
1       # Sports Data Collector ? Delivery / Ingestion Report (summary)
2
3       ## Run
4
5       | Field | Value |
6       | --- | --- |
7       | Video | smoke2 |
8       | Report generated | 2026-06-07T07:46:17Z |
9
10      ## Video
11
12      _(none)_
13
14      ## Rally summary
15
16      - **2** highlights
17      - **10.0s** of play
18
19      ## Things to review
20
21      - ? Rally 1 (ends 7.00s) overlaps Rally 2 (starts 6.00s) by 1.00s ? rallies must be
disjoint.
22
23      ## Verdict
24
25      This delivery has blocking issues that must be fixed before it can be imported ? see
'Things to review'.
26
```

304. assistant (2026-06-07T07:46:52.157Z)

The collector customer summary is correct, but it calls rallies "highlights" ? wrong noun for a data-annotation delivery. Let me make the highlights section's item noun configurable in the library (last lib polish), so each tool uses its own word:

305. user (2026-06-07T07:46:55.270Z)

The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\spec.py has been updated successfully. (file state is current in your context ? no need to Read it back)

306. user (2026-06-07T07:47:05.014Z)

The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\render\markdown.py has been updated successfully. (file state is current in your context ? no need to Read it back)

307. user (2026-06-07T07:47:12.949Z)

The file C:\Users\avidu\Projects\sports-obsreport\src\obsreport\render\markdown.py has been updated successfully. (file state is current in your context ? no need to Read it back)

308. assistant (2026-06-07T07:47:21.999Z)

Bump to v0.1.3, add a test, regen schema, run, reinstall:

309. user (2026-06-07T07:47:24.209Z)

bumped

```
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

310. user (2026-06-07T07:47:31.437Z)

```
The file C:\Users\avidu\Projects\sports-obsreport\tests\test_render_markdown.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

311. user (2026-06-07T07:47:45.106Z)

```
.......................................................................... [ 97%]
..                                                                       [100%]
74 passed in 0.36s
warning: in the working copy of 'src/obsreport/render/markdown.py', LF will be replaced by CRLF
the next time Git touches it
warning: in the working copy of 'src/obsreport/spec.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'tests/test_render_markdown.py', LF will be replaced by CRLF the
next time Git touches it
[notice] To update, run: python.exe -m pip install --upgrade pip
reinstalled 0.1.3
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

312. assistant (2026-06-07T07:47:51.068Z)

Now point the collector's summary section at the `rally` noun and bump the dep pins to v0.1.3:

313. user (2026-06-07T07:47:52.479Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\src\reporting\spec.yaml has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

314. user (2026-06-07T07:48:00.616Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\pyproject.toml has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

315. user (2026-06-07T07:48:05.752Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\requirements.txt has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

316. assistant (2026-06-07T07:48:10.935Z)

Re-run both suites to confirm green against lib v0.1.3, then commit Phase B:

317. user (2026-06-07T07:48:27.820Z)

```
=== collector ===
.......................................................................... [ 63%]
.....................................                                    [100%]
113 passed in 1.86s
=== indexer ===
.......................................................................... [ 64%]
.......................................................................... [ 97%]
......                                                                   [100%]
222 passed in 11.79s
```

318. assistant (2026-06-07T07:48:39.252Z)

Both green (collector 113, indexer 222). Committing Phase B:

319. user (2026-06-07T07:48:41.624Z)

```
warning: in the working copy of 'nonexistent.report.json', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'nonexistent.report.md', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'nonexistent.summary.md', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'src/reporting/__init__.py', LF will be replaced by CRLF the
next time Git touches it
warning: in the working copy of 'src/reporting/adapter.py', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'src/reporting/spec.yaml', LF will be replaced by CRLF the next
time Git touches it
warning: in the working copy of 'tests/test_reporting.py', LF will be replaced by CRLF the next
time Git touches it
de5c0ff feat(reporting): per-video observability report (collector)
```

320. assistant (2026-06-07T07:49:02.151Z)

Stray test artifacts (`nonexistent.report.*`) got committed to the collector root ? a test wrote a report whose path resolved to the repo cwd. Let me find the source and clean it up:

321. user (2026-06-07T07:49:03.267Z)

```
nonexistent.report.json:18:    "video_path": "nonexistent.mp4",
nonexistent.report.md:9:| Video path | nonexistent.mp4 |
nonexistent.summary.md:7:| Video | nonexistent.mp4 |
tests\test_cli.py:39:        sport=None, video_path="nonexistent.mp4", annotations_path=None,
tests\test_validator.py:13:    return DatasetValidator(config_path="nonexistent_config.yaml")
```

322. assistant (2026-06-07T07:49:35.706Z)

The existing `test_cli.py` exercises `import-local` with `video_path="nonexistent.mp4"`, so my emit wrote the report to the repo cwd. Let me look at that test and fix the behavior (don't litter cwd) + gitignore the artifacts:

323. user (2026-06-07T07:49:36.924Z)

```
1       """End-to-end coverage for the SportsDataCLI commands.
2
3       Each test runs against a throwaway config + SQLite DB in tmp_path, so the real
4       data/ directory is never touched. Network (yt-dlp) and the browser are mocked.
5       """
6       import csv as _csv
7       import json
8       from types import SimpleNamespace
9       from unittest import mock
10
11      import pytest
12      import yaml
13
14      from src.cli.curate import SportsDataCLI
15      from src.core.schemas.base import VideoMetadata, SportType, LicenseType, CurationStatus
16      from src.core.schemas.badminton import BadmintonRally
17
18
19      @pytest.fixture
20      def cli(tmp_path):
21          cfg = {
22              "database_path": str(tmp_path / "test.db"),
23              "video_storage": {
24                  "local_dir": str(tmp_path / "videos"),
25                  "max_download_size_mb": 100,
26                  "auto_download": False,
27                  "max_height": None,
```

```
            },
            "commercial_licenses": ["MIT", "Apache-2.0", "BSD", "CC0", "CC-BY", "Public-
Domain", "Proprietary"],
            "supported_sports": ["badminton", "tennis", "tabletennis", "pickleball",
"cricket"],
        }
        cfg_path = tmp_path / "config.yaml"
        cfg_path.write_text(yaml.safe_dump(cfg), encoding="utf-8")
        return SportsDataCLI(config_path=str(cfg_path))


    def _import_args(**kw):
        base = dict(
            sport=None, video_path="nonexistent.mp4", annotations_path=None,
            match_name=None, video_id=None, license="Proprietary", fps=30.0,
resolution="1080p",
        )
        base.update(kw)
        return SimpleNamespace(**base)


    def _write_csv(path, header, rows):
        with open(path, "w", newline="", encoding="utf-8") as f:
            w = _csv.writer(f)
            w.writerow(header)
            for r in rows:
                w.writerow(r)


    # ---------------------------------------------------------------------------- #
    # import-local: every sport, both JSON and CSV (covers dispatch + parse)       #
    # ---------------------------------------------------------------------------- #

    def test_import_local_badminton_json(cli, tmp_path):
        p = tmp_path / "bad.json"
        p.write_text(json.dumps([
            {"rally_number": 1, "start_time": 10.0, "end_time": 20.0, "shots_count": 5,
"ending_reason": "winner"},
            {"rally_number": 2, "start_time": 25.0, "end_time": 30.0, "shots_count": 1,
"ending_reason": "service_fault"},
        ]), encoding="utf-8")
        cli.import_local(_import_args(sport="badminton", annotations_path=str(p),
video_id="bad1", match_name="Bad"))
        assert len(cli.db.get_badminton_rallies("bad1")) == 2


    def test_import_local_pickleball_csv(cli, tmp_path):
        p = tmp_path / "pb.csv"
        _write_csv(p, ["rally_number", "start_time", "end_time", "shots_count",
"ending_reason"],
                   [[1, 10.0, 20.0, 6, "winner"], [2, 25.0, 27.0, 1, "service_fault"]])
        cli.import_local(_import_args(sport="pickleball", annotations_path=str(p),
video_id="pb1", match_name="PB"))
        assert len(cli.db.get_pickleball_rallies("pb1")) == 2


    def test_import_local_tabletennis_json(cli, tmp_path):
        p = tmp_path / "tt.json"
        p.write_text(json.dumps([
            {"rally_number": 1, "start_time": 5.0, "end_time": 9.0, "shots_count": 7,
"ending_reason": "winner"},
        ]), encoding="utf-8")
        cli.import_local(_import_args(sport="tabletennis", annotations_path=str(p),
video_id="tt1", match_name="TT"))
        assert len(cli.db.get_tabletennis_rallies("tt1")) == 1
```

```
83
84
85      def test_import_local_tennis_csv(cli, tmp_path):
86          p = tmp_path / "tn.csv"
87          _write_csv(p, ["point_number", "start_time", "end_time", "server", "ending_type"],
88                     [[1, 10.0, 20.0, "player1", "winner"]])
89          cli.import_local(_import_args(sport="tennis", annotations_path=str(p),
video_id="tn1", match_name="TN"))
90          assert len(cli.db.get_tennis_points("tn1")) == 1
```

324. assistant (2026-06-07T07:50:05.997Z)

Confirmed ? those tests register with a non-existent video path, so the report landed in cwd. The right behavior: when the video file doesn't exist on disk, write the report next to the **annotations file** (which always exists) instead of cwd. Let me fix `emit_import_report` and gitignore the artifacts:

325. user (2026-06-07T07:50:07.696Z)

The file C:\Users\avidu\Projects\sports-data-collector\src\reporting__init__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

326. user (2026-06-07T07:50:17.189Z)

The file C:\Users\avidu\Projects\sports-data-collector\src\reporting__init__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

327. user (2026-06-07T07:50:23.003Z)

The file C:\Users\avidu\Projects\sports-data-collector\src\cli\curate.py has been updated successfully. (file state is current in your context ? no need to Read it back)

328. assistant (2026-06-07T07:50:29.607Z)

Now gitignore the report artifacts in both repos and remove the stray committed files:

329. user (2026-06-07T07:50:29.992Z)

```
1       # ---- Python ----
2       __pycache__/
3       *.py[cod]
4       *$py.class
5       *.egg-info/
6       .eggs/
7       build/
8       dist/
9       .installed.cfg
10      *.egg
11
12      # ---- Test / coverage ----
13      .pytest_cache/
14      .coverage
15      .coverage.*
16      htmlcov/
17      .tox/
18      .cache/
19
20      # ---- Virtual environments ----
21      .venv/
22      venv/
23      env/
24      ENV/
25
26      # ---- Secrets / local config ----
```

```
27      .env
28      .env.*
29
30      # ---- OS / editor cruft ----
31      .DS_Store
32      Thumbs.db
33      .idea/
34      .vscode/
35      *.swp
36
37      # ---- Project: large media kept outside git (see README) ----
38      # The video corpus is hundreds of MB+; only metadata (the SQLite DB) and the
39      # small source annotation CSVs are version-controlled. Re-fetch videos from the
40      # stored video_url / local_path per the manifest.
41      data/videos/
42
43      # ---- Generated eval/training exports (regenerated via `curate export`) ----
44      data/eval_export*/
45
46      # ---- Generated temp artifacts (crawler scratch space; regenerated each run) ----
47      data/temp/
48
```

330. user (2026-06-07T07:50:35.235Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\.gitignore has been updated successfully.
(file state is current in your context ? no need to Read it back)
```

331. user (2026-06-07T07:50:43.307Z)

```
.................................................................... [ 63%]
.....................................                              [100%]
113 passed in 1.87s
=== stray files at root? ===
none (clean)
```

332. assistant (2026-06-07T07:50:51.472Z)

No more cwd litter. Let me fold these fixes into the B commit (unpushed) and add the same gitignore guard to the indexer:

333. user (2026-06-07T07:50:53.202Z)

```
warning: in the working copy of 'src/reporting/__init__.py', LF will be replaced by CRLF the
next time Git touches it
d4f9c15 feat(reporting): per-video observability report (collector)
 .gitignore                                    |   5 +
 data/roora_badminton_canonical.csv            |  25 ++++
 data/roora_badminton_issues.txt               |  16 +++
 data/sports_metadata.db                       | Bin 184320 -> 200704 bytes
 data/validation/pilot_badminton_01_report.json |  45 ++++++++
 data/validation/pilot_badminton_01_report.md   |  16 +++
 data/validation/pilot_badminton_01_review.csv  |  25 ++++
 docs/INGESTION_PIPELINE.md                    |  22 ++++
 pyproject.toml                                |  11 +-
 src/cli/curate.py                             |  39 +++++++
 src/parsers/cvat/rally_cvat.py                |   8 +-
 src/reporting/__init__.py                     |  77 ++++++++++++
 src/reporting/adapter.py                      | 151 ++++++++++++++++++++++++
 src/reporting/spec.yaml                       |  47 ++++++++
 tests/test_cvat_parser.py                     |  15 +++
 tests/test_reporting.py                       | 120 ++++++++++++++++++++
 16 files changed, 619 insertions(+), 3 deletions(-)
=== indexer stray check ===
```

```
 M backend/reporting/spec.yaml
 M requirements.txt
```

334. assistant (2026-06-07T07:51:26.218Z)

I made a mistake ? `git add -A` swept in **pre-existing working-tree changes that aren't mine**
(the `data/roora_*` artifacts, the SQLite DB, and modifications to
`rally_cvat.py`/`test_cvat_parser.py` that I never touched). I must not commit those. Let me
undo the commit (keeping all changes) and inspect what was already in the tree:

335. user (2026-06-07T07:51:27.970Z)

```
=== status ===
 M .gitignore
 M data/sports_metadata.db
 M docs/INGESTION_PIPELINE.md
 M pyproject.toml
 M src/cli/curate.py
 M src/parsers/cvat/rally_cvat.py
 M tests/test_cvat_parser.py
?? data/roora_badminton_canonical.csv
?? data/roora_badminton_issues.txt
?? data/validation/
?? src/reporting/
?? tests/test_reporting.py
=== diff: rally_cvat.py (NOT mine?) ===
diff --git a/src/parsers/cvat/rally_cvat.py b/src/parsers/cvat/rally_cvat.py
index 6b9c595..6b18266 100644
--- a/src/parsers/cvat/rally_cvat.py
+++ b/src/parsers/cvat/rally_cvat.py
@@ -439,8 +439,12 @@ def load_canonical_csv(csv_path: str) -> List[RallyRecord]:
             rn = safe_int(row.get("rally_number"), None)
             if rn is None:
                 continue
-            start = _parse_secs(row.get("start_time")) or _parse_secs(row.get("start_mmss"))
-            end = _parse_secs(row.get("end_time")) or _parse_secs(row.get("end_mmss"))
+            start = _parse_secs(row.get("start_time"))
+            if start is None:
+                start = _parse_secs(row.get("start_mmss"))
+            end = _parse_secs(row.get("end_time"))
+            if end is None:
+                end = _parse_secs(row.get("end_mmss"))
             if start is None or end is None:
                 continue
             records.append(RallyRecord(
```

336. assistant (2026-06-07T07:52:37.137Z)

The `rally_cvat.py` change is exactly the falsy-zero fix from my spawned task (the background
agent applied it here, with a regression test in `test_cvat_parser.py`). That's good ? but it's
a **separate concern** from the observability feature. Let me verify the test, then commit them
as two distinct commits, leaving the unrelated `data/*` artifacts uncommitted:

337. user (2026-06-07T07:52:38.636Z)

```
diff --git a/tests/test_cvat_parser.py b/tests/test_cvat_parser.py
index 4c241f8..11fb310 100644
--- a/tests/test_cvat_parser.py
+++ b/tests/test_cvat_parser.py
@@ -204,6 +204,21 @@ def test_load_canonical_csv_decimal_and_mmss(tmp_path):
     assert recs[1].start_time == 29.0 and recs[1].end_time == 32.0   # mm:ss fallback


+def test_load_canonical_csv_keeps_rally_starting_at_zero(tmp_path):
```

```
+    # Regression: a rally that legitimately starts at 0.0s must not be dropped.
+    # The old `start_time or start_mmss` fallback treated the falsy 0.0 as "missing"
+    # and skipped the row when there was no start_mmss column to fall back to.
+    p = tmp_path / "c.csv"
+    p.write_text(
+        "rally_number,start_time,end_time,shots_count,ending_reason,last_shot_outcome\n"
+        "1,0.0,4.0,8,winner,in\n"
+        "2,10.0,14.0,6,forced_error,out\n"
+        "3,20.0,24.0,5,unforced_error,out\n", encoding="utf-8")
+    recs = load_canonical_csv(str(p))
+    assert len(recs) == 3                      # the zero-start row 1 is not lost
+    assert recs[0].start_time == 0.0 and recs[0].end_time == 4.0
+
+
 def test_validation_artifacts(tmp_path):
     recs = [_rec(1, 1.0, 5.0), _rec(2, 4.0, 8.0, reason="into_net")]  # overlap + bad vocab
     issues = detect_issues(recs)
```

338. assistant (2026-06-07T07:52:50.605Z)

Both changes are clean and tested. Committing them separately, leaving the unrelated `data/*` artifacts untouched:

339. user (2026-06-07T07:52:53.791Z)

```
warning: in the working copy of 'src/reporting/__init__.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/reporting/adapter.py', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'src/reporting/spec.yaml', LF will be replaced by CRLF the next time Git touches it
warning: in the working copy of 'tests/test_reporting.py', LF will be replaced by CRLF the next time Git touches it
=== log ===
028937b fix(parser): keep a rally that starts at 0.0s in load_canonical_csv
bb2b5be feat(reporting): per-video observability report (collector)
d68c5da Merge pull request #9 from avidullu/feat/validate-delivery
=== remaining unstaged (should be data/* only) ===
 M data/sports_metadata.db
?? data/roora_badminton_canonical.csv
?? data/roora_badminton_issues.txt
?? data/validation/
```

340. assistant (2026-06-07T07:54:24.155Z)

Clean now ? only pre-existing `data/*` artifacts remain unstaged. The parser fix landed as its own cherry-pickable commit, so I'll dismiss its chip (done this session). Marking Phase B complete and starting **Phase D** (debuggability eval + golden regression).

341. user (2026-06-07T07:54:25.639Z)

Task task_7a3d8081 was already started by the user ? it's no longer pending and can't be withdrawn. Nothing was changed.

342. user (2026-06-07T07:54:26.085Z)

Updated task #6 status

343. user (2026-06-07T07:54:26.412Z)

Updated task #7 status

344. assistant (2026-06-07T07:55:23.611Z)

The chip was already started by the user in a separate session ? so the parser fix is being handled there too. My copy is an isolated, cherry-pickable commit on the feature branch; I'll leave it (removing it risks clobbering the parallel session's files) and flag the potential duplicate for merge-time dedupe. Let me note this in memory, then proceed with Phase D.

345. user (2026-06-07T07:55:41.311Z)

The file c:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

346. user (2026-06-07T07:56:03.744Z)

[Request interrupted by user]

347. user (2026-06-07T07:56:03.764Z)

Make sure to sync your repo to remote. I have started a parallel branch of work with Grok. I trust you more with merge conflicts than Grok :)

Do remember to "... that you iterate on the quality of the report by evaluating it as if "someone who is reading the README file and some FAQs is running the tool and he should be able to make sense of the report and do an initial debugging if required" as a stretch goal" -- I would want you to do this exercise after the first version is built and maybe find some refinements in it, you can take a screenshot image of the report and "eyeball" for sure!

348. assistant (2026-06-07T07:56:47.761Z)

Understood on all three. Let me first get everything into a clean, committed state and check remotes/auth before pushing. I noticed the indexer still has uncommitted spec/requirements tweaks from the lib bumps.

349. user (2026-06-07T07:56:51.586Z)

```
=== gh auth ===
github.com
  ? Logged in to github.com account avidullu (keyring)
  - Active account: true
  - Git operations protocol: https
  - Token: gho_**********************************
=== indexer remote + status ===
origin  https://github.com/avidullu/badminton-highlight-indexer.git (fetch)
origin  https://github.com/avidullu/badminton-highlight-indexer.git (push)
 M backend/reporting/spec.yaml
 M requirements.txt
=== collector remote + status ===
origin  https://github.com/avidullu/sports-data-collector.git (fetch)
origin  https://github.com/avidullu/sports-data-collector.git (push)
 M data/sports_metadata.db
?? data/roora_badminton_canonical.csv
?? data/roora_badminton_issues.txt
?? data/validation/
=== obsreport remote (expect none) + branch ===
main
```

350. assistant (2026-06-07T07:57:14.588Z)

Auth and remotes look good (gh as `avidullu`, https). The indexer has the `customer_verdict` + dep-pin tweaks still uncommitted. Let me commit those, then fetch both origins to base my branches on the latest HEAD before pushing.

351. user (2026-06-07T07:57:21.881Z)

warning: in the working copy of 'backend/reporting/spec.yaml', LF will be replaced by CRLF the

```
next time Git touches it
indexer committed
indexer: local feat vs origin/master:
5       2
collector: origin branches:
  origin/feat/cvat-rally-adapter
  origin/feat/last-shot-outcome
  origin/feat/validate-delivery
  origin/master
  origin/refactor/ending-reason-forced-unforced
collector: origin/master vs HEAD:
0       2
```

352. assistant (2026-06-07T07:57:35.337Z)

The indexer's `origin/master` is **5 commits ahead** of my branch base ? either I branched from a stale local master or the parallel work landed there. Let me inspect those commits before rebasing:

353. user (2026-06-07T07:57:38.085Z)

```
=== origin/master recent ===
ce6a0e9 Merge pull request #39 from avidullu/feat/config-pydantic-package
da6ea5c feat(config): introduce backend/config/ package as first low-regret deliverable
f4c1b2d docs: checkpoint approved structure + sequencing decisions (Config first, atomic
refactor follow-up)
9029415 docs: checkpoint approved structure + sequencing decisions (Config first, atomic
refactor follow-up)
8718f1f docs: add Commercial Readiness Review + Low-Regret Moves starter plan (Config as
Deliverable 1) (#37)
8aae6b0 docs: quantitative audio (E1) findings + reusable lessons (#36)
64ccc50 feat(audio): shuttle-hit detector + E1 go/no-go probe (ADR-007) (#35)
=== my branch's 2 commits ===
bc0fe21 chore(reporting): per-tool customer_verdict + bump obsreport pin to v0.1.3
9dbc764 feat(reporting): always-on per-video observability report (indexer)
8aae6b0 docs: quantitative audio (E1) findings + reusable lessons (#36)
=== merge-base ===
8aae6b0 docs: quantitative audio (E1) findings + reusable lessons (#36)
=== local master vs origin/master ===
0       5
```

354. assistant (2026-06-07T07:57:53.156Z)

Clear picture: `origin/master` advanced 5 commits (the "Config-first / low-regret" work ? a new `backend/config/` package + docs, likely touching `main.py`). My branch is based on the older `#36`. Let me rebase onto `origin/master` and resolve any conflicts (you trust me here).

355. user (2026-06-07T07:57:53.943Z)

```
Auto-merging backend/main.py
CONFLICT (content): Merge conflict in backend/main.py
Rebasing (1/2)
error: could not apply 9dbc764... feat(reporting): always-on per-video observability report
(indexer)
hint: Resolve all conflicts manually, mark them as resolved with
hint: "git add/rm <conflicted_files>", then run "git rebase --continue".
hint: You can instead skip this commit: run "git rebase --skip".
hint: To abort and get back to the state before "git rebase", run "git rebase --abort".
hint: Disable this message with "git config advice.mergeConflict false"
Could not apply 9dbc764... feat(reporting): always-on per-video observability report (indexer)
```

356. assistant (2026-06-07T07:58:34.316Z)

A real conflict: the parallel work migrated `global_config` to a **typed `AppConfig`** (Pydantic

model with attribute access), while my reporting layer assumes a **dict**. I need to reconcile both correctly. Let me understand the new config package and see the full conflict + what else the config commit touched.

357. user (2026-06-07T07:58:35.826Z)

```
=== what da6ea5c changed ===
commit da6ea5c21e4c5df762aa60e27fd6336850e9bc2a
Author: avidullu <avidullu@users.noreply.github.com>
Date:   Sun Jun 7 07:38:12 2026 +0000

    feat(config): introduce backend/config/ package as first low-regret deliverable

    - backend/config/models.py: full Pydantic AppConfig + nested models (single source of truth
for defaults).
    - backend/config/loader.py: load_config + save_default_config with file override + graceful
migration (extra=allow).
    - backend/config/__init__.py: clean public API.
    - Minimal dict-like .get / [] compat on AppConfig for transition period (legacy call sites
that pass config objects).
    - Updated setup.py:init_default_config to delegate to the canonical loader.
    - Updated backend/main.py: typed global_config (AppConfig), attribute access in hot paths,
legacy load_config wrapper still returns dict for diagnose_clip etc.
    - Added tests/test_config.py covering defaults, overrides, extra keys, roundtrip.

    This is the code implementation of Deliverable 1 from the approved plan.
    Downstream consumers (validators, segmenters, etc.) will be migrated to typed access in
follow-up PRs / during the structure refactor.

    Co-authored with avidullu.

 backend/config/__init__.py |  19 +++++++
 backend/config/loader.py   |  72 ++++++++++++++++++++++
 backend/config/models.py   | 139 ++++++++++++++++++++++++++++++++++++++++++++++++++++++++++++
 backend/main.py            |  50 ++++++++++++------
 setup.py                   |  62 +++++++++++++++++----
 tests/test_config.py       |  85 ++++++++++++++++++++++++++++
 6 files changed, 364 insertions(+), 63 deletions(-)
=== backend/config files ===
__init__.py
loader.py
models.py
```

358. assistant (2026-06-07T07:58:46.393Z)

The commit message says AppConfig has **dict-like `.get`/`[]` compat** for the transition ? so my `config.get(...)` calls may already work. Let me verify the depth of that compat (does nested `.get("indexing").get("default_segmenter")` work?) before resolving:

359. user (2026-06-07T07:58:47.056Z)

```
1       """Pydantic models for the application configuration.
2
3       This module defines the canonical, validated, typed configuration for the
4       Badminton Highlight Indexer. Built-in defaults live here (single source of truth).
5
6       The models support graceful migration from old config.json files via extra="allow".
7       """
8
9       from __future__ import annotations
10
11      from typing import Any, Literal
12
13      from pydantic import BaseModel, Field, field_validator
```

```
14
15      Sport = Literal["badminton", "tennis", "pickleball", "table_tennis", "cricket"]
16
17
18      class ValidationRelevanceConfig(BaseModel):
19          court_green_lower: list[int] = Field(default=[35, 40, 40])
20          court_green_upper: list[int] = Field(default=[85, 255, 255])
21          court_pixels_min_ratio: float = Field(default=0.05, ge=0.0, le=1.0)
22
23
24      class ValidationConfig(BaseModel):
25          min_resolution_width: int = 1280
26          min_resolution_height: int = 720
27          min_fps: float = 29.9
28          min_blur_threshold: float = 20.0
29          max_scene_cuts_per_minute: float = 0.5
30          relevance: ValidationRelevanceConfig =
Field(default_factory=ValidationRelevanceConfig)
31
32
33      class TrackNetConfig(BaseModel):
34          wsl_distro: str = "Ubuntu"
35          repo_dir: str = "~/models/TrackNetV4"
36          conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
37          conda_env: str = "TrackNetV4"
38          weights_path: str = ""
39          queue_length: int = 5
40          stage_in_wsl: bool = True
41          wsl_stage_dir: str = "~/clips"
42          timeout_sec: int = 0
43          velocity_threshold: float = 0.02
44
45
46      class IndexingConfig(BaseModel):
47          default_segmenter: str = "yolo_hybrid"
48          use_gpu: bool = True
49          motion_threshold: float = 0.08
50          min_segment_duration: float = 3.0
51          dead_time_merge_gap: float = 2.0
52          chunk_duration: float = 90.0
53          chunk_overlap: float = 10.0
54          detector_model: str = "fasterrcnn_resnet50_fpn"
55          detector_score_threshold: float = 0.5
56          yolo_velocity_threshold: float = 0.15
57          yolo_frame_skip: int = 3
58          yolo_tracker: str = "bytetrack.yaml"
59          yolo_parallel_chunks: bool = False
60          yolo_chunk_workers: int = 2
61          min_action_window_duration: float = 2.0
62          log_yolo_candidates: bool = True
63          store_yolo_candidates: bool = True
64          ai_handoff_padding: float = 2.0
65          ai_max_workers: int = 5
66
67          tracknet: TrackNetConfig = Field(default_factory=TrackNetConfig)
68
69          @field_validator("default_segmenter")
70          @classmethod
71          def segmenter_must_be_registered(cls, v: str) -> str:
72              # Registry check happens at runtime in main/segmenter selection.
73              # Here we just allow any string for forward compatibility.
74              return v
75
76
77      class OutputConfig(BaseModel):
```

```
78          default_highlights_name: str = "highlights.mp4"
79          db_name: str = "sports_indexer.db"
80
81
82      class StitchingConfig(BaseModel):
83          begin_padding: float = 0.5
84          end_padding: float = 0.5
85          use_cache: bool = False
86          min_shot_count: int = 1
87
88
89      class AppConfig(BaseModel):
90          """Root configuration object.
91
92          All runtime configuration should be accessed via an instance of this class
93          (or the loader) rather than raw dict lookups.
94          """
95
96          sport: Sport = "badminton"
97          ai_provider: str = "gemini"
98          ai_model: str = "gemini-2.5-flash"
99
100         validation: ValidationConfig = Field(default_factory=ValidationConfig)
101         indexing: IndexingConfig = Field(default_factory=IndexingConfig)
102         output: OutputConfig = Field(default_factory=OutputConfig)
103         stitching: StitchingConfig = Field(default_factory=StitchingConfig)
104
105         # Allow extra keys during transition so existing config.json files do not break.
106         model_config = {
107             "extra": "allow",
108             "validate_assignment": True,
109         }
110
111         # Temporary dict-like access for legacy code that does config.get("foo", {})
112         # and passes the config object around. Will be removed after consumers are migrated.
113         def get(self, key: str, default: Any = None) -> Any:
114             try:
115                 return getattr(self, key)
116             except AttributeError:
117                 # Also support nested "indexing.default_segmenter" style? For now simple
top-level.
118                 if key in self.model_fields:
119                     return getattr(self, key)
120                 # Try to find in sections
121                 for section_name in ("validation", "indexing", "output", "stitching"):
122                     section = getattr(self, section_name, None)
123                     if section is not None:
124                         try:
125                             return getattr(section, key)
126                         except AttributeError:
127                             pass
128                 return default
129
130         def __getitem__(self, key: str) -> Any:
131             val = self.get(key)
132             if val is None and key not in self.model_fields:
133                 # Re-raise like a real dict for unknown top-level
134                 raise KeyError(key)
135             return val
136
137
138     # Convenience type alias for callers that want to annotate
139     Config = AppConfig
```

360. assistant (2026-06-07T07:59:20.802Z)

The dict-compat is **top-level only** ? `config.get("indexing")` returns a nested model (no `.get`), so my `config.get("indexing").get("default_segmenter")` would break on an `AppConfig`. The robust fix is to **normalize config to a plain dict at the reporting boundary**. But first I need to see the full `main.py` (CLI path + `main()`) to know whether `global_config` is actually an `AppConfig` or the dict wrapper at runtime. Let me read it all:

361. user (2026-06-07T07:59:21.198Z)

```
1       import os
2       import sys
3       import argparse
4       import hashlib
5       import json
6       import uvicorn
7       from dotenv import load_dotenv
8
9       load_dotenv()
10      from fastapi import FastAPI, HTTPException
11      from fastapi.middleware.cors import CORSMiddleware
12      from pydantic import BaseModel
13      from typing import List, Dict, Any, Optional
14
15      from fastapi.staticfiles import StaticFiles
16      from fastapi.responses import HTMLResponse
17      from backend.validators.base import registry
18      from backend.database import Database
19      from backend.indexer.base import segmenter_registry, BasePlaySegmenter
20      from backend.stitcher.compiler import VideoCompiler
21      <<<<<<< HEAD
22      from backend.config import load_config as load_app_config, AppConfig  # new typed config
23      =======
24      from backend.reporting import emit_indexer_report
25      >>>>>>> 9dbc764 (feat(reporting): always-on per-video observability report (indexer))
26
27      app = FastAPI(title="Sports Highlight Indexer API")
28
29      # Enable CORS for local web interface
30      app.add_middleware(
31          CORSMiddleware,
32          allow_origins=["*"],
33          allow_credentials=True,
34          allow_methods=["*"],
35          allow_headers=["*"],
36      )
37
38      # Serves static frontend files directly
39      os.makedirs("backend/static", exist_ok=True)
40      app.mount("/static", StaticFiles(directory="backend/static"), name="static")
41
42      @app.get("/", response_class=HTMLResponse)
43      def serve_index():
44          index_path = "backend/static/index.html"
45          if os.path.exists(index_path):
46              with open(index_path, "r", encoding="utf-8") as f:
47                  return f.read()
48          return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/static/</h3>"
49
50      # Global variables initialized at startup
51      db_instance: Optional[Database] = None
52      global_config: Optional[AppConfig] = None  # now typed (Pydantic model)
53
54      def get_file_md5(filepath: str) -> str:
55          """Calculates file hash to uniquely identify videos and prevent duplicate
indexes."""
```

```
56          hasher = hashlib.md5()
57          with open(filepath, "rb") as f:
58              # Read in chunk sizes of 64kb
59              for chunk in iter(lambda: f.read(65536), b""):
60                  hasher.update(chunk)
61          return hasher.hexdigest()
62
63      # Legacy thin wrapper for any remaining direct calls during transition.
64      # New code should import load_app_config / AppConfig from backend.config directly.
65      def load_config(config_path: str = "config.json") -> Dict[str, Any]:
66          cfg = load_app_config(config_path)
67          return cfg.model_dump(mode="json")
68
69      # --- API Models ---
70      class ProcessRequest(BaseModel):
71          video_path: str
72          player_pool: Optional[List[str]] = None
73          max_frames: Optional[int] = None
74          segmenter: Optional[str] = None
75
76      class QueryRequest(BaseModel):
77          video_id: str
78          server: Optional[str] = None
79          receiver: Optional[str] = None
80          duration_min: Optional[float] = None
81          event_type: Optional[str] = None
82
83      class CompileRequest(BaseModel):
84          video_id: str
85          segment_ids: List[int]
86          output_filename: str
87
88      # --- API Endpoints ---
89      @app.get("/api/config")
90      def get_config():
91          return global_config
92
93      @app.post("/api/process")
94      def process_video(req: ProcessRequest):
95          if not os.path.exists(req.video_path):
96              raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
97
98          video_id = get_file_md5(req.video_path)
99          was_reingest = db_instance.get_video(video_id) is not None
100
101         # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
report)
102         print(f"[API] Running validations for {req.video_path}...")
103         val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
104         failures = [r for r in val_results if not r.passed]
105
106     <<<<<<< HEAD
107         # 2. Run segmenter & indexer
108         print(f"[API] Video passed checks. Segmenting and indexing...")
109         db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
110
111         # Use attribute access on the typed AppConfig (with fallback for safety)
112         default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
113         seg_name = req.segmenter or default_seg
114         segmenter_cls = segmenter_registry.get(seg_name)
115         if not segmenter_cls:
```

```
116             available = list(segmenter_registry.list_available().keys())
117             raise HTTPException(
118                 status_code=400,
119                 detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
120             )
121 
122         print(f"[API] Using segmenter: {seg_name}")
123         # Pass the Pydantic model; downstream will be migrated to typed access over time.
124         # For now we also support legacy dict access via the model.
125         segmenter = segmenter_cls(db_instance, global_config)
126         segments = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
127     =======
128         seg_name = req.segmenter or global_config.get("indexing",
{}).get("default_segmenter", "motion")
129         segmenter_actual = None
130         segmentation_ran = False
131         response: Optional[Dict[str, Any]] = None
132         try:
133             if failures:
134                 db_instance.add_video(video_id, req.video_path, "failed", [r.dict() for r in
val_results])
135                 response = {
136                     "video_id": video_id,
137                     "status": "failed",
138                     "message": "Video failed validation checks.",
139                     "results": [r.dict() for r in val_results],
140                 }
141                 return response
142     >>>>>>> 9dbc764 (feat(reporting): always-on per-video observability report (indexer))
143 
144             # 2. Run segmenter & indexer
145             print(f"[API] Video passed checks. Segmenting and indexing...")
146             db_instance.add_video(video_id, req.video_path, "processed", [r.dict() for r in
val_results])
147 
148             segmenter_cls = segmenter_registry.get(seg_name)
149             if not segmenter_cls:
150                 available = list(segmenter_registry.list_available().keys())
151                 raise HTTPException(
152                     status_code=400,
153                     detail=f"Segmenter '{seg_name}' not found. Available segmenters:
{available}"
154                 )
155 
156             print(f"[API] Using segmenter: {seg_name}")
157             segmenter_actual = seg_name
158             segmenter = segmenter_cls(db_instance, global_config)
159             segments = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
160             segmentation_ran = True
161 
162             response = {
163                 "video_id": video_id,
164                 "status": "success",
165                 "message": f"Successfully indexed {len(segments)} play segments using
'{seg_name}' segmenter.",
166                 "results": [r.dict() for r in val_results],
167                 "play_segments": segments,
168             }
169             return response
170         finally:
171             # Always emit the per-video observability report (next to the video, or to
172             # report.output_dir). Never raises. Surface the paths in the response.
```

```
            paths = emit_indexer_report(
                db=db_instance, video_id=video_id, video_path=req.video_path,
config=global_config,
                val_results=val_results, val_context=val_context,
                segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
                was_reingest=was_reingest, segmentation_ran=segmentation_ran,
            )
            if paths and isinstance(response, dict):
                response["reports"] = paths

    @app.post("/api/query")
    def query_play_segments(req: QueryRequest):
        filters = {
            "server": req.server,
            "receiver": req.receiver,
            "duration_min": req.duration_min,
            "event_type": req.event_type
        }
        segments = db_instance.query_play_segments(req.video_id, filters)
        return {"play_segments": segments}

    @app.post("/api/compile")
    def compile_highlights(req: CompileRequest):
        video = db_instance.get_video(req.video_id)
        if not video:
            raise HTTPException(status_code=404, detail="Indexed video record not found.")

        # Retrieve matching play segments from db
        all_segments = db_instance.query_play_segments(req.video_id, {})
        matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]

        if not matched_segments:
            raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")

        # Extract start/end time pairs
        time_pairs = [(s["start_time"], s["end_time"]) for s in matched_segments]

        # Run compiler
        output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
or "output"
        compiler = VideoCompiler(output_dir)
        try:
            out_path = compiler.compile_highlights(video["filepath"], time_pairs,
req.output_filename)
            return {"status": "success", "filepath": out_path}
        except Exception as e:
            raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}")

    # --- CLI Debug Utility & Orchestration ---
    def run_cli_diagnose_clip(video_path: str, timestamp: float):
        """CLI debugging utility to examine segment properties around a timestamp."""
        print("=" * 60)
        print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
        print("=" * 60)

        cfg = load_config()  # legacy wrapper still works, returns dict for now

        # 1. Validation check diagnostics
        print("[DIAGNOSTIC] Running validation framework...")
        results = registry.run_all(video_path, cfg)
        for r in results:
            status_symbol = "[PASS]" if r.passed else "[FAIL]"
            print(f"  {status_symbol} {r.validator_name}: {r.message}")
```

```
            if r.details:
                print(f"         Details: {r.details}")

        # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
        print("-" * 60)
        print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
        import cv2
        cap = cv2.VideoCapture(video_path)
        if not cap.isOpened():
            print("[FAIL] OpenCV could not open video.")
            return

        fps = cap.get(cv2.CAP_PROP_FPS)
        total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)

        start_frame = max(0, int((timestamp - 3.0) * fps))
        end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))

        # Measure frame-by-frame changes in sample region
        cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
        prev_gray = None
        motions = []

        for f in range(start_frame, end_frame):
            ret, frame = cap.read()
            if not ret:
                break
            gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
            gray = cv2.GaussianBlur(gray, (21, 21), 0)

            if prev_gray is not None:
                diff = cv2.absdiff(prev_gray, gray)
                _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
                motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
                motions.append(motion_ratio)
            prev_gray = gray

        cap.release()

        if motions:
            avg_motion = sum(motions) / len(motions)
            # Support both legacy dict (from wrapper) and future direct model
            if hasattr(cfg, "indexing"):
                threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
            else:
                threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08)  # type:
ignore[attr-defined]
            print(f"  Sample Frame Count: {len(motions)}")
            print(f"  Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
            if avg_motion > threshold:
                print("  Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
            else:
                print("  Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
        else:
            print("  [ERROR] No visual frames were extracted in the sample range.")

        print("=" * 60)

    def main():
        parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
```

```
292         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
293         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
294         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
295         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
296         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
297         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
298         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
299
300         available_segmenters = list(segmenter_registry.list_available().keys())
301         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
302         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
303         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
304         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
305
306         args = parser.parse_args()
307
308         if args.setup:
309             # Invoke setup diagnostics
310             import subprocess
311             print("[INFO] Invoking setup.py...")
312             subprocess.run([sys.executable, "setup.py"])
313             return
314
315         if args.trim_start is not None and args.trim_end is not None:
316             if not args.video_path:
317                 print("[ERROR] Please provide --video-path to extract a segment.")
318                 return
319
320             # Determine output path
321             out_filename = args.trim_output or "trimmed_segment.mp4"
322             if os.path.isabs(out_filename):
323                 out_path = out_filename
324                 out_dir = os.path.dirname(out_path)
325                 out_name = os.path.basename(out_path)
326             else:
327                 out_dir = args.output_dir
328                 out_path = os.path.join(out_dir, out_filename)
329                 out_name = out_filename
330
331             os.makedirs(out_dir, exist_ok=True)
332             print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
333
334             compiler = VideoCompiler(out_dir)
335             try:
336                 res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
337                 print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
338             except Exception as e:
339                 print(f"[ERROR] Failed to compile trimmed segment: {e}")
340             return
341
342         if args.debug_clip is not None:
343             if not args.video_path:
344                 print("[ERROR] Please provide --video-path to debug a specific clip.")
```

```
                return
            run_cli_diagnose_clip(args.video_path, args.debug_clip)
            return

        # Initialize Global Config and DB
        global global_config, db_instance
        global_config = load_app_config()  # returns typed AppConfig

        # output_dir is a runtime concern (not stored in config.json).
        # We attach it to the config object for downstream compatibility during transition.
        # Downstream code that does dict access will still work via model_dump() if needed,
        # but new code should read cfg.output.output_dir after we extend the model.
        if not hasattr(global_config, "_runtime_overrides"):
            global_config._runtime_overrides = {}
        global_config._runtime_overrides["output_dir"] = args.output_dir

        # For code that still does global_config.get("output", {}) we provide a thin compat
layer
        # by using the model + overrides when necessary. For now we compute db_path
directly.
        os.makedirs(args.output_dir, exist_ok=True)

        db_name = getattr(getattr(global_config, "output", None), "db_name",
"sports_indexer.db")
        db_path = os.path.join(args.output_dir, db_name)
        db_instance = Database(db_path)

        # CLI processing mode (no web UI needed)
        if args.video_path and not args.server:
            print(f"[CLI] Processing video file: {args.video_path}")
            if not os.path.exists(args.video_path):
                print(f"[FAIL] Video file not found: {args.video_path}")
                return

            video_id = get_file_md5(args.video_path)
            was_reingest = db_instance.get_video(video_id) is not None

            # Validate (with-context variant surfaces ffprobe_meta for the report)
            print("[CLI] Validating...")
            val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
            failures = [r for r in val_results if not r.passed]

            # Save video status
            status = "failed" if failures else "processed"
            db_instance.add_video(video_id, args.video_path, status, [r.dict() for r in
val_results])

    <<<<<<< HEAD
            if failures:
                print("[FAIL] Video failed checks. Indexing halted.")
                return

            # Index
            default_seg = getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
            seg_name = args.segmenter or default_seg
            segmenter_cls = segmenter_registry.get(seg_name)
            if not segmenter_cls:
                print(f"[FAIL] Segmenter '{seg_name}' not found.")
                return

            print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
            segmenter = segmenter_cls(db_instance, global_config)
            segments = segmenter.process_video(video_id, args.video_path,
```

```
max_frames=args.max_frames)
404             print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments inside
database: {db_path}")
405     =======
406             seg_name = args.segmenter or global_config.get("indexing",
{}).get("default_segmenter", "motion")
407             segmenter_actual = None
408             segmentation_ran = False
409             try:
410                 print("\n" + "="*40)
411                 print("VALIDATION SUMMARY")
412                 print("="*40)
413                 for r in val_results:
414                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
415                     print(f"{status_symbol} {r.validator_name}: {r.message}")
416                 print("="*40 + "\n")
417
418                 if failures:
419                     print("[FAIL] Video failed checks. Indexing halted.")
420                     return
421
422                 # Index
423                 segmenter_cls = segmenter_registry.get(seg_name)
424                 if not segmenter_cls:
425                     print(f"[FAIL] Segmenter '{seg_name}' not found.")
426                     return
427
428                 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
429                 segmenter_actual = seg_name
430                 segmenter = segmenter_cls(db_instance, global_config)
431                 segments = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
432                 segmentation_ran = True
433                 print(f"[SUCCESS] Process complete. Indexed {len(segments)} play segments
inside database: {db_path}")
434             finally:
435                 # Always emit the per-video observability report. Never raises.
436                 paths = emit_indexer_report(
437                     db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
438                     val_results=val_results, val_context=val_context,
439                     segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
440                     was_reingest=was_reingest, segmentation_ran=segmentation_ran,
441                 )
442                 if paths:
443                     print(f"[REPORT] Technical report : {paths.get('technical_md')}")
444                     print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
445                     print(f"[REPORT] Machine JSON     : {paths.get('json')}")
446     >>>>>>> 9dbc764 (feat(reporting): always-on per-video observability report (indexer))
447             return
448
449         # Start Local Server Mode
450         if args.server or not args.video_path:
451             print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
452             uvicorn.run(app, host="127.0.0.1", port=args.port)
453
454     if __name__ == "__main__":
455         main()
456
```

362. user (2026-06-07T07:59:29.877Z)

[Request interrupted by user]

363. user (2026-06-07T07:59:29.887Z)

The other session also made a fix PR but I have held it

364. assistant (2026-06-07T08:01:15.248Z)

Understood ? thanks, that removes the double-merge pressure on the parser fix; I'll keep mine on the branch and you can reconcile later.

Now resolving the `main.py` conflict. The picture is clear: `global_config` is now a typed `AppConfig` (attribute access), so I'll **keep the parallel work's typed access for segmenter selection** and **keep my reporting wiring**, and make the reporting layer normalize `AppConfig`?dict at its boundary (since my harvest does nested `.get`). Resolving the three conflict hunks:

365. user (2026-06-07T08:01:17.464Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

366. user (2026-06-07T08:01:30.641Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

367. user (2026-06-07T08:01:48.941Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\main.py has been updated successfully. (file state is current in your context ? no need to Read it back)

368. assistant (2026-06-07T08:01:55.584Z)

Now make the reporting layer normalize a typed `AppConfig` to a dict at its boundary (so harvest's nested `.get` works):

369. user (2026-06-07T08:01:56.639Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting__init__.py has been updated successfully. (file state is current in your context ? no need to Read it back)

370. assistant (2026-06-07T08:02:03.388Z)

Let me verify no conflict markers remain, then continue the rebase:

371. user (2026-06-07T08:02:05.513Z)

```
=== conflict markers? ===
none
main.py parses OK
You must edit all merge conflicts and then
mark them as resolved using git add
```

372. user (2026-06-07T08:02:13.231Z)

```
interactive rebase in progress; onto ce6a0e9
Last command done (1 command done):
   pick 9dbc764 feat(reporting): always-on per-video observability report (indexer)
Next command to do (1 remaining command):
   pick bc0fe21 chore(reporting): per-tool customer_verdict + bump obsreport pin to v0.1.3
  (use "git rebase --edit-todo" to view and edit)
You are currently rebasing branch 'feat/observability-reports' on 'ce6a0e9'.
  (all conflicts fixed: run "git rebase --continue")
```

```
Changes to be committed:
  (use "git restore --staged <file>..." to unstage)
        modified:   README.md
        modified:   backend/main.py
        new file:   backend/reporting/__init__.py
        new file:   backend/reporting/harvest.py
        new file:   backend/reporting/spec.yaml
        modified:   backend/validators/base.py
        modified:   requirements.txt
        new file:   tests/test_reporting_harvest.py
        new file:   tests/test_validators_base.py
```

373. assistant (2026-06-07T08:02:20.433Z)

Conflicts resolved (git confirms "all conflicts fixed"). Continuing the rebase with a non-interactive editor:

374. user (2026-06-07T08:02:22.015Z)

```
You must edit all merge conflicts and then
mark them as resolved using git add
=== log ===
ce6a0e9 Merge pull request #39 from avidullu/feat/config-pydantic-package
da6ea5c feat(config): introduce backend/config/ package as first low-regret deliverable
f4c1b2d docs: checkpoint approved structure + sequencing decisions (Config first, atomic
refactor follow-up)
9029415 docs: checkpoint approved structure + sequencing decisions (Config first, atomic
refactor follow-up)
=== status ===
M  README.md
M  backend/main.py
AM backend/reporting/__init__.py
A  backend/reporting/harvest.py
A  backend/reporting/spec.yaml
M  backend/validators/base.py
M  requirements.txt
A  tests/test_reporting_harvest.py
A  tests/test_validators_base.py
```

375. user (2026-06-07T08:02:37.069Z)

```
(above = remaining unmerged, should be empty)
 create mode 100644 backend/reporting/__init__.py
 create mode 100644 backend/reporting/harvest.py
 create mode 100644 backend/reporting/spec.yaml
 create mode 100644 tests/test_reporting_harvest.py
 create mode 100644 tests/test_validators_base.py
Rebasing (2/2)

Successfully rebased and updated refs/heads/feat/observability-reports.
=== HEAD ===
02aa99a chore(reporting): per-tool customer_verdict + bump obsreport pin to v0.1.3
f1839b8 feat(reporting): always-on per-video observability report (indexer)
ce6a0e9 Merge pull request #39 from avidullu/feat/config-pydantic-package
```

376. assistant (2026-06-07T08:02:44.939Z)

Rebase succeeded ? my two commits now sit cleanly on top of `origin/master`. Let me run the full indexer suite (now including the parallel work's config tests) to confirm the reconciliation is sound:

377. user (2026-06-07T08:02:59.691Z)

```
    def test_save_default_config_roundtrip():
```

```
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "saved.json")
            saved = save_default_config(p)
>           assert saved == p
E           AssertionError: assert
WindowsPath('C:/Users/avidu/AppData/Local/Temp/tmpzdbctxbv/saved.json') ==
'C:\\Users\\avidu\\AppData\\Local\\Temp\\tmpzdbctxbv\\saved.json'

tests\test_config.py:73: AssertionError
========================== short test summary info ==========================
FAILED tests/test_config.py::test_load_default_config - AssertionError: asser...
FAILED tests/test_config.py::test_save_default_config_roundtrip - AssertionEr...
2 failed, 225 passed in 12.52s
```

378. assistant (2026-06-07T08:03:30.650Z)

225 passed ? all my reporting tests green. The 2 failures are in the **parallel work's `test_config.py`** (a Windows `Path` vs `str` mismatch in `save_default_config`), unrelated to my changes and pre-existing on `origin/master`. Let me look at the loader to confirm and decide whether it's a safe trivial fix:

379. user (2026-06-07T08:03:31.035Z)

```
1       """Configuration loader and saver.
2
3       Provides a single place to load validated AppConfig with built-in defaults
4       merged with optional file overrides (config.json).
5
6       This replaces scattered json.load + .get() patterns throughout the codebase.
7       """
8
9       from __future__ import annotations
10
11      import json
12      from pathlib import Path
13      from typing import Any
14
15      from .models import AppConfig
16
17      DEFAULT_CONFIG_PATH = Path("config.json")
18
19
20      def _get_builtin_defaults() -> dict[str, Any]:
21          """Return a plain dict of the current built-in defaults.
22
23          The AppConfig model is the single source of truth for defaults.
24          """
25          return AppConfig().model_dump(mode="json")
26
27
28      def load_config(path: str | Path | None = None) -> AppConfig:
29          """Load configuration with the following precedence:
30
31          1. Built-in defaults defined in the Pydantic models.
32          2. Values from the JSON file (if present and readable).
33          3. (Future) environment / CLI overrides.
34
35          Missing file or unreadable file ? returns a fully defaulted AppConfig.
36          Unknown keys in the file are tolerated (extra="allow" on the model).
37          """
38          cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
39
40          file_data: dict[str, Any] = {}
41          if cfg_path.exists():
42              try:
```

```
                with open(cfg_path, "r", encoding="utf-8") as f:
                    file_data = json.load(f)
            except Exception as exc:
                # Non-fatal: continue with defaults + whatever we could parse.
                # In production we might log here.
                print(f"[CONFIG WARNING] Failed to read {cfg_path}: {exc}. Using defaults +
partial data.")

        # Merge: start from defaults, overlay file data (Pydantic will validate/coerce).
        merged = {**_get_builtin_defaults(), **file_data}

        return AppConfig.model_validate(merged)


    def save_default_config(path: str | Path | None = None) -> Path:
        """Write a fresh default config.json.

        Used by setup.py diagnostics and the --setup CLI path.
        Overwrites if the file exists.
        """
        cfg_path = Path(path) if path is not None else DEFAULT_CONFIG_PATH
        cfg = AppConfig()
        cfg_path.parent.mkdir(parents=True, exist_ok=True)
        cfg_path.write_text(cfg.model_dump_json(indent=4), encoding="utf-8")
        return cfg_path


    # Backwards-compatible alias used by some older call sites during transition.
    def get_default_config() -> dict[str, Any]:
        """Return defaults as a plain dict (for transitional use only)."""
        return _get_builtin_defaults()

```

380. assistant (2026-06-07T08:03:59.378Z)

`save_default_config` correctly returns a `Path` (annotated `-> Path`), but the parallel work's tests compare it to a `str` ? which fails on every platform (a `Path` never `==` a `str`). It's a clear test bug, now in my branch's suite because I rebased onto their master. To keep the checkpoint PR green, I'll fix the **test assertions** (not the loader) as a clearly-labeled separate commit. Let me read them:

381. user (2026-06-07T08:04:00.314Z)

```
            data = {
                "sport": "badminton",
                "indexing": {"default_segmenter": "yolo_hybrid"},
                "some_future_key": "ignored_for_now",
                "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
            }
            with open(p, "w") as f:
                json.dump(data, f)

            cfg = load_config(p)
            assert cfg.indexing.default_segmenter == "yolo_hybrid"
            assert cfg.validation.relevance.court_pixels_min_ratio == 0.1


    def test_save_default_config_roundtrip():
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "saved.json")
            saved = save_default_config(p)
            assert saved == p
            assert os.path.exists(p)

            cfg = load_config(p)
```

```
            assert cfg.indexing.default_segmenter == "yolo_hybrid"
            assert cfg.validation.min_blur_threshold == 20.0


    def test_model_is_the_source_of_truth():
        """Changing a default in the model should be reflected without touching json."""
        # This is a documentation test more than anything.
        cfg = AppConfig()
```

382. user (2026-06-07T08:04:08.563Z)

```
    """Basic tests for the typed configuration package.

    These ensure:
    - Defaults are sensible and come from the models.
    - File overrides work.
    - Old config files with extra keys are tolerated.
    - save_default_config produces loadable output.
    """

    import json
    import tempfile
    import os

    import pytest

    from backend.config import load_config, save_default_config, AppConfig


    def test_load_default_config():
        cfg = load_config()
        assert isinstance(cfg, AppConfig)
        assert cfg.sport == "badminton"
        assert cfg.indexing.default_segmenter == "yolo_hybrid"
        assert cfg.validation.min_blur_threshold == 20.0  # updated default from review
        assert cfg.indexing.ai_max_workers == 5


    def test_file_override():
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "override.json")
            data = {
                "indexing": {
                    "default_segmenter": "gemini",
                    "ai_max_workers": 3,
                },
                "validation": {
                    "min_blur_threshold": 42.0,
                },
            }
            with open(p, "w") as f:
                json.dump(data, f)

            cfg = load_config(p)
            assert cfg.indexing.default_segmenter == "gemini"
            assert cfg.indexing.ai_max_workers == 3
            assert cfg.validation.min_blur_threshold == 42.0
            # Unspecified keys fall back to defaults
            assert cfg.sport == "badminton"


    def test_extra_keys_tolerated():
        """Old or future keys in config.json must not break loading."""
        with tempfile.TemporaryDirectory() as td:
            p = os.path.join(td, "with_extra.json")
```

383. assistant (2026-06-07T08:04:34.061Z)

The roundtrip test compares a `Path` to a `str` (objectively a test bug). Let me see the exact failure for the other one before deciding (it reads the repo's `config.json`, which may simply differ from the model defaults):

384. user (2026-06-07T08:04:38.518Z)

```
F                                                                        [100%]
================================== FAILURES ===================================
_____________________________ test_load_default_config _____________________________

    def test_load_default_config():
        cfg = load_config()
        assert isinstance(cfg, AppConfig)
        assert cfg.sport == "badminton"
>       assert cfg.indexing.default_segmenter == "yolo_hybrid"
E       AssertionError: assert 'gemini' == 'yolo_hybrid'
E
E         - yolo_hybrid
E         + gemini

tests\test_config.py:23: AssertionError
=========================== short test summary info ===========================
FAILED tests/test_config.py::test_load_default_config - AssertionError: asser...
1 failed in 0.34s
=== checked-in config.json default_segmenter / blur / workers ===
default_segmenter= gemini
min_blur= 20.0
ai_max_workers= 5
```

385. assistant (2026-06-07T08:05:29.952Z)

Both config-test failures are **parallel-work bugs unrelated to my PR**: `test_load_default_config` fails because the checked-in `config.json` sets `default_segmenter: gemini` while the test expects the model default `yolo_hybrid` (the "diverging defaults" footgun!), and the roundtrip test compares a `Path` to a `str`. Since you have **active parallel work on config (Grok)**, I'll *not* touch their tests ? I'll flag them in the PR rather than risk conflicting with Grok's changes.

I also notice the **collector working tree got reverted** (my Phase B changes aren't showing) ? likely the parallel session switched branches there. My commits should still be safe on the branch; I'll verify after securing the indexer.

First, let me lock my config-reconciliation with a test (typed `AppConfig` ? report still emits), then run the suite:

386. user (2026-06-07T08:05:35.097Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py has been updated successfully. (file state is current in your context ? no need to Read it back)

387. user (2026-06-07T08:05:59.965Z)

```
..............                                                           [100%]
14 passed, 214 deselected in 4.54s
=== full suite tally ===
=========================== short test summary info ===========================
FAILED tests/test_config.py::test_load_default_config - AssertionError: asser...
FAILED tests/test_config.py::test_save_default_config_roundtrip - AssertionEr...
2 failed, 226 passed in 11.90s
```

388. assistant (2026-06-07T08:06:11.808Z)

My reporting suite is green (14 passed incl. the typed-config test); the only 2 failures are the pre-existing config ones. Committing the reconciliation test, then I'll create the `obsreport` GitHub repo and push:

389. user (2026-06-07T08:06:13.678Z)

```
fa60efa test(reporting): cover typed AppConfig normalization in emit (post-#39 rebase)
02aa99a chore(reporting): per-tool customer_verdict + bump obsreport pin to v0.1.3
f1839b8 feat(reporting): always-on per-video observability report (indexer)
ce6a0e9 Merge pull request #39 from avidullu/feat/config-pydantic-package
```

390. assistant (2026-06-07T08:06:23.257Z)

Now creating the private `sports-obsreport` repo on GitHub and pushing it (main + all tags):

391. user (2026-06-07T08:06:31.972Z)

```
=== state ===
1f72c1c feat: configurable highlights item_noun (v0.1.3)
=== create + push ===
https://github.com/avidullu/sports-obsreport
branch 'main' set up to track 'origin/main'.
To https://github.com/avidullu/sports-obsreport.git
 * [new branch]      HEAD -> main
=== remote tags ===
cba67f29d261903e03026fe5fefa56ade63860a4        refs/tags/v0.1.1^{}
75aea1b807a79cf3669534808c3d48fb01753975        refs/tags/v0.1.2
2f5a214bf5051c386875a0e81bc724383b51d7e9        refs/tags/v0.1.2^{}
821a664f5d2861397d5de7ebe803e72f71e9964a        refs/tags/v0.1.3
1f72c1c2dd471d4e3ed3130f0a4040a6d4813042        refs/tags/v0.1.3^{}
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

392. assistant (2026-06-07T08:06:48.282Z)

`obsreport` is live on GitHub (private, tags v0.1.0?v0.1.3). Now pushing the indexer branch and opening a checkpoint PR:

393. user (2026-06-07T08:06:57.748Z)

```
remote: Create a pull request for 'feat/observability-reports' on GitHub by visiting:
remote:      https://github.com/avidullu/badminton-highlight-
indexer/pull/new/feat/observability-reports
remote:
https://github.com/avidullu/badminton-highlight-indexer/pull/41
```

394. assistant (2026-06-07T08:07:06.124Z)

Indexer checkpoint PR is up: [indexer#41](https://github.com/avidullu/badminton-highlight-indexer/pull/41). Now let me investigate the collector ? its working tree showed reverted content, so I need to find where my Phase B commits went:

395. user (2026-06-07T08:07:08.296Z)

```
=== current branch ===
feat/observability-reports
=== local branches ===
* feat/observability-reports   028937b fix(parser): keep a rally that starts at 0.0s in
load_canonical_csv
  fix/canonical-csv-zero-start f7cab11 [origin/fix/canonical-csv-zero-start] fix(parser): keep a
rally that starts at 0.0s in load_canonical_csv
  master                       d68c5da [origin/master] Merge pull request #9 from
avidullu/feat/validate-delivery
=== feat branch log (my commits?) ===
028937b fix(parser): keep a rally that starts at 0.0s in load_canonical_csv
```

```
bb2b5be feat(reporting): per-video observability report (collector)
d68c5da Merge pull request #9 from avidullu/feat/validate-delivery
2882a03 feat(ingest): validate-delivery ? per-video QA gate + structured report
=== working tree status ===
 M data/sports_metadata.db
?? data/roora_badminton_canonical.csv
?? data/roora_badminton_issues.txt
?? data/validation/
```

396. assistant (2026-06-07T08:07:32.512Z)

Good ? I'm still on `feat/observability-reports` and **my commits are intact** (bb2b5be feat + 028937b parser fix). I also see Grok's `fix/canonical-csv-zero-start` branch (the held PR). The earlier reminders may have been mid-operation snapshots. Let me verify my committed changes are actually present in the working tree right now:

397. user (2026-06-07T08:07:35.977Z)

```
=== curate.py emit wiring present? ===
6
=== src/reporting exists? ===
__init__.py
__pycache__
adapter.py
spec.yaml
=== INGESTION_PIPELINE observability FAQ present? ===
2
=== pyproject build-backend ===
build-backend = "setuptools.build_meta"
=== git diff HEAD on tracked source (should be empty) ===
```

398. assistant (2026-06-07T08:08:17.763Z)

All my collector changes are intact (the reminders were stale snapshots). Collector `origin/master` is unchanged, so no rebase needed. I see Grok's dedicated `fix/canonical-csv-zero-start` branch exists (your held PR). To keep my observability PR **clean and non-overlapping**, I'll drop my duplicate parser-fix commit (the fix lives safely on Grok's branch) so my PR is purely observability. The branch is local-only, so this is safe:

399. user (2026-06-07T08:08:20.288Z)

```
=== confirm fix exists on Grok branch before dropping mine ===
f7cab11 fix(parser): keep a rally that starts at 0.0s in load_canonical_csv
2
=== drop my parser-fix commit (top of feat) ===
HEAD is now at bb2b5be feat(reporting): per-video observability report (collector)
=== feat log now ===
bb2b5be feat(reporting): per-video observability report (collector)
d68c5da Merge pull request #9 from avidullu/feat/validate-delivery
=== rally_cvat reverted to pre-fix on my branch? (expect falsy-or line) ===
442:            start = _parse_secs(row.get("start_time")) or _parse_secs(row.get("start_mmss"))
```

400. assistant (2026-06-07T08:08:31.510Z)

Clean ? my collector branch is now purely observability. Running the suite to confirm green, then pushing + PR:

401. user (2026-06-07T08:08:34.859Z)

```
.................................................................... [ 64%]
.....................................                              [100%]
112 passed in 1.89s
```

402. assistant (2026-06-07T08:08:48.505Z)

112 passed. Pushing the collector branch and opening its checkpoint PR:

403. user (2026-06-07T08:08:56.685Z)

```
remote: Create a pull request for 'feat/observability-reports' on GitHub by visiting:
remote:      https://github.com/avidullu/sports-data-collector/pull/new/feat/observability-
reports
remote:
https://github.com/avidullu/sports-data-collector/pull/11
```

404. assistant (2026-06-07T08:09:31.296Z)

Both checkpoint PRs are up and `obsreport` is on GitHub. Let me record the sync in memory, then run the debuggability evaluation you asked for.

Synced to remote:
- `obsreport` lib ? github.com/avidullu/sports-obsreport (private, tags v0.1.0?v0.1.3)
- Indexer ? [PR #41](https://github.com/avidullu/badminton-highlight-indexer/pull/41) (rebased onto your `#39` config work; conflicts resolved)
- Collector ? [PR #11](https://github.com/avidullu/sports-data-collector/pull/11) (observability-only; left Grok's parser fix to its own held branch)

405. user (2026-06-07T08:09:39.102Z)

The file c:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

406. assistant (2026-06-07T08:10:14.831Z)

Now the debuggability evaluation. First I'll generate real broken-run reports for both tools (the fixtures a "fresh reader" will debug from), then fan out fresh-reader agents via a Workflow. Writing the indexer fixture generator:

407. assistant (2026-06-07T08:10:25.739Z)

```
"""Generate broken-run indexer reports (technical markdown) for the debuggability eval."""
import json, os, sys
sys.path.insert(0, os.getcwd())  # indexer repo root
import obsreport
from backend.reporting import harvest
from backend.validators.base import ValidationResult

OUT = r"C:\Users\avidu\AppData\Local\Temp\obseval\indexer_fixtures.json"

FFPROBE_OK = {
    "streams": [{"width": 1920, "height": 1080, "r_frame_rate": "60000/1001", "codec_name":
"hevc"}],
    "format": {"format_name": "mov,mp4,m4a", "duration": "676.0", "size": "5500000000"},
}

def vr(name, passed, msg="ok"):
    return ValidationResult(validator_name=name, passed=passed, message=msg)

def tech(env, spec):
    return obsreport.render_technical(env, spec)

scenarios = []

# S1: no API key + 0 segments -> silent_provider_noop
os.environ.pop("GEMINI_API_KEY", None)
rec = harvest.record_run(
```

```
    video_id="md5a", video_path="/videos/match.mp4",
    config={"ai_provider": "gemini", "ai_model": "gemini-flash", "indexing":
{"default_segmenter": "wasb_hybrid"}},
    val_results=[vr("format_check", True), vr("relevance_check", True)],
val_context={"ffprobe_meta": FFPROBE_OK},
    play_segments=[], candidates=[{"id": 1}],
    segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid", was_reingest=False,
segmentation_ran=True)
scenarios.append({"id": "indexer_no_api_key", "tool": "indexer", "report_md": tech(rec.envelope,
rec.spec),
                  "truth": "The run produced 0 highlight segments because no AI-provider API key
was set, so the AI segmenter silently no-op'd. It is NOT that the video has no rallies. Fix: set
GEMINI_API_KEY (e.g. in a .env file) and re-run.",
                  "expected_faq": "Q7"})
```

S2: motion segmenter -> synthetic_motion_metadata

```
os.environ["GEMINI_API_KEY"] = "x"
rec = harvest.record_run(
    video_id="md5b", video_path="/videos/match.mp4",
    config={"indexing": {"default_segmenter": "motion"}},
    val_results=[vr("format_check", True), vr("relevance_check", True)],
val_context={"ffprobe_meta": FFPROBE_OK},
    play_segments=[{"duration": 5.0, "metadata": {"detector": "motion"}}], candidates=[],
    segmenter_requested="motion", segmenter_actual="motion", was_reingest=False,
segmentation_ran=True)
scenarios.append({"id": "indexer_motion_synthetic", "tool": "indexer", "report_md":
tech(rec.envelope, rec.spec),
                  "truth": "The 'motion' segmenter was used; its per-segment metadata/events are
heuristic/synthetic, not measured, so they must not be trusted as ground truth. Use yolo_hybrid
or gemini for real shot metadata.",
                  "expected_faq": "Q10"})
```

S3: validation failed (blur)

```
rec = harvest.record_run(
    video_id="md5c", video_path="/videos/match.mp4",
    config={"indexing": {"default_segmenter": "wasb_hybrid"}, "ai_provider": "gemini"},
    val_results=[vr("format_check", True), vr("blur_check", False, "Focus check failed. Average
sharpness 0.0 below threshold 20.0."), vr("relevance_check", True)],
    val_context={"ffprobe_meta": FFPROBE_OK}, play_segments=[], candidates=[],
    segmenter_requested=None, segmenter_actual=None, was_reingest=False, segmentation_ran=False)
scenarios.append({"id": "indexer_validation_failed", "tool": "indexer", "report_md":
tech(rec.envelope, rec.spec),
                  "truth": "Indexing was halted because a validation check failed: the
blur/focus check (video too blurry / sharpness below the min threshold). Fix the footage quality
or lower validation.min_blur_threshold in config.json, then re-run.",
                  "expected_faq": "Q13"})
```

S4: fps fallback (no ffprobe meta but segmented)

```
rec = harvest.record_run(
    video_id="md5d", video_path="/videos/match.mp4",
    config={"indexing": {"default_segmenter": "wasb_hybrid"}, "ai_provider": "gemini"},
    val_results=[vr("format_check", True)], val_context={},
    play_segments=[{"duration": 5.0, "metadata": {"detector": "wasb-hybrid"}}],
candidates=[{"id": 1}],
    segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid", was_reingest=False,
segmentation_ran=True)
scenarios.append({"id": "indexer_fps_fallback", "tool": "indexer", "report_md":
tech(rec.envelope, rec.spec),
                  "truth": "fps was not probed from the file (a fallback default was used), so
motion thresholds and frame->second timing may be miscalibrated. Confirm the real fps with
ffprobe and ensure validation ran.",
                  "expected_faq": "Q5"})
```

```
with open(OUT, "w", encoding="utf-8") as f:
    json.dump(scenarios, f, indent=2)
print(f"wrote {len(scenarios)} indexer scenarios -> {OUT}")
```

408. user (2026-06-07T08:10:25.766Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\obseval\gen_indexer.py (file
state is current in your context ? no need to Read it back)
```

409. assistant (2026-06-07T08:10:40.846Z)

Now the collector fixture generator:

410. assistant (2026-06-07T08:10:44.516Z)

```
"""Generate broken-run collector reports (technical markdown) for the debuggability eval."""
import json, os, sys
sys.path.insert(0, os.getcwd())  # collector repo root
from types import SimpleNamespace
import obsreport
from src.reporting import adapter
from src.parsers.cvat.rally_cvat import Issue, RallyRecord

OUT = r"C:\Users\avidu\AppData\Local\Temp\obseval\collector_fixtures.json"

def rec_(n, s, e, shots=5):
    return RallyRecord(rally_number=n, start_time=s, end_time=e, shots_count=shots,
                       ending_reason="winner", last_shot_outcome="in", score_before=None,
                       score_after=None, notes=None, frame_min=int(s*60), frame_max=int(e*60),
n_boxes=3)

def tech(rec):
    return obsreport.render_technical(rec.envelope, rec.spec)

scenarios = []
```

C1: overlap blocker

```
records = [rec_(1, 2.0, 7.0), rec_(2, 6.0, 11.0)]
issues = [Issue("blocker", "overlap", "Rally 1 (ends 7.00s) overlaps Rally 2 (starts 6.00s) by
1.00s ? rallies must be disjoint.", 2)]
rec = adapter.build_delivery_recorder(records=records, issues=issues, fps=59.94,
source="pilot_overlap", video_duration_s=600.0)
scenarios.append({"id": "collector_overlap", "tool": "collector", "report_md": tech(rec),
                  "truth": "The delivery is blocked because two rallies overlap in time (rally 1
ends at 7.00s but rally 2 starts at 6.00s). Rallies must be disjoint; the vendor/reviewer must
fix rally 1's end or rally 2's start before it can be imported.",
                  "expected_faq": "what-does-validation-check"})
```

C2: possible missed rally (gap)

```
records = [rec_(1, 2.0, 7.0), rec_(2, 10.0, 14.0), rec_(3, 40.0, 44.0)]
issues = [Issue("medium", "possible_missed_rally", "26.0s of unlabeled time between Rally 2
(ends 14.0s) and Rally 3 (starts 40.0s) ? check for an unlabeled rally in the gap.", 3)]
rec = adapter.build_delivery_recorder(records=records, issues=issues, fps=59.94,
source="pilot_gap", video_duration_s=600.0)
scenarios.append({"id": "collector_missed_rally", "tool": "collector", "report_md": tech(rec),
                  "truth": "There is a long unlabeled gap (~26s) between rally 2 and rally 3,
which may hide an omitted/missed rally. The reviewer should watch that span and add the rally if
one is there. It is a review flag, not a blocker.",
                  "expected_faq": "what-does-validation-check"})
```

C3: non-commercial license on import

```
vm = SimpleNamespace(video_id="cc_match", local_path="data/videos/cc_match.mp4", fps=30.0,
```

```
resolution="1080p",
                      license_type=SimpleNamespace(value="CC-BY-NC-SA"), is_commercial=False,
                      status=SimpleNamespace(value="curated"))
anns = [SimpleNamespace(start_time=1.0, end_time=6.0), SimpleNamespace(start_time=8.0,
end_time=12.0)]
rec = adapter.build_import_recorder(video_meta_obj=vm, annotations=anns, is_valid=True,
errors=[], count=2,
                                    sport="badminton", source_format="csv")
scenarios.append({"id": "collector_noncommercial", "tool": "collector", "report_md": tech(rec),
                  "truth": "The imported video has a non-commercial license, so it is EXCLUDED
from the default commercial-only export. It stays in the DB but won't appear downstream unless
exported with --include-noncommercial.",
                  "expected_faq": "will-a-non-commercial-license-block-export"})

with open(OUT, "w", encoding="utf-8") as f:
    json.dump(scenarios, f, indent=2)
print(f"wrote {len(scenarios)} collector scenarios -> {OUT}")
```

411. user (2026-06-07T08:10:44.538Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\obseval\gen_collector.py (file
state is current in your context ? no need to Read it back)
```

412. user (2026-06-07T08:10:51.384Z)

```
wrote 4 indexer scenarios -> C:\Users\avidu\AppData\Local\Temp\obseval\indexer_fixtures.json
wrote 3 collector scenarios -> C:\Users\avidu\AppData\Local\Temp\obseval\collector_fixtures.json
```

413. assistant (2026-06-07T08:11:18.037Z)

7 broken-run reports generated. Let me write each to a `.md` file and build a small manifest (paths + ground truth) so the Workflow's fresh-reader agents can read the report + the real README/FAQ by path ? exactly simulating a user, without bloating the script:

414. assistant (2026-06-07T08:11:22.259Z)

```
import json, os

BASE = r"C:\Users\avidu\AppData\Local\Temp\obseval"
REPORTS = os.path.join(BASE, "reports")
os.makedirs(REPORTS, exist_ok=True)

DOC = {
    "indexer": r"C:\Users\avidu\Projects\badminton-highlight-indexer\README.md",
    "collector": r"C:\Users\avidu\Projects\sports-data-collector\docs\INGESTION_PIPELINE.md",
}

manifest = []
for fn in ("indexer_fixtures.json", "collector_fixtures.json"):
    with open(os.path.join(BASE, fn), encoding="utf-8") as f:
        for sc in json.load(f):
            rpath = os.path.join(REPORTS, sc["id"] + ".report.md")
            with open(rpath, "w", encoding="utf-8") as rf:
                rf.write(sc["report_md"])
            manifest.append({
                "id": sc["id"], "tool": sc["tool"],
                "report_path": rpath, "doc_path": DOC[sc["tool"]],
                "truth": sc["truth"], "expected_faq": sc["expected_faq"],
            })

with open(os.path.join(BASE, "manifest.json"), "w", encoding="utf-8") as f:
    json.dump(manifest, f, indent=2)
print(json.dumps(manifest, indent=2))
```

415. user (2026-06-07T08:11:22.326Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\obseval\combine.py (file state
is current in your context ? no need to Read it back)
```

416. user (2026-06-07T08:11:28.822Z)

```
Exit code 1
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import sys,json;
m=json.load(open(r'C:\Users\avidu\AppData\Local\Temp\obseval\manifest.json'));
print('scenarios:', [x['id'] for x in m])
~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
FileNotFoundError: [Errno 2] No such file or directory:
'C:\\Users\\avidu\\AppData\\Local\\Temp\\obseval\\manifest.json'
```

417. user (2026-06-07T08:11:37.767Z)

```
exit=0
    "doc_path": "C:\\Users\\avidu\\Projects\\sports-data-
collector\\docs\\INGESTION_PIPELINE.md",
    "truth": "The imported video has a non-commercial license, so it is EXCLUDED from the
default commercial-only export. It stays in the DB but won't appear downstream unless exported
with --include-noncommercial.",
    "expected_faq": "will-a-non-commercial-license-block-export"
  }
]
=== manifest exists? ===
manifest.json

reports/:
collector_missed_rally.report.md
collector_noncommercial.report.md
collector_overlap.report.md
indexer_fps_fallback.report.md
indexer_motion_synthetic.report.md
indexer_no_api_key.report.md
indexer_validation_failed.report.md
```

418. assistant (2026-06-07T08:11:46.165Z)

```
Manifest + 7 report files written. Let me read the manifest to pass as the Workflow's args:
```

419. user (2026-06-07T08:11:46.271Z)

```
1       [
2         {
3           "id": "indexer_no_api_key",
4           "tool": "indexer",
5           "report_path":
"C:\\Users\\avidu\\AppData\\Local\\Temp\\obseval\\reports\\indexer_no_api_key.report.md",
6           "doc_path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\README.md",
7           "truth": "The run produced 0 highlight segments because no AI-provider API key was
set, so the AI segmenter silently no-op'd. It is NOT that the video has no rallies. Fix: set
GEMINI_API_KEY (e.g. in a .env file) and re-run.",
8           "expected_faq": "Q7"
9         },
10        {
11          "id": "indexer_motion_synthetic",
12          "tool": "indexer",
13          "report_path":
"C:\\Users\\avidu\\AppData\\Local\\Temp\\obseval\\reports\\indexer_motion_synthetic.report.md",
14          "doc_path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\README.md",
```

```
15          "truth": "The 'motion' segmenter was used; its per-segment metadata/events are
heuristic/synthetic, not measured, so they must not be trusted as ground truth. Use yolo_hybrid
or gemini for real shot metadata.",
16          "expected_faq": "Q10"
17        },
18        {
19          "id": "indexer_validation_failed",
20          "tool": "indexer",
21          "report_path":
"C:\\Users\\avidu\\AppData\\Local\\Temp\\obseval\\reports\\indexer_validation_failed.report.md",
22          "doc_path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\README.md",
23          "truth": "Indexing was halted because a validation check failed: the blur/focus
check (video too blurry / sharpness below the min threshold). Fix the footage quality or lower
validation.min_blur_threshold in config.json, then re-run.",
24          "expected_faq": "Q13"
25        },
26        {
27          "id": "indexer_fps_fallback",
28          "tool": "indexer",
29          "report_path":
"C:\\Users\\avidu\\AppData\\Local\\Temp\\obseval\\reports\\indexer_fps_fallback.report.md",
30          "doc_path": "C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\README.md",
31          "truth": "fps was not probed from the file (a fallback default was used), so motion
thresholds and frame->second timing may be miscalibrated. Confirm the real fps with ffprobe and
ensure validation ran.",
32          "expected_faq": "Q5"
33        },
34        {
35          "id": "collector_overlap",
36          "tool": "collector",
37          "report_path":
"C:\\Users\\avidu\\AppData\\Local\\Temp\\obseval\\reports\\collector_overlap.report.md",
38          "doc_path": "C:\\Users\\avidu\\Projects\\sports-data-
collector\\docs\\INGESTION_PIPELINE.md",
39          "truth": "The delivery is blocked because two rallies overlap in time (rally 1 ends
at 7.00s but rally 2 starts at 6.00s). Rallies must be disjoint; the vendor/reviewer must fix
rally 1's end or rally 2's start before it can be imported.",
40          "expected_faq": "what-does-validation-check"
41        },
42        {
43          "id": "collector_missed_rally",
44          "tool": "collector",
45          "report_path":
"C:\\Users\\avidu\\AppData\\Local\\Temp\\obseval\\reports\\collector_missed_rally.report.md",
46          "doc_path": "C:\\Users\\avidu\\Projects\\sports-data-
collector\\docs\\INGESTION_PIPELINE.md",
47          "truth": "There is a long unlabeled gap (~26s) between rally 2 and rally 3, which
may hide an omitted/missed rally. The reviewer should watch that span and add the rally if one
is there. It is a review flag, not a blocker.",
48          "expected_faq": "what-does-validation-check"
49        },
50        {
51          "id": "collector_noncommercial",
52          "tool": "collector",
53          "report_path":
"C:\\Users\\avidu\\AppData\\Local\\Temp\\obseval\\reports\\collector_noncommercial.report.md",
54          "doc_path": "C:\\Users\\avidu\\Projects\\sports-data-
collector\\docs\\INGESTION_PIPELINE.md",
55          "truth": "The imported video has a non-commercial license, so it is EXCLUDED from
the default commercial-only export. It stays in the DB but won't appear downstream unless
exported with --include-noncommercial.",
56          "expected_faq": "will-a-non-commercial-license-block-export"
57        }
58      ]
```

420. assistant (2026-06-07T08:12:41.405Z)

Now I'll run the **debuggability evaluation as a Workflow** (ultracode): for each of the 7 broken-run scenarios, a fresh-reader agent gets *only* the README/FAQ + the report (no source) and must self-diagnose; a tough grader then checks whether they found the root cause and the right FAQ, and proposes concrete refinements; a final agent synthesizes.

421. user (2026-06-07T08:12:59.739Z)

Workflow launched in background. Task ID: w8ghqp2hd
Summary: Fresh-reader debuggability eval of per-video observability reports across broken-run scenarios
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_63677f5b-055
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-sports-data-collector\56b7f849-5d56-4770-93e0-b3b4da943700\workflows\scripts\report-debuggability-eval-wf_63677f5b-055.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-sports-data-collector\56b7f849-5d56-4770-93e0-b3b4da943700\workflows\scripts\report-debuggability-eval-wf_63677f5b-055.js"} to iterate without resending the script.)
Run ID: wf_63677f5b-055
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-sports-data-collector\56b7f849-5d56-4770-93e0-b3b4da943700\workflows\scripts\report-debuggability-eval-wf_63677f5b-055.js", resumeFromRunId: "wf_63677f5b-055"}) ? completed agents return cached results.

You will be notified when it completes. Use /workflows to watch live progress.

422. assistant (2026-06-07T08:13:11.973Z)

The workflow failed instantly ? `args` arrived as a JSON string, not an array. Let me make the script robust to both and re-invoke via its saved path:

423. user (2026-06-07T08:13:13.661Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

424. user (2026-06-07T08:13:18.066Z)

```
86          `- confusing_parts: ${reader ? JSON.stringify(reader.confusing_parts || []) :
'[]'}\n\n` +
87          `Also read the actual report for context: ${it.report_path}\n\n` +
88          `Judge strictly: did the reader land on the correct ROOT CAUSE (cause_correct)? Did
they find/consult the ` +
89          `expected FAQ (faq_correct)? How large is the remaining gap (none/minor/major)? Then
propose ONE concrete, ` +
90          `specific refinement to the REPORT wording, the spec rule message, or the FAQ that
would have made this scenario ` +
91          `unambiguous to self-debug ? or "none" if it is already excellent. Don't be
generous; if anything was confusing, propose a fix.`,
92          { label: `grade:${it.id}`, phase: 'Grade', schema: GRADER_SCHEMA },
93        ).then((g) => ({ ...g, scenario: it.id, tool: it.tool, reader })),
94      )
95
```

425. user (2026-06-07T08:13:23.119Z)

```
68          `(do not open source code or any other file):\n` +
69          `1. The README/FAQ: ${it.doc_path}\n` +
70          `2. The tool's technical report for the suspicious run: ${it.report_path}\n\n` +
71          `Using ONLY those two, tell me: what went wrong, exactly what you'd do to fix it,
and which FAQ section ` +
```

```
72          `(by heading text or Q-number) you used. Be honest about whether you could truly
self-debug this from the ` +
73          `report alone, and list anything in the report that was confusing, ambiguous, or
under-explained.`,
74          { label: `read:${it.id}`, phase: 'Read', agentType: 'Explore', schema:
FRESH_READER_SCHEMA },
75        ),
76        (reader, it) => agent(
77          `You are a tough evaluator of how DEBUGGABLE an observability report is for a
README-only reader.\n` +
```

426. user (2026-06-07T08:13:28.422Z)

```
62
63      const graded = await pipeline(
64        items,
65        (it) => agent(
66          `You are a USER of a sports-video tool who has read ONLY its README/FAQ ? you have
NEVER seen the source code. ` +
```

427. user (2026-06-07T08:13:33.513Z)

```
59      }
60
61      const items = args
62
```

428. user (2026-06-07T08:13:38.522Z)

```
The file C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-sports-data-
collector\56b7f849-5d56-4770-93e0-b3b4da943700\workflows\scripts\report-debuggability-eval-
wf_63677f5b-055.js has been updated successfully. (file state is current in your context ? no
need to Read it back)
```

429. assistant (2026-06-07T08:13:54.390Z)

```
Fixed. Re-invoking the workflow via its saved script path:
```

430. user (2026-06-07T08:13:58.667Z)

```
Workflow launched in background. Task ID: wwkbitg1n
Summary: Fresh-reader debuggability eval of per-video observability reports across broken-run
scenarios
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_49729ae7-962
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-sports-data-
collector\56b7f849-5d56-4770-93e0-b3b4da943700\workflows\scripts\report-debuggability-eval-
wf_63677f5b-055.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-sports-data-
collector\56b7f849-5d56-4770-93e0-b3b4da943700\workflows\scripts\report-debuggability-eval-
wf_63677f5b-055.js"} to iterate without resending the script.)
Run ID: wf_49729ae7-962
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--
Users-avidu-Projects-sports-data-
collector\56b7f849-5d56-4770-93e0-b3b4da943700\workflows\scripts\report-debuggability-eval-
wf_63677f5b-055.js", resumeFromRunId: "wf_49729ae7-962"}) ? completed agents return cached
results.

You will be notified when it completes. Use /workflows to watch live progress.
```

431. assistant (2026-06-07T08:14:16.890Z)

```
I see PR #10 (the parser fix) was merged to the collector master by the parallel flow, and my
```

memory was updated to reflect it. My collector PR #11 (observability-only) is now behind that merge. While the debuggability workflow runs in the background, let me **sync PR #11 from head** ? rebase it onto the updated master (no conflict expected since I don't touch `rally_cvat.py`):

432. user (2026-06-07T08:14:20.392Z)

```
=== origin/master head ===
99dfa22 Merge pull request #10 from avidullu/fix/canonical-csv-zero-start
f7cab11 fix(parser): keep a rally that starts at 0.0s in load_canonical_csv
d68c5da Merge pull request #9 from avidullu/feat/validate-delivery
=== my branch vs origin/master ===
2       1
```

433. assistant (2026-06-07T08:14:28.008Z)

Master has #10's parser fix (2 commits ahead); my 1 observability commit needs rebasing on top. Rebasing and force-pushing PR #11 to keep it current:

434. user (2026-06-07T08:14:34.875Z)

```
Rebasing (1/1)

Successfully rebased and updated refs/heads/feat/observability-reports.
=== log ===
038424d feat(reporting): per-video observability report (collector)
99dfa22 Merge pull request #10 from avidullu/fix/canonical-csv-zero-start
f7cab11 fix(parser): keep a rally that starts at 0.0s in load_canonical_csv
=== quick test ===
.................................................................... [ 63%]
.......................................                             [100%]
113 passed in 1.98s
pushed PR #11 (rebased on master incl #10)
```

435. assistant (2026-06-07T08:14:58.512Z)

PR #11 is now rebased on master (113 tests, the parser fix's regression test now in ancestry) and force-pushed ? current with head. The debuggability workflow is running in the background.

While it runs, let me prep the "eyeball" rendering (MD?styled HTML?screenshot) so I can visually review the *refined* reports once the eval lands. Checking the markdown lib + loading the screenshot tools:

436. user (2026-06-07T08:14:59.764Z)

```
markdown 3.10.2
```

437. assistant (2026-06-07T08:15:25.826Z)

Good ? `markdown` 3.10 is available, and Preview works via a static dev server. Let me pre-write the MD?HTML render script now (independent of the eval) so I can quickly screenshot the *refined* reports once the workflow lands:

438. assistant (2026-06-07T08:15:34.082Z)

```
"""Render report .md files into one self-contained styled HTML page for eyeballing.

Usage: python render_html.py <out_html> <title::md_path> [<title::md_path> ...]
"""
import sys, os, markdown

CSS = """
body{font-family:-apple-system,Segoe UI,Roboto,Helvetica,Arial,sans-serif;line-height:1.5;
  color:#1b1f24;max-width:1180px;margin:24px auto;padding:0 20px;background:#f6f8fa;}
.cards{display:flex;gap:20px;flex-wrap:wrap;align-items:flex-start;}
```

```
.card{background:#fff;border:1px solid #d0d7de;border-radius:10px;padding:20px 24px;
  flex:1 1 520px;box-shadow:0 1px 3px rgba(27,31,36,.08);}
.card h1{font-size:20px;margin-top:0;border-bottom:1px solid #d8dee4;padding-bottom:8px;}
.card h2{font-size:15px;color:#57606a;margin:18px 0 6px;text-transform:uppercase;letter-
spacing:.03em;}
table{border-collapse:collapse;margin:6px 0 12px;font-size:14px;}
th,td{border:1px solid #d0d7de;padding:5px 12px;text-align:left;}
th{background:#f6f8fa;}
code{background:#eff1f3;padding:1px 5px;border-radius:5px;font-size:13px;}
li{margin:3px 0;}
.tag{display:inline-block;font-size:11px;font-weight:600;padding:2px 8px;border-radius:999px;
  margin-bottom:10px;}
.tech{background:#ddf4ff;color:#0969da;}
.cust{background:#dafbe1;color:#1a7f37;}
"""

def main():
    out = sys.argv[1]
    cards = []
    for spec in sys.argv[2:]:
        title, path = spec.split("::", 1)
        md = open(path, encoding="utf-8").read()
        html = markdown.markdown(md, extensions=["tables", "fenced_code"])
        cls = "cust" if "summary" in title.lower() or "customer" in title.lower() else "tech"
        cards.append(f'<div class="card"><span class="tag {cls}">{title}</span>{html}</div>')
    doc = f"<!doctype html><html><head><meta charset='utf-8'><style>{CSS}</style></head>" \
          f"<body><div class='cards'>{''.join(cards)}</div></body></html>"
    os.makedirs(os.path.dirname(out), exist_ok=True)
    open(out, "w", encoding="utf-8").write(doc)
    print(f"wrote {out}")

if __name__ == "__main__":
    main()
```

439. user (2026-06-07T08:15:34.120Z)

File created successfully at: C:\Users\avidu\AppData\Local\Temp\obseval\render_html.py (file state is current in your context ? no need to Read it back)

440. assistant (2026-06-07T08:15:48.484Z)

Everything is synced and the evaluation is in flight. Status while the workflow runs:

Synced to remote (all current with head):
- `obsreport` lib ? github.com/avidullu/sports-obsreport (private, tags v0.1.0?v0.1.3)
- Indexer ? [PR #41](https://github.com/avidullu/badminton-highlight-indexer/pull/41) (rebased onto your `#39` typed-config work; `main.py` conflict resolved ? kept your typed config access + my reporting, reporting normalizes `AppConfig`?dict)
- Collector ? [PR #11](https://github.com/avidullu/sports-data-collector/pull/11) (observability-only, **rebased onto master including the merged `#10` parser fix**; 113 tests green)

Merge-conflict notes for you: the only real conflict was indexer `main.py` (your config package vs my reporting) ? resolved cleanly. Indexer PR carries **2 pre-existing red tests from `#39`** (`test_config.py`: `config.json` default = `gemini` vs model default `yolo_hybrid`, and a `Path == str` Windows comparison) ? I left them untouched so they don't collide with Grok's config work; flagged in the PR body.

Debuggability eval (running): a multi-agent workflow is putting all 7 broken-run reports in front of "fresh reader" agents that may see *only* the README/FAQ + the report (no source), then a tough grader checks whether each could self-diagnose and cites the right FAQ. I'll apply the synthesized refinements, **render the refined reports to an image and eyeball them**, add the golden faq-resolution tests, and push to the PRs once it lands.

441. user (2026-06-07T08:15:50.049Z)

Can you resync from remote and run athe entire test suite to validate that the directory and Config changes are all well.

442. assistant (2026-06-07T08:16:03.097Z)

On it. Let me re-sync both repos from remote ? fetch latest master (which may now include the directory restructure that was sequenced after Config), see how far my branches are behind, then rebase and run the full suites.

443. user (2026-06-07T08:16:08.680Z)

```
=== INDEXER origin/master head ===
0355417 Merge pull request #40 from avidullu/refactor/approved-directory-structure
1e96e07 test: fix mock patch targets after structure refactor and config test assertions
8615534 refactor: adopt approved layered directory structure
ce6a0e9 Merge pull request #39 from avidullu/feat/config-pydantic-package
da6ea5c feat(config): introduce backend/config/ package as first low-regret deliverable
f4c1b2d docs: checkpoint approved structure + sequencing decisions (Config first, atomic
refactor follow-up)
=== my feat vs origin/master (behind/ahead) ===
3       3

=== COLLECTOR origin/master head ===
99dfa22 Merge pull request #10 from avidullu/fix/canonical-csv-zero-start
f7cab11 fix(parser): keep a rally that starts at 0.0s in load_canonical_csv
d68c5da Merge pull request #9 from avidullu/feat/validate-delivery
2882a03 feat(ingest): validate-delivery ? per-video QA gate + structured report
23f2ed0 Merge pull request #8 from avidullu/feat/last-shot-outcome
=== my feat vs origin/master (behind/ahead) ===
0       1
```

444. assistant (2026-06-07T08:16:23.801Z)

Significant change on the **indexer**: PR #40 merged a **directory restructure** (`refactor/approved-directory-structure`) ? and it also fixed the 2 config test assertions I'd flagged. The collector is unchanged (already current). My indexer branch needs rebasing onto the restructured master, which will mean relocating my reporting module + reconciling moved files. Let me first understand the new layout before rebasing:

445. user (2026-06-07T08:16:25.626Z)

```
=== refactor commit file moves (renames) ===
commit 8615534e261ea7bc355468667772b272820eea6a
Author: avidullu <avidullu@users.noreply.github.com>
Date:   Sun Jun 7 07:47:14 2026 +0000

    refactor: adopt approved layered directory structure

    - Move to professional layered layout per checkpointed plan:
      - backend/api/static/ (frontend)
      - backend/pipeline/{validators,segmenters,stitcher,audio,detectors}
      - backend/infrastructure/database.py
    - indexer/ renamed to segmenters/ inside pipeline for clarity.
    - Updated all imports (absolute backend.xxx paths) across source, tests, run_*.py.
    - Fixed static serving paths in main.py.
    - Updated CODE_MAP.md references and high-level structure description.
    - This is the atomic high-pri follow-up refactor after the Config package (Deliverable 1).

    All moved modules now live under clear layers. No behavior change intended.
    Co-authored with avidullu.

 backend/{ => api}/static/app.js                            | 0
```

```
 backend/{ => api}/static/index.html                          | 0
 backend/{ => api}/static/style.css                           | 0
 backend/{ => infrastructure}/database.py                     | 0
 backend/{ => pipeline}/audio/__init__.py                     | 0
 backend/{ => pipeline}/audio/hit_detector.py                 | 0
 backend/{ => pipeline}/detectors/__init__.py                 | 0
 backend/{ => pipeline}/detectors/rally_gate.py               | 0
 backend/{ => pipeline}/detectors/tracknet_runner.py          | 0
 backend/{ => pipeline}/detectors/wasb_infer.py               | 0
 backend/{ => pipeline}/detectors/wasb_runner.py              | 2 +-
 backend/{indexer => pipeline/segmenters}/base.py             | 2 +-
 backend/{indexer => pipeline/segmenters}/gemini.py           | 4 ++--
 backend/{indexer => pipeline/segmenters}/motion.py           | 4 ++--
 backend/{indexer => pipeline/segmenters}/tracknet_hybrid.py | 4 ++--
 backend/{indexer => pipeline/segmenters}/wasb_hybrid.py      | 4 ++--
 backend/{indexer => pipeline/segmenters}/yolo_hybrid.py      | 2 +-
 backend/{ => pipeline}/stitcher/compiler.py                  | 0
 backend/{ => pipeline}/validators/base.py                    | 0
 backend/{ => pipeline}/validators/blur.py                    | 2 +-
 backend/{ => pipeline}/validators/format.py                  | 2 +-
 backend/{ => pipeline}/validators/pts_continuity.py          | 2 +-
 backend/{ => pipeline}/validators/relevance.py               | 2 +-
 backend/{ => pipeline}/validators/resolution_fps.py          | 2 +-
 backend/{ => pipeline}/validators/scene_cut.py               | 2 +-
 25 files changed, 17 insertions(+), 17 deletions(-)
```

446. assistant (2026-06-07T08:17:01.949Z)

The indexer got a **layered directory restructure** (#40): `backend/validators/` → `backend/pipeline/validators/`, `backend/indexer/` → `backend/pipeline/segmenters/`, `backend/database.py` → `backend/infrastructure/database.py`, etc. My branch modifies some of those moved files, so this rebase needs care. Let me confirm a clean working tree and attempt the rebase:

447. user (2026-06-07T08:17:04.285Z)

```
=== working tree ===
=== rebase onto origin/master ===
Rebasing (1/3)
Rebasing (2/3)
Rebasing (3/3)

Successfully rebased and updated refs/heads/feat/observability-reports.
```

448. assistant (2026-06-07T08:17:25.064Z)

The rebase applied cleanly (git's rename detection merged my `main.py` reporting wiring onto the restructured imports). But my **test files still import the old paths** (`backend.validators.base`). Let me find stale references in my added files and verify the moved validator got my change:

449. user (2026-06-07T08:17:28.000Z)

```
=== run_all_with_context in new location? ===
2
=== old backend/validators exists? (should be gone) ===
__pycache__
=== stale imports in MY files ===
tests/test_validators_base.py:3:from backend.validators.base import ValidationRegistry,
ValidationResult, VideoValidator
tests/test_reporting_harvest.py:13:from backend.validators.base import ValidationResult  # noqa:
E402
```

450. assistant (2026-06-07T08:17:34.696Z)

Two fixes needed ? my test files import the pre-move path. Let me update them to the new `backend.pipeline.validators.base`:

451. user (2026-06-07T08:17:36.080Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_validators_base.py has been updated successfully. (file state is current in your context ? no need to Read it back)

452. user (2026-06-07T08:17:40.961Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_reporting_harvest.py has been updated successfully. (file state is current in your context ? no need to Read it back)

453. assistant (2026-06-07T08:17:48.050Z)

Now run the **full indexer suite** to validate the directory + config changes integrate cleanly with my reporting:

454. user (2026-06-07T08:18:03.085Z)

```
.................................................................... [ 31%]
.................................................................... [ 63%]
.................................................................... [ 94%]
............                                                         [100%]
228 passed in 12.63s
```

455. assistant (2026-06-07T08:18:23.737Z)

All **228 indexer tests pass** ? the directory restructure + typed config + my reporting all integrate cleanly, and the 2 previously-flagged config tests are now green (fixed by #40). Let me do a bare import smoke (tests mock heavily), commit the import-path fixes, and update the PR:

456. user (2026-06-07T08:18:34.475Z)

```
import smoke OK
pushed indexer PR #41 (rebased on #40 restructure)
```

457. assistant (2026-06-07T08:18:40.787Z)

Indexer PR #41 updated and green. Now the **collector** and **obsreport** full suites (the user asked for the entire suite across the changes):

458. user (2026-06-07T08:18:46.135Z)

```
=== COLLECTOR full suite ===
.................................................................... [ 63%]
.....................................                                [100%]
113 passed in 1.97s

=== OBSREPORT full suite ===
.................................................................... [ 97%]
..                                                                   [100%]
74 passed in 0.36s
Shell cwd was reset to C:\Users\avidu\Projects\badminton-highlight-indexer
```

459. assistant (2026-06-07T08:19:21.743Z)

All three suites are green. Let me also confirm the restructured + typed-config pipeline runs **end-to-end at runtime** (tests mock heavily), via a quick CLI smoke that exercises my reporting in the `finally`:

460. user (2026-06-07T08:19:31.348Z)

```
VALIDATION SUMMARY
[PASS] format_check: Video format is valid MP4 container.
[FAIL] resolution_fps_check: Validator threw an unexpected error: 'ValidationConfig' object has
no attribute 'get'
[PASS] pts_continuity_check: Video is too short to perform a PTS continuity check.
[FAIL] scene_cut_check: Validator threw an unexpected error: 'ValidationConfig' object has no
attribute 'get'
[FAIL] blur_check: Validator threw an unexpected error: 'ValidationConfig' object has no
attribute 'get'
[FAIL] relevance_check: Validator threw an unexpected error: 'ValidationConfig' object has no
attribute 'get'
[FAIL] Video failed checks. Indexing halted.
[REPORT] Technical report :
C:\Users\avidu\AppData\Local\Temp\obs_restructure_smoke\smoke.report.md
[REPORT] Customer summary :
C:\Users\avidu\AppData\Local\Temp\obs_restructure_smoke\smoke.summary.md
[REPORT] Machine JSON     :
C:\Users\avidu\AppData\Local\Temp\obs_restructure_smoke\smoke.report.json
---
C:\Users\avidu\AppData\Local\Temp/obs_restructure_smoke/smoke.report.json
C:\Users\avidu\AppData\Local\Temp/obs_restructure_smoke/smoke.report.md
C:\Users\avidu\AppData\Local\Temp/obs_restructure_smoke/smoke.summary.md
REPORT FILES EMITTED OK
```

461. assistant (2026-06-07T08:20:59.881Z)

Re-sync + full validation done across all three repos. But the runtime smoke surfaced something important ? and notably, **my observability report is what caught it.**

Resync + test validation

Repo	Synced onto	Full suite
Indexer	rebased onto master incl. **#40 directory restructure** + **#39 config**	**228 passed, 0 failed** ?
Collector	rebased onto master incl. **#10 parser fix**	**113 passed** ?
obsreport	(mine)	**74 passed** ?

- The rebase onto the layered restructure (`backend/pipeline/{validators,segmenters,stitcher,detectors,audio}`, `backend/infrastructure/database.py`, `backend/api/static/`) was clean ? git's rename detection merged my `main.py` reporting wiring onto the new imports; I only had to repoint two of my test files. PR #41 force-pushed and green.
- The 2 config-test failures I'd flagged earlier are now **fixed by #40** (so the indexer PR is fully green now).

?? Directory changes: well. Config change: a real runtime bug.

A bare-import smoke and the test suite pass, but an **actual `python -m backend.main` run** fails 4 validators:

```
[FAIL] resolution_fps_check: Validator threw an unexpected error: 'ValidationConfig' object has
no attribute 'get'
[FAIL] scene_cut_check:      ... 'ValidationConfig' object has no attribute 'get'
[FAIL] blur_check:           ... 'ValidationConfig' object has no attribute 'get'
[FAIL] relevance_check:      ... 'ValidationConfig' object has no attribute 'get'
```

Root cause: `main()` now passes the typed `AppConfig` to validators/segmenters, but `AppConfig.get()` is dict-compatible only at the **top level** ? `config.get("validation")` returns a `ValidationConfig` *model*, and the validators then call `.get("min_resolution_width",

...)` on it, which a Pydantic model doesn't support. So every consumer that does nested `config.get("section").get("key")` (all the validators, and likely the segmenters) breaks on a real video. Tests don't catch it because they pass plain dicts directly.

This is a **master-level bug from the config migration (#39), not fixed by #40, and not introduced by my PRs** ? my reporting layer already guards itself (it normalizes `AppConfig`?dict), which is exactly why it ran fine and emitted the report that exposed the failure.

The minimal fix lives in Grok's `backend/config/models.py`: have `AppConfig.get()` return `model_dump()` (a real dict) for nested model sections, so legacy `config.get("validation").get(...)` works again (1?2 lines, low risk). Since config is Grok's active area:

462. user (2026-06-07T08:21:14.955Z)

Your questions have been answered: "The typed AppConfig from #39 breaks all the validators (and likely segmenters) on a real video run ? nested config.get("section").get("key") fails. Want me to fix it, or leave it to the config work?"="Fix it now (small PR)". You can now continue with these answers in mind.

463. assistant (2026-06-07T08:21:58.821Z)

You chose to fix it now as a small PR. Let me branch off the indexer master (so the PR is just the config fix), confirm my feat branch is clean first:

464. user (2026-06-07T08:22:01.176Z)

```
=== feat clean? ===
branch 'fix/appconfig-nested-get' set up to track 'origin/master'.
Switched to a new branch 'fix/appconfig-nested-get'
=== on branch ===
fix/appconfig-nested-get
=== confirm reporting NOT here (master-based) ===
__pycache__
```

465. assistant (2026-06-07T08:22:07.972Z)

On the fix branch. Let me read the current `AppConfig.get()` and apply the nested-section fix:

466. user (2026-06-07T08:22:08.838Z)

```
110
111         # Temporary dict-like access for legacy code that does config.get("foo", {})
112         # and passes the config object around. Will be removed after consumers are migrated.
113         def get(self, key: str, default: Any = None) -> Any:
114             try:
115                 return getattr(self, key)
116             except AttributeError:
117                 # Also support nested "indexing.default_segmenter" style? For now simple
top-level.
118                 if key in self.model_fields:
119                     return getattr(self, key)
120                 # Try to find in sections
121                 for section_name in ("validation", "indexing", "output", "stitching"):
122                     section = getattr(self, section_name, None)
123                     if section is not None:
124                         try:
125                             return getattr(section, key)
126                         except AttributeError:
127                             pass
128                 return default
129
130         def __getitem__(self, key: str) -> Any:
```

```
131             val = self.get(key)
132             if val is None and key not in self.model_fields:
133                 # Re-raise like a real dict for unknown top-level
134                 raise KeyError(key)
135             return val
136 
137 
138     # Convenience type alias for callers that want to annotate
139     Config = AppConfig
```

467. user (2026-06-07T08:22:19.778Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

468. assistant (2026-06-07T08:22:39.133Z)

Now validate the fix ? full suite on this branch + the CLI smoke (validators should now execute their real logic instead of erroring on `.get`):

469. user (2026-06-07T08:22:54.623Z)

```
=== full suite on fix branch ===
============================ short test summary info ============================
ERROR tests/test_config.py
!!!!!!!!!!!!!!!!!!!! Interrupted: 1 error during collection !!!!!!!!!!!!!!!!!!!!
1 error in 4.78s

=== CLI smoke: validators should run real logic, no '.get' errors ===
```

470. assistant (2026-06-07T08:23:00.790Z)

Collection error in `test_config.py` ? let me see it:

471. user (2026-06-07T08:23:01.762Z)

```
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import backend.config.models
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\__init__.py", line 9,
in <module>
    from .loader import load_config, save_default_config
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\loader.py", line 15,
in <module>
    from .models import AppConfig
  File "C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py", line 121
    val: Any = _MISSING = object()
                        ^
SyntaxError: invalid syntax
```

472. assistant (2026-06-07T08:23:08.813Z)

Syntax error ? can't combine an annotated assignment with chaining. Let me split it:

473. user (2026-06-07T08:23:09.960Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py has been
updated successfully. (file state is current in your context ? no need to Read it back)
```

474. user (2026-06-07T08:23:37.735Z)

```
import OK
=== full suite ===
```

```
.................................................................... [ 33%]
.................................................................... [ 67%]
....................................................................  [100%]
214 passed in 12.01s

=== CLI smoke ===
[PASS] format_check: Video format is valid MP4 container.
[PASS] resolution_fps_check: Video resolution and frame rate meet minimum standards.
[PASS] pts_continuity_check: Video is too short to perform a PTS continuity check.
[PASS] scene_cut_check: Scene continuity check passed. No abnormal visual cuts detected.
[FAIL] blur_check: Video quality rejected: Focus check failed. Average sharpness score of 0.0 is
below the minimum required threshold of 20.0.
[PASS] relevance_check: Video identified as relevant to badminton (badminton court structures
found).
[FAIL] Video failed checks. Indexing halted.
```

475. assistant (2026-06-07T08:23:53.826Z)

The fix works ? validators now execute their real logic (resolution/scene-cut/relevance **PASS**; blur legitimately fails on the textureless green clip), no more `.get` errors. 214 tests green. Let me lock it with a regression test. Reading the current (post-#40) test_config.py first:

476. user (2026-06-07T08:23:54.665Z)

```
60                  "validation": {"relevance": {"court_pixels_min_ratio": 0.1}},
61              }
62              with open(p, "w") as f:
63                  json.dump(data, f)
64
65              cfg = load_config(p)
66              assert cfg.indexing.default_segmenter == "yolo_hybrid"
67              assert cfg.validation.relevance.court_pixels_min_ratio == 0.1
68
69
70      def test_save_default_config_roundtrip():
71          with tempfile.TemporaryDirectory() as td:
72              p = os.path.join(td, "saved.json")
73              saved = save_default_config(p)
74              assert str(saved) == p
75              assert os.path.exists(p)
76
77              cfg = load_config(p)
78              assert cfg.indexing.default_segmenter == "yolo_hybrid"
79              assert cfg.validation.min_blur_threshold == 20.0
80
81
82      def test_model_is_the_source_of_truth():
83          """Changing a default in the model should be reflected without touching json."""
84          # This is a documentation test more than anything.
85          cfg = AppConfig()
86          assert cfg.indexing.tracknet.velocity_threshold == 0.02
87
```

477. user (2026-06-07T08:24:08.076Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py has been updated successfully. (file state is current in your context ? no need to Read it back)

478. user (2026-06-07T08:24:27.126Z)

```
  C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py:123:
PydanticDeprecatedSince211: Accessing the 'model_fields' attribute on the instance is
deprecated. Instead, you should access this attribute from the model class. Deprecated in
```

```
Pydantic V2.11 to be removed in V3.0.
    if key in self.model_fields or hasattr(self, key):

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
6 passed, 4 warnings in 0.18s
=== full suite ===

-- Docs: https://docs.pytest.org/en/stable/how-to/capture-warnings.html
215 passed, 4 warnings in 11.68s
```

479. assistant (2026-06-07T08:24:35.715Z)

215 passed. Let me silence a Pydantic deprecation warning (instance `model_fields` access) that my rewrite carried over ? access it from the class instead:

480. user (2026-06-07T08:24:37.475Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

481. user (2026-06-07T08:24:50.020Z)

```
......                                                                   [100%]
6 passed in 0.25s
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/fix/appconfig-
nested-get
remote:
pushed
```

482. assistant (2026-06-07T08:25:02.474Z)

Clean (no warnings). Opening the focused config-fix PR:

483. user (2026-06-07T08:25:08.041Z)

```
https://github.com/avidullu/badminton-highlight-indexer/pull/42
=== back to feat branch ===
Switched to branch 'feat/observability-reports'
```

484. assistant (2026-06-07T08:25:38.308Z)

PR #42 opened for the config fix. (Back on the feat branch, `models.py` correctly shows master's original ? the fix lives on its own branch/PR.) Let me checkpoint memory, then give you the full status.

485. user (2026-06-07T08:25:47.024Z)

The file c:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

486. assistant (2026-06-07T08:26:11.439Z)

Done ? re-synced and validated across everything, plus I caught and fixed a real config bug that the parallel work introduced.

? Re-sync + full-suite validation

```
| Repo | Synced onto latest master | Full suite |
|---|---|---|
| **Indexer** | **#39** (typed config) + **#40** (layered directory restructure) | **228
passed** ? |
| **Collector** | **#10** (parser fix) | **113 passed** ? |
```

| **obsreport** | ? | **74 passed** ? |

- **Directory restructure: all well.** The rebase onto the new `backend/pipeline/{validators,segmenters,stitcher,detectors,audio}` / `backend/infrastructure/` / `backend/api/static/` layout was clean ? git's rename-detection merged my `main.py` reporting wiring onto the relocated imports; I only repointed two of my test files. The 2 config-test failures I flagged earlier are now fixed by #40.

? Config change: **was *not* all well ? found & fixed**

A real `python -m backend.main` run failed **4 validators** with `'ValidationConfig' object has no attribute 'get'`. The typed `AppConfig` from #39 is passed to validators/segmenters that do legacy chained access `config.get("validation", {}).get("key")`, but `AppConfig.get()` returned the nested *Pydantic model* (no `.get`) ? dict-compat was top-level only. Tests passed because they inject plain dicts, so it slipped through CI; a real video run is broken on master.

- **Fixed** in [indexer PR #42](https://github.com/avidullu/badminton-highlight-indexer/pull/42): `AppConfig.get()` now returns nested sections as dicts (`model_dump`) ? chained access works again; + regression test; **215 green**; CLI smoke now runs all validators. Nicely, **my observability report is what surfaced it** (it normalizes config itself, so it ran and reported the validator failures).

Open PRs

- [indexer #41](https://github.com/avidullu/badminton-highlight-indexer/pull/41) ? observability (green, rebased on #40)
- [indexer #42](https://github.com/avidullu/badminton-highlight-indexer/pull/42) ? the config fix (green)
- [collector #11](https://github.com/avidullu/sports-data-collector/pull/11) ? observability (green, rebased on #10)

Suggested merge order: **#42 ? #41** (so #41's runtime path isn't broken), **#11** any time.

Still in flight: the Phase-D debuggability workflow (fresh-reader agents over the 7 broken-run reports). When it lands I'll apply the wording refinements, render a report to an image to eyeball, add the golden FAQ-resolution tests, and push to #41/#11. I'll pick that up on its completion notification.

487. user (2026-06-07T08:26:11.541Z)

Do you see any issues with #41? Can I merge it and then you can rebase and keep going ?

488. assistant (2026-06-07T08:26:50.255Z)

Let me verify #41's actual merge state on GitHub before giving you a grounded answer, and check whether CI exists:

489. user (2026-06-07T08:26:56.106Z)

{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE","title":"feat: per-video observability reports (indexer) + shared obsreport lib"}
=== CI checks on #41 (if any) ===
no checks reported on the 'feat/observability-reports' branch
=== files #41 touches vs #42 (overlap check) ===
--#41--
README.md
backend/main.py
backend/pipeline/validators/base.py
backend/reporting/__init__.py
backend/reporting/harvest.py
backend/reporting/spec.yaml
requirements.txt
tests/test_reporting_harvest.py
tests/test_validators_base.py

--#42--
backend/config/models.py
tests/test_config.py

490. assistant (2026-06-07T08:27:29.294Z)

Two things resolved at once. First, your question about **#41**:

No issues ? safe to merge. GitHub reports it `MERGEABLE` / `CLEAN`, no CI checks are configured (nothing to go red), and **#41 and #42 touch disjoint files** (`backend/reporting/*` + `main.py` + `pipeline/validators/base.py` vs. `backend/config/models.py`) ? so they merge independently in any order. 228 tests green. One design note: the reports only *emit* when `obsreport` is installed (soft dep) ? it's editable-installed on your machine so it works; a fresh clone/CI no-ops gracefully. Recommend merging **#42** too so real video runs aren't broken by the validator bug.

Go ahead and merge #41 ? I'll rebase #42 and my follow-on work onto the new master.

Also, the **debuggability workflow just finished**. Let me read the full results:

491. user (2026-06-07T08:27:30.346Z)


```
1       {
2         "summary": "Fresh-reader debuggability eval of per-video observability reports across
broken-run scenarios",
3         "agentCount": 15,
4         "logs": [],
5         "result": {
6           "scores": [
7             {
8               "cause_correct": true,
9               "faq_correct": true,
10              "gap": "minor",
11              "refinement": "In the silent_provider_noop finding, name the active segmenter
and tie it to the AI handoff so the reader does not get distracted wondering whether the
segmenter name is the fault. E.g.: \"The 'wasb_hybrid' segmenter found 1 candidate window but
produced 0 confirmed segments because it hands those windows to the AI provider (gemini), and no
API key was set (api_key_present=False) ? so the provider silently did nothing. This is a
missing credential, NOT 'the video has no rallies'. Fix: set GEMINI_API_KEY in a .env file in
the project root (see README#Q7) and re-run.\"",
12              "notes": "Reader landed exactly on the root cause (missing GEMINI_API_KEY ->
silent AI provider no-op, 0 segments, NOT a no-rallies video) and consulted the expected FAQ
(README#Q7, which exists at README.md:197). Fix proposed by reader matches ground truth.
could_self_debug=true is justified.\n\nReport quality is high: the finding explicitly rules out
the false 'no rallies' interpretation, gives the GEMINI_API_KEY/.env fix, points to Q7, and
cites evidence (api_key_present=False, segment_count=0).\n\nRemaining gap is minor and stems
from one report detail: the segmentation stage prominently shows
_segmenter_requested_/_actual_/_config_default_ = wasb_hybrid with
_matches_config_default_=True, but never connects wasb_hybrid to the AI handoff. This led the
reader to a side-quest ? believing 'wasb_hybrid' is undocumented/invalid and must be changed to
yolo_hybrid/gemini/motion. In fact wasb_hybrid is a real, registered segmenter
(backend/indexer/wasb_hybrid.py) that routes through the gemini provider; it is just absent from
Q7's enumeration. The reader's 'invalid segmenter' worry is therefore a false lead that did not
block self-debug but did add noise.\n\nTwo smaller nits the reader raised: (1) 'silent provider
no-op' is jargon, though the finding immediately glosses it; (2) the finding says '.env file'
but not where ? the project-root location lives only in Q7. The proposed refinement folds the
segmenter-to-handoff link and the explicit project-root path into the finding so the reader
never has to leave it.\n\nSecondary doc-gap worth a follow-up (not blocking this scenario):
README Q7 lists only yolo_hybrid/gemini/motion and omits wasb_hybrid, which is what made the
reader think it was invalid.",
13              "scenario": "indexer_no_api_key",
14              "tool": "indexer",
15              "reader": {
16                "diagnosis": "The tool ran the segmenter with AI handoff enabled but the
Gemini API key was not set (GEMINI_API_KEY missing from .env). The AI provider silently exited
```

without processing any segments, resulting in 0 confirmed segments and making it incorrectly appear as if \"the video has no rallies.\" This is a silent failure caused by a missing credential, not a video content issue.",
17 "fix": "Create a .env file in the project root (C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\) with the line GEMINI_API_KEY=your_actual_key. Obtain an API key from Google AI Studio. Verify config.json has a valid segmenter name (yolo_hybrid, gemini, or motion ? not wasb_hybrid). Re-run the processing command.",
18 "faq_section": "README#Q7 ? How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?",
19 "could_self_debug": true,
20 "confusing_parts": [
21 "The segmenter name 'wasb_hybrid' is not documented and appears invalid ? the report does not explain where this came from or how to correct it to a valid option (yolo_hybrid, gemini, or motion).",
22 "The term 'silent provider no-op' is technical jargon ? clearer phrasing for users would be 'The AI provider exited without running because the API key was missing.'",
23 "The error message does not explicitly state the expected .env file path, forcing the user to refer back to Q7 for the 'project root' location."
24]
25 }
26 },
27 {
28 "cause_correct": true,
29 "faq_correct": true,
30 "gap": "minor",
31 "refinement": "In the segmentation stage block, relabel the misleading success-looking counters so the synthetic nature is visible at the data level, not just in the WARN prose. Concretely, rename/annotate the segment fields, e.g. add a line `segment_count: 1 (metadata=SYNTHETIC/unmeasured)` and `metadata_source: heuristic (NOT measured)`, and reword the WARN's lead from \"is HEURISTIC / synthetic, not measured\" to a front-loaded \"are PLACEHOLDER values (unmeasured) ? these fields are fabricated by the motion heuristic, not derived from the video.\" This addresses the reader's two real friction points: (1) PASS + segment_count=1 looks like success until you read the buried WARN, and (2) \"synthetic\" reads ambiguously where \"placeholder/unmeasured\" reads as a problem immediately.",
32 "notes": "Reader nailed both the root cause (motion segmenter's intensity/speed/events/server-receiver metadata is heuristic/synthetic, not measured) and the fix (switch to yolo_hybrid or gemini). They consulted the exact expected FAQ Q10, which the report explicitly links via \"see README#Q10\" plus evidence pointer segmenter_actual=motion. could_self_debug=true is justified. Gap is minor, not none: the reader flagged genuine observability friction ? Verdict PASS (with warnings) + segment_count=1 + candidate_windows=0 surface-reads as success, and \"synthetic\" is softer than \"placeholder/unmeasured.\" The report's WARN is correct and well-pointed, but the synthetic-ness lives only in prose; the structured stage data still presents segment metadata as if trustworthy. Surfacing it at the field level would make this unambiguous without requiring careful reading of the warning. candidate_windows:0 vs segment_count:1 also went unexplained ? a secondary confusion the reader noted but did not let derail them.",
33 "scenario": "indexer_motion_synthetic",
34 "tool": "indexer",
35 "reader": {
36 "diagnosis": "The `motion` segmenter was used, which is a zero-dependency OpenCV pixel-difference heuristic. While it successfully detected 1 segment (5 seconds of play), its per-segment metadata?intensity, speed, events (serve/smash/foul/winner), and server/receiver fields?is synthetic/heuristic, NOT measured ground truth. These are placeholder values only.",
37 "fix": "Switch to a measured-metadata segmenter: either `yolo_hybrid` (default, cheaper) or `gemini` (more expensive, full-video). Update config.json to set \"default_segmenter\": \"yolo_hybrid\" and ensure GEMINI_API_KEY is in .env, then re-run indexing with --segmenter yolo_hybrid (or let it use the config default).",
38 "faq_section": "Q10: Can I trust the per-segment metadata from the `motion` segmenter?",
39 "could_self_debug": true,
40 "confusing_parts": [
41 "The report shows validation PASS and 1 segment detected, which appears successful on the surface, so the problem is not obvious without reading the warning section",
42 "The term 'synthetic' could be clearer upfront?phrases like 'placeholder' or

```
'unmeasured' would signal the problem faster than burying 'not measured' at the end of a
sentence",
                  "The report shows candidate_windows: 0 but segment_count: 1, which is
unintuitive?unclear if this is success or failure without understanding the motion segmenter's
behavior"
                ]
              }
            },
            {
              "cause_correct": true,
              "faq_correct": true,
              "gap": "minor",
              "refinement": "Make the blur_check failure line in the validation stage self-
remediating by naming the config knob and both remedies, e.g.: \"FAILED blur_check: Average
sharpness 0.0 below threshold 20.0 (validation.min_blur_threshold). Fix: improve footage
sharpness/lighting, or lower validation.min_blur_threshold in config.json, then re-run. A
reading of 0.0 may also indicate undecodable frames ? re-encode (ffmpeg -i in.mp4 -c:v libx264
out.mp4) before lowering the threshold.\" This removes the reader's ambiguity about re-encode
vs. lower-threshold without them having to invent it.",
              "notes": "Reader landed on the correct root cause (blur/focus validation check
failed, sharpness 0.0 < threshold 20.0 ? verbatim from report line 27) and consulted the
expected FAQ (README#Q13, cited on report line 49). Both cause_correct and faq_correct are true.
Gap is minor, not none, for two report-induced reasons: (1) The reader over-extended the
diagnosis ? they speculated 0.0 means frame corruption and made re-encoding the PRIMARY fix,
relegating the ground-truth remedy (lower validation.min_blur_threshold) to \"last resort.\"
Ground truth treats it as a tunable threshold violation OR footage-quality fix; the report gives
no guidance to disambiguate the 0.0 case, so the reader filled the void with plausible-but-
unverified steps. (2) Real report defect the reader flagged:
stages.segmentation._config_default_=wasb_hybrid is not a documented segmenter (README only
lists yolo_hybrid/gemini/motion), which damaged the reader's trust in the report even though it
didn't change the diagnosis ? worth fixing separately. Also Q13 is generic (\"fix the underlying
cause\") with no blur-specific remediation, so the reader couldn't confirm their fix from the
FAQ. The single highest-value fix is making the blur failure line itself carry the config knob
name + both remedies, since that one line carries the entire diagnosis.",
              "scenario": "indexer_validation_failed",
              "tool": "indexer",
              "reader": {
                "diagnosis": "The blur validation check failed because the video's average
frame Laplacian variance (sharpness metric) measured as 0.0, well below the configured threshold
of 20.0. A Laplacian value of 0.0 indicates either completely undecodable frames or corrupted
frame data in the video file, even though the container metadata (duration, resolution, fps) are
readable by FFprobe.",
                "fix": "First, verify the video file is not corrupted by playing it in a media
player. If it plays but still fails the blur check, re-encode it to a fresh MP4 file using:
ffmpeg -i /videos/match.mp4 -c:v h264 -c:a aac output.mp4. This fixes most container/frame-data
issues. Only as a last resort should you lower validation.min_blur_threshold in config.json,
which masks the underlying corruption.",
                "faq_section": "README#Q13 (cited directly in the report's findings block) ?
\"A validation check failed and indexing was halted ? what do I do?\" ? which lists common
validation failures and remediation strategies, though it does not specifically detail blur
failure solutions.",
                "could_self_debug": true,
                "confusing_parts": [
                  "The report lists _config_default_: wasb_hybrid, which is not a valid
segmenter in the README (only yolo_hybrid, gemini, motion exist) ? this creates doubt about
whether the report is trustworthy or up-to-date.",
                  "A Laplacian value of 0.0 is catastrophic and not tunable like
resolution/fps, but the report treats it as just another threshold violation without emphasizing
the severity or pointing to frame corruption as the root cause.",
                  "Q13 says 'fix the underlying cause' but does not give specific steps for
blur failures ? re-encoding the video is the real remedy, but is not mentioned in the FAQ."
                ]
              }
            },
            {
```

```
            "cause_correct": true,
            "faq_correct": true,
            "gap": "minor",
            "refinement": "Repoint the `fps_fallback_used` rule's FAQ from Q5 to a fps-specific entry (and add that entry). Q5 in the indexer README (C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\README.md:187) is titled \"Why did the video fail with 'No keyframes / packet timestamps could be extracted?'\" and its body is ENTIRELY about HEVC/GDR/CRA demuxer keyframe extraction ? it never mentions fps probing, fps fallback, or timing miscalibration. Concretely: in C:\\Users\\avidu\\Projects\\badminton-highlight-indexer\\backend\\reporting\\spec.yaml:64 change `faq_ref: \"README#Q5\"` to a new `README#Q14` titled e.g. \"Why does the report say fps was a 'fallback' / not probed?\", whose body states the internal default (harvest.py:182 documents it as 30fps/1280px ? so the report's `FPS | ? (fallback)` row should also print that numeric value, e.g. `FPS | 30 (fallback)`), explains that no fps-probing validator ran (which is why validation can show 1/1 passed yet fps still fell back), and gives the fix: confirm real fps with ffprobe and re-run so a probing validator populates fps. Without this, a reader following the cited FAQ is misdirected toward HEVC keyframe debugging (and, as this reader did, toward an unrelated Q1 ffprobe-not-found cross-reference).",
            "notes": "Reader recovered the correct ROOT CAUSE precisely (fps fallback -> miscalibrated motion thresholds + frame->second timing; ffprobe + re-run + ensure validation ran) ? but this came from the self-explanatory `fps_fallback_used` warning message (spec.yaml:60-63), NOT from the FAQ. The cited FAQ (Q5) is genuinely mistargeted: it covers HEVC keyframe/packet-timestamp extraction, not fps fallback. faq_correct=true only in the literal sense that they consulted the expected Q5; the expectation itself is the defect. The reader independently flagged this exact mismatch plus two other real issues: (1) the fallback numeric value is hidden as `FPS | ? (fallback)` though harvest.py knows it's 30fps; (2) validation shows 1/1 passed while fps still fell back, with no visible explanation that no fps-probing validator ran in this run. Gap rated minor because correct self-debug was achieved, but the FAQ pointer would misdirect a reader who trusts it.",
            "scenario": "indexer_fps_fallback",
            "tool": "indexer",
            "reader": {
              "diagnosis": "The FPS (frames-per-second) value was not probed from the video file and instead fell back to a default hardcoded value. This means all motion thresholds, frame-to-second timing conversions, and velocity calculations in the segmentation pipeline are using incorrect baseline assumptions. The report shows FPS | ? (fallback) in the Video section and the warning fps_fallback_used explicitly states fps was assumed (a fallback default), not probed from the file. Motion thresholds and frame->second timing may be miscalibrated.",
              "fix": "1. Run ffprobe directly on the video to extract the actual FPS: ffprobe -v error -select_streams v:0 -show_entries stream=r_frame_rate -of default=noprint_wrappers=1:nokey=1:noesc=1 /videos/match.mp4. 2. If ffprobe returns empty or an error, the file likely uses HEVC/H.265 encoding (common in GoPro footage) where standard FFmpeg frame-skipping fails ? see README Q5. 3. Once the actual FPS is confirmed, re-run the indexing pipeline so that FPS is properly probed from the file during validation. 4. Alternatively, if FFprobe cannot extract FPS from this file format, manually set the correct FPS in config.json under validation.min_fps based on the confirmed value, then re-run.",
              "faq_section": "README#Q5 (Why did the video fail with \"No keyframes / packet timestamps could be extracted\"?)",
              "could_self_debug": true,
              "confusing_parts": [
                "Missing fallback value ? The report shows FPS | ? (fallback) but never states which numeric fallback was actually used (30? 60?). Showing the actual fallback value (e.g., FPS | 30 (fallback)) would immediately help validate if the assumption is wildly wrong.",
                "Validation vs. FPS probing linkage ? The report shows validation passed (checks_passed: 1/1), but doesn't clarify whether FPS probing is part of validation or a separate step. If it's part of validation, why did validation pass but FPS still fall back?",
                "No error context ? The report doesn't include the underlying error from FFprobe/FFmpeg (e.g., FFprobe not in PATH, stream has no r_frame_rate field, HEVC keyframe parsing failed). Including that would pinpoint the root cause immediately.",
                "Q5 reference mismatch ? Q5 is titled about keyframes, but the actual fix for FPS fallback might be a different problem entirely. The warning message could benefit from citing Q1 (ffprobe not found) as an alternative possibility."
              ]
            }
          },
```

```
88           {
89             "cause_correct": true,
90             "faq_correct": true,
91             "gap": "minor",
92             "refinement": "Add frame-level detail to the overlap finding so a CVAT/frame-
based reviewer can act without manual conversion. Rewrite line 37 as: \"overlap ? Rally 1 (ends
7.00s / frame 420) overlaps Rally 2 (starts 6.00s / frame 360) by 1.00s (~60 frames @ 59.94fps)
? rallies must be disjoint; fix one boundary so Rally 1.end_frame <= Rally 2.start_frame.\" This
kills the reader's biggest friction (hand-computing frames = 1.00 x 59.94) and the disjointness
invariant becomes a copy-pasteable rule.",
93             "notes": "Reader nailed the exact root cause (1.00s overlap, rallies must be
disjoint, input-data error) and consulted the expected FAQ \"what-does-validation-check\" ? both
correct. could_self_debug=false, but the report+FAQ gave them the full cause AND the complete
fix space (move Rally1.end earlier OR Rally2.start later, then re-run validate-delivery), which
they articulated precisely. Two of three \"confusing parts\" are inherent to the data, not
report defects: which boundary is actually wrong genuinely requires opening the video ? the
report cannot and should not guess. The \"rally_number=2\" evidence ambiguity is real but the
FAQ correctly frames either boundary as a candidate. The one genuinely fixable friction is the
lack of frame-level numbers (report has FPS=59.94 and knows source_format=cvat, which is frame-
based), forcing the reader to convert seconds->frames by hand. Hence gap=minor, not none. Report
file: C:\\Users\\avidu\\AppData\\Local\\Temp\\obseval\\reports\\collector_overlap.report.md
(overlap finding on line 37).",
94             "scenario": "collector_overlap",
95             "tool": "collector",
96             "reader": {
97               "diagnosis": "Rally 1 ends at 7.00 seconds, and Rally 2 starts at 6.00
seconds. This means their time intervals overlap by 1.00 second (Rally 1 should have ended
before Rally 2 began). The `overlap` validation check is a blocker because two rallies'
intervals must be disjoint ? the end time of one rally must not be greater than the start time
of the next rally. This is a data error in the input annotations: the rally boundaries have not
been correctly separated in time.",
98               "fix": "Correct the CSV so that the two rallies' time intervals no longer
overlap. Either: (1) move Rally 1's end_time to 6.00 seconds or earlier, or (2) move Rally 2's
start_time to 7.00 seconds or later. Then re-run `validate-delivery` to confirm the blocker is
gone before proceeding to `import-local`.",
99               "faq_section": "What does validation check? (the table with `overlap` listed
as a blocker severity check)",
100              "could_self_debug": false,
101              "confusing_parts": [
102                "The report says 'Rally 1 (ends 7.00s) overlaps Rally 2 (starts 6.00s)' but
does not explicitly state whether you should shorten Rally 1 or extend Rally 2's start, or offer
any guidance on which is more likely correct. You must open the video yourself to see which
rally boundary is actually wrong.",
103                "The report gives the overlap duration (1.00s) but no frame-level detail, so
if you're working from frame numbers in CVAT, you'd need to convert back (frames = seconds × fps
= 1.00 × 59.94 ? 60 frames) to know how many frames to adjust.",
104                "The report references 'rally_number=2' in the evidence but does not clarify
whether that means Rally 2 is the problem or both are. From the FAQ wording, either boundary
could be wrong ? you must inspect the video to judge."
105              ]
106            }
107          },
108          {
109            "cause_correct": true,
110            "faq_correct": false,
111            "gap": "minor",
112            "refinement": "Change the report finding line to encode severity + non-guarantee
+ the right anchor, e.g.: \"[WARN · medium · review-only] possible_missed_rally ? 26.0s
unlabeled gap between Rally 2 (ends 14.0s) and Rally 3 (starts 40.0s). HEURISTIC, not a
certainty: watch 14.0s-40.0s and ADD a rally only if one is actually there (the gap may be a
legitimate break). Not a blocker ? gate still PASSes.\" This stops a reader from asserting \"the
export is missing an entire rally\" as fact and from treating it as \"the worst error.\"",
113            "notes": "Report at
C:\\Users\\avidu\\AppData\\Local\\Temp\\obseval\\reports\\collector_missed_rally.report.md; FAQ
at C:\\Users\\avidu\\Projects\\sports-data-
```

collector\\docs\\INGESTION_PIPELINE.md.\n\ncause_correct=true: reader nailed the root cause ? possible_missed_rally warning, 26s gap between Rally 2 (ends 14.0s) and Rally 3 (starts 40.0s), interpreted as a likely-omitted rally; prescribed action (watch the span, add the rally if present, re-validate, document in review sheet, import-local) matches ground truth exactly.\n\nfaq_correct=false (strict): the report's finding cites the anchor #what-does-validation-check (the validation-check table, which has the possible_missed_rally row). The reader instead consulted/cited the sibling section \"How are *missed rallies* caught? (the worst error)\". That section is adjacent and explains the same case well, but it is NOT the expected anchor, so strictly they did not consult the cited FAQ. Notably the report points to one section while the more explanatory prose lives in another ? a small split that the refinement also addresses.\n\ngap=minor: two wording-level slips, both traceable to the report omitting severity/uncertainty. (1) Over-assertion: reader stated \"The vendor's CVAT export is missing an entire rally\" as fact, but the FAQ is explicit that these checks \"surface regions to review, they don't certify completeness\" and \"Neither is a guarantee.\" (2) Over-escalation: reader leaned on the \"worst error\" framing and did not note this is a non-blocking medium review flag (verdict = PASS with warnings). Diagnosis and remediation are still correct, so the reader could self-debug ? hence minor, not major.",
114   "scenario": "collector_missed_rally",
115   "tool": "collector",
116   "reader": {
117   "diagnosis": "The report shows a `possible_missed_rally` warning indicating a 26-second unlabeled gap between Rally 2 (ends at 14.0s) and Rally 3 (starts at 40.0s). According to the FAQ, this is a \"clean omission\" ? a real hole in the timeline where a rally was likely forgotten during annotation. The vendor's CVAT export is missing an entire rally in that time period.",
118   "fix": "Open the video at the 14.0s to 40.0s interval and verify whether a rally occurs there. If one exists, add it to the canonical CSV with the correct start_time, end_time, shots_count, ending_reason, and last_shot_outcome. Re-run `validate-delivery` to confirm the gap is resolved, then use the review sheet to document the correction before importing with `import-local`.",
119   "faq_section": "How are *missed rallies* caught? (the worst error)",
120   "could_self_debug": true,
121   "confusing_parts": []
122   }
123   },
124   {
125   "cause_correct": true,
126   "faq_correct": true,
127   "gap": "none",
128   "refinement": "none",
129   "notes": "Report: C:\\Users\\avidu\\AppData\\Local\\Temp\\obseval\\reports\\collector_noncommercial.report.md. The WARN finding `license_noncommercial` (line 38) states the full causal chain in one sentence ? non-commercial license -> EXCLUDED from default commercial-only export -> stays in DB -> pass --include-noncommercial ? plus the exact FAQ anchor (#will-a-non-commercial-license-block-export) and machine-checkable evidence (is_commercial=False). The registration stage details (lines 32-34: license=CC-BY-NC-SA, is_commercial=False) corroborate it. The reader nailed root cause (correct), consulted the exact expected FAQ (correct), and proposed the right remedy. Gap is none. Only nuance: the reader supplied a full example command (python -m src.cli.curate export ... --include-noncommercial) that the report does not contain ? it names only the flag. The inference is reasonable and correct, so not a defect; the report is already self-debuggable. No refinement warranted.",
130   "scenario": "collector_noncommercial",
131   "tool": "collector",
132   "reader": {
133   "diagnosis": "The video cc_match is licensed under CC-BY-NC-SA (non-commercial license). The report correctly identifies this with a license_noncommercial warning. The video will be excluded from the default export because is_commercial=False, as stated in the report and documented in the FAQ.",
134   "fix": "Run the export command with the --include-noncommercial flag to include this video in the export: python -m src.cli.curate export --out-dir data/eval_export --include-noncommercial",
135   "faq_section": "Will a non-commercial license block export?",
136   "could_self_debug": true,
137   "confusing_parts": []

```
            }
          }
        ],
        "synthesis": {
          "overall": "All seven scenarios are self debuggable. A reader with only the README
and FAQ can debug a bad run today. Two confirmed doc defects dominate: the fps fallback rule
points at the wrong FAQ, and the README segmenter list omits the registered wasb_hybrid
segmenter.",
          "can_readers_debug": true,
          "refinements": [
            {
              "change": "Add wasb_hybrid and tracknet_hybrid to the README Q7 segmenter list
so docs match the registry, and note that wasb_hybrid and yolo_hybrid both route candidate
windows to the gemini provider. wasb_hybrid is valid and shown in the report segmentation stage,
but Q7 lists only yolo_hybrid, gemini, and motion, so readers think it is invalid and try to
change it.",
              "where": "FAQ doc: README.md Q7 segmenter list near lines 197 to 217 and
config line 155",
              "priority": "high",
              "scenarios": [
                "indexer_no_api_key",
                "indexer_validation_failed",
                "indexer_fps_fallback"
              ]
            },
            {
              "change": "Repoint the fps_fallback_used rule away from README Q5, which is
about HEVC keyframe extraction and never mentions fps. Point it to a new fps FAQ that gives the
assumed default value, explains no fps-probing validator ran, and the fix: confirm fps with
ffprobe and re-run.",
              "where": "spec rule: spec.yaml line 64 faq_ref plus a new README FAQ",
              "priority": "high",
              "scenarios": [
                "indexer_fps_fallback"
              ]
            }
          ]
        }
      }
    }
```


492. assistant (2026-06-07T08:29:15.624Z)

The eval is in and the result is strong:

Debuggability eval ? verdict: **all 7 scenarios self-debuggable** ?

`can_readers_debug: true`. Every fresh reader (given *only* README/FAQ + the report) landed on the **correct root cause**; 6/7 also cited the exact expected FAQ. Gaps were all "minor" (one "none"). So the reports already pass your bar ? but the agents found concrete polish, two of them genuinely important:

High-priority (real doc defects):
1. **`fps_fallback_used` cites the wrong FAQ** ? it points at README Q5, which is about HEVC keyframes and never mentions fps. Needs its own fps FAQ. *(A faq existence-test wouldn't catch this ? Q5 exists; it's semantically wrong. Good catch by the eval.)*
2. **README Q7 omits `wasb_hybrid`/`tracknet_hybrid`** ? readers see `wasb_hybrid` in the report and waste effort thinking it's an invalid segmenter (false lead in 3 scenarios).

Minor polish: name the active segmenter + explicit `.env` path in `silent_provider_noop`; show the numeric fps fallback (`30 (fallback)` not `? (fallback)`); flag motion metadata as synthetic at the data level, not just prose; make the blur failure line name `validation.min_blur_threshold` + remedies; add frame-level numbers to the collector overlap finding; mark `possible_missed_rally` as review-only/non-guarantee.

Now let me honor your "screenshot and eyeball" ? generating a representative technical +
customer pair and rendering them to an image:

493. assistant (2026-06-07T08:29:23.166Z)

```
import sys, os
sys.path.insert(0, os.getcwd())
import obsreport
from backend.reporting import harvest
from backend.pipeline.validators.base import ValidationResult

OUT = r"C:\Users\avidu\AppData\Local\Temp\obseval\html_src"
os.makedirs(OUT, exist_ok=True)
FF = {"streams": [{"width": 1920, "height": 1080, "r_frame_rate": "60000/1001", "codec_name":
"hevc"}],
      "format": {"format_name": "mov,mp4,m4a", "duration": "676.0", "size": "5500000000"}}

def vr(n, p, m="ok"):
    return ValidationResult(validator_name=n, passed=p, message=m)

os.environ.pop("GEMINI_API_KEY", None)
rec = harvest.record_run(
    video_id="9f3a1c?", video_path="/videos/Adarsh and Vasanth.mp4",
    config={"ai_provider": "gemini", "ai_model": "gemini-flash-latest", "indexing":
{"default_segmenter": "wasb_hybrid"}},
    val_results=[vr("format_check", True), vr("resolution_fps_check", True), vr("blur_check",
True),
                 vr("relevance_check", True)],
    val_context={"ffprobe_meta": FF}, play_segments=[], candidates=[{"id": i} for i in
range(64)],
    segmenter_requested="wasb_hybrid", segmenter_actual="wasb_hybrid", was_reingest=False,
segmentation_ran=True)
open(os.path.join(OUT, "indexer.report.md"), "w",
encoding="utf-8").write(obsreport.render_technical(rec.envelope, rec.spec))
open(os.path.join(OUT, "indexer.summary.md"), "w",
encoding="utf-8").write(obsreport.render_customer(rec.envelope, rec.spec))
print("wrote indexer eyeball md")
```

494. user (2026-06-07T08:29:23.180Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\obseval\gen_eyeball_indexer.py
(file state is current in your context ? no need to Read it back)
```

495. assistant (2026-06-07T08:29:35.004Z)

```
import sys, os
sys.path.insert(0, os.getcwd())
import obsreport
from src.reporting import adapter
from src.parsers.cvat.rally_cvat import Issue, RallyRecord

OUT = r"C:\Users\avidu\AppData\Local\Temp\obseval\html_src"
os.makedirs(OUT, exist_ok=True)

def r(n, s, e, shots=5, reason="winner", out="in"):
    return RallyRecord(rally_number=n, start_time=s, end_time=e, shots_count=shots,
ending_reason=reason,
                       last_shot_outcome=out, score_before=None, score_after=None, notes=None,
                       frame_min=int(s*60), frame_max=int(e*60), n_boxes=3)

records = [r(1, 2.0, 7.0, 9), r(2, 6.0, 11.0, 7), r(3, 40.0, 44.0, 2)]
issues = [Issue("blocker", "overlap", "Rally 1 (ends 7.00s) overlaps Rally 2 (starts 6.00s) by
1.00s ? rallies must be disjoint.", 2),
          Issue("medium", "possible_missed_rally", "26.0s of unlabeled time between Rally 2
```

```
(ends 11.0s) and Rally 3 (starts 40.0s) ? check for an unlabeled rally in the gap.", 3),
          Issue("medium", "shot_density", "Rally 3: 2 shots over 4.0s (0.50/s) is implausibly
low ? the span may be inflated by a stray box.", 3)]
rec = adapter.build_delivery_recorder(records=records, issues=issues, fps=59.94,
source="pilot_badminton_03",
                                      video_duration_s=600.0)
open(os.path.join(OUT, "collector.report.md"), "w",
encoding="utf-8").write(obsreport.render_technical(rec.envelope, rec.spec))
open(os.path.join(OUT, "collector.summary.md"), "w",
encoding="utf-8").write(obsreport.render_customer(rec.envelope, rec.spec))
print("wrote collector eyeball md")
```

496. user (2026-06-07T08:29:35.033Z)

```
File created successfully at: C:\Users\avidu\AppData\Local\Temp\obseval\gen_eyeball_collector.py
(file state is current in your context ? no need to Read it back)
```

497. user (2026-06-07T08:29:43.366Z)

```
wrote indexer eyeball md
wrote collector eyeball md
wrote C:/Users/avidu/AppData/Local/Temp/obseval/html/index.html
```

498. assistant (2026-06-07T08:29:54.381Z)

Now serve the HTML and screenshot it. Creating a throwaway static-server config (untracked; I'll remove it after):

499. assistant (2026-06-07T08:29:55.963Z)

```
{
  "version": "0.0.1",
  "configurations": [
    {
      "name": "obseval-report-preview",
      "runtimeExecutable": "python",
      "runtimeArgs": ["-m", "http.server", "8731", "--directory",
"C:/Users/avidu/AppData/Local/Temp/obseval/html"],
      "port": 8731
    }
  ]
}
```

500. user (2026-06-07T08:29:55.987Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\.claude\launch.json (file state is current in your context ? no need to Read it back)
```

501. user (2026-06-07T08:30:02.827Z)

```
{
  "serverId": "9dd33862-4c8c-4d15-b578-61bbaf6f5328",
  "port": 8731,
  "name": "obseval-report-preview",
  "reused": false
}
Server started successfully on port 8731.
```

502. assistant (2026-06-07T08:30:18.088Z)

The render looks clean (tables, fps shown as "59.94 fps (probed)"). Let me widen the viewport to capture the full reports ? the cards are laid out side by side ? and grab the key sections (Findings + customer summaries):

503. user (2026-06-07T08:30:24.548Z)

```
Viewport set to 1260x1180.
```

504. user (2026-06-07T08:30:34.924Z)

```
""
```

505. user (2026-06-07T08:31:09.022Z)

```
preview_screenshot timed out after 30s. The preview window may be stuck (modal dialog,
navigation hang, or unresponsive renderer). Check preview_console_logs for errors.
```

506. user (2026-06-07T08:31:44.405Z)

```
preview_screenshot timed out after 30s. The preview window may be stuck (modal dialog,
navigation hang, or unresponsive renderer). Check preview_console_logs for errors.
```

507. assistant (2026-06-07T08:31:58.183Z)

The screenshot tool hung after resize (I already have one clean capture confirming the visual layout renders well). Rather than fight it, let me eyeball the two **customer summaries** (the "customer-friendly video report" you care most about) directly from the rendered source, then clean up the preview:

508. user (2026-06-07T08:31:58.704Z)

```
Server 9dd33862-4c8c-4d15-b578-61bbaf6f5328 stopped
```

509. user (2026-06-07T08:31:59.172Z)

```
1       # Badminton Highlight Indexer ? Video Report (summary)
2
3       ## Run
4
5       | Field | Value |
6       | --- | --- |
7       | Video | Adarsh and Vasanth.mp4 |
8       | Report generated | 2026-06-07T08:29:42Z |
9
10      ## Video
11
12      | Field | Value |
13      | --- | --- |
14      | Duration | 11m16s |
15      | Resolution | 1920x1080 |
16      | Frame rate | 59.94 fps |
17
18      ## Highlights
19
20      - **0** highlights
21      - **0.0s** of play
22      - **0%** of the video is highlights
23      - No highlights were produced for this video.
24
25      ## Things to know
26
27      - ? We couldn't generate highlights for this video due to a configuration issue on our
side. Please contact support.
28
29      ## Verdict
30
31      ?? We couldn't finish generating highlights for this video. Please contact support.
32
```

510. user (2026-06-07T08:31:59.753Z)

```
1       # Sports Data Collector ? Delivery / Ingestion Report (summary)
2
3       ## Run
4
5       | Field | Value |
6       | --- | --- |
7       | Video | pilot_badminton_03 |
8       | Report generated | 2026-06-07T08:29:43Z |
9
10      ## Video
11
12      | Field | Value |
13      | --- | --- |
14      | Duration | 10m00s |
15      | Frame rate | 59.94 fps |
16
17      ## Rally summary
18
19      - **3** rallies
20      - **14.0s** of play
21      - **2.3%** of the video is rallies
22
23      ## Things to review
24
25      - ? Rally 1 (ends 7.00s) overlaps Rally 2 (starts 6.00s) by 1.00s ? rallies must be
disjoint.
26      - ?? 26.0s of unlabeled time between Rally 2 (ends 11.0s) and Rally 3 (starts 40.0s) ?
check for an unlabeled rally in the gap.
27      - ?? Rally 3: 2 shots over 4.0s (0.50/s) is implausibly low ? the span may be inflated
by a stray box.
28
29      ## Verdict
30
31      This delivery has blocking issues that must be fixed before it can be imported ? see
'Things to review'.
32
```

511. user (2026-06-07T08:32:40.272Z)

```
launch.json cleaned; working tree:
0
```

512. assistant (2026-06-07T08:33:02.531Z)

```
Cleaned up the throwaway preview config ? working tree is clean.
```

Eyeball verdict (screenshot + rendered customer summaries)

```
The rendered reports look clean and professional: tidy tables, `Video` shows **`59.94 fps
(probed)`** (the probe-vs-fallback distinction is visible), and the **customer summaries strip
all internals** ? they show only video metadata, highlight/rally counts, plain-language "things
to know/review," and a gentle audience-appropriate verdict:
- **Indexer customer:** *"We couldn't finish generating highlights for this video. Please
contact support."* (no codes, no jargon) ?
- **Collector delivery summary:** *3 rallies · 14.0s of play · "Rally 1 overlaps Rally 2 by
1.00s?"* + *"This delivery has blocking issues that must be fixed?"* (vendor-actionable, no
internal codes) ?

No layout/visual problems ? the only improvements are the content refinements the eval surfaced.
```

Bottom line on your question

```
**#41 has no issues ? merge away.** It's `MERGEABLE`/`CLEAN`, 228 tests green, and touches files
disjoint from #42, so order doesn't matter. Recommend merging **#42** (config fix) too so real
```

video runs work.

After you merge ? my plan (I'll rebase and keep going)

1. **Rebase** #42 (and anything else) onto the new master.
2. **Apply the eval's refinements** as a focused follow-up:
   - *High:* repoint `fps_fallback_used` to a new fps FAQ (not the keyframe Q5); add `wasb_hybrid`/`tracknet_hybrid` to README Q7's segmenter list.
   - *Minor:* numeric fps fallback (`30 (fallback)`), motion-metadata "synthetic" surfaced at the data level, self-remediating blur line, frame-level detail on the collector overlap finding, review-only framing on `possible_missed_rally`.
   - Push indexer refinements as a new PR; push collector refinements to **#11** (still open).
3. **Add the golden regression tests** (faq-ref-resolution: every flag's cited anchor exists in its doc; + expected flag-set per fixture) to both repos.

Go ahead and merge #41 (and #42) whenever you're ready ? ping me and I'll rebase and roll straight into the refinements + golden tests.

513. user (2026-06-07T08:33:04.469Z)

Can you review the comments on #42 and respond?

514. assistant (2026-06-07T08:33:11.513Z)

Let me pull the comments on PR #42:

515. user (2026-06-07T08:33:13.231Z)

```
=== ISSUE/REVIEW COMMENTS ===
{"author":"avidullu","body":""}
```

516. assistant (2026-06-07T08:33:19.998Z)

The top-level view shows little ? the comments are likely **inline review comments** on specific lines. Let me fetch those via the API:

517. user (2026-06-07T08:33:22.446Z)

```
=== INLINE REVIEW COMMENTS on #42 ===

=== REVIEW SUMMARIES ===
?? avidullu [PENDING]
```

518. assistant (2026-06-07T08:33:37.073Z)

A `PENDING` review by you means draft comments not yet submitted ? those aren't visible via the API until you submit the review. Let me check the other comment channels (PR conversation, commit comments) to be sure I'm not missing anything:

519. user (2026-06-07T08:33:42.264Z)

```
=== PR conversation (issue) comments ===
[end issue comments]

=== all reviews (id/state/body) ===
id 4444704980 state PENDING author avidullu body= ''

=== review comments incl pending (via review id) ===
inline comment count: 0
```

520. assistant (2026-06-07T08:33:51.586Z)

There are no submitted comments, but there's a **pending review** (your draft, id `4444704980`). Since I'm authenticated as your account, let me try to read that pending review's draft comments

directly:

521. user (2026-06-07T08:33:54.316Z)

```
=== pending review draft comments (as author) ===
count: 5
?? backend/config/models.py : None
Good catch on the real-run breakage vs unit tests. The previous .get() only handled top-level
and didn't dump nested Pydantic models to dicts, which is exactly what broke the legacy chained
access in validators/segmenters.

The updated logic + docstring clearly explains the intent for legacy compat. Nice.

?? backend/config/models.py : None
The fallback loop for leaf keys inside sections is still limited to the four known sections. If
we ever add another top-level config section (e.g. `audio` or `jobs` in the future), keys inside
it won't be discoverable via the legacy .get() fallback.

Worth a small comment noting this, or we can make the section list dynamic from model_fields
later.

?? backend/config/models.py : None
Minor: the `get()` method is accumulating more and more legacy compat logic. Consider adding a
short class or method-level comment like:
```

Temporary migration shim for legacy dict-style access.
Will be removed once all call sites (validators, segmenters, etc.) are migrated to attribu

```
This sets expectations for future cleanup.

?? tests/test_config.py : None
The new regression test is excellent and directly targets the exact failure mode (chained .get
on nested sections including relevance).

One small suggestion: you could also add a quick assertion that a top-level scalar with a
default still works (you have one for the missing key at the end, but making the scalar case
explicit is nice for coverage).

?? backend/config/models.py : None
Overall this is a solid, minimal fix that restores the promised legacy compat without changing
the happy path for attribute access.

The PR description is clear and links the context well. Good work surfacing this during the end-
to-end validation.

No blocking issues from my side. A couple of nits above for polish.
```

522. assistant (2026-06-07T08:34:32.524Z)

I can see your 5 pending-review comments. Three are actionable nits (2 praise). Since the cleanest way to "respond" to review feedback is to address it, let me apply the nits to #42's branch. Switching to it and checking master state:

523. user (2026-06-07T08:34:36.202Z)

```
Switched to branch 'fix/appconfig-nested-get'
=== is #41/#42 merged? origin/master head ===
0355417 Merge pull request #40 from avidullu/refactor/approved-directory-structure
1e96e07 test: fix mock patch targets after structure refactor and config test assertions
8615534 refactor: adopt approved layered directory structure
=== this branch vs origin/master ===
0       1
```

524. assistant (2026-06-07T08:34:59.488Z)

Master hasn't moved (neither #41 nor #42 merged yet) ? no rebase needed. Let me apply the three actionable nits. First, make the section scan dynamic (nit #2) and sharpen the shim comment (nit #3):

525. user (2026-06-07T08:35:05.538Z)

<tool_use_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write it.</tool_use_error>

526. assistant (2026-06-07T08:35:11.640Z)

Let me re-read the current state of the file on this branch before editing:

527. user (2026-06-07T08:35:12.624Z)

```
110
111         # Temporary dict-like access for legacy code that does config.get("foo", {})
112         # and passes the config object around. Will be removed after consumers are migrated.
113         #
114         # A nested *section* (e.g. config.get("validation")) is returned as a plain dict
115         # via model_dump, so legacy callers that chain dict access ?
116         # config.get("validation", {}).get("min_resolution_width", ...) ? keep working.
117         # (Returning the Pydantic section model instead breaks them, since a model has
118         # no .get; that bit the validators on every real run.) Scalar leaves are returned
119         # as-is.
120         def get(self, key: str, default: Any = None) -> Any:
121             _MISSING = object()
122             val: Any = _MISSING
123             if key in type(self).model_fields or hasattr(self, key):
124                 val = getattr(self, key, _MISSING)
125             if val is _MISSING:
126                 # Fall back to a leaf key living inside a known section (legacy
convenience).
127                 for section_name in ("validation", "indexing", "output", "stitching"):
128                     section = getattr(self, section_name, None)
129                     if section is not None and hasattr(section, key):
130                         val = getattr(section, key)
131                         break
132             if val is _MISSING:
133                 return default
134             if isinstance(val, BaseModel):
135                 return val.model_dump(mode="json")
136             return val
137
138         def __getitem__(self, key: str) -> Any:
139             val = self.get(key)
140             if val is None and key not in self.model_fields:
141                 # Re-raise like a real dict for unknown top-level
142                 raise KeyError(key)
143             return val
144
145
146     # Convenience type alias for callers that want to annotate
147     Config = AppConfig
```

528. user (2026-06-07T08:35:31.494Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\config\models.py has been updated successfully. (file state is current in your context ? no need to Read it back)

529. assistant (2026-06-07T08:35:36.954Z)

Now nit #4 ? add the explicit scalar-with-default assertion to the regression test, plus a dynamic-section-discovery assertion to lock the nit #2 improvement:

530. user (2026-06-07T08:35:37.558Z)

```
88
89      def test_get_returns_dict_for_nested_sections_legacy_compat():
90          """Regression: legacy consumers (validators/segmenters) do
91          ``config.get("validation", {}).get("min_resolution_width", ...)``. The dict-compat
92          .get must return nested SECTIONS as plain dicts (not Pydantic models), or that
93          chained access raises 'X object has no attribute get' on every real run."""
94          cfg = AppConfig()
95          validation = cfg.get("validation", {})
96          assert isinstance(validation, dict)
97          assert validation.get("min_resolution_width") == 1280
98          # nested-of-nested is also dict-accessible
99          assert isinstance(validation.get("relevance"), dict)
100         assert validation["relevance"].get("court_pixels_min_ratio") == 0.05
101         assert cfg.get("indexing", {}).get("default_segmenter") == "yolo_hybrid"
102         # scalar leaves are returned unchanged; missing keys fall back to the default
103         assert cfg.get("ai_provider") == "gemini"
104         assert cfg.get("does_not_exist", "fallback") == "fallback"
105
```

531. user (2026-06-07T08:35:45.748Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\tests\test_config.py has been updated successfully. (file state is current in your context ? no need to Read it back)

532. user (2026-06-07T08:36:05.322Z)

```
......                                                                   [100%]
6 passed in 0.19s
=== full suite (no regressions) ===
.......................................................................... [ 66%]
...........................................................................  [100%]
215 passed in 11.94s
```

533. assistant (2026-06-07T08:36:29.482Z)

All green (215; no warnings). Let me commit and push to #42:

534. user (2026-06-07T08:36:36.601Z)

pushed; #42 updated

535. assistant (2026-06-07T08:36:59.271Z)

Done ? addressed all the actionable feedback and pushed to #42. Here's my point-by-point response to your 5 review comments:

1. "Good catch on real-run breakage vs unit tests? nice." ? Thanks. The key was that the old `.get()` only handled top-level and returned the nested *model*; dumping sections to dicts is what restores the chained legacy access. ? (no change needed)

2. "Fallback loop is limited to the four known sections? new section like `audio`/`jobs` won't be discoverable; make it dynamic from `model_fields`." ? **Done, the better option.** The section scan now iterates `type(self).model_fields` and recurses into any field whose value is a `BaseModel`, so a future top-level section is covered automatically with no hardcoded list to maintain. Added test assertions proving leaf keys resolve from `validation` *and* `stitching` via the dynamic scan.

3. "`get()` is accumulating legacy compat logic; add a 'temporary migration shim' comment." ? **Done.** Reworded the comment on `get()` to: *"Temporary migration shim for legacy dict-style

access? Will be removed once all call sites (validators, segmenters, etc.) move to typed attribute access."* Sets the cleanup expectation explicitly.

4. "Regression test is excellent; also assert a top-level scalar with a default still works." ? **Done.** Added `assert cfg.get("ai_provider", "ignored-default") == "gemini"` (present scalar ignores the default arg), alongside the existing missing-key?default case.

5. "Solid minimal fix, no blocking issues, nits for polish." ? Appreciated. All nits applied; also switched `__getitem__` to class-level `model_fields` to drop the last Pydantic instance-access deprecation warning. **215 tests pass, no warnings.**

Pushed as commit "review: address #42 nits ?" ? your pending line comments will show as outdated since those lines changed.

Two notes:
- Your review is still **`PENDING`** (draft) ? the comments aren't publicly submitted yet, so I responded here rather than posting threaded replies (which aren't possible until you submit). If you'd like, I can post a single summary comment on #42 recording these resolutions ? say the word and I'll add it (it'd appear under your account, since that's the authenticated identity).
- #42 is still based on the pre-#41 master (they're disjoint, so it merges cleanly in any order). Whenever you merge #41/#42, ping me and I'll rebase and roll into the eval refinements + golden tests.

536. user (2026-06-07T08:41:40.546Z)

I submitted the comments sa well. So now we are good to merge this?

537. assistant (2026-06-07T08:41:52.817Z)

Let me verify #42's current state ? the submitted review, mergeability, and that my fixes are on the branch:

538. user (2026-06-07T08:41:58.053Z)

```
=== reviews (state) ===
avidullu: COMMENTED

=== mergeable / state ===
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE","reviewDecision":""}

=== any NEW unaddressed inline comments? ===
backend/config/models.py:115 [avidullu] Good catch on the real-run breakage vs unit tests. The previous .get() only hand
backend/config/models.py:122 [avidullu] The fallback loop for leaf keys inside sections is still limited to the four kno
backend/config/models.py:112 [avidullu] Minor: the `get()` method is accumulating more and more legacy compat logic. Con
tests/test_config.py:100 [avidullu] The new regression test is excellent and directly targets the exact failure mode
backend/config/models.py:1 [avidullu] Overall this is a solid, minimal fix that restores the promised legacy compat wi
[end]

=== PR branch HEAD on origin == my fix? ===
1358521ce3587b684a1bdb8b8ff3680e9ff7b463
1358521ce3587b684a1bdb8b8ff3680e9ff7b463
```

539. assistant (2026-06-07T08:42:32.311Z)

Yes ? **#42 is good to merge.** Verified:

- **All 5 review comments addressed** and pushed (origin HEAD = my fix commit `1358521`). The submitted review is **`COMMENTED`** (non-blocking ? your own note said "No blocking issues from my side"). The inline comments now show against changed lines, so they'll render as *outdated*.

No new/unaddressed feedback.
- **`MERGEABLE` / `CLEAN`** against current master.
- **215 tests pass, no warnings.**
- The two actionable nits are genuinely improved: dynamic section discovery (future-proof) + the temporary-shim comment; plus the extra test assertions.

Merge whenever you like. Quick notes:
- **Order doesn't matter for conflicts** (#42 and #41 touch disjoint files), but #42 is the one that **unbreaks real video runs** (the validators), so merging it is pure upside.
- After you merge, I'll **rebase #41 onto the new master**, then roll into the eval refinements + golden tests as we planned.

Want me to merge it for you now (`gh pr merge 42`)? I can do #42 and, if you want, #41 too ? just say go, and tell me your preference (squash / merge commit / rebase). Otherwise click merge and ping me, and I'll pick up the rebase.

540. user (2026-06-07T08:42:55.172Z)

I merged 42. Can I merge 41 now?

541. assistant (2026-06-07T08:43:21.440Z)

Let me verify before you do ? I'll confirm #42 landed, rebase #41 onto the new master, and run the full suite so we know the *combined* state is green (not just that GitHub says mergeable):

542. user (2026-06-07T08:43:25.868Z)

```
Switched to branch 'feat/observability-reports'
=== master now has #42? ===
ab7f895 Merge pull request #42 from avidullu/fix/appconfig-nested-get
1358521 review: address #42 nits ? dynamic section scan, shim comment, scalar-default test
d02554e fix(config): AppConfig.get returns nested sections as dicts (legacy dict-compat)
=== #41 vs new master (behind/ahead) ===
3       4
=== rebase #41 onto new master ===
Rebasing (1/4)
Rebasing (2/4)
Rebasing (3/4)
Rebasing (4/4)

Successfully rebased and updated refs/heads/feat/observability-reports.
```

543. assistant (2026-06-07T08:43:32.088Z)

#42 is on master, and #41 rebased onto it cleanly (disjoint, no conflicts). Now the decisive check ? full suite on the combined state, then I'll make the PR current:

544. user (2026-06-07T08:43:50.015Z)

```
.......................................................................... [ 62%]
.......................................................................... [ 94%]
.............                                                            [100%]
229 passed in 12.35s

=== #41 mergeable now? ===
{"mergeStateStatus":"CLEAN","mergeable":"MERGEABLE"}
```

545. assistant (2026-06-07T08:44:03.142Z)

Yes ? merge #41 now. Verified on the combined state, not just GitHub's word:

- Rebased #41 onto the new master (which now includes **#42** `ab7f895`) ? **clean, no conflicts** (disjoint files).
- **229 tests pass** on master + #42 + #41 together (incl. the typed-`AppConfig`?report test now

running against the *fixed* config). No warnings.
- GitHub: **`MERGEABLE` / `CLEAN`**, and the PR branch is now current (force-pushed the rebase).

So the full pipeline is sound end-to-end: validators run (thanks to #42), and every run emits the report (#41). Go ahead and merge.

Once it's in, ping me and I'll:
1. Rebase remaining work onto the post-#41 master,
2. Apply the eval's refinements (the fps-FAQ repoint, README Q7 segmenter list, + the minor wording/data-surfacing fixes) as a follow-up PR, and
3. Add the golden regression tests (faq-ref-resolution + expected flag-set) ? plus push the collector-side refinements to **#11**.

546. user (2026-06-07T08:44:05.083Z)

oh great you know just merge it yourself if its good to go!

547. assistant (2026-06-07T08:44:34.777Z)

On it ? merging #41 myself.

548. user (2026-06-07T08:44:42.094Z)

```
Switched to branch 'master'
=== sync local master ===
 create mode 100644 tests/test_config.py
 create mode 100644 tests/test_reporting_harvest.py
 create mode 100644 tests/test_validators_base.py
=== master head ===
98bb1d9 Merge pull request #41 from avidullu/feat/observability-reports
22d6410 chore(test): update reporting/validator test imports for layered package layout
(post-#40 restructure)
56ca0e0 test(reporting): cover typed AppConfig normalization in emit (post-#39 rebase)
```

549. assistant (2026-06-07T08:44:51.633Z)

#41 is merged to master (`98bb1d9`). Let me confirm it's clean and green post-merge:

550. user (2026-06-07T08:45:09.583Z)

```
=== #41 state ===
{"mergedAt":"2026-06-07T08:44:39Z","state":"MERGED"}
=== full suite on merged master ===
.................................................................... [ 94%]
.............                                                        [100%]
229 passed in 12.67s
=== branches (feat gone?) ===
  fix/appconfig-nested-get
* master
remote feat:
```

551. assistant (2026-06-07T08:45:38.314Z)

Both PRs are in. **#41 is `MERGED`**, master is green (**229 passed**), and the feature branch is cleaned up. Let me tidy the stale local config-fix branch and checkpoint memory:

552. user (2026-06-07T08:45:39.937Z)

```
Deleted branch fix/appconfig-nested-get (was 1358521).
=== remaining branches ===
* master
=== recent master ===
98bb1d9 Merge pull request #41 from avidullu/feat/observability-reports
22d6410 chore(test): update reporting/validator test imports for layered package layout
```

```
(post-#40 restructure)
56ca0e0 test(reporting): cover typed AppConfig normalization in emit (post-#39 rebase)
b8904dc chore(reporting): per-tool customer_verdict + bump obsreport pin to v0.1.3
```

553. user (2026-06-07T08:45:55.724Z)

```
The file c:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

554. assistant (2026-06-07T08:46:29.277Z)

Both merged, master green (229), branches cleaned. ? The indexer pipeline is now whole: validators run (via #42) and every run emits the report (via #41).

Now I'll **keep going** with the debuggability refinements the eval surfaced. Starting the indexer follow-up branch and applying the high-value fixes. First, mark a new chapter and branch:

555. user (2026-06-07T08:46:29.935Z)

```
Chapter marked: "Debuggability refinements". Continue your current work.
```

556. user (2026-06-07T08:46:36.606Z)

```
Switched to a new branch 'feat/report-debuggability-refinements'
=== confirm reporting spec + README present on master ===
backend/reporting/spec.yaml
166:## ? Frequently Asked Questions (FAQ)
229:### Q9: How can I debug/test the Gemini Segmenter on a small portion of a video without
uploading a huge multi-GB file?
242:### Q10: Can I trust the per-segment metadata from the `motion` segmenter?
```

557. user (2026-06-07T08:46:44.319Z)

````
197     ### Q7: How do I configure and run an AI-powered segmenter (Gemini / YOLO-hybrid)?
198     Both `yolo_hybrid` (default) and `gemini` need an API key for the AI provider. To use
Google Gemini:
199     1. Create a local `.env` file in the project root (this is already in `.gitignore` to
prevent secret leaks).
200     2. Set your Google AI Studio API key in the file:
201        ```env
202        GEMINI_API_KEY=your_key_here
203        ```
204     3. Run with whichever segmenter you want:
205        ```bash
206        # Default: cheap candidate filtering with YOLO, then precise boundaries via Gemini
207        python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter yolo_hybrid
208
209        # Full-video Gemini analysis (more expensive, highest semantic precision per call)
210        python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter gemini
211
212        # Pure CV baseline ? no API key needed
213        python -m backend.main --video-path "C:/path/to/video.mp4" --segmenter motion
214        ```
215        *Or set `"default_segmenter": "yolo_hybrid"` (or `"gemini"`/`"motion"`) under
`"indexing"` in `config.json`.*
216
217     The AI provider used by `yolo_hybrid` and `gemini` is controlled by `ai_provider` +
`ai_model` at the top level of `config.json`. Today only `"gemini"` is registered; new providers
can be added under `backend/providers/`.
218
219     ### Q8: How does the `yolo_hybrid` pipeline actually save money?
220     Sending a multi-gigabyte video to a hosted LLM is slow and expensive. `yolo_hybrid` runs
````

in four phases:

558. assistant (2026-06-07T08:46:43.629Z)

Let me read the README FAQ region (Q7 + Q13) to apply the segmenter-list and new fps-FAQ edits precisely:

559. assistant (2026-06-07T08:47:00.446Z)

Now apply the **Q7 segmenter-list fix** (the #1 false-lead from the eval) ? note `wasb_hybrid`/`tracknet_hybrid` are valid:

560. user (2026-06-07T08:47:04.119Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\README.md has been updated successfully. (file state is current in your context ? no need to Read it back)

561. assistant (2026-06-07T08:47:09.682Z)

Now read the Q13 region to add the new fps FAQ (Q14) after it:

562. user (2026-06-07T08:47:10.189Z)

```
255
256     To read the technical report: scan **Verdict** first (PASS / PASS-with-warnings / FAIL), then **Findings & warnings** ? each finding has a plain-English message, evidence, and a `? see README#Qn` pointer. To turn reports off or relocate them, set `"report": {"enabled": false}` or `"report": {"output_dir": "..."}` in `config.json`.
257
258     ### Q13: A validation check failed and indexing was halted ? what do I do?
259     Open the report's **Processing stages ? validation** block (or the `VALIDATION SUMMARY` printed to the console). Each failed check states the exact reason. Common cases: **relevance** (the court color didn't match ? see [Q2](#q2-why-does-my-video-fail-the-relevance-check)); **resolution/fps** (below the configured minimums ? adjust `validation.min_*` in `config.json`); **format** (not an MP4 container); **keyframes/PTS** (HEVC/GoPro keyframe quirks ? see [Q5](#q5-why-did-the-video-fail-with-no-keyframes--packet-timestamps-could-be-extracted)). Fix the underlying cause or relax the relevant threshold in `config.json`, then re-run.
260
261     ---
262
263     ## ? Annotation & Evaluation
264
265     Once a video is processed, you can measure **how well the pipeline detected its rallies** against hand-annotated ground truth. You annotate each rally's `[start, end)` time range in a small CSV; the harness scores the detected rallies straight from the SQLite DB ? no API calls, fully deterministic.
266
267     ```bash
268     python run_eval.py --scaffold --annotations rallies.csv          # make an empty template
269     # ...fill in start,end rows, then:
270     python run_eval.py --video "C:/videos/match.mp4" --annotations rallies.csv --show-failures
271     ```
272
273     It reports precision/recall/F1, mean IoU, and boundary error for **two stages** ? raw detector `candidates` vs. AI-confirmed `final` segments ? which pinpoints whether a weak result is a *detection* problem or a *segmentation* problem.
274
275     > Rally timestamps evaluate and tune **segmentation**; they do **not** train the shuttle detector (that needs per-frame coordinate labels). See **[docs/EVALUATION.md](docs/EVALUATION.md)** for the annotation format, CLI flags, and how to read the output.
276
```

563. user (2026-06-07T08:47:26.902Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\README.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

564. assistant (2026-06-07T08:47:40.489Z)

Now update the spec: repoint `fps_fallback_used` to the new Q14, and sharpen the `silent_provider_noop` + `synthetic_motion_metadata` wording. Reading the current spec:

565. user (2026-06-07T08:47:40.882Z)

```
1       # Observability report spec for the badminton-highlight-indexer.
2       # The report ENGINE (config language, renderers) lives in the shared `obsreport`
3       # library; this file is the indexer-specific layout + footgun rules. Every
4       # actionable rule cites a README/FAQ section so a reader can self-debug.
5       name: indexer
6       title: Badminton Highlight Indexer ? Video Report
7
8       # Customer here = whoever receives the highlight reel.
9       customer_verdict:
10        pass: "? Your highlights are ready."
11        pass_with_warnings: "? Your highlights are ready. (A couple of minor notes above.)"
12        fail: "?? We couldn't finish generating highlights for this video. Please contact
support."
13
14      sections:
15        - { id: run,        title: "Run",                kind: identity,   audience: both }
16        - { id: video,      title: "Video",              kind: video_meta, audience: both }
17        - { id: highlights, title: "Highlights",         kind: highlights, audience: customer
}
18        - { id: cust_notes, title: "Things to know",     kind: flags,      audience: customer
}
19        - { id: stages,     title: "Processing stages",  kind: stages,     audience:
technical }
20        - { id: findings,   title: "Findings & warnings", kind: flags,     audience:
technical }
21        - { id: verdict,    title: "Verdict",            kind: verdict,    audience: both }
22
23      rules:
24        # The classic silent failure: 0 segments because the AI provider no-op'd on a
25        # missing API key ? NOT because the video has no rallies.
26        - code: silent_provider_noop
27          severity: error
28          when:
29            - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: false }
30            - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
31          message: >-
32            0 segments AND no AI provider API key was set ? this is almost certainly a
33            silent provider no-op, not 'the video has no rallies'. Set the provider key
34            (e.g. GEMINI_API_KEY in a .env file) and re-run.
35          faq_ref: "README#Q7"
36          customer_message: >-
37            We couldn't generate highlights for this video due to a configuration issue
38            on our side. Please contact support.
39
40        # API key WAS present but an AI segmenter still produced nothing ? likely a
41        # provider error / quota / WSL healthcheck issue rather than a truly empty video.
42        - code: ai_empty_vs_no_rallies
43          severity: warn
44          when:
45            - { path: "stages.ai_handoff.details.ai_based", op: eq, value: true }
46            - { path: "stages.ai_handoff.details.api_key_present", op: eq, value: true }
47            - { path: "stages.segmentation.metrics.segment_count", op: eq, value: 0 }
48          message: >-
```

```
            An AI-based segmenter returned 0 segments even though the API key is set.
            Empty can mean a provider error, exhausted quota, or (for WASB/TrackNet) a
            failed WSL healthcheck ? these all return [] silently. Re-run with -u for
            unbuffered logs and check the provider/WSL output before concluding 'no rallies'.
          faq_ref: "README#Q7"

        # fps was assumed rather than probed -> velocity thresholds + timing may be wrong.
        - code: fps_fallback_used
          severity: warn
          when:
            - { path: "video_meta.fps_source", op: eq, value: fallback }
          message: >-
            fps was assumed (a fallback default), not probed from the file. Motion
            thresholds and frame->second timing may be miscalibrated. Confirm the real
            fps with `ffprobe` and ensure validation ran (it probes the file).
          faq_ref: "README#Q5"

        # The motion baseline fabricates its per-segment metadata/events ? they are not
        # measured. Warn so nobody trusts them as ground truth.
        - code: synthetic_motion_metadata
          severity: warn
          when:
            - { path: "stages.segmentation.details.segmenter_actual", op: eq, value: motion }
          message: >-
            The 'motion' segmenter was used. Its per-segment metadata (intensity, speed,
            events, server/receiver) is HEURISTIC / synthetic, not measured ? do not
            treat it as ground truth. Use yolo_hybrid/gemini for real shot metadata.
          faq_ref: "README#Q10"

        # The segmenter that ran differs from config's default -> you may be looking at a
        # different detector than expected (config/setup/CLI defaults are known to diverge).
        - code: segmenter_divergence
          severity: info
          when:
            - { path: "stages.segmentation.details.matches_config_default", op: eq, value:
false }
            - { path: "stages.segmentation.details.segmenter_explicitly_requested", op: eq,
value: false }
          message: >-
            The segmenter that ran differs from config.json's default_segmenter, and no
            explicit --segmenter was given ? so a fallback default kicked in. The indexer's
            defaults are known to diverge across config.json / setup.py / the CLI; confirm
            you ran the detector you meant.
          faq_ref: "README#Q7"

        # Re-processing an already-indexed video replaced its prior segments (INSERT OR
        # REPLACE cascades). Note it so a changed result isn't mistaken for instability.
        - code: reingest_replaced_prior
          severity: info
          when:
            - { path: "stages.segmentation.details.was_reingest", op: eq, value: true }
          message: >-
            This video was already in the database; re-processing REPLACED its previous
            segments (videos are keyed by file MD5; re-ingest cascades a delete). Expected
            when you intend to re-run; just know the earlier results are gone.
          faq_ref: "README#Q11"

        # A validation check failed -> segmentation was halted. Point the reader at the
        # report's Validation section + the per-check FAQ.
        - code: validation_failed
          severity: error
          when:
            - { path: "stages.validation.status", op: eq, value: error }
          message: >-
            One or more validation checks failed, so indexing was halted. See the
```

```
112           Processing stages > validation block above: each failed check states the
113           reason (relevance -> README#Q2, fps/keyframes -> README#Q5).
114         faq_ref: "README#Q13"
115         customer_message: >-
116           This video couldn't be processed because it didn't pass our quality checks
117           (for example: too blurry, too low resolution, or not a badminton court).
118
```

566. assistant (2026-06-07T08:47:49.542Z)

Applying the spec refinements ? `silent_provider_noop` (tie to AI handoff + explicit `.env` path), `fps_fallback_used` (?Q14, reworded), and `synthetic_motion_metadata` (front-load "placeholder/unmeasured"):

567. user (2026-06-07T08:47:51.453Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml has been updated successfully. (file state is current in your context ? no need to Read it back)

568. user (2026-06-07T08:48:00.480Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml has been updated successfully. (file state is current in your context ? no need to Read it back)

569. user (2026-06-07T08:48:08.279Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\spec.yaml has been updated successfully. (file state is current in your context ? no need to Read it back)

570. assistant (2026-06-07T08:48:19.544Z)

Now the harvest refinements ? show the numeric fps fallback value, and surface motion's synthetic metadata at the data level. Reading harvest.py:

571. user (2026-06-07T08:48:20.474Z)

```
100             st.add_metric("total_play_s", total_play, unit="s")
101             # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
102             # pick it, and it differs from config's default (the genuine surprise case).
103             # An explicit --segmenter override or a halted run is not a divergence.
104             matches_default = True
105             if ran and segmenter_actual and config_default:
106                 matches_default = (segmenter_actual == config_default)
107             st.details.update({
108                 "segmenter_requested": segmenter_requested,
109                 "segmenter_actual": segmenter_actual,
110                 "config_default": config_default,
111                 "segmenter_explicitly_requested": segmenter_requested is not None,
112                 "matches_config_default": matches_default,
113                 "was_reingest": was_reingest,
114                 "detector": detector_tag,
115             })
116
117
118     def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
119                   play_segments: List[Dict[str, Any]], ran: bool) -> None:
120         """Only meaningful for AI-based segmenters. Records whether the provider key was
121         present ? the signal the silent_provider_noop rule keys on."""
122         ai_based = segmenter_actual in _AI_SEGMENTERS
123         provider = (config.get("ai_provider") or "gemini")
124         key_env = f"{provider.upper()}_API_KEY"
125         with rec.stage("ai_handoff") as st:
126             if not ran or not ai_based:
```

```
                st.status = "not_run" if not ran else "skipped"
            st.details.update({
                "ai_based": ai_based,
                "provider": provider if ai_based else None,
                "model": config.get("ai_model") if ai_based else None,
                "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
            })
            if ai_based:
                st.add_metric("confirmed_segments", len(play_segments))


    def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
                    video_meta: Dict[str, Any]) -> None:
        total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
        coverage = None
        dur = video_meta.get("duration_s")
        if dur:
            coverage = round(100.0 * total / dur, 1)
        notes = []
        if not play_segments:
            notes.append("No highlights were produced for this video.")
        rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
                           coverage_pct=coverage, notes=notes)


    # --- top-level ----------------------------------------------------------
    def record_run(
        *,
        video_id: str,
        video_path: Optional[str],
        config: Dict[str, Any],
        val_results: List[Any],
        val_context: Optional[Dict[str, Any]],
```

572. assistant (2026-06-07T08:48:31.907Z)

Let me surface motion's synthetic metadata as a stage field, and set the numeric fps fallback. First the segmentation stage detail:

573. user (2026-06-07T08:48:34.365Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py has been updated successfully. (file state is current in your context ? no need to Read it back)

574. user (2026-06-07T08:48:40.360Z)

```
        **recorder_kwargs: Any,
    ) -> "obsreport.RunRecorder":
        """Assemble a finalized recorder for an indexer run (does NOT write)."""
        rec = obsreport.RunRecorder(
            "badminton-highlight-indexer",
            tool_version=tool_version,
            video_id=video_id,
            video_id_kind="file_md5",
            video_path=video_path,
            spec=spec if spec is not None else load_spec(),
            **recorder_kwargs,
        )
        vm = video_meta_from_context(val_context, video_path)
        # If we never probed an fps but segmentation still ran, the segmenter must have
        # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is.
        if vm.get("fps_source") == "unknown" and segmentation_ran:
            vm["fps_source"] = "fallback"
        rec.set_video_meta(**vm)
```

```
193
194         _validation_stage(rec, val_results)
195         config_default = (config.get("indexing") or {}).get("default_segmenter")
196         _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
```

575. user (2026-06-07T08:48:48.408Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\backend\reporting\harvest.py has
been updated successfully. (file state is current in your context ? no need to Read it back)
```

576. assistant (2026-06-07T08:49:20.999Z)

Now the **golden FAQ-resolution test** ? the committed guarantee that every flag's cited anchor actually exists in the README (this is what makes the reports durably self-debuggable):

577. assistant (2026-06-07T08:49:29.971Z)

```
"""Golden guard: every footgun rule's faq_ref must resolve to a real README heading.

This is the committed backbone of report debuggability ? if a rule cites README#Qn,
that section must exist, so a reader who follows the pointer always lands somewhere.
(A fresh-reader eval found a rule citing the wrong-but-existing section; an existence
test can't catch *semantic* mismatch, but it locks the floor: no dangling anchors.)
"""
import re
from pathlib import Path

import pytest

pytest.importorskip("obsreport")
from obsreport import load_spec  # noqa: E402

ROOT = Path(__file__).resolve().parent.parent
SPEC = ROOT / "backend" / "reporting" / "spec.yaml"
README = ROOT / "README.md"


def _headings(md: str):
    return [ln.lstrip("#").strip() for ln in md.splitlines() if ln.lstrip().startswith("#")]


def _slug(h: str) -> str:
    s = re.sub(r"[^\w\s-]", "", h.lower())
    return re.sub(r"\s+", "-", s.strip())


def _anchor_resolves(anchor: str, headings) -> bool:
    a = anchor.lower()
    for h in headings:
        hl = h.lower()
        if hl == a or hl.startswith(a + ":") or hl.startswith(a + " ") or hl.startswith(a +
".."):
            return True  # Q-number style: "Q7" -> "### Q7: ..."
        if _slug(h) == a:
            return True  # full GitHub slug match
    return False


def test_every_rule_faq_ref_resolves_to_a_readme_heading():
    spec = load_spec(SPEC)
    headings = _headings(README.read_text(encoding="utf-8"))
    unresolved = []
    for rule in spec.rules:
        if not rule.faq_ref:
```

```
            continue
        file_part, _, anchor = rule.faq_ref.partition("#")
        assert file_part in ("README", "README.md"), f"{rule.code}: unexpected faq file
'{file_part}'"
        if not _anchor_resolves(anchor, headings):
            unresolved.append((rule.code, rule.faq_ref))
    assert not unresolved, f"faq_ref anchors with no matching README heading: {unresolved}"


def test_actionable_rules_have_faq_refs():
    """error/warn findings must cite a FAQ (info may omit). Belt-and-suspenders on top
    of obsreport's own spec-load validation."""
    spec = load_spec(SPEC)
    missing = [r.code for r in spec.rules if r.severity in ("error", "warn") and not r.faq_ref]
    assert not missing, f"actionable rules missing faq_ref: {missing}"
```

578. user (2026-06-07T08:49:29.983Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\tests\test_reporting_spec_faq.py (file state is current in your context ? no need to
Read it back)
```

579. assistant (2026-06-07T08:49:39.677Z)

Run the full suite + the new golden test:

580. user (2026-06-07T08:49:41.755Z)

```
..                                                                       [100%]
2 passed in 0.24s
```

581. assistant (2026-06-07T08:49:48.724Z)

All faq_refs resolve. Now the full suite, then I'll regenerate a couple of refined reports to confirm the wording lands:

582. user (2026-06-07T08:50:03.043Z)

```
.......................................................................... [ 62%]
.......................................................................... [ 93%]
...............                                                          [100%]
231 passed in 12.21s
```

583. assistant (2026-06-07T08:50:18.760Z)

231 passed (229 + 2 new). Let me eyeball the refined wording on the three affected scenarios (fps numeric, motion data-level, silent-provider):

584. user (2026-06-07T08:50:23.380Z)

```
===== FPS FALLBACK (no ffprobe meta) =====
   | FPS | 30 fps (fallback) |
   - [WARN] **fps_fallback_used** ? fps was NOT probed from the file ? a fallback default (shown
as the fps value) was used. Because rally timing and motion thresholds scale with fps,
boundaries and motion sensitivity may be miscalibrated. This usually means validation didn't run
(it probes the file) or ffprobe couldn't read r_frame_rate. Confirm the real fps with ffprobe
and re-run.
   ? see **README#Q14**
===== MOTION (synthetic data-level) =====
   - _metadata_source_: synthetic/heuristic ? NOT measured
   - [WARN] **synthetic_motion_metadata** ? The 'motion' segmenter was used: its per-segment
metadata (intensity, speed, events, server/receiver) are PLACEHOLDER values (unmeasured) ?
fabricated by the heuristic, NOT derived from the video ? so a PASS with segments here does not
mean the metadata is trustworthy. Use yolo_hybrid/gemini for real shot metadata.
```

```
===== SILENT PROVIDER NOOP =====
   silent_provider_noop** ? 0 segments AND no AI provider API key was set. The segmenter (see
Processing stages > segmentation > segmenter_actual) hands its candidate windows to the AI
provider, which silently did nothing without a key ? so this is a MISSING CREDENTIAL, not 'the
video has no rallies'. Fix: put GEMINI_API_KEY=<key> in a .env file in the project root and re-
run (see README#Q7).
     ? s
```

585. assistant (2026-06-07T08:50:44.644Z)

All three refinements landed cleanly: **`FPS | 30 fps (fallback)`**, motion `metadata_source: synthetic/heuristic ? NOT measured` at the data level, and the silent-provider finding now ties the segmenter?AI-handoff with the explicit `.env` path. Committing + opening the indexer refinements PR:

586. user (2026-06-07T08:50:55.089Z)

```
warning: in the working copy of 'tests/test_reporting_spec_faq.py', LF will be replaced by CRLF
the next time Git touches it
M  README.md
M  backend/reporting/harvest.py
M  backend/reporting/spec.yaml
A  tests/test_reporting_spec_faq.py
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/feat/report-
debuggability-refinements
remote:
https://github.com/avidullu/badminton-highlight-indexer/pull/43
```

587. assistant (2026-06-07T08:51:26.654Z)

Indexer refinements ? [PR #43](https://github.com/avidullu/badminton-highlight-indexer/pull/43). Now the collector side (push to #11) ? switching repos and checking out its branch:

588. user (2026-06-07T08:51:31.451Z)

```
Already on 'feat/observability-reports'
=== collector master vs my branch ===
1       0
=== #11 state ===
{"mergeable":"UNKNOWN","state":"MERGED"}
```

589. assistant (2026-06-07T08:51:42.276Z)

PR #11 is already **MERGED** ? collector observability is on master too. So the collector refinements go on a fresh branch off the updated master. Syncing and branching:

590. user (2026-06-07T08:51:48.709Z)

```
Switched to branch 'master'
 create mode 100644 src/reporting/spec.yaml
 create mode 100644 tests/test_reporting.py
=== master head ===
94fec51 Merge pull request #11 from avidullu/feat/observability-reports
038424d feat(reporting): per-video observability report (collector)
99dfa22 Merge pull request #10 from avidullu/fix/canonical-csv-zero-start
Switched to a new branch 'feat/collector-report-refinements'
=== reporting present? ===
__init__.py
__pycache__
adapter.py
spec.yaml
.................................................................... [ 63%]
.......................................                             [100%]
113 passed in 1.88s
```

591. assistant (2026-06-07T08:51:55.935Z)

Baseline green (113). Let me read the current adapter to apply the frame-evidence + review-framing refinements:

592. user (2026-06-07T08:51:56.632Z)

```
28          """Map a rally_cvat.Issue onto a report Flag. Actionable issues (blocker/high/
29          medium) also carry a customer_message so the delivery summary shows the vendor
30          exactly what to fix, in plain language."""
31          sev = _SEVERITY.get(issue.severity, "info")
32          customer = issue.message if sev in ("error", "warn") else None
33          evidence = {"rally_number": issue.rally_number} if issue.rally_number is not None
else {}
34          return obsreport.Flag(
35              code=issue.code,
36              severity=sev,
37              message=issue.message,
38              stage="validation",
39              faq_ref=_ISSUE_FAQ,
40              customer_message=customer,
41              evidence=evidence,
42          )
43
44
45      def _recorder(source: str, video_path: Optional[str], spec) -> "obsreport.RunRecorder":
46          return obsreport.RunRecorder(
47              "sports-data-collector",
48              video_id=source,
49              video_id_kind="human_id",
50              video_path=video_path,
51              spec=spec if spec is not None else load_spec(),
52          )
53
54
55      def build_delivery_recorder(
56          *,
57          records: List[Any],
58          issues: List[Any],
59          fps: Optional[float],
60          source: str,
61          video_path: Optional[str] = None,
62          video_duration_s: Optional[float] = None,
63          source_format: str = "cvat",
64          step: Optional[int] = None,
65          spec=None,
66      ) -> "obsreport.RunRecorder":
67          """Envelope for `validate-delivery` / `convert-cvat`: parse + validation stages,
68          one flag per delivery issue, a rally summary."""
69          rec = _recorder(source, video_path, spec)
70          if fps:
71              rec.set_video_meta(fps=fps, fps_source="declared")
72          if video_duration_s:
73              rec.set_video_meta(duration_s=round(video_duration_s, 1))
74
75          with rec.stage("parse") as st:
76              st.add_metric("rallies_parsed", len(records))
77              st.details.update({"source_format": source_format, "fps": fps, "sampling_step":
step})
78
79          blockers = [i for i in issues if i.severity == "blocker"]
80          with rec.stage("validation") as st:
81              for sev in ("blocker", "high", "medium", "low"):
82                  st.add_metric(f"issues_{sev}", sum(1 for i in issues if i.severity == sev))
83              st.add_metric("n_blockers", len(blockers))
```

```
84              st.details["gate"] = "BLOCKED" if blockers else "PASS"
85              st.status = "error" if blockers else ("warn" if issues else "ok")
86
87          for issue in issues:
88              rec.envelope.flags.append(issue_to_flag(issue))
89
```

593. assistant (2026-06-07T08:52:19.295Z)

Let me enrich `issue_to_flag` with frame-level evidence (for CVAT reviewers) and review-only framing for heuristic checks, then wire it in the delivery recorder:

594. user (2026-06-07T08:52:24.014Z)

The file C:\Users\avidu\Projects\sports-data-collector\src\reporting\adapter.py has been updated successfully. (file state is current in your context ? no need to Read it back)

595. user (2026-06-07T08:52:30.886Z)

The file C:\Users\avidu\Projects\sports-data-collector\src\reporting\adapter.py has been updated successfully. (file state is current in your context ? no need to Read it back)

596. assistant (2026-06-07T08:52:49.382Z)

Now the collector golden faq-resolution test (mirrors the indexer's), covering both the issue-FAQ and the spec rules:

597. assistant (2026-06-07T08:52:55.178Z)

```
"""Golden guard: every faq_ref the collector reports emit must resolve to a real
heading in the cited doc ? both the per-issue FAQ and the spec rules' faq_refs.
Locks the floor: no dangling debug pointers in delivery/ingestion reports."""
import re
from pathlib import Path

import pytest

pytest.importorskip("obsreport")
from obsreport import load_spec  # noqa: E402

from src.reporting import adapter  # noqa: E402

ROOT = Path(__file__).resolve().parent.parent
SPEC = ROOT / "src" / "reporting" / "spec.yaml"


def _headings(md: str):
    return [ln.lstrip("#").strip() for ln in md.splitlines() if ln.lstrip().startswith("#")]


def _slug(h: str) -> str:
    s = re.sub(r"[^\w\s-]", "", h.lower())
    return re.sub(r"\s+", "-", s.strip())


def _resolves(faq_ref: str) -> bool:
    file_part, _, anchor = faq_ref.partition("#")
    doc = ROOT / file_part
    if not doc.exists():
        return False
    if not anchor:
        return True
    return anchor.lower() in {_slug(h) for h in _headings(doc.read_text(encoding="utf-8"))}
```

```
def test_all_collector_faq_refs_resolve():
    refs = {adapter._ISSUE_FAQ}
    for rule in load_spec(SPEC).rules:
        if rule.faq_ref:
            refs.add(rule.faq_ref)
    unresolved = [r for r in sorted(refs) if not _resolves(r)]
    assert not unresolved, f"faq_ref anchors with no matching heading in their doc:
{unresolved}"


def test_actionable_rules_have_faq_refs():
    spec = load_spec(SPEC)
    missing = [r.code for r in spec.rules if r.severity in ("error", "warn") and not r.faq_ref]
    assert not missing, f"actionable collector rules missing faq_ref: {missing}"
```

598. user (2026-06-07T08:52:55.203Z)

```
File created successfully at: C:\Users\avidu\Projects\sports-data-
collector\tests\test_reporting_faq.py (file state is current in your context ? no need to Read
it back)
```

599. assistant (2026-06-07T08:53:04.944Z)

Let me run the collector suite (incl. the new faq test) to confirm nothing broke and the anchors resolve:

600. user (2026-06-07T08:53:06.794Z)

```
..........                                                               [100%]
10 passed in 0.28s
```

601. assistant (2026-06-07T08:53:20.007Z)

All green. Let me add a behavior test for the new frame-evidence + review-framing, then run the full suite:

602. user (2026-06-07T08:53:22.571Z)

<tool_use_error>File has not been read yet. Read it first before writing to it.</tool_use_error>

603. user (2026-06-07T08:53:28.323Z)

```
25      # --- issue -> flag ----------------------------------------------------------
26      def test_issue_to_flag_severity_and_customer_message():
27          blocker = adapter.issue_to_flag(Issue("blocker", "overlap", "Rally 1 overlaps Rally
2.", 2))
28          assert blocker.severity == "error"
29          assert blocker.customer_message == "Rally 1 overlaps Rally 2."  # actionable ->
shown to vendor
30          assert blocker.faq_ref.endswith("#what-does-validation-check")
31          assert blocker.evidence == {"rally_number": 2}
32
33          low = adapter.issue_to_flag(Issue("low", "outcome_vocab", "odd outcome", 3))
34          assert low.severity == "info"
35          assert low.customer_message is None  # cosmetic -> not surfaced to the vendor
36
37
38      # --- delivery envelope ------------------------------------------------------
39      def test_delivery_with_blocker_fails_and_surfaces_to_customer():
40          records = [_rec(1, 0.0, 5.0), _rec(2, 4.0, 9.0)]
41          issues = [Issue("blocker", "overlap", "Rally 1 (ends 5.00s) overlaps Rally 2 (starts
4.00s).", 2)]
42          rec = adapter.build_delivery_recorder(records=records, issues=issues, fps=59.94,
43                                                source="pilot_01", video_duration_s=600.0)
```

```
44          env = rec.envelope
45          assert env.verdict == "fail"
46          assert env.get_stage("validation").status == "error"
47          assert "overlap" in _codes(env)
48          assert env.highlights.segment_count == 2
49          assert env.lint() == []
50
51          from obsreport import render_customer, render_technical
52          tech = render_technical(env, rec.spec)
53          cust = render_customer(env, rec.spec)
54          assert "**overlap**" in tech  # bolded issue CODE visible technically
55          assert "**overlap**" not in cust  # code hidden from the customer/vendor view
56          assert "overlaps Rally 2" in cust  # but the plain message is shown
57          assert "blocking issues" in cust  # collector-specific customer verdict (not the
generic default)
58
59
60      def test_delivery_clean_passes():
61          records = [_rec(1, 0.0, 5.0), _rec(2, 6.0, 9.0)]
62          rec = adapter.build_delivery_recorder(records=records, issues=[], fps=59.94,
source="clean")
63          assert rec.envelope.verdict == "pass"
64          assert _codes(rec.envelope) == set()
```

604. user (2026-06-07T08:53:40.270Z)

```
The file C:\Users\avidu\Projects\sports-data-collector\tests\test_reporting.py has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

605. user (2026-06-07T08:53:48.361Z)

```
.................................................................... [ 62%]
........................................                           [100%]
116 passed in 1.95s
```

606. assistant (2026-06-07T08:54:00.905Z)

```
116 passed. Committing + opening the collector refinements PR:
```

607. user (2026-06-07T08:54:08.516Z)

```
warning: in the working copy of 'tests/test_reporting_faq.py', LF will be replaced by CRLF the
next time Git touches it
remote:      https://github.com/avidullu/sports-data-collector/pull/new/feat/collector-report-
refinements
remote:
https://github.com/avidullu/sports-data-collector/pull/12
```

608. assistant (2026-06-07T08:54:27.036Z)

```
Collector refinements ? [PR #12](https://github.com/avidullu/sports-data-collector/pull/12).
**Phase D is complete.** Let me mark it done and checkpoint memory:
```

609. user (2026-06-07T08:54:27.046Z)

```
Updated task #7 status
```

610. user (2026-06-07T08:54:32.039Z)

```
The file c:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\current-state.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

611. assistant (2026-06-07T08:54:55.478Z)

The full feature is shipped end-to-end. Here's where everything landed.

? Per-video observability ? delivered across both tools + shared lib

Merged to master:
- **`obsreport`** ? new shared, MIT, permissively-licensed "report config language" lib ? github.com/avidullu/sports-obsreport (private, v0.1.3, 74 tests)
- **Indexer**: [#41](https://github.com/avidullu/badminton-highlight-indexer/pull/41) (observability) + [#42](https://github.com/avidullu/badminton-highlight-indexer/pull/42) (the config runtime bug I caught & fixed)
- **Collector**: [#11](https://github.com/avidullu/sports-data-collector/pull/11) (observability) + [#10](https://github.com/avidullu/sports-data-collector/pull/10) (parser fix)

Every processing/ingestion run now writes, next to the video (configurable): a machine `*.report.json`, a **technical** `*.report.md` (stages + findings, each citing a FAQ), and a **customer-friendly** `*.summary.md` (metadata + meta-highlights, internals stripped, gentle audience-appropriate verdict).

? Debuggability validated (your stretch goal)

A 15-agent workflow put all 7 broken-run reports in front of "fresh reader" agents that saw **only the README/FAQ + the report** ? verdict: **all 7 self-debuggable**, every reader found the root cause and cited the right FAQ. I also rendered the reports to HTML and eyeballed them (clean tables, fps source shown, customer summaries free of jargon).

? Two follow-up PRs open (green, awaiting your merge)

The eval surfaced concrete polish, now applied:
- **Indexer [#43](https://github.com/avidullu/badminton-highlight-indexer/pull/43)** (231 tests): README Q7 now lists `wasb_hybrid`/`tracknet_hybrid` as valid (killed a false lead); new fps **Q14** + repointed `fps_fallback_used` off the wrong Q5; numeric `30 (fallback)`; motion "synthetic" surfaced at the data level; sharper silent-provider + blur guidance; **faq-resolution golden test**.
- **Collector [#12](https://github.com/avidullu/sports-data-collector/pull/12)** (116 tests): frame-level evidence on delivery flags; "review only, not a blocker" framing for heuristics; **faq-resolution golden test**.

Both are disjoint follow-ups ? merge whenever.

Notes for the record

- **Config bug I found:** the typed `AppConfig` from your `#39` broke all validators on real runs (nested `config.get(...).get(...)`); my report is what surfaced it, and #42 fixed it (your review nits addressed). The directory restructure (#40) was clean ? I rebased onto it without issues.
- **Merge-conflict help:** resolved the indexer `main.py` conflict (your config package vs my reporting) and kept the parser fix to its own dedicated branch so PRs stayed non-overlapping.
- **Optional future** (noted in memory): an HTML customer-report renderer; threading real AI-provider timing through the segmenters (currently recorded as `not_captured`); pinning `obsreport` via `git+ssh` once CI/SSH access is set up (it's a soft dep today, so nothing breaks meanwhile).

Memory/handoff updated. Want me to merge #43 and #12 too, or leave them for your review?

612. user (2026-06-07T08:54:57.034Z)

I merged #11 as well on the collector. Can you bring both the repos at a clean slate and quickly tell me whether the low regret moves pan in the docs/ folder seems a good starter project list for grok? I think your work will have some intersect but those are mostly syntactic sugars and robust reporting.
oh and I hope you are checkpointing the session handoffs and current state and other "carry forward package for claude to resume quickly" suite of updates since you are already at >70% context util (700k tokens)

#43 is also merged

613. assistant (2026-06-07T08:55:16.117Z)

Got it ? and yes, I'll lock in the carry-forward updates (handoff + current-state) given the context level. Let me first bring both repos to a clean slate and check PR states, then assess the low-regret doc for Grok.

614. user (2026-06-07T08:55:30.603Z)

```
######## INDEXER ########
Switched to branch 'master'
master head:
491c909 Merge pull request #43 from avidullu/feat/report-debuggability-refinements
51b1f8d feat(reporting): debuggability refinements from fresh-reader eval + faq-resolution
golden test
98bb1d9 Merge pull request #41 from avidullu/feat/observability-reports
PR states #41/#42/#43:
#41: MERGED
#42: MERGED
#43: MERGED
local branches:
  feat/report-debuggability-refinements
* master

######## COLLECTOR ########
Switched to branch 'master'
master head:
94fec51 Merge pull request #11 from avidullu/feat/observability-reports
038424d feat(reporting): per-video observability report (collector)
99dfa22 Merge pull request #10 from avidullu/fix/canonical-csv-zero-start
PR states #10/#11/#12:
#10: MERGED
#11: MERGED
#12: OPEN
local branches:
  feat/collector-report-refinements
  feat/observability-reports
* master
```

615. assistant (2026-06-07T08:55:52.738Z)

Clean-slate cleanup ? pruning the merged local branches (keeping collector `#12` since it's still open):

616. user (2026-06-07T08:55:57.034Z)

INDEXER cleanup ###

```
Deleted branch feat/report-debuggability-refinements (was 51b1f8d).
* master
uncommitted files: 0
```

COLLECTOR cleanup ###

```
Deleted branch feat/observability-reports (was 038424d).
(keeping feat/collector-report-refinements ? PR #12 still OPEN)
  feat/collector-report-refinements
* master
uncommitted files: 0
```

find low-regret doc (indexer) ###

```
docs/AUDIO_E1_FINDINGS.md docs/AUDIO_RALLY_DETECTION_PLAN.md docs/BACKLOG.md docs/CODE_MAP.md
docs/COMMERCIAL_READINESS_REVIEW.md docs/DECISIONS.md docs/EVALUATION.md
docs/GOLDEN_SET_IMPLEMENTATION.md docs/GOLDEN_SET_PHASE1_VIDEOS.md docs/GOLDEN_SET_VENDORING.md
docs/HOW_RALLY_DETECTION_WORKS.md docs/LOW_REGRET_MOVES_PLAN.md docs/MONETIZATION_AUDIT.md
```

docs/RALLY_DETECTION_RESEARCH.md docs/ROADMAP.md docs/TRACKNET_WSL_SETUP.md
docs/VIDEO_RECORDING_GUIDELINES.md

617. assistant (2026-06-07T08:56:04.662Z)

Both repos are clean (on `master`, no uncommitted files, merged branches pruned; kept the open #12 branch). The doc is `docs/LOW_REGRET_MOVES_PLAN.md`. Let me read it (and peek at its companion review):

618. user (2026-06-07T08:56:05.026Z)

```
1       # Low-Regret Moves Plan ? Config Model First
2
3       **Status:** Approved (checkpointed)
4       **Date:** 2026-06-07 (updated 2026-06-07)
5       **Owner / Driver:** Grok (second-in-command) + avidullu + Claude
6       **Related:** [COMMERCIAL_READINESS_REVIEW.md](COMMERCIAL_READINESS_REVIEW.md),
[BACKLOG.md](BACKLOG.md), [CODE_MAP.md](CODE_MAP.md), [DECISIONS.md](DECISIONS.md)
7
8       ---
9
10      ## 1. Purpose & Philosophy
11
12      These are **low-regret moves**: changes that are high-confidence improvements to code
health today while directly de-risking the commercial and self-improving vision (offline learned
segmenter, hosted path, multi-sport, data flywheel).
13
14      **Guiding principles**
15      - Small, reviewable increments (one or two files per PR where possible).
16      - Payoff within the same or next session.
17      - Improve the developer + product experience immediately.
18      - Lay infrastructure that makes future items cheaper and safer.
19      - Respect the existing culture (registries, eval harness, ADRs, excellent docs).
20
21      **Rule:** Every item must be traceable to a concrete pain/risk from the Commercial
Readiness Review or existing BACKLOG/ADRs.
22
23      ---
24
25      ## 2. Overall Sequence (Approved Order)
26
27      1. **Config Model (Pydantic) + Central Loader** (as a proper package) ? foundational.
28      2. **Directory Structure Refactor** (high-pri, atomic follow-up PR) ? move to the
approved layered layout.
29      3. Error / Result Contract standardization.
30      4. Logging standardization + removal of stray `print()` in hot paths.
31      5. Packaging modernization (`pyproject.toml` + clean entry points).
32      6. Detector Runner Abstraction (start de-risking WSL).
33      7. Unify / extract shared logic from the two hybrid segmenters.
34      8. Input Quality UX (surface validator + audio readiness signals).
35      9. (Stretch) First slice of async job model.
36
37      **Important sequencing decision (approved):** Complete the Config work **first** (this
introduces `backend/config/` as a package immediately). Then perform the broader directory
refactor as a **separate, atomic, high-priority PR**.
38
39      Rationale:
40      - Keeps the Config PR small and focused.
41      - Allows the structural change to land atomically with minimal merge conflicts.
42      - Gives Claude a clean window with no concurrent file moves.
43      - Still delivers the "level of indirection" benefit early via the `config/` package.
44
45      ---
46
47      ## 3. Approved Directory Structure Direction
```

Current flat layout under `backend/` (many sibling dirs + root .py files) feels unprofessional for long-term maintenance. We will trend toward clear layers without creating 15+ tiny single-file directories.

Target layered structure (directional, not all at once):

```
backend/
??? __init__.py
??? main.py                     # Thin CLI + FastAPI bootstrap only
?
??? config/                     # ? Start here with Deliverable 1
?   ??? __init__.py
?   ??? models.py               # Pydantic AppConfig + nested models
?   ??? loader.py               # load_config, save_default_config, etc.
?
??? api/                        # Presentation layer
?   ??? __init__.py
?   ??? app.py                  # FastAPI instance, middleware, routes
?   ??? static/                 # Frontend assets (HTML/JS/CSS)
?
??? pipeline/                   # Core processing engine
?   ??? validators/
?   ??? segmenters/             # (renamed/moved from current indexer/)
?   ??? stitcher/
?   ??? audio/
?   ??? detectors/
?
??? infrastructure/             # External adapters
?   ??? database.py             # (moved from root)
?
??? providers/                  # AI providers (group under infrastructure later if desired)
??? sports/
??? annotations/
?
??? eval/                       # Keep prominent (data flywheel / measurement is first-class)
?
??? core/                       # Shared primitives (introduce only as needed)
??? utils/                      # Low-level cross-cutting helpers (keep small)
```

This provides professional indirection and natural homes for future commercial features while respecting the current "backend" import root for now.

---

4. Deliverable 1: Config Model (The Starter Project)

4.1 Problem Statement
- Raw `json.load` + repeated dict `.get()` chains.
- Three divergent default sources (`config.json`, `setup.py`, `main.py` fallbacks).
- No validation at load ? problems surface late on long videos.
- No single source of truth for the config shape.
- Future needs (per-job overrides, hosted settings, experiments) have no clean home.

4.2 Goals for v1
- One canonical, validated, typed `AppConfig`.
- Built-in defaults (in code) + file overrides (`config.json`).
- Graceful migration (old files continue to work).
- Typed access + IDE support.
- Central loader usable from CLI, server, tests, `setup.py`.
- Start with a proper **package** (`backend/config/`) to align with the approved

structure.

    ### 4.3 Design (as Package)

    Location: `backend/config/` (package, not single file)

    - `backend/config/models.py` ? All Pydantic models (`AppConfig`, `ValidationConfig`, `IndexingConfig`, `TrackNetConfig`, etc.).
    - `backend/config/loader.py` ? `load_config(path=None) -> AppConfig`, `save_default_config(...)`, internal default merging, `extra="allow"` for transition.
    - `backend/config/__init__.py` ? Public API re-exports: `from backend.config import load_config, AppConfig, ...`

    Consumers will do:
    ```python
    from backend.config import load_config, AppConfig
    cfg: AppConfig = load_config()
    print(cfg.indexing.ai_max_workers)
    ```

    The models are the single source of truth for defaults. `setup.py` will call the new save function. `main.py` will use the typed loader early.

    ### 4.4 Migration & Rollout (Config PR scope)
    - Create the `backend/config/` package.
    - Update `setup.py` to use the new loader/saver.
    - Update `main.py` (CLI + API paths) to use typed config (minimal surface at first).
    - Add `tests/test_config.py`.
    - Update `CODE_MAP.md` and this plan.
    - Verify no behavior change on existing runs (`--setup`, small video processing).
    - **Deliberately do not** move other directories in this PR.

    ### 4.5 Success Criteria
    - Typed config is the only way new code reads settings.
    - Existing `config.json` files (and `setup.py` output) continue to work unchanged.
    - `backend/config/` package exists and sets the pattern for the larger structure.
    - One focused, reviewable PR.

    ## 5. Next High-Priority Item: Directory Structure Refactor (Separate Atomic PR)

    After the Config PR lands and is verified:
    - Create a dedicated branch/PR whose only job is the layout change.
    - Move files into `api/`, `pipeline/`, `infrastructure/` etc.
    - Update all imports, entry points, docs (`CODE_MAP.md`, README, tests), and any hard-coded paths.
    - Goal: one clean atomic change with no one else (Claude) needing to resolve conflicts at the same time.

    This item is now tracked at the top of the sequence and in BACKLOG.

    ## 6. Later Low-Regret Items (after the above two)

    - Error / Result Contract standardization (small, high signal).
    - Logging standardization.
    - Packaging modernization.
    - Detector Runner Abstraction.
    - Unify hybrid segmenter logic.
    - Input Quality UX surfacing.
    - Async job model slice.

    See BACKLOG for status tracking.

```
166
167     ---
168
169     **Checkpoint note (2026-06-07):**
170     The directory structure proposal and the "Config first, then atomic refactor" sequencing
decision were reviewed and approved. This document + the corresponding BACKLOG update serve as
the official checkpoint before code changes begin.
171
172     Next step after this docs PR merges: implement Deliverable 1 (the `backend/config/`
package) in a code PR.
```

619. assistant (2026-06-07T08:57:03.665Z)

```
Both repos are clean. The plan is `docs/LOW_REGRET_MOVES_PLAN.md`. Before I give the assessment,
let me lock in the carry-forward (you're right about context level) ? refreshing the session
handoff to reflect the shipped feature:
```

620. assistant (2026-06-07T08:57:20.474Z)

```
---
name: session-handoff
description: "Living \"you are here\" handoff for this project ? current state, next steps, open
decisions, ramp-up kit. AUTO-REFRESHED at meaningful checkpoints (PR merged, key decision/ADR,
phase shift, session wrap); \"update the handoff\" forces an immediate full rewrite."
metadata:
  node_type: memory
  type: project
  originSessionId: eb4b7b53-2f77-4c4d-83e1-4fa5e77f3e3f
---
```

Session handoff ? badminton-highlight-indexer (+ siblings: sports-data-collector, rally

```
**Last updated:** 2026-06-07 (observability feature shipped). **Read [[current-state]] first**
for the
blow-by-blow; this is the ~1-page orientation. Refresh at meaningful checkpoints.
```

You are here (2026-06-07)

```
- **NEW FEATURE SHIPPED: per-video OBSERVABILITY REPORTS across BOTH tools** (indexer +
collector), powered by a
  **new shared library `obsreport`** (its own private GitHub repo `github.com/avidullu/sports-
obsreport`, MIT, tag
  **v0.1.3**, editable-installed locally). Every process/ingest run writes, next to the video
(override
  `report.output_dir`): `*.report.json` (machine), `*.report.md` (TECHNICAL ? stages + footgun
flags, each citing a
  FAQ), `*.summary.md` (CUSTOMER-friendly ? metadata + meta-highlights, internals stripped).
`obsreport` is a **SOFT
  dependency** (pipeline no-ops if absent; report writes NEVER raise).
- **MERGED to master:** indexer **#41** (observability) + **#42** (a config runtime-bug fix I
found ? the typed
  AppConfig broke validators' nested `.get`) + **#43** (debuggability refinements + golden
test). Collector **#11**
  (observability). **OPEN:** collector **#12** (collector refinements + golden test, green,
awaiting merge).
- **Both repos CLEAN on master**, merged branches pruned (collector keeps `feat/collector-
report-refinements` = #12).
  Suites green: indexer 231, collector 116 (on #12), obsreport 74.
- **Debuggability validated** (your stretch goal): a 15-agent Workflow showed all 7 broken-run
scenarios are
  self-debuggable by a README+FAQ-only reader; refinements applied (#43/#12) + faq-resolution
golden tests lock it.
- **Parallel track (Grok):** the **low-regret moves** in `docs/LOW_REGRET_MOVES_PLAN.md` ? items
**1 (Config pkg, #39)
```

+ 2 (directory restructure, #40) DONE**; items 3?9 remain. My work intersects mainly item **8 (Input-Quality UX =
largely my reports already)** and synergizes with **3 (Result contract ? stronger report signals)** + **7 (unify
hybrid segmenters ? enables the deferred AI-timing capture)**. See assessment in [[current-state]] / chat.

Next steps / open threads

1. **Merge collector #12** (observability refinements; green).
2. **Optional obsreport follow-ups** (noted, not blocking): HTML customer-report renderer; thread REAL AI-provider
timing through the segmenters (currently recorded `not_captured` ? easiest once Grok unifies the twins, item 7);
pin `obsreport` via `git+ssh` tag in both repos' deps once SSH/CI access exists (soft dep until then).
3. **Grok's low-regret items 3?9** ? good list (see assessment). Coordinate on item 8 (don't rebuild quality
surfacing ? extend the report) and item 7 (I'll add the AI-timing recorder hook there).
4. **Original project (parked):** rally-detection modeling still waits on GOLDEN LABELS (owner self-rates via the VLC
annotator). Audio E1 done (~2.2× ceiling, not adopted). See the older checkpoints in [[current-state]].

Key decisions (don't relitigate)

- **obsreport = shared "report config language"** lib (pydantic+Jinja2+PyYAML; permissive only). Soft dep; reports
next-to-video; ONE typed envelope ? technical + customer views (customer shows only curated `customer_message`
flags + per-tool `customer_verdict`). Heavy frameworks (Soda=ELv2, GE/Evidently) rejected (license/shape).
- **Reporting normalizes config to a dict at its boundary** (works with dict OR typed AppConfig).
- Frequent small PRs; continuous [[current-state]] updates; never fabricate provenance ([[working-agreement-attribution]]).

Ramp-up kit (read to get current)

- [[current-state]] (blow-by-blow), this handoff, [[working-agreement-checkpoints]], [[working-agreement-attribution]].
- **The lib:** `C:\Users\avidu\Projects\sports-obsreport\` (README + `src/obsreport/` + `specs/example.yaml`).
- **Indexer reporting:** `backend/reporting/{__init__,harvest}.py` + `backend/reporting/spec.yaml`; hooks in
`backend/main.py`; `run_all_with_context` in `backend/pipeline/validators/base.py`; README FAQ Q7/Q10?Q14.
- **Collector reporting:** `src/reporting/{__init__,adapter}.py` + `spec.yaml`; wired in `src/cli/curate.py`;
`docs/INGESTION_PIPELINE.md` FAQ.
- **CODE/architecture:** `docs/CODE_MAP.md` (? pre-#40 paths ? regenerate; layout is now `backend/pipeline/...`,
`backend/infrastructure/`, `backend/api/`, `backend/config/`). Strategy: `docs/DECISIONS.md`, `docs/ROADMAP.md`,
`docs/LOW_REGRET_MOVES_PLAN.md`, `docs/COMMERCIAL_READINESS_REVIEW.md`.
- Eval artifacts (regenerable): `%TEMP%\obseval\` (manifest + reports/ + gen_*.py + render_html.py).

621. user (2026-06-07T08:57:20.475Z)

<tool_use_error>File has been modified since read, either by the user or by a linter. Read it again before attempting to write it.</tool_use_error>

622. assistant (2026-06-07T08:57:27.500Z)

The handoff was modified externally ? let me re-read before rewriting:

623. user (2026-06-07T08:57:28.204Z)

```
1       ---
2       name: session-handoff
3       description: "Living \"you are here\" handoff for this project ? current state, next
steps, open decisions, ramp-up kit. AUTO-REFRESHED at meaningful checkpoints (PR merged, key
decision/ADR, phase shift, session wrap); \"update the handoff\" forces an immediate full
rewrite."
4       metadata:
5         node_type: memory
6         type: project
7         originSessionId: eb4b7b53-2f77-4c4d-83e1-4fa5e77f3e3f
8       ---
9
10      # Session handoff ? badminton-highlight-indexer (+ siblings: sports-data-collector,
rally-annotator, sports-obsreport)
11
12      **Last updated:** 2026-06-07 (observability phase) · **Refresh policy: AUTO-refresh at
meaningful checkpoints** ?
13      PR merged, key decision/ADR, phase shift, session wrap-up (NOT on-request-only). "update
the handoff" forces an
14      immediate full rewrite. **Staleness tripwire:** if ?3 substantive [[current-state]]
checkpoints ? or any
15      PR-merge / new ADR / phase shift ? have landed since the "Last updated" date above,
refresh this handoff now.
16      Pointer, not a duplicate. **For the freshest blow-by-blow state, read [[current-state]]
first**
17      (updated every micro-checkpoint); this handoff is the slower ~1-page orientation.
18
19      ## You are here (2026-06-07) ? ? OBSERVABILITY-REPORTS feature mid-flight (multi-
session/concurrent)
20      - **Active feature:** always-on **per-video observability reports** across BOTH tools
(indexer + collector),
21        powered by a NEW shared lib **`sports-obsreport`** (pkg `obsreport`) ? a small "report
config language" on
22        permissive primitives (pydantic/Jinja2/PyYAML). Each tool emits a **technical** report
(debuggable via README+FAQ)
23        AND a **customer-friendly summary** (internals stripped). Reports written **next to
the video** by default
24        (override `report.output_dir`). Approved plan: `C:\Users\avidu\.claude\plans\drifting-
snuggling-squirrel.md` (read it to resume).
25      - **Status by piece:**
26        - ? **obsreport lib** built + on GitHub **(PRIVATE)** github.com/avidullu/sports-
obsreport (main + tags v0.1.0?v0.1.3).
27          Editable-installed into Python 3.13. ~74 lib tests green. Modules:
envelope/spec/rules/render/io/recorder/diff/schema.
28        - ? **Indexer Phase A** ? **PR #41 OPEN** (`feat/observability-reports`, rebased onto
#39 typed-config `backend/config/`).
29          `backend/reporting/` (soft-import, never-raises), wired CLI+API. My reporting: **226
tests pass.**
30          ?? **2 PRE-EXISTING red tests from #39** (NOT mine, left untouched):
`test_config.py::test_load_default_config`
31          + `test_save_default_config_roundtrip` (Path==str on Windows).
32        - ? **Collector Phase B** ? **PR #11 OPEN** (`feat/observability-reports`,
observability-ONLY). `src/reporting/`
33          (adapter Issue?Flag + recorders), wired curate validate-delivery/convert-
cvat/import-local. **112 tests pass.**
34        - ? **Collector parser zero-start fix** ? **PR #10 MERGED** to master (`99dfa22`).
Separate from the reporting PR;
35          DUP RISK resolved (fix landed once; PR #11 carries no parser fix). See [[current-
state]].
36      - **?? NEXT: Phase D ? debuggability eval** (ultracode Workflow: fresh-reader agents
over broken-run reports ? refine
37        wording; render a report to an image + eyeball) + committed golden tests (faq_ref-
resolution + flag-set) in both repos.
```

```
38
39      ## Standing project context (rally-detection stack ? stable, mostly merged; PARKED
pending golden data)
40      - WASB shuttle-trajectory substrate is resumable/checkpointed; on top = rally layer
(windowing thresholds +
41        net-crossing "gate A" + distilled classifier `distill_local`); eval harness
`calibrate_wasb`/`calibration.py` (LOVO).
42      - **Honest core finding:** WASB-only threshold tuning **ceilings ~F1 0.44**; learned
distilled model **underperforms
43        baseline at n=2 videos**. Blocker = **golden data + WASB candidate recall**, not the
method.
44      - **Audio E1 ran ? classical-onset = ~2.2× discrimination ceiling, NOT adopted** (band-
pass didn't help; detector kept,
45        indexer PR #35 merged). Gains/losses + reusable lessons in [[audio-e1-findings]]. Next
audio path = E2 learned timbre (needs labeled hits).
46      - **Golden data path:** owner self-rates clips in the standalone **VLC annotator**
(github.com/avidullu/rally-annotator,
47        MIT, multi-sport) ? CSV ? in-repo `--labels` loaders / collector `import-local`.
48
49      ## Key decisions (don't relitigate without reason)
50      - **Observability:** always-on; reports next-to-video; technical + customer views from
ONE typed envelope; shared
51        schema via the `obsreport` lib (reusable config language); build-backend MUST be
`setuptools.build_meta` (underscore).
52      - **Strategy (ADR-007):** Gemini = offline teacher only; **AUDIO** was the next local
cue, **fused in OUR rally layer**
53        (WASB stays a frozen vision-only detector); **pose/stance parked #2**; **Option B**
(one OWNED end-to-end multimodal
54        model) = eventual end-state ? modular cues are how we *earn* it (each generates the
labels it needs).
55      - **Local-only inference = privacy lever; permissive licensing only (ADR-005); distilled
> threshold tuning but needs
56        more videos (n=2 overfits); per-video adaptation = calibration on held-out, never mix
train/eval.**
57      - **Working style:** frequent **small PRs** + continuous [[current-state]] updates (this
handoff auto-refreshed at checkpoints).
58
59      ## Ramp-up kit (read these to get current)
60      - [[current-state]] (live blow-by-blow ? read FIRST), this handoff, [[working-agreement-
checkpoints]],
61        [[working-agreement-attribution]], [[feedback-tail-long-job-logs]],
[[audio-e1-findings]].
62      - **Observability plan:** `C:\Users\avidu\.claude\plans\drifting-snuggling-squirrel.md`.
Lib at `C:\Users\avidu\Projects\sports-obsreport\`.
63      - **CODE/architecture:** `docs/CODE_MAP.md` FIRST (dense map). See [[code-map-
artifact]].
64      - **Strategy/decisions:** `docs/DECISIONS.md` (esp. **ADR-007** audio),
`docs/RALLY_DETECTION_RESEARCH.md`,
65        `docs/AUDIO_RALLY_DETECTION_PLAN.md`, `docs/VIDEO_RECORDING_GUIDELINES.md`.
66      - **Key code:** `backend/reporting/` (indexer) + `src/reporting/` (collector);
`backend/eval/{distill_local,
67        calibrate_local,calibrate_wasb,gemini_refine,metrics,calibration}.py`;
`backend/detectors/{wasb_infer,wasb_runner,rally_gate,tracknet_runner}.py`.
68      - Memory notes: golden-set-vendoring, collector-indexer-bridge, candidate-segments-
finetuning, sibling-internship-repo, monetization-plan.
69
```

624. user (2026-06-07T08:58:01.334Z)

```
The file c:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\session-handoff.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

625. assistant (2026-06-07T08:58:39.150Z)

Carry-forward locked in ? ? both `current-state.md` (FEATURE COMPLETE checkpoint) and `session-handoff.md` (rewritten to the shipped state + the low-regret/Grok context + ramp-up kit) are current. A fresh Claude can resume from those alone.

Clean slate

- **Indexer:** `master` (#41/#42/#43 all merged), 0 uncommitted, merged branch pruned.
- **Collector:** `master` (#10/#11 merged), 0 uncommitted; kept `feat/collector-report-refinements` (that's the open **#12**).

Low-regret plan assessment ? for Grok

Verdict: yes, it's a genuinely good starter list. The philosophy (small reviewable PRs, each traceable to a concrete pain, payoff within a session) and the sequencing are sound ? and "Config-first, then atomic restructure" was exactly right (it gave my reporting work a clean window; the rebase onto #40 was conflict-free). Items **1 (Config #39) and 2 (restructure #40) are already done.**

Quick read on the remaining 3?9, with where my work touches them:

#	Item	As a Grok starter	Intersect with my observability work
3	Error/Result contract	? great, safe-parallel	**Synergy** ? today my report *infers* failures (e.g. `0 segs + no key`). If providers/segmenters return a structured Result instead of a silent `[]`, the report can read the real reason as a first-class flag. Worth coordinating.
4	Logging + kill stray `print()`	? great, pure hygiene	~none. Reports are artifacts; logs are the stream. (CODE_MAP flags `print()` in compiler/motion.)
5	Packaging (`pyproject` + entry points)	? good	minor ? model it on `obsreport`'s pyproject; when adding the `obsreport` dep, use the `git+ssh@tag` pin. (I already fixed the collector's `build-backend` typo.)
6	Detector runner abstraction (de-risk WSL)	? good, but **meatier/architectural** ? more "main project" than "starter"	low
7	Unify the two hybrid segmenters	? good, medium	**Synergy** ? they're copy-paste twins (CODE_MAP). Unify first, then there's *one* place to thread the **real AI-provider timing** I deferred (currently `not_captured`). I'll add that recorder hook as part of / right after #7.
8	Input-Quality UX (surface validator + audio readiness)	?? **largely subsumed by my reports**	**Biggest overlap.** The report already surfaces validation stages + flags + verdict. Grok should *extend the report* (add an audio-readiness stage/flag to the obsreport spec), **not** build a parallel mechanism.
9	Async job model (stretch)	? fine as a stretch	each per-run report slots in naturally as a per-job artifact

Your instinct is right ? most of the intersect is hygiene/"syntactic sugar" (4, 5) or pure synergy (3). The two to flag explicitly so we don't collide or double-build:
- **#8 is mostly already done** by the observability feature ? reframe it as "extend the report," not a new surface.
- **#7 should land before/with the AI-timing capture** ? I'll pick that up there rather than touching the twins now.

Net: hand Grok 3 ? 4 ? 5 as the safe quick wins (near-zero conflict with me), then 6/7 as the de-risking work (coordinate on 7), and rewrite 8 to build on the report. Want me to drop these coordination notes into `LOW_REGRET_MOVES_PLAN.md` / `BACKLOG.md` so Grok sees them in-repo?

626. user (2026-06-07T08:58:40.903Z)

#12 is also merged so I hope we are good? Any other pending items for you ? The starter project is complete? Can I see a sample report which the user (without debug info) would see?

627. assistant (2026-06-07T08:59:03.349Z)

Let me sync the collector (now that #12 is merged), confirm both repos are at a clean slate, then I'll show you the customer-facing report.

628. user (2026-06-07T08:59:11.520Z)

```
Already on 'master'
Deleted branch feat/collector-report-refinements (was b8d40c8).
=== collector master head ===
0e4ccc7 Merge pull request #12 from avidullu/feat/collector-report-refinements
b8d40c8 feat(reporting): delivery-report debuggability refinements + faq-resolution golden test
=== #12 state ===
MERGED
=== branches / clean ===
* master
uncommitted: 0

=== INDEXER clean check ===
* master
uncommitted: 0
```

629. assistant (2026-06-07T08:59:31.928Z)

Both repos clean (`master` only, 0 uncommitted), and **all PRs merged** (#10?#12 collector, #41?#43 indexer). Now here's exactly what an end user sees ? the customer-facing summary for a **successful** run (no system internals):

630. user (2026-06-07T08:59:37.786Z)

Badminton Highlight Indexer ? Video Report (summary)

Run

Field	Value
Video	Adarsh and Vasanth.mp4
Report generated	2026-06-07T08:59:37Z

Video

Field	Value
Duration	11m16s
Resolution	1920x1080
Frame rate	59.94 fps

Highlights

- **12** highlights
- **4m11s** of play
- **37.2%** of the video is highlights

Things to know

Nothing needs your attention.

Verdict

? Your highlights are ready.

631. assistant (2026-06-07T09:00:11.365Z)

That's the complete end-user view above ?? ? **no codes, no stages, no internals**: just the video's metadata, what was produced (12 highlights · 4m11s of play · 37% of the video), a plain "nothing needs your attention," and **"? Your highlights are ready."** (When something goes wrong, that same report instead shows a gentle line under *Things to know* + *"?? We couldn't finish generating highlights? please contact support"* ? never a stack trace or an internal flag

code.)

Are we good? ? Yes. ?

- **All PRs merged:** indexer #41/#42/#43, collector #10/#11/#12. Both repos clean on `master`, branches pruned, 0 uncommitted. `obsreport` on GitHub (private, v0.1.3).

Is the starter project complete? ? Yes.

Everything you asked for shipped:
- ? Rich per-video observability in **both** tools (indexer + collector) ? every run emits a machine JSON + a **technical** debuggable report + a **customer-friendly** summary, written next to the video.
- ? Catches inconsistencies / silent failures (it literally surfaced the config bug ? #42) and is diffable across runs/tools.
- ? Debuggability bar met ? a 15-agent eval confirmed a README+FAQ-only reader can self-diagnose all 7 broken-run cases; refinements + golden faq-resolution tests locked it in.
- ? The customer report (shown above).
- ? Reusable shared "report config language" lib in its own repo.

Pending items for me? ? None.

Nothing is blocked or half-done. A few **optional** future opportunities (not pending ? flagging for when you want them):
1. **`obsreport` git+ssh pin** in both repos' deps (it's a soft/commented dep today ? works on your machine via the editable install; pin to the tag once SSH/CI access is set up).
2. **Real AI-provider timing** in the report (currently `not_captured`) ? cleanest to land alongside Grok's item #7 (unify the hybrid segmenters).
3. **HTML customer-report renderer** + **audio-readiness signal** (Grok's reframed item #8) in `obsreport`.
4. **`docs/CODE_MAP.md` is stale** ? it still references pre-#40 paths (`backend/validators`, `backend/indexer`, `backend/database`); worth regenerating after the restructure.

Two quick offers if useful: I can (a) drop the Grok coordination notes into `LOW_REGRET_MOVES_PLAN.md` / `BACKLOG.md` in-repo, and/or (b) regenerate `CODE_MAP.md` for the new layout. Otherwise ? the starter project is done, and the handoff/current-state are fully checkpointed for a clean resume. ?

632. user (2026-06-07T09:07:23.161Z)

pls do (a) (b) .. and also can you add a neat run_all_tests in this repo so that grok can easily find and runn full test suites before sending PRs

633. assistant (2026-06-07T09:08:48.380Z)

On it ? (a) coordination notes, (b) CODE_MAP regen, and (c) a discoverable `run_all_tests`. Since (b) is the substantive piece and the CODE_MAP must be accurate against the restructured tree, I'll regenerate it with a multi-agent workflow (parallel subsystem readers ? synthesize ? adversarially verify every anchor), and do (a)/(c) directly while it runs.

First, branch + launch the CODE_MAP regeneration workflow:

634. user (2026-06-07T09:08:50.280Z)

```
Switched to a new branch 'chore/grok-handoff-codemap-runner'
=== current backend layout (for the workflow) ===
backend
backend/__pycache__
backend/annotations
backend/annotations/__pycache__
backend/api
backend/api/static
backend/audio
backend/audio/__pycache__
backend/config
backend/config/__pycache__
```

```
backend/detectors
backend/detectors/__pycache__
backend/eval
backend/eval/__pycache__
backend/eval/audio
backend/indexer
backend/indexer/__pycache__
backend/infrastructure
backend/infrastructure/__pycache__
backend/pipeline
backend/pipeline/audio
backend/pipeline/detectors
backend/pipeline/segmenters
backend/pipeline/stitcher
backend/pipeline/validators
backend/providers
backend/providers/__pycache__
backend/reporting
backend/reporting/__pycache__
backend/sports
backend/sports/__pycache__
backend/stitcher
backend/stitcher/__pycache__
backend/tools
backend/tools/__pycache__
backend/utils
backend/utils/__pycache__
backend/validators
backend/validators/__pycache__
```

635. assistant (2026-06-07T09:09:20.939Z)

Both old and new dirs appear, but the old ones are just stale `__pycache__` (the `.py` files moved in #40). Let me get the authoritative **git-tracked** layout so the workflow maps reality, not leftovers:

636. user (2026-06-07T09:09:22.582Z)

```
=== tracked python files (backend + root + tests) ===
backend/annotations/__init__.py
backend/annotations/automated_provider.py
backend/annotations/base.py
backend/annotations/file_provider.py
backend/config/__init__.py
backend/config/loader.py
backend/config/models.py
backend/eval/__init__.py
backend/eval/annotations.py
backend/eval/audio/__init__.py
backend/eval/audio/e1_probe.py
backend/eval/batch.py
backend/eval/calibrate_local.py
backend/eval/calibrate_wasb.py
backend/eval/calibration.py
backend/eval/classifier.py
backend/eval/distill_local.py
backend/eval/export_wasb_segments.py
backend/eval/gemini_refine.py
backend/eval/harness.py
backend/eval/metrics.py
backend/eval/regression.py
backend/eval/training.py
backend/infrastructure/database.py
backend/main.py
backend/pipeline/audio/__init__.py
```

backend/pipeline/audio/hit_detector.py
backend/pipeline/detectors/__init__.py
backend/pipeline/detectors/rally_gate.py
backend/pipeline/detectors/tracknet_runner.py
backend/pipeline/detectors/wasb_infer.py
backend/pipeline/detectors/wasb_runner.py
backend/pipeline/segmenters/__init__.py
backend/pipeline/segmenters/base.py
backend/pipeline/segmenters/gemini.py
backend/pipeline/segmenters/motion.py
backend/pipeline/segmenters/tracknet_hybrid.py
backend/pipeline/segmenters/wasb_hybrid.py
backend/pipeline/segmenters/yolo_hybrid.py
backend/pipeline/stitcher/compiler.py
backend/pipeline/validators/__init__.py
backend/pipeline/validators/base.py
backend/pipeline/validators/blur.py
backend/pipeline/validators/format.py
backend/pipeline/validators/pts_continuity.py
backend/pipeline/validators/relevance.py
backend/pipeline/validators/resolution_fps.py
backend/pipeline/validators/scene_cut.py
backend/providers/__init__.py
backend/providers/base.py
backend/providers/gemini.py
backend/reporting/__init__.py
backend/reporting/harvest.py
backend/sports/__init__.py
backend/sports/badminton.py
backend/sports/base.py
backend/tools/__init__.py
backend/tools/rally_slicer.py
backend/utils/__init__.py
backend/utils/geometry.py
backend/utils/video.py
run_eval.py
run_phase3_eval.py
run_process_and_stitch.py
run_regression.py
scratch/analyze_motion.py
scratch/clean_state.py
scratch/diagnose_rally.py
scratch/gemini_video_debugger.py
scratch/list_models.py
scratch/test_ai_segmenter.py
scratch/test_thresholds.py
scripts/generate_khelacharya_flyer.py
setup.py
tests/__init__.py
tests/test_annotation_providers.py
tests/test_automated_provider.py
tests/test_base.py
tests/test_calibrate_local.py
tests/test_calibrate_wasb.py
tests/test_calibration.py
tests/test_config.py
tests/test_database.py
tests/test_distill_local.py
tests/test_eval_annotations.py
tests/test_eval_batch.py
tests/test_eval_classifier.py
tests/test_eval_harness.py
tests/test_eval_metrics.py
tests/test_eval_regression.py
tests/test_eval_training.py

```
tests/test_export_wasb_segments.py
tests/test_gemini.py
tests/test_gemini_refine.py
tests/test_geometry.py
tests/test_hit_detector.py
tests/test_provider_gemini.py
tests/test_providers.py
tests/test_rally_gate.py
tests/test_rally_slicer.py
tests/test_reporting_harvest.py
tests/test_reporting_spec_faq.py
tests/test_sport_profiles.py
tests/test_tracknet_hybrid.py
tests/test_tracknet_runner.py
tests/test_validators_base.py
tests/test_video.py
tests/test_wasb_cache.py
tests/test_yolo_hybrid.py
```

637. assistant (2026-06-07T09:10:14.951Z)

I have the authoritative layout. Launching the CODE_MAP regeneration workflow (8 parallel subsystem readers ? synthesize against the old map ? adversarial anchor verification ? fix), running in the background:

638. user (2026-06-07T09:10:30.777Z)

```
Workflow launched in background. Task ID: wrtpil6w6
Summary: Regenerate docs/CODE_MAP.md for the layered restructure (#40) + config/reporting
subsystems
Transcript dir: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\56b7f849-5d56-4770-93e0-b3b4da943700\subagents\workflows\wf_d88615a4-f56
Script file: C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\56b7f849-5d56-4770-93e0-b3b4da943700\workflows\scripts\regen-code-map-wf_d88615a4-f56.js
(Edit this file with Write/Edit and re-invoke Workflow with {scriptPath:
"C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\56b7f849-5d56-4770-93e0-b3b4da943700\workflows\scripts\regen-code-map-
wf_d88615a4-f56.js"} to iterate without resending the script.)
Run ID: wf_d88615a4-f56
To resume after editing the script: Workflow({scriptPath: "C:\Users\avidu\.claude\projects\C--
Users-avidu-Projects-badminton-highlight-
indexer\56b7f849-5d56-4770-93e0-b3b4da943700\workflows\scripts\regen-code-map-
wf_d88615a4-f56.js", resumeFromRunId: "wf_d88615a4-f56"}) ? completed agents return cached
results.

You will be notified when it completes. Use /workflows to watch live progress.
```

639. assistant (2026-06-07T09:10:52.900Z)

Workflow running. While it regenerates the map, let me do **(c)** ? a discoverable full-suite runner at the repo root (matching the `run_*.py` convention):

640. assistant (2026-06-07T09:10:58.494Z)

```
#!/usr/bin/env python
"""Run the FULL test suite for the badminton-highlight-indexer.

A single, discoverable entry point so you (or a reviewer / CI) can verify the
whole suite before opening a PR:

    python run_all_tests.py             # full suite, quiet
    python run_all_tests.py -v          # verbose (passthrough to pytest)
    python run_all_tests.py -k config   # filter by keyword (passthrough)
    python run_all_tests.py --cov=backend --cov-report=term-missing   # coverage
```

```
Anything after the script name is passed straight through to pytest.

Notes
-----
- The suite is fully mocked (no GPU, no network, no WSL) and runs offline.
- The observability **reporting** tests skip cleanly if the optional ``obsreport``
  package isn't installed (they use ``pytest.importorskip``). To exercise them too:
      pip install -e ../sports-obsreport --no-build-isolation
- Exit code is pytest's own, so this is CI / pre-PR friendly (0 = all green).
"""
from __future__ import annotations

import importlib.util
import subprocess
import sys


def _installed(module: str) -> bool:
    try:
        return importlib.util.find_spec(module) is not None
    except (ImportError, ValueError):
        return False


def main() -> int:
    args = list(sys.argv[1:])
    # Default to quiet unless the caller chose a verbosity.
    if not any(a in ("-q", "-v", "-vv", "-vvv", "--quiet", "--verbose") for a in args):
        args = ["-q", *args]
    cmd = [sys.executable, "-m", "pytest", *args]

    obs = "available" if _installed("obsreport") else "NOT installed ? reporting tests will
SKIP"
    bar = "=" * 72
    print(bar)
    print(" Full test suite:  " + " ".join(cmd))
    print(f" obsreport (optional reporting dep): {obs}")
    print(bar, flush=True)

    rc = subprocess.run(cmd).returncode

    print(bar)
    print(" RESULT: " + ("PASS - all tests green" if rc == 0 else f"FAIL - pytest exit {rc}"))
    print(bar)
    return rc


if __name__ == "__main__":
    raise SystemExit(main())
```

641. user (2026-06-07T09:10:58.505Z)

```
File created successfully at: C:\Users\avidu\Projects\badminton-highlight-
indexer\run_all_tests.py (file state is current in your context ? no need to Read it back)
```

642. assistant (2026-06-07T09:11:07.207Z)

Let me add a README pointer to it. Finding the test section:

643. user (2026-06-07T09:11:08.368Z)

```
288:## ? Running the Test Suite
292:pip install pytest pytest-cov --user
293:pytest                              # run everything
```

```
294:pytest tests/test_yolo_hybrid.py    # run a single file
295:pytest -k yolo --maxfail=1 -v       # filter by keyword, fail fast, verbose
296:pytest --cov=backend --cov-report=term-missing   # coverage report (uses .coveragerc)
```

644. user (2026-06-07T09:11:14.896Z)

```
288     ## ? Running the Test Suite
289     The test suite uses `pytest` and lives in `tests/`. From the project root:
290
291     ```bash
292     pip install pytest pytest-cov --user
293     pytest                              # run everything
294     pytest tests/test_yolo_hybrid.py    # run a single file
295     pytest -k yolo --maxfail=1 -v       # filter by keyword, fail fast, verbose
296     pytest --cov=backend --cov-report=term-missing   # coverage report (uses .coveragerc)
297     ```
298
299     The suite mocks external dependencies (the Gemini client, ffprobe/ffmpeg, OpenCV
`VideoCapture`, the YOLO model) so it runs offline and without a GPU. The tests cover:
```

645. user (2026-06-07T09:11:25.972Z)

```
The file C:\Users\avidu\Projects\badminton-highlight-indexer\README.md has been updated
successfully. (file state is current in your context ? no need to Read it back)
```

646. assistant (2026-06-07T09:11:32.682Z)

```
Now **(a)** ? the Grok coordination notes. Reading BACKLOG to append in its style:
```

647. user (2026-06-07T09:11:33.258Z)

```
1       # Backlog ? deferred items to revisit
2
3       Single tracker for work we consciously deferred, so nothing gets lost between sessions.
Each item records **what**, **why it's deferred**, and **the trigger to revisit**. Every entry
traces to a real decision/PR/doc (no speculative work).
4
5       > Maintained as a version-controlled doc for now. Say the word and I'll mirror any/all
of these to **GitHub issues** instead (or in addition).
6       > Status legend: ? blocks something · ? do when triggered · ? nice-to-have/cleanup.
7
8       _Last updated: 2026-06-07._
9
10      ---
11
12      ## Monetization / licensing
13      *(source: `docs/MONETIZATION_AUDIT.md` §7, ADR-005)*
14
15      - ? **Ultralytics Enterprise pricing** ? get a quote **only if** the
torchvision/ByteTrack detector (PR #8) proves insufficient on real footage. Currently *not
needed* (swap done). Quote-only pricing. **Revisit:** if detector quality forces reconsidering
YOLOv8.
16      - ? **Codec patent thresholds (AVC/HEVC/AAC)** ? confirm exact Via-LA (AVC) / Access
Advance (HEVC) royalty rates & free thresholds **with legal counsel** before relying on the
"small SaaS is under threshold" reading. Mitigation already chosen: emit royalty-free AV1/Opus
output. **Revisit:** before commercial launch.
17      - ? **Per-checkpoint weight/dataset licenses** ? if we borrow pretrained weights (YOLOX,
ActionFormer, WASB, TrackNetV3), the *code* license ? the distributed *weights/dataset* terms.
Either train our own or confirm each checkpoint. **Revisit:** when adopting any borrowed
weights.
18      - ? **Real GPU $/video & latency** ? benchmark the three LLM-step options (offline
ActionFormer vs self-hosted Qwen2.5-VL-7B vs Gemini paid-tier) on the target VM before pricing
the product. **Revisit:** when choosing the production LLM step.
19
```

Detector swap follow-ups
(source: PR #8, ADR-005)

- ? **`supervision` ByteTrack deprecation** ? `sv.ByteTrack` is deprecated as of supervision 0.28.0 and **removed in 0.30.0**; `requirements.txt` is pinned `<0.30.0` as a stopgap. Migrate to `ByteTrackTracker` from the separate `trackers` package before lifting the cap. **Revisit:** before bumping supervision past 0.30.
- ? **Real-video validation of the swap** ? the torchvision + ByteTrack Stage-1 is unit-tested with mocks but **not yet validated on actual footage** for detection/tracking quality. (ADR-004 separately found player pixel-motion is a weak rally proxy on broadcast video, so expect tuning.) **Revisit:** when a real test clip + the pipeline are run together.
- ? **`yolo_hybrid` naming** ? the registry key, `yolo_*` config keys, and `yolo-hybrid-*` DB detector tags keep the "yolo" name for back-compat despite no YOLO remaining. Optional rename. **Revisit:** if/when a clean break is worth the migration.

Golden-set subproject
(source: `docs/GOLDEN_SET_IMPLEMENTATION.md`, `docs/GOLDEN_SET_VENDORING.md`)

- ? **Annotation guideline** ? **DONE (2026-06-02):** authored as the Roora brief + illustrated guide in the collector repo `docs/annotation/` ? rally CSV schema; ignore dead time / include every rally (incl. faults & lets) / shot = one contact; `ending_reason` (forced/unforced-error taxonomy, shared across net-separated sports); singles & doubles for badminton & pickleball.
- ? **Internal gold key** ? hand-annotate 5?10 licensed clips to near-perfect as the vendor answer key. **Unblocked.** **Revisit:** anytime.
- ? **GS-3 ? `HumanVendorProvider`** ? concrete adapter for the chosen annotation vendor (submit/poll/fetch + DPA/secure-link controls). **Vendor selected for the initial pilot: Roora** (Bengaluru; iMerit / Taskmonk / TELUS remain the fallback shortlist). **Revisit:** once the Roora pilot confirms quality, then build the adapter.
- ? **GS-4 oracle model** ? `AutomatedProvider` interface is stubbed (this work) with a deterministic fake. Wire a *concrete* fast-tier auto-labeler later. **Oracle rule:** it MUST be a different model lineage than production, or the regression test is blind to shared failures. **Revisit:** when a production LLM step is chosen (pick the oracle's lineage to differ).
- ? **GS-5 ? regression triggers** ? wire `regress()` into CI (pre-merge gate), on-demand CLI (the "surprise" case), and a scheduled run; include a deliberate re-baseline path. **Revisit:** after GS-4.
- ? **Vendor pilot (Roora)** ? run the paid pilot with **Roora** as a **single-vendor pilot first** (not a multi-vendor bake-off): assess quality-per-rupee against the §4 bar; escalate to the iMerit/Taskmonk fallback only if it misses. **Revisit:** the pilot doubles as calibration once the gold key exists.
- ? **Phase-1 video collection** ? gather the round-1 golden corpus per `docs/GOLDEN_SET_PHASE1_VIDEOS.md`: **static self-recorded** footage (mirror deployment, not broadcast), 3 sports incl. singles & doubles, across the diversity matrix; ~12 clips/sport with **? held out by match**. **Unblocked.**
- ? **Train/eval separation guardrails** ? train & eval videos live in **separate catalogs** today, kept apart by convention only. Add **hard guardrails** (permissions / explicit approval / re-confirmation at ingest & export) so a video can **never** land in both train and eval. **Revisit:** dedicated design session with the user.

Recording best practices
(source: `docs/VIDEO_RECORDING_GUIDELINES.md`)

- ? **Validate recording hypotheses** ? empirically test which capture conditions (resolution, fps, camera angle/height, lighting) actually move precision/recall, then promote the winners into user-facing "how to record" guidance. Only the static-camera rule is currently evidence-backed (ADR-004). **Revisit:** once a controlled corpus + the eval loop can measure per-condition deltas.

Code & test organization
(source: owner review comments on PR #35, 2026-06-07)

- ? **`tests/` reorganization to reduce fragmentation** ? owner wants to isolate test components on a grouping
  metric (by subsystem/component) so `tests/` is less fragmented, mirroring the new `backend/audio/` +

```
49        `backend/eval/audio/` layout (e.g. audio tests grouped under `tests/audio/`, eval
tests under `tests/eval/`).
50        The exact grouping **metric is the owner's design choice** (consolidate small files
vs. add per-area dirs ? the
51        "lesser fragmentation" goal slightly tensions with adding dirs), so this is left as a
**dedicated refactor PR**,
52        not guessed here. **Revisit:** when the owner specifies the grouping scheme. (The
`backend/eval/audio/` half of
53        the request ? comment 1 ? was done in PR #35.)
54
55      ## Commercial Readiness & Low-Regret Moves ? Approved Decisions Checkpoint (2026-06-07)
56
57      *(source: Commercial Readiness Review + structure/sequencing discussion with Grok as
second-in-command)*
58
59      - ? **Docs-only checkpoint PR** ? This update + the refreshed
`docs/LOW_REGRET_MOVES_PLAN.md` formally record the approved path before any code changes. The
initial review docs (PR #37) introduced the findings; this entry locks the execution decisions.
60      - ? **Approved directory structure** ? Adopt a layered approach under `backend/` for
professionalism and future growth (no flat dumping of new modules, no explosion of 15+ single-
file dirs):
61        - `backend/config/` (package ? start with this)
62        - `backend/api/` (FastAPI app, routes, static UI)
63        - `backend/pipeline/` (validators + segmenters + stitcher + audio + detectors)
64        - `backend/infrastructure/` (database + external adapters)
65        - Keep `backend/eval/` prominent as a first-class concern.
66        - Logical grouping for providers, sports, annotations, utils, and a small `core/` only
when needed.
67      - ? **Sequencing (Config first, then atomic refactor)** ? Implement Deliverable 1
(Config) **first**, introducing the `backend/config/` package immediately. Perform the full
directory structure refactor as a **separate high-pri atomic PR** afterwards. This minimizes
conflicts, keeps PRs focused, and gives Claude a clean window with no concurrent moves.
68      - ? **Deliverable 1 (Config)** ? `backend/config/` package (`models.py` + `loader.py`)
instead of a single `config.py`. Typed `AppConfig`, central loader with built-in defaults + file
override, `extra="allow"` for migration. Update `setup.py` + `main.py`. Add tests. Land this
before the layout move.
69      - ? **Subsequent items** ? See the refreshed `LOW_REGRET_MOVES_PLAN.md` for the full
ordered list (Error contracts, logging, packaging, detector abstraction, etc.). The structure
refactor is now explicitly tracked as its own high-pri step right after Config.
70
71      **Status legend reminder:** ? = approved / in progress tracking, ? = do when triggered.
72
```

648. assistant (2026-06-07T09:12:02.414Z)

Appending the coordination notes to both docs ? BACKLOG first:

649. user (2026-06-07T09:12:10.545Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\BACKLOG.md has been updated successfully. (file state is current in your context ? no need to Read it back)

650. user (2026-06-07T09:12:32.187Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\LOW_REGRET_MOVES_PLAN.md has been updated successfully. (file state is current in your context ? no need to Read it back)

651. assistant (2026-06-07T09:12:39.878Z)

Let me verify `run_all_tests.py` works end-to-end (it also confirms the branch is green):

652. user (2026-06-07T09:12:54.366Z)

```
======================================================================
```

```
 Full test suite:  C:\Users\avidu\AppData\Local\Programs\Python\Python313\python.exe -m pytest
-q
 obsreport (optional reporting dep): available
======================================================================
..................................................................... [ 31%]
..................................................................... [ 62%]
..................................................................... [ 93%]
...............                                                      [100%]
231 passed in 12.14s
======================================================================
 RESULT: PASS - all tests green
======================================================================
```

653. assistant (2026-06-07T09:13:12.563Z)

`run_all_tests.py` works ? clean banner (flags the optional `obsreport` dep), full suite **231 passed**, clear PASS verdict.

Status on your three asks:
- **(c) `run_all_tests.py`** ? ? discoverable root-level wrapper (matches the `run_*.py` convention), passes args through to pytest, reports PASS/FAIL with pytest's exit code, and flags whether the optional `obsreport` dep is present. README's test section now points to it.
- **(a) Grok coordination notes** ? ? added to both `docs/BACKLOG.md` and `docs/LOW_REGRET_MOVES_PLAN.md`: items 1?2 marked done; the key intersections flagged (item 8 = subsumed ? extend the report; item 7 = land before AI-timing capture; item 3 = Result-contract synergy; items 4/5/6 safe to do independently; pin the `obsreport` tag later).
- **(b) CODE_MAP regeneration** ? ? running as a background workflow now (8 parallel subsystem readers over the current layered tree ? synthesize against the old map ? adversarially verify every `@path::symbol` anchor ? fix). I'll write the regenerated `docs/CODE_MAP.md`, then commit all three and open a single PR once it lands.

I'll finalize and open the PR when the workflow completes ? sit tight.

654. user (2026-06-07T09:13:14.290Z)

pls merge all the PRs by yourelf and I will resume later. Thanks for maing this more user friendly!

655. assistant (2026-06-07T09:13:37.223Z)

Thank you ? glad it's more debuggable now! Let me finish and merge. First, the CODE_MAP regeneration workflow needs to be done before I can open/merge that last PR ? let me check its status:

656. user (2026-06-07T09:13:41.173Z)

```
Exit code 1
=== workflow COMPLETE ===
Traceback (most recent call last):
  File "<string>", line 1, in <module>
    import json;d=json.load(open(r'C:/Users/avidu/AppData/Local/Temp/claude/C--Users-avidu-
Projects-badminton-highlight-indexer/56b7f849-5d56-4770-93e0-
b3b4da943700/tasks/wrtpil6w6.output'));print('keys:',list(d.get('result',{}).keys()));v=d['resul
t'].get('verify',{});print('verify ok:',v.get('ok'),'problems:',len(v.get('problems',[])));cm=d[
'result'].get('codemap','');print('codemap chars:',len(cm));print('starts:',cm[:60])
                  ~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\json\__init__.py", line 298,
in load
    return loads(fp.read(),
        cls=cls, object_hook=object_hook,
        parse_float=parse_float, parse_int=parse_int,
        parse_constant=parse_constant, object_pairs_hook=object_pairs_hook, **kw)
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\json\__init__.py", line 352,
```

```
in loads
    return _default_decoder.decode(s)
           ~~~~~~~~~~~~~~~~~~~~~~~~^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\json\decoder.py", line 345,
in decode
    obj, end = self.raw_decode(s, idx=_w(s, 0).end())
               ~~~~~~~~~~~~~~~^^^^^^^^^^^^^^^^^^^^^^
  File "C:\Users\avidu\AppData\Local\Programs\Python\Python313\Lib\json\decoder.py", line 363,
in raw_decode
    raise JSONDecodeError("Expecting value", s, err.value) from None
json.decoder.JSONDecodeError: Expecting value: line 1 column 1 (char 0)
```

657. assistant (2026-06-07T09:13:57.282Z)

The output file is empty ? the workflow is still running (output not yet written). Let me check its live progress:

658. user (2026-06-07T09:13:57.852Z)

```
output file size: 0 bytes
=== subagent transcripts (progress) ===
agent-a3139ab4edb6a20a5.jsonl
agent-a3139ab4edb6a20a5.meta.json
agent-a46cae3f0f615dfc3.jsonl
agent-a46cae3f0f615dfc3.meta.json
agent-a56929604f77b9d41.jsonl
agent-a56929604f77b9d41.meta.json
agent-a6029042d6f6bbf07.jsonl
agent-a6029042d6f6bbf07.meta.json
agent-a646f11db19db3854.jsonl
agent-a646f11db19db3854.meta.json
agent-a98897151b5da076f.jsonl
agent-a98897151b5da076f.meta.json
agent-aac4a0e5db2cfe047.jsonl
agent-aac4a0e5db2cfe047.meta.json
agent-ab90acd72923cd2a5.jsonl
agent-ab90acd72923cd2a5.meta.json
agent-abb109c8dd5e01539.jsonl
agent-abb109c8dd5e01539.meta.json
journal.jsonl
```

659. assistant (2026-06-07T09:14:25.263Z)

The workflow is mid-run (8 readers done; synthesize/verify/fix still to go) ? it'll notify me when finished. All previously-opened PRs are already merged; the only remaining piece is this last docs+tooling PR, which I'll create and merge once the map lands. Let me confirm there are no other open PRs I should be aware of (so "merge all" is accurate):

660. user (2026-06-07T09:14:29.016Z)

```
=== INDEXER open PRs ===
=== COLLECTOR open PRs ===
=== OBSREPORT open PRs ===
```

661. assistant (2026-06-07T09:14:48.306Z)

Confirmed ? **zero open PRs** across all three repos; everything to date is already merged. The only outstanding item is the final docs+tooling PR (regenerated `CODE_MAP.md` + `run_all_tests.py` + the Grok notes), and its CODE_MAP regeneration workflow is still running.

Here's what I'll do **autonomously the moment the workflow finishes** (no need for you to be here):
1. Write the regenerated `docs/CODE_MAP.md`,
2. Commit (a)+(b)+(c) on the `chore/grok-handoff-codemap-runner` branch,

3. Run `run_all_tests.py` to confirm green,
4. Push, open the PR, and **merge it myself** (merge commit + delete branch), then fast-forward master,
5. Update the memory/handoff checkpoint.

After that, all work is merged and both repos are clean on `master` ? nothing left hanging. I'll report the final state here when it's done. Enjoy the break ? and thanks again! ?

662. user (2026-06-07T09:20:30.910Z)

<task-notification>
<task-id>wrtpil6w6</task-id>
<tool-use-id>toolu_01WebmnkMYMacbi5Jfu2iC1E</tool-use-id>
<output-file>C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\tasks\wrtpil6w6.output</output-file>
<status>completed</status>
<summary>Dynamic workflow "Regenerate docs/CODE_MAP.md for the layered restructure (#40) + config/reporting subsystems" completed</summary>
<result>{"codemap":"Now I have the full picture of the existing structure. Let me regenerate the complete CODE_MAP.md, applying the path migrations, new subsystems (config, reporting), refreshed test map, and corrected gotchas/ramp paths.\n\n# CODE_MAP.md\n\n**Purpose:** cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.\n\n**Regenerate:** rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).\n\n**LEGEND**\n- `@path` = file (always repo-relative)\n- `::sym` = symbol (class/func/const) in the nearest preceding `@path`\n- `->` = dataflow / call / \"produces\"\n- `§` = data contract (canonical shape)\n- `?` = gotcha / footgun\n- `?` = registry / plugin extension point\n- `WSL` = runs inside WSL2 conda env, NOT the Windows Python process\n\n---\n\n## 1. PIPELINE\n\n### 1A. Runtime: video file -> highlight reel\n```\ntyped config load (built-in defaults -> config.json overrides; absent/parse-fail -> all-defaults, never fatal)\n @backend/config/loader.py::load_config -> @backend/config/models.py::AppConfig\n -> video_id = MD5(whole file)   @backend/main.py::get_file_md5 (? chunk size varies per entrypoint: 64KB main, 1MB run_process, 8KB run_eval, 64KB phase3 ? all hash WHOLE file so values agree)\n -> validate @backend/pipeline/validators/base.py::registry.run_all_with_context(video_path, global_config)\n   FormatValidator -> ctx['ffprobe_meta'] -> ResolutionFPSValidator -> PTSContinuity -> SceneCut -> Blur -> Relevance   (? ACTUAL registration/exec order ? see @backend/pipeline/validators/__init__.py)\n   [validation FAIL -> persist status=\"failed\", return early; report STILL emitted in finally]\n -> db.add_video(video_id, filepath, status, [r.dict()])   @backend/infrastructure/database.py\n -> seg_name = req.segmenter or global_config.indexing.default_segmenter (fallback 'motion')\n -> seg = segmenter_registry.get(seg_name)(db, global_config)   ? @backend/pipeline/segmenters/base.py::segmenter_registry (400 + available list if missing)\n -> segments = seg.process_video(video_id, video_path, player_pool?, max_frames?)\n   [STOP POINT: segments-only ? predictions now in DB; eval/regression operate here without stitching]\n -> finally: emit_indexer_report(...)   @backend/reporting/__init__.py (? ALWAYS runs ? even on validation failure / exception; NEVER raises; grafts response[\"reports\"])\n -> select segment ids -> [(start,end)] (+ stitching padding)\n -> VideoCompiler.compile_highlights(filepath, time_pairs, out_name) @backend/pipeline/stitcher/compiler.py\n   per-clip ffmpeg re-encode (libx264/superfast/crf22) -> concat -c copy -> output/<name>.mp4\n```\n\nHybrid segmenter internal 4-phase (yolo_hybrid / tracknet_hybrid / wasb_hybrid ? same shape):\n```\nP1 chunk video (yolo only; tracknet/wasb = whole-video single pass, chunk_index=0)\n -> P2 candidate \"action windows\" from motion/trajectory [STOP: candidates persisted via db.add_candidate_segment]\n -> P3 pad clip (ai_handoff_padding) -> ThreadPoolExecutor -> provider.analyze_video(clip, prompt)   ? provider_registry\n -> P4 db.add_play_segment + db.link_candidate_to_segment\n```\n\nPure-AI path `gemini`: no P2; chunk-or-single -> provider -> heavy validate (clamp/dedup) -> add_play_segment.\nLegacy `motion`: pixel-diff -> segments + **fabricated** metadata/events (? not ground truth).\n\n### 1B. WASB shuttle substrate (P2 of wasb_hybrid)\n```\nWasbHybridSegmenter.process_video @backend/pipeline/segmenters/wasb_hybrid.py\n -> WasbRunner.healthcheck() (gate; not ok -> return [])   @backend/pipeline/detectors/wasb_runner.py\n ->

WasbRunner.run_predict(video, out_dir)\n   stage video + wasb_infer.py into WSL fs -> `wsl bash -c 'source conda.sh && python wasb_infer.py ...'`\n   -> @backend/pipeline/detectors/wasb_infer.py::run (WSL): sliding windows -> GPU detector pass (CHECKPOINTED det_raw.jsonl) -> CPU tracker (always full re-run) -> trajectory CSV\n -> TrackNetRunner.parse_trajectory_csv(csv)   @backend/pipeline/detectors/tracknet_runner.py (shared, re-exported by WasbRunner)\n -> TrackNetRunner.trajectory_to_action_windows(points, fps, frame_width, chunk_start, velocity_thresh, merge_gap, min_window_duration, chunk_index) [the model-agnostic WINDOWING BRAIN]\n```\n(`tracknet_hybrid` identical, swapping WasbRunner->TrackNetRunner; `_probe_fps_width` fallback (30.0, 1280.0).)\n\n### 1C. Calibration / eval flow (NO pipeline run unless told)\n```\nGT CSV -> annotations.load_annotations / calibrate_wasb.load_golden_gts -> List[Interval]\nDB predictions -> harness.get_predictions(db, video_id, stage) -> List[Interval]   stage ? {candidates, final}\n -> metrics.match_intervals (greedy temporal-IoU) -> metrics.score -> Score\n -> harness.evaluate -> EvalReport -> report_to_dict\n   -> batch.aggregate (corpus micro/macro)   @backend/eval/batch.py\n   -> regression.regress(_with_provider) vs baseline   @backend/eval/regression.py [CI gate]\nCalibration: calibrate_wasb.run -> calibration.{select_best, improvement_report, cross_validate} @backend/eval/calibration.py\nDistill Gemini->local: calibrate_local (thresholds) OR distill_local (learned classifier) @backend/eval/{calibrate_local,distill_local}.py\nLearned segmenter (offline): training.build_training_set(X,y) -> classifier.LogisticRegression.fit/save\nGemini refinement driver (tuning, standalone): gemini_refine.{refine_window,full_video_rallies} @backend/eval/gemini_refine.py\nAudio probe (diagnostic only, NOT in live pipeline): audio/e1_probe.run @backend/eval/audio/e1_probe.py\n```\nEntrypoints: `@run_eval.py` (single video score), `@run_phase3_eval.py` (manifest batch, can segment), `@run_regression.py` (baseline gate, exit 1=regression), `@backend/eval/calibrate_wasb.py` + `@backend/eval/export_wasb_segments.py` (WASB tuning drivers).\n\n---\n\n## 2. SUBSYSTEMS\n\n### 2.0 Orchestration & config\n- `@backend/main.py` ? Thin CLI + FastAPI bootstrap (orchestrator; modes via FLAGS). Static UI mounts `/static` from `backend/api/static`.\n - `::app` FastAPI; `::db_instance`,`::global_config` (`Optional[AppConfig]`) module globals set ONLY in `main()`. ? importing `app` for ASGI without `main()` -> both `None` -> every endpoint NPEs.\n - `::process_video` `@POST /api/process` -> validate -> add_video -> segment -> `finally:` always `emit_indexer_report` (never raises). `::compile_highlights` `@POST /api/compile`. `::query_play_segments` `@POST /api/query`. `::get_config` `@GET /api/config`. `::run_cli_diagnose_clip` `--debug-clip`: lazy `import cv2`, samples ±3s vs `indexing.motion_threshold` (dual model/dict read).\n - `::load_config` ? ? legacy thin wrapper returning a **dict** (`load_app_config(path).model_dump(...)`); kept for transition ? new code imports `load_app_config`/`AppConfig` directly. `::ProcessRequest`/`::QueryRequest`/`::CompileRequest` pydantic bodies.\n - ? `main()` runtime-override hack: `global_config._runtime_overrides = {\"output_dir\": args.output_dir}` ? output dir lives ONLY on this dynamic attr (works because `AppConfig` is `extra=\"allow\"`), NOT in any config model; `db_path = os.path.join(args.output_dir, output.db_name)`.\n - ? `compile_highlights` reads `getattr(global_config.output,\"output_dir\",\"output\")` but `OutputConfig` has **no `output_dir`** -> ALWAYS falls back to `\"output\"` (the `_runtime_overrides
... (truncated 49723 chars, full result in C:\Users\avidu\AppData\Local\Temp\claude\C--Users-avidu-Projects-badminton-highlight-indexer\56b7f849-5d56-4770-93e0-b3b4da943700\tasks\wrtpil6w6.output)</result>
<usage><agent_count>10</agent_count><subagent_tokens>649486</subagent_tokens><tool_uses>149</tool_uses><duration_ms>599969</duration_ms></usage>
</task-notification>

663. assistant (2026-06-07T09:21:06.759Z)

The CODE_MAP regeneration is done (and it even surfaced two real config-migration bugs as gotchas ? nice dogfood). The result has some agent preamble before the markdown; let me extract the clean map, write it, and sanity-check it (no stale paths, new subsystems present):

664. user (2026-06-07T09:21:14.251Z)

verify ok: True | problems: 0
written chars: 56432 | lines: 325
stale-path hits: 0 []
has backend/config: True | has backend/reporting: True | has backend/infrastructure: True | has run_all_tests: True

---- first 5 lines ----
CODE_MAP.md

Purpose: cold-start navigation map for the badminton-highlight-indexer ? a self-hosted video ? highlight-reel pipeline (CV/AI rally detection ? SQLite ? ffmpeg stitch) with a segmenter-agnostic eval/calibration stack, typed config, and per-video observability reporting.

Regenerate: rerun the `build-code-map` workflow (parallel subsystem readers ? this doc). Re-run after any registry/contract/WSL-interface change, or after a directory restructure (this revision follows #40).

665. assistant (2026-06-07T09:21:30.820Z)

CODE_MAP regenerated cleanly ? **verify passed (0 broken anchors)**, 325 lines, zero stale paths, includes config/reporting/infrastructure + `run_all_tests`. The regen also surfaced two real config-migration bugs worth flagging for Grok ? adding them to BACKLOG:

666. user (2026-06-07T09:21:34.821Z)

The file C:\Users\avidu\Projects\badminton-highlight-indexer\docs\BACKLOG.md has been updated successfully. (file state is current in your context ? no need to Read it back)

667. assistant (2026-06-07T09:21:46.851Z)

Now confirm green, then commit everything, push, open the PR, and merge it:

668. user (2026-06-07T09:22:05.229Z)

```
========================================================================
 RESULT: PASS - all tests green
========================================================================
=== staged files ===
warning: in the working copy of 'run_all_tests.py', LF will be replaced by CRLF the next time
Git touches it
M  README.md
M  docs/BACKLOG.md
M  docs/CODE_MAP.md
M  docs/LOW_REGRET_MOVES_PLAN.md
A  run_all_tests.py
remote:      https://github.com/avidullu/badminton-highlight-indexer/pull/new/chore/grok-
handoff-codemap-runner
remote:
pushed
```

669. assistant (2026-06-07T09:22:20.542Z)

Now open the PR and merge it (then sync master):

670. user (2026-06-07T09:22:34.123Z)

```
https://github.com/avidullu/badminton-highlight-indexer/pull/44
Switched to branch 'master'
=== master head ===
e03820a Merge pull request #44 from avidullu/chore/grok-handoff-codemap-runner
5b39068 docs+tooling: regenerate CODE_MAP for layered layout + run_all_tests.py + Grok low-
regret notes
=== clean? ===
* master
uncommitted: 0
```

671. assistant (2026-06-07T09:22:50.542Z)

PR #44 created and merged. Master is at `e03820a`, clean (0 uncommitted, branch deleted).

Final verification across all repos + then I'll checkpoint memory:

672. user (2026-06-07T09:22:56.326Z)

```
=== OPEN PRs (should be none anywhere) ===
badminton-highlight-indexer: 0 open
sports-data-collector: 0 open
sports-obsreport: 0 open
=== both repos clean on master? ===
badminton-highlight-indexer: master | uncommitted 0 | local branches: * master
sports-data-collector: master | uncommitted 0 | local branches: * master
```

673. assistant (2026-06-07T09:23:10.863Z)

All clean ? **0 open PRs anywhere; both repos on `master`, 0 uncommitted, no stray branches.**
Let me finalize the carry-forward memory:

674. user (2026-06-07T09:23:14.749Z)

The file c:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\memory\current-state.md has been updated successfully. (file state is current in your context ? no need to Read it back)

675. assistant (2026-06-07T09:23:46.898Z)

Now the handoff ? let me read its current state to fix the now-stale lines (#12 merged, #44 added, CODE_MAP regenerated):

676. user (2026-06-07T09:23:47.608Z)

```
19      ## You are here (2026-06-07) ? ? OBSERVABILITY-REPORTS feature SHIPPED (both tools + shared lib)
20      - **Shipped:** always-on **per-video observability reports** across BOTH tools, powered by the NEW shared lib
21        **`obsreport`** (own private repo `github.com/avidullu/sports-obsreport`, MIT, **v0.1.3**, editable-installed). Each
22        run writes next to the video (override `report.output_dir`): `*.report.json` (machine), `*.report.md` (TECHNICAL ?
23        stages + footgun flags, each citing a FAQ), `*.summary.md` (CUSTOMER ? metadata + meta-highlights, internals
24        stripped). **SOFT dependency** (no-op if absent; writes NEVER raise). Plan: `C:\Users\avidu\.claude\plans\drifting-snuggling-squirrel.md`.
25      - **MERGED to master:** indexer **#41** (observability) + **#42** (config runtime-bug fix: typed AppConfig broke
26        validators' nested `.get` ? `AppConfig.get` now returns nested sections as dicts) + **#43** (debuggability
27        refinements + faq-resolution golden test). Collector **#10** (parser zero-start fix) + **#11** (observability).
28      - **OPEN:** collector **#12** (`feat/collector-report-refinements` ? frame-level evidence + review-only framing +
29        golden test; green, awaiting merge). Both repos CLEAN on master; merged branches pruned. Suites: indexer **231**,
30        collector **116** (on #12), obsreport **74**.
31      - **Debuggability validated (stretch goal):** 15-agent Workflow ? all 7 broken-run scenarios self-debuggable by a
32        README+FAQ-only reader; eyeballed via rendered HTML screenshot + customer summaries. Golden faq-resolution tests lock it.
33      - **Parallel track (Grok) = `docs/LOW_REGRET_MOVES_PLAN.md`:** items **1 (Config pkg #39) + 2 (directory restructure
34        #40) DONE**; items 3?9 remain (good list ? assessment in [[current-state]]). Intersections: item **8 (Input-Quality
35        UX) largely subsumed by my reports**; **3 (Result contract)** + **7 (unify hybrid segmenters)** synergize (7 enables
36        the deferred real AI-timing capture, currently `not_captured`).
```

```
37      - **?? NEXT:** merge collector **#12**; then optional obsreport follow-ups (HTML
customer renderer; thread real
38        AI-provider timing via #7; pin obsreport `git+ssh` once SSH/CI access exists).
Original rally-detection modeling
39        still PARKED pending golden labels.
40
```

677. user (2026-06-07T09:24:06.486Z)

```
The file c:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\session-handoff.md has been updated successfully. (file state is current in your
context ? no need to Read it back)
```

678. assistant (2026-06-07T09:24:22.644Z)

```
Let me update the CODE_MAP memory pointer to record the #44 regeneration (provenance accuracy):
```

679. user (2026-06-07T09:24:25.881Z)

```
The file c:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-
indexer\memory\code-map-artifact.md has been updated successfully. (file state is current in
your context ? no need to Read it back)
```

680. assistant (2026-06-07T09:24:47.534Z)

```
All done ? everything's merged and both repos are pristine. ?
```

Final state

```
- **Every PR merged. Zero open PRs across all three repos.** Both repos on `master`, 0
uncommitted, no stray branches.
  - **Indexer:** #41 (observability) · #42 (config fix) · #43 (debuggability refinements) ·
**#44** (regenerated CODE_MAP + `run_all_tests.py` + Grok notes ? created & merged just now)
  - **Collector:** #10 (parser fix) · #11 (observability) · #12 (delivery-report refinements)
  - **obsreport:** on GitHub (private, v0.1.3)
- **Suites green:** indexer 231 · collector 116 · obsreport 74 ? all runnable via `python
run_all_tests.py`.
```

What #44 delivered (your a/b/c)

```
- **(b) `docs/CODE_MAP.md` regenerated** for the layered layout via the multi-agent build-code-
map workflow ? every `@path::symbol` anchor adversarially verified against the real tree (**0
broken**), now covering `backend/pipeline/?`, `backend/infrastructure/`, `backend/config/`,
`backend/reporting/`.
- **(c) `run_all_tests.py`** at the repo root ? one command, clear PASS/FAIL + pytest exit code,
flags the optional `obsreport` dep; README points to it.
- **(a) Grok coordination notes** in `BACKLOG.md` + `LOW_REGRET_MOVES_PLAN.md` (items 1?2 done;
8 subsumed; 7 before AI-timing; 3 synergy; 4/5/6 independent).
```

One thing to hand Grok (not blocking)

```
The CODE_MAP regeneration **surfaced two real config-migration bugs** (from #39/#40, *not* mine
? left for Grok's config pass, documented in `BACKLOG.md` + `CODE_MAP.md §2.0`):
1. `/api/compile` **ignores `--output-dir`** ? `OutputConfig` has no `output_dir` field, so it
always falls back to `"output"`.
2. `--output-dir` lives only on a dynamic `global_config._runtime_overrides` attr (fragile).

Fix is small: add `output_dir` to a config/runtime model and read it consistently.
```

Carry-forward checkpointed

```
`current-state.md`, `session-handoff.md`, and the `code-map-artifact` note are all updated to
the final shipped state ? a fresh session can resume cleanly from them.

Thanks for the great back-and-forth ? the observability layer is fully shipped, debuggable-by-
design, and validated. Enjoy the break! ?
```